HIP04 处理器用户指南

文档版本 1.0
发布日期 20140421

版权所有 © 深圳市海思半导体有限公司 2013。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明

、HISILICON、海思和其他海思商标均为深圳市海思半导体有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受海思公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，海思公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

深圳市海思半导体有限公司

地址： 深圳市龙岗区坂田华为基地华为电气生产中心 邮编：518129

网址： http://www.hisilicon.com

客户服务电话： +86-755-28788858

客户服务传真： +86-755-28357515

客户服务邮箱： support@hisilicon.com

海思专有和保密信息
版权所有 © 深圳市海思半导体有限公司

目　录

目　录 3

1 芯片综述 1

1.1 特性列表 1

1.1.1 处理器特性 1

1.1.2 存储特性 1

1.1.3 加速器特性 2

1.1.4 虚拟化特性 2

1.1.5 低功耗特性 2

1.1.6 DFX 特性 3

1.1.7 其他特性 4

1.1.8 遵从的标准与协议 4

1.2 全局数据结构 7

1.2.1 报文解析描述符结构 7

1.2.2 报文发送描述符结构 10

1.3 数据大小端模式 13

1.3.1 大小端处理单元之间的数据交互： 13

1.3.2 通讯接口的大小端处理区别： 13

1.3.3 在工作过程中各个阶段的描述 15

1.3.4 HIP04 接收到的报文在各个阶段的存储示意 16

2 模块详述 18

2.1 CPU Core 18

2.1.1 概述 18

2.1.2 特性 18

2.1.3 功能描述 19

2.2 CPU Cluster 20

2.2.1 概述 20

2.2.2 特性 20

2.2.3 关键术语 20

2.2.4 功能描述 20

2.2.5 工作方式 23

2.2.6 调试（Debug） 24
2.2.7 注意事项 24
2.3 TIMER 25
2.3.1 概述 25
2.3.2 特性 25
2.3.3 关键术语 25
2.3.4 功能描述 26
2.3.5 工作方式 27
2.4 WatchDog 31
2.4.1 概述 31
2.4.2 特性 31
2.4.3 功能描述 31
2.4.4 工作方式 32
2.4.5 关闭 WatchDog 33
2.4.6 注意事项 33
2.5 L3 Cache 35
2.5.1 概述 35
2.5.2 特性 35
2.5.3 关键术语 36
2.5.4 功能描述 36
2.5.5 工作方式 41
2.5.6 调试（Debug） 43
2.6 中断控制模块（GIC） 45
2.6.1 概述 45
2.6.2 特性 45
2.6.3 关键术语 45
2.6.4 功能描述 46
2.6.5 工作方式 52
2.6.6 性能 54
2.6.7 调试（Debug） 54
2.7 系统存储管理单元（SMMU） 55
2.7.1 概述 55
2.7.2 特性 55
2.7.3 关键术语 55
2.7.4 功能描述 56
2.7.5 工作方式 60
2.7.6 性能 61
2.7.7 调试（Debug） 62
2.7.8 注意事项 62
2.8 DDR 控制器（DDRC） 63

2.8.1 概述 63
2.8.2 功能描述 63
2.9 包解析引擎（PPE） 64
2.9.1 概述 64
2.9.2 特性 64
2.9.3 关键术语 65
2.9.4 功能描述 66
2.9.5 工作方式 68
2.9.6 性能 72
2.9.7 调试（Debug） 72
2.10 GE 网口（GE） 73
2.10.1 概述 73
2.10.2 特性 73
2.10.3 关键术语 74
2.10.4 功能描述 74
2.10.5 工作方式 76
2.11 USB 2.0 Host 89
2.11.1 概述 89
2.11.2 特性 89
2.11.3 接口信号 89
2.11.4 功能描述 90
2.11.5 工作方式 92
2.12 USB2.0 Device 94
2.12.1 概述 94
2.12.2 特性 94
2.12.3 关键术语 94
2.12.4 接口信号 95
2.12.5 功能描述 95
2.12.6 工作方式 97
2.13 MMC/eMMC/SD/SDIO 存储控制接口（MMC） 100
2.13.1 概述 100
2.13.2 特性 100
2.13.3 关键术语 100
2.13.4 接口信号 101
2.13.5 功能描述 101
2.14 Local Bus 105
2.14.1 概述 105
2.15 NAND 存储控制接口（NANDC） 106
2.15.1 概述 106
2.15.2 特性 106

2.15.3 关键术语 106
2.15.4 接口信号 107
2.15.5 功能描述 109
2.15.6 工作方式 116
2.15.7 注意事项 118
2.16 SPI FLASH 接口（SFC） 119
2.16.1 概述 119
2.16.2 特性 119
2.16.3 关键术语 119
2.16.4 接口信号 119
2.16.5 功能描述 120
2.16.6 工作方式 123
2.17 MDIO 接口（MDIO） 124
2.17.1 概述 124
2.17.2 特性 124
2.17.3 关键术语 124
2.17.4 接口信号 124
2.17.5 功能描述 125
2.18 UART 接口（UART） 127
2.18.1 概述 127
2.18.2 特性 127
2.18.3 关键术语 127
2.18.4 接口信号 128
2.18.5 功能描述 128
2.18.6 工作方式 132
2.19 I2C 接口（I2C） 136
2.19.1 概述 136
2.19.2 特性 136
2.19.3 关键术语 136
2.19.4 接口信号 136
2.19.5 功能描述 137
2.19.6 工作方式 140
2.20 GPIO 接口（GPIO） 144
2.20.1 概述 144
2.20.2 特性 144
2.20.3 关键术语 144
2.20.4 功能描述 144
2.20.5 工作方式 146
2.21 JTAG 接口（JTAG） 149

2.21.1 概述 149
2.21.2 特性 149
3 存储与中断映射 150
3.1 存储映射 150
3.1.1 概述 150
3.1.2 特性 150
3.1.3 关键术语 151
3.2 中断映射 155
3.2.1 概述 155
3.2.2 特性 155
3.2.3 关键术语 156
3.2.4 中断向量表 156
3.2.5 安全扩展 160
3.2.6 1-N 中断分发 160
3.2.7 N-N 中断分发 160
3.2.8 使用 SGI 实现核间消息通信 161
3.2.9 中断优先级 161
3.2.10 中断状态 161
3.2.11 特殊中断号 162
3.2.12 电平触发和沿触发 162
4 时钟与复位 163
4.1 时钟 163
4.1.1 概述 163
4.1.2 特性 163
4.1.3 关键术语 163
4.1.4 时钟控制 164
4.1.5 时钟结构 165
4.1.6 PLL 功能描述 165
4.1.7 时钟配置 166
4.1.8 系统时钟选择配置 167
4.1.9 低功耗的时钟管理 170
4.1.10 时钟门控 170
4.2 复位 171
4.2.1 概述 171
4.2.2 特性 171
4.2.3 关键术语 171
4.2.4 复位接口信号 171
4.2.5 复位控制 172
4.2.6 复位配置 173

4.2.7 低功耗的复位管理 176
5 系统初始化 177
5.1 概述 177
5.2 特性 177
系统初始化包括以下特性： 177
5.3 关键术语 177
5.4 BOOT 启动流程 178
5.4.1 概述 178
5.4.2 NORFLASH BOOT 178
5.4.3 NANDFLASH BOOT 179
6 系统功耗 181
6.1 概述 181
6.2 特性 181
6.3 关键术语 182
6.4 功耗分布 182
6.5 功耗组成和计算 184
6.6 功耗控制 185
6.6.1 Power Supply domain 185
6.6.2 Clock Gating & DFS 186
6.6.3 Power Gating 187
6.6.4 pd_core 子电源域 188
6.6.5 AVS 188
6.6.6 低功耗控制结构 198
6.6.7 Power Down 调试 199
6.6.8 PMU 接口 200
6.7 工作方式 202
6.7.1 静态 power gating 202
6.7.2 动态 Power Gating 202
6.7.3 AVS 动态温度补偿配置流程 1 – IMVP 接口 202
6.7.4 AVS 动态温度补偿配置流程 1 – PMBus 接口 203
6.7.5 AVS0 HPM 静态测量 204

1 芯片综述

1.1 特性列表

1.1.1 处理器特性

- 集成 16 个 Cortex-A15 CPU Core，最高工作频率为 1.5GHz，可提供最大 84000 DMIPS 的处理性能；
- 支持 16 个 CPU 任意组合配置为 AMP/SMP；
- 支持 16 个 CPU 任意组合实现虚拟化；
- 数据小端模式或大端模式可配置，指令固定为小端；
- 支持 ARMv7-A 指令集，支持浮点运算的 VFPv4 指令集；
- CPU Core 支持 1TB 地址直接映射，可访问所有的寄存器、片内/片外 Memory；
- 支持灵活路由的中断控制；
- 每个 CPU Core 包含 4 个 64-bits 定时器；
- 系统提供 32 个 32 bits Timer 和 16 个 64 bits Timer，共 48 个公共 Timer；
- 系统提供 16 个 WatchDog，每个 CPU Core 独占一个；
- 全局中断管理 GIC，完成外部中断，内部中软及软件中断的选择，路由至 16 CPU Cores 之一处理，支持安全扩展和虚拟化扩展；
- 支持 CPU Core Trace 调试。

1.1.2 存储特性

- 支持 3 级 Cache，每个 CPU Core 内包括指令和数据各 32KB 的 L1 Cache，每 4 个 CPU Core，即 CPU Cluster 共享一个 1MB L2 Cache，芯片所有主设备，包括 CPU、IO、加速器再共享一个 8MB 的 L3 Cache；
- 支持基于 AMBA 协议的互联，基于 MOESI 模式，以 Directory 结构支持片内 L1/L2/L3 Cache、Memory、IO、Accelerator 的完全数据一致性互联；
- 提供两个 64 bits DDR3 存储器接口，最高数据率 1600Mbps，可以访问最大 64GB 外存空间；
- 支持全局的双指令（Exclusive）原子操作，可完成加、减、与、或、SWAP 读-改-写原子操作；

- 提供 1MB 的片上 SRAM，SRAM 额外支持加、减、与、或的单指令原子操作加速（单次写完成原子操作），支持配置固定空间作为片上安全存储；
- 提供两个独立的 DMA 单元，加速各 Memory/IO 之间的数据搬运；
- 全系统存储空间的访问支持 QoS 进行流量管理；

1.1.3 加速器特性

- 提供硬件包解析模块 PPE，用以在数据通路的 GE/XGE 接口线速对包基本类型进行解析，将解析结果按照规定格式写入包描述符中，可根据系统缓存状态，对高低优先级报文在端口上采用不同的丢弃策略；模块实现 OAM Y.1731 标准所必须的硬件统计功能；
- 数据通路支持 4 个 GE Serdes 接口，包含 4 个 Lane，速率支持 10M/100M/1G/2.5Gbps/5Gbps（TBD）；
- 提供丰富的通用/调试接口包括：
  - 1 个 NANDC 接口，可以对接 NAND FLASH；
  - 1 个 Local Bus 接口，可以对接 NOR FLASH 或 Intel/Moto 时序器件；
  - 1 个 SFC 接口，可以对接 SPI FLASH；
  - 1 个独立的 10M/100Mbps MII 接口；
  - 1 个独立的 10M/100M/1Gbps RGMII 接口；
  - 1 个 MII 和 RGMII 复用的网口，可以配成 MII 或者 RGMII 使用；
  - 2 个 USB2.0 Host 接口和 1 个 USB 2.0 Device 接口；
  - 3 个 UART 接口；
  - 2 个 I2C 接口；
  - 1 个 SPI 接口，每个 SPI 提供 4 个片选；
  - 1 个 MMC 接口，可以配置成 EMMC 或 MMC 接口；
  - 1 个 JTAG 接口，支持 IEEE1149.1/IEEE1149.6 边界扫描，CPU 软件调试及芯片内硬件调试，通过 JTAG 接口进行芯片内硬件调试参见下面中的“DJTAG”章节；
  - 2 个 MDIO 接口能控制最多 16 个 PHY 器件。

1.1.4 虚拟化特性

- 提供系统虚拟化，包括 CPU 虚拟化、中断虚拟化、页表虚拟化、IO 设备虚拟化等多项虚拟化技术，使得多个业务的操作系统可以运行在一个中间层（Hypervisor）之上并共用一套硬件配置，每个操作系统按照原有的方式运行并只看到属于自己的资源，互相不干涉对方的资源，Hypervisor 按一定策略在不同的时间片切换运行于其上的 GuestOS，并使得旧系统迁移到虚拟化系统上的移植过程变得相对简单。

1.1.5 低功耗特性

- 支持 Power Gating：
  - 每个 Cortex A15 核支持 Power Gating；
  - L3 Cache 中的 SRAM、各个加速引擎中集成的 8KB 以上的 SRAM、片上 SRAM 支持 Power Gating；

  - DDRC（及 DDR 接口）模块可独立 Power Gating；
- 支持 Clock Gating：
  - 数据通路各个模块在不使用时，均可独立关断模块时钟以节省功耗；
  - 数据通路各个模块在运行中，均可自动关闭模块内未使用逻辑的时钟；
- 支持 DFS：
  - 支持 Cortex A15 核与 DDR/ACC 之间的频率按照 N:1 方式动态切换，切换过程中 DDR 将有 10000 DDR Cycles 不可使用；
- 支持 AVS：
  - 支持工艺静态补偿，即支持通过 Speed monitor 频率测试结合频率筛选测试为不同的芯片选择不同的默认工作电压；
  - 支持运行过程中的温度补偿，即当 Speed monitor 检测到在保证当前工作频率不变时可升/降压时自动调整当前工作电压；支持 AVS 保护机制；
- 支持 SRAM 的低功耗自动管理，数据通路上集成的 SRAM 支持 Retention Mode 和 Light Sleep 模式（能够在几个 Cycle 内唤醒）：
  - 在模块不使能时，SRAM 自动进入 Retention Mode，功耗降低至只保留数据信息的水平；
  - 在模块使能，但暂时没有数据处理时，SRAM 自动进入 Light Sleep，可降低 SRAM 大部份逻辑功耗。

1.1.6 DFX 特性

- 提供统一的整芯片 Memory BIST 功能，支持片内所有 SRAM、ROM 及片外 DDR 的 BIST 检测；
  - 片内 SRAM、ROM 支持 MBIST 测试，L1/L2/L3 Cache 及总线 Directory 额外支持直接读写、错误注入、访问次数统计、捕获设定的访问地址功能；
  - DDR 支持初始化、遍历测试、直接读写功能；
  - Memory BIST 功能由 CPU 和 JTAG 控制；
  - 支持 Memory Repair 功能；
- GE 提供网络报文调试功能；
  - 支持包捕获功能，包括基于特殊协议帧类型配置其捕获、Jumbo 帧捕获、异常报文捕获；
  - 支持报文在业务运行中，CPU 指定的无损环回；
  - 支持报文在 MAC 层的内环回和外环回；
- 片上 L1 Instruction Cache 使用的 SRAM 支持 Parity，L1 Data Cache、L2 Cache、L3 Cache、片上 SRAM 及重要模块使用的 SRAM 支持 SECDED ECC；DDR 支持 SECDED ECC；
  - ECC 出错时，支持错误地址记录；
  - L1/L2/L3 Cache 及总线 Directory 和 DDR 支持 ECC 不使能时直接读写 ECC RAM；
- 可提供总线及 CACHE 相关行为统计，协助分析系统性能；
- TSensor 可以探测芯片实时结温，并进行数字量化，CPU 可以读取 TSensor 获取送出 8 bits 量化信息。还可通过设定温度门限，以中断方式实现过热保护功能；

1.1.7 其他特性

- 时钟特性如下：
  - 系统时钟由外部晶振通过输入时钟管脚 CLKIN 提供，为 25~75MHz；
  - 系统内部各模块高频工作时钟由内部 PLL 基于晶振时钟 CLKIN 生成；
  - 系统上电默认工作在晶振时钟，PLL 锁定后可动态切换到高频工作时钟；
  - 系统内 RGMII 等 IO 接口时钟固定，由专用 PLL 生成；
- 复位特性如下：
  - 上电复位由片外提供，输入一个低电平脉冲，触发系统整体复位，上电复位信号支持滤毛刺处理，有效复位低电平维持时间需要大于 64 个 CLKIN 时钟周期；
  - 上电复位期间，输出复位信号 HRESET（低电平），平维持时间大于 512 个 CLKIN 时钟周期，HRESET 信号为 Open Drain 信号；

  支持片内 WDG 超时触发软复位；
- 物理特性如下：
  - 封装结构为 37.5mm x 37.5mm；
- 可靠性特性如下：
  - 使用寿命为 10 年；
  - ESD：HBM +/-2000V，CDM +/-200V
  - Latch Up：Itrigger 100mA，Vtrigger 1.5 倍 Vdd；
  - 潮敏等级为 3 级；
  - 工作环境温度为工业级标准：-40～85 ℃；
  - 最大工作结温为 125°C，长期工作结温为 105°C（不受降额影响）；
- 功耗特性如下：
  - CPU 运行频率为 1.5GHz 时，全芯片典型功耗（Tj = 105℃）约为 18W；

1.1.8 遵从的标准与协议

- DDRC 遵从如下标准：
  - JESD79-3E; July 2010
  - JESD79-2E; April 2008

- JTAG 遵从如下标准：
  - 1149.6TM IEEE Standard for Boundary-Scan Testing of Advanced Digital Networks

1.2 全局数据结构

1.2.1 报文解析描述符结构

报文解析描述符数据结构如下表所示。

<table>
<tr><td>31</td><td>30</td><td>29</td><td>28</td><td>27</td><td>26</td><td>25</td><td>24</td><td>23</td><td>22</td><td>21</td><td>20</td><td>19</td><td>18</td><td>17</td><td>16</td><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="8">Buf_Num</td><td colspan="8">Node_Code</td><td colspan="16">Packet_Len</td></tr>
<tr><td colspan="8">Back</td><td colspan="2">Reserved</td><td colspan="6">Pool</td><td colspan="16">Size</td></tr>
<tr><td colspan="32">Next_Buf_Addr</td></tr>
<tr><td colspan="16">Reserved</td><td colspan="16">Offset</td></tr>
<tr><td colspan="8">IP_Offset</td><td>Reserved</td><td>FRAG</td><td colspan="6">Reserved</td><td colspan="6">Reserved</td><td>DMAC</td><td>Reserved</td><td>L3E</td><td>L2E</td><td>DROP</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">Reserved</td></tr>
<tr><td colspan="32">说 明</td></tr>
</table>

Packet_Len	报文的总长度，单位：字节；高 1 位是保留位。
Node_Code	7 位消息源设备号。
Buf_Num	存储整个报文消耗的缓存个数，如果个数超过 1 个，表示使用了链表结构来存储整段报文。
Size	表示当前该缓存中存储报文数据的长度，单位：字节；高 1 位是保留位。
Pool	表示当前这块缓存属于内存池中的哪个 Pool，用于释放缓存。
Back	表示当前缓存起始地址到解析描述符的偏移，以 Cache-Line（16 个 word）为单位，用于释放缓存。
Offset	表示当前报文数据相对于报文指针的偏移，单位：字节。
Next_Buf_Addr	对于链表方式存储的报文，该域表示下一段链表节点的缓存中存放报文数据的地址；对于非链表存储结构的报文，该域为 0。
Offset	报文数据相对于当前报文地址的偏移，报文地址是当前 BD 提供的下一缓存的报文地址。
DROP	接收的报文被截断标志：
	■ 1’b0：报文完整被接收；
	■ 1’b1：报文接收不完整。
L2E	二层报文解析错误标志：
	■ 1’b0：无二层错误；
	■ 1’b1：二层出错。
L3E	三层报文解析错误标志：
	■ 1’b0：无三层错误；
	■ 1’b1：三层出错。
DMAC	二层报文类别编码：
	■ 2’b00：单播报文；

	■ 2’b01：多播报文；
	■ 2’b10：广播报文；
	■ 2’b11：保留。
FRAG	IP 分片报文标识：
	■ 1’b0：非 IP 分片报文；
	■ 1’b1：IP 分片报文。
IP_Offset	报文头到 IP 头的偏移，单位为字节。

下一链表结构

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Back								Reserved			Pool					Size															
Next_Buf_Addr																															
Reserved																Offset															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															

Reserved	
Reserved	
Reserved	
Reserved	
说 明	
Size	表示当前该缓存中存储报文数据的长度，单位：字节。
Pool	表示当前这块缓存属于内存池中的哪个 Pool，用于释放缓存。
Back	表示当前缓存起始地址到解析描述符的偏移，以 Cache-Line（16 个 word）为单位，用于释放缓存。
Offset	表示当前报文数据相对于报文指针的便宜，单位：字节。
Next_Buf_Addr	对于链表方式存储的报文，该域表示下一段链表节点的缓存中存放报文数据的地址；对于非链表存储结构的报文，该域为 0。

1.2.2 报文发送描述符结构

报文发送描述符数据结构如下表所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Send_Addr																															
Back								RI	RA	FM	Pool					Send_Size															
Next_Buf_Addr																													Reserved		
IP_Offset								DS								Reserved		SD	LB	Reserved		Reserved							L3CS	CL	Reserved
WB_Addr[31:24]								WB_Addr[23:16]								WB_Addr[15:8]						WB_Addr[7:0]									
说 明																															

Send_Addr	发送数据的开始地址。
Send_Size	当前缓冲区中发送数据长度，单位：字节。
Pool	释放数据的 pool ID。
FM	发送完成释放缓存空间。
RA	读分配（Read-Allocate）标志，控制是否发送数据读入 L3 cache：
	■ 1’b0：数据不读入 L3-Cache；
	■ 1’b1：数据读入 L3-Cache。
RI	在 PPE 将报文发送出去以后，可以选择是否对 Cache 进行 Invalidate 操作：
	■ 1’b0：报文发送完成后不对 Cache 进行 Invalidate 操作；
	■ 1’b1：报文发送完成后对 Cache 中相应位置进行 Invalidate 操作。
Back	数据所在 cache line 首地址与缓存首地址之间距离，以 Cache-Line（64 个字节）为单位。
Next_Buf_Addr	对于链表方式存储的报文，该域表示下一链表结构的起始地址；对于非链表存储结构的报文，该域为 0。
CL	发送完成后是否需要向指定地址“WB_Addr”写 0：
	■ 1’b0：不进行回写操作；
	■ 1’b1：报文发送完成后对“WB_Addr”域指定的地址回写 0x0。
L3CS	报文发送时，控制是否对三层附加 Checksum：
	■ 1’b0：不对三层附加 Checksum；
	■ 1’b1：对三层附加 Checksum。
LB	环回报文标志。
DS	表示数据段个数，当 SD 域为 1 时有效。
SD	为 0 时，帧只使用了一个缓存；为 1 时，帧使用了多个缓存。

IP_Offset	IP 数据包头在数据包中的偏移位置，单位：字节。
WB_Addr	发送数据完成回写清 0 地址。

下一链表结构

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Send_Addr																															
Back								RI	RA	FM	Pool					Send_Size															
Next_Buf_Addr																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
Reserved																															
说 明																															

Send_Addr	发送数据的开始地址。
Send_Size	缓冲区中发送数据长度，单位：字节。
Pool	释放数据的 pool ID。
FM	发送完成释放缓存空间。
C	发送数据读入 L3 cache。
RI	在 PPE 将报文发送出去以后，可以选择是否对 Cache 进行 Invalidate 操作。
Back	数据所在 cache line 首地址与缓存首地址之间距离，以 Cache-Line（64 个字节）为单位。
Offset	数据头距离数据头所在 cache line 首地址的偏移。
Next_Buf_Addr	对于链表方式存储的报文，该域表示下一链表结构的起始地址；对于非链表存储结构的报文，该域为 0。

1.3 数据大小端模式

Hip04 整芯片的数据存储要求既支持大端也支持小端模式与 Power-PC 的大端模式兼容；对于指令，ARM 处理器比较特殊：指令恒为小端模式。

除了处理器核之外， 对于外设的编程或者配置接口（即寄存器接口），原设计一般都是按小端解析，寄存器描述时按照小端方式编写；CPU 工作在大端模式访问寄存器接口时，需要转换成小端（字节序反转）进行配置，软件编写时如果需要支持大端模式则通过宏控制转换为小端后访问寄存器接口，如果整系统工作在小端模式则不反向配置。

Hip04 中的大端是 BE-8 格式。

1.3.1 大小端处理单元之间的数据交互：

数据以大端或者小端在主存中的存储格式，跟数据位宽有关。

不同处理单元如果以不同的大小端模式工作，在读取数据时应该转换。

在 HiP04 中，均按照 32 位的数据格式存储，即将数据按照 Word 写入和读取（即 32 位，4 个字节）。

1.3.2 通讯接口的大小端处理区别：

如果按照大端模式收发，则接收时将先收到的字节存放在一个 16 位/32 位/64 位（16 位、32 位或者 64 位是通讯接口内部和访问内存的并行数据宽度，每处理完一个 16 位、32 位或者 64 位数据后地址总是递增的）位数据的低字节上，写满一个 16 位、32 位或者

64 位数据后直接写入内存不进行字节序转换（注意这是在内存存储看到的数据，而不是程序员编程看到的数据），同样发送也是先发送低字节，此为大端模式；如果是小端方式的话将先收到的字节存放在高位，发送也先发送高位上的字节；如果是 UART、I2C 和 SPI 等这类字节流的话与大小端无关。

Hip04 芯片内部总线和桥中涉及到数据拼接和拆分的地方也需要注意按照同样的方式处理。

Hip04 各个模块的大小端特性如所示。

表1-1 Hip04 各个模块的大小端特性表

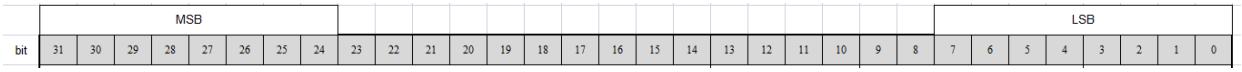
模块	Slave 编程接口	Master 访问内存中的 BD	Master 访问内存中的数据
A15	不涉及	大小端可编程	大小端可编程，网络应用为大端
GMAC/PPE	仅支持小端	大小端可编程	只支持大端
I2C	仅支持小端	不涉及	不涉及
UART	仅支持小端	不涉及	不涉及
SPI	仅支持小端	不涉及	不涉及
NANDC	仅支持小端	不涉及	不涉及
Local Bus	仅支持小端	不涉及	不涉及
SMMU	仅支持小端	不涉及	页表大小端可编程，Hardware table walk 支持大小端可编程
GIC	仅支持小端	不涉及	不涉及
Timer	仅支持小端	不涉及	不涉及
GPIO	仅支持小端	不涉及	不涉及
IOCFG	仅支持小端	不涉及	不涉及
SYSCTRL	仅支持小端	不涉及	不涉及
DDRC	仅支持小端	不涉及	不涉及

L2/L3 Cache	仅支持小端	不涉及	不涉及
DDR/Cache/SRAM	不涉及	不涉及	不涉及
NAND/NOR/Local Bus	对于 FLASH，烧写指令需要底软转换，FLASH 文件系统不需要修改；	不涉及	不涉及
Tsensor	仅支持小端	不涉及	不涉及

1.3.3 在工作过程中各个阶段的描述

1.3.3.1 寄存器访问

寄存器统一采用小端访问方式，其中各个 bit 和 MSB、LSB 的映射关系如下：

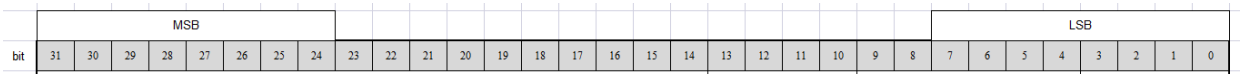


在总线上的行为：0x0 地址就是对应于 LSB，0x3 地址就是对应于 MSB。

工作在数据大端的 A15，在访问寄存器时，需要采用指令转换后配置在寄存器接口上；

1.3.3.2 存储在 DDR 中的数据结构

对应于全局数据结构，MSB 和 LSB 的定义如下：



1.3.3.2.1 在工作在大端模式时

MSB 存放在 0x0 地址，LSB 存放在 0x3 地址，对应于 AXI 总线上也是如此，即对于 128bit 的数据线，其中 bit[7:0]存放的是 MSB，bit[31:24]存放的是 LSB；

1.3.3.2.2 在工作在小端模式时

MSB 存放在 0x3 地址，LSB 存放在 0x0 地址，对应于 AXI 总线上也是如此，即对于 128bit 的数据线，其中 bit[7:0]存放的是 LSB，bit[31:24]存放的是 MSB；

1.3.3.3 存储在 DDR 中的报文数据

对于报文数据，MSB 和 LSB 的定义如下：其中 bit0 是在网口上收到的报文数据的第 1 个 bit。

HIP04 中，存储在 DDR 中的报文只支持大端方式存储。

	MSB																															LSB
网口接受bit顺序	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
IPv4报文举例	Version				Header				TOS								Total Length															
	ID																Flags			Fragment Offset												

注意

上面图中的各个 bit 标号是接口上各个比特接收的顺序，和内部存储的 bit0、bit1 等没有关系。

1.3.3.3.1 当工作在大端模式时

MSB 存放在 0x0 地址，LSB 存放在 0x3 地址，对应于 AXI 总线上也是如此，即对于 128bit 的数据线，其中 bit[7:0]存放的是 MSB（对应于报文数据的第 24 到 31 个 bit），bit[31;24]存放的是 LSB（对应于报文数据的前 8 个比特）；后续报文以 32bit 为单位类推。

1.3.3.4 对存放于 DDR 中的数据统一采用大端模式或者小端模式

A15 工作时，数据配置为大端或者小端；存放在 DDR 中的数据都统一成大端或者小端模式，那么 A15 访问时，就不需区分数据进行指令级别的字节倒换。

1.3.4 HIP04 接收到的报文在各个阶段的存储示意

步骤 1 在 PPE 接口上，其中 Version 中的 msb(most significant bit)先收到，为 bit0；

	MSB																															LSB
网口接受bit顺序	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
IPv4报文举例	Version（0x1）				Header(0x2)				TOS(0x34)								Total Length(0x5678)															
	ID(0x9abc)																Flags(0x7)			Fragment Offset(0x0def)												

步骤 2 经过 PPE 存储之后，在 DDR 中的存储的数据结构如下

地址	bit31~24	bit23~16	bit15~8	bit7~0
0x0	Total Length lsb (0x78)	Total Length msb (0x56)	TOS (0x34)	Version, Header (0x12)
0x4	fragment offset lsb (oxef)	flags, fragment offset msb (0xed)	ID LSB (0xbc)	ID MSB (0x9a)

步骤 3 在上一步中，看到报文在 DDR 中按照大端存储，bit7~0 对应于接收到第一个报文字节；工作在数据大端模式下，A15 会在一个 Word（32 位）内，字节进行翻转，此时软件看到的数据结构就变为：

	MSB																															LSB
软件看到的域段	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IPv4报文举例	Version（0x1）				Header(0x2)				TOS(0x34)								Total Length(0x5678)															
	ID(0x9abc)																Flags(0x7)			Fragment Offset(0x0def)												

步骤 4 各个加速器按照类似方式进行解析和使用数据。

2 模块详述

2.1 CPU Core

2.1.1 概述

Cortex-A15 是基于 ARMv7-A 架构的一款高性能、低功耗的处理器,详见 ARM 文档。

2.1.2 特性

Cortex-A15支持以下特性：

- 完全支持 ARMv7-A 架构指令集；
- 3 发射的超标量结构，支持乱序执行；
- 支持动态分支预测，包含 Branch Target Buffer (BTB) 和 Global History Buffer (GHB)；
- 集成 MMU，包括：3 个独立的容量为 32 项的全相联一级快表（TLB）分别用于指令、数据读、数据写，同时支持 4 路组相联包含 512 个表项的二级快表；
- 集成 32K/32K Bytes 大小的一级数据/指令 Cache，数据 Cache 支持 SECDED ECC 校验，指令 Cache 支持奇偶校验；
- 提供一个数据和指令混合的总线接口,接口能够通过 Coherency 总线保证多个 CPU 间以及与 IO 和加速器的数据一致性，读写数据位宽各为 128 bits；
- 最大支持 40 bits 物理地址；
- 数据处理支持大、小端模式，大端模式为 BE-8。CPU 上电时的端模式管脚可配，上电后寄存器可配。指令处理固定为小端；
- 支持对 Memory 非对齐访问；
- 支持基于 PMUv2 架构的性能检测单元（PMU）；
- 集成 VFP，支持 VFPv4 指令集；
- 支持 TrustZone 安全扩展；
- 基于 ARMv7.1 调试架构，支持 Coresight；
- 支持 PTM(Program Trace Macrocell)接口；
- 集成 4 个 64 bits 的定时器；

- 集成双重页表硬件转换 MMU 和中断路由，支持 CPU 虚拟化；
- 支持功能模式下启动 MBIST 测试。

2.1.3 功能描述

2.1.3.1 参考文档

Cortex-A15 Core L1 Cache、L2 Cache、MMU、CP14 寄存器、CP15 寄存器描述等，详见 ARM 提供的文档：《DDI0438I_cortex_a15_r4p0_trm.pdf》。

ARM Architecture 指令集详见 ARM 提供的文档：《DDI0406C2c_arm_architecture_reference_manual.pdf》。

2.2 CPU Cluster

2.2.1 概述

Cortex-A15 Cluster 包括 4 个 Cortex-A15 CPU Core、及 4 核共享的 512KB L2 Cache。

2.2.2 特性

Cortex-A15 Cluster 具有以下特性：

- 一个 Cluster 内包含 4 个 Cortex-A15 CPU Core 共享一个 512KB L2 Cache；
- 每个 Cluster 和总线之间，支持时钟比例动态调整为 N:1（N 为正整数）；
- L2 Cache 支持 SECDED ECC 校验；
- 支持多核交互调试（Cross Trigger）；
- 支持多核多电源域，可控制单核掉电，降低功耗。

2.2.3 关键术语

如表 2-1 的术语将在本章节表示特殊的含义。

表2-1 关键术语描述

术语	描述
SCU	Snoop Control Unit。Cortex-A15 多核 CPU 中控制 Cache 一致性的逻辑单元。
Cluster	多 CPU 核的集合。包含多个 CPU core、一致性控制逻辑和相关外设。
CRG	Clock and Reset Generator。时钟复位发生器，控制系统中各个模块的时钟和复位。
UNK	Unknown。在寄存器描述处，表示此寄存器值不定，软件操作不可依赖于此寄存器值。

2.2.4 功能描述

2.2.4.1 架构框图

Cluster 的架构如图 2-1 所示。主要包含四个 Cortex-A15 CPU core、一个 L2 Cache、定时器和调试模块。每个 Cortex-A15 CPU core 各自带有 32KB 的数据和指令 Cache，L2 Cache 控制器内包含一致性控制逻辑。一致性控制逻辑内部包含 L1 Cache 的 TAG 备份，用于加速一致性。Trace 和 Debug 用于 Cortex-A15 的调试。每个 CPU core 配备一个定时器。

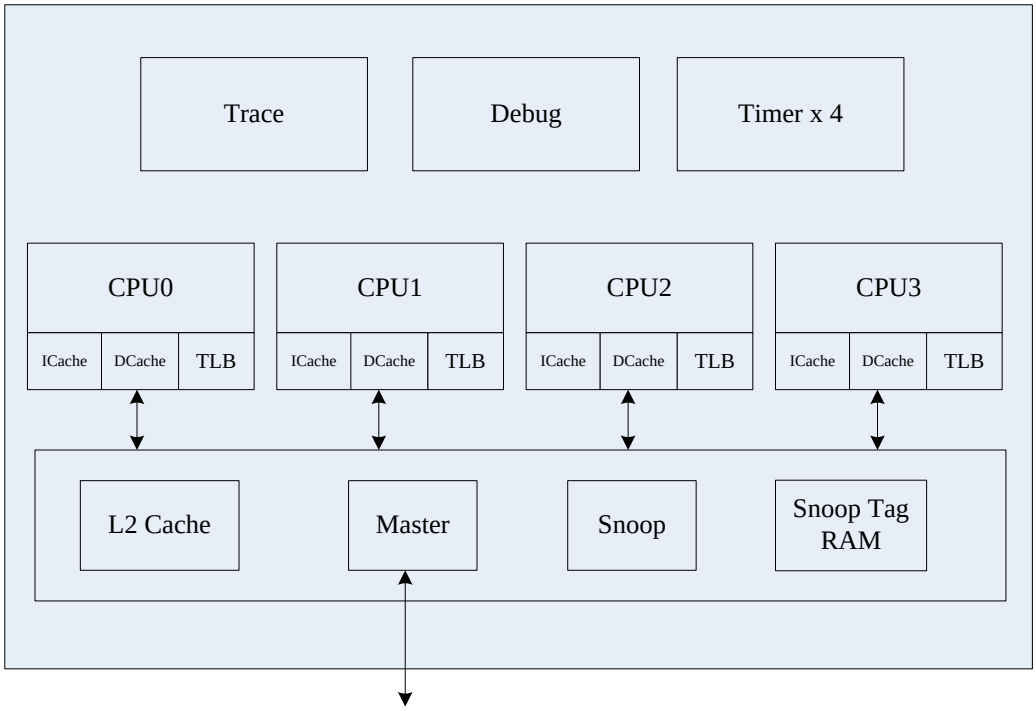


图2-1 Cortex-A15 多核结构图

2.2.4.2 时钟和复位

Cortex-A15 Cluster 中的多个 CPU core 工作在相同频率，可根据应用需要单独关闭每个 CPU core 的时钟。

多个 CPU core 支持单独软复位，复位可由主核通过系统控制寄存器控制 CRG 完成。

CPU 和 DDR/ACC 支持 N:1 的频率比例，N 为正整数，在该比例关系下，系统 CPU 频率可在初始化任意设置，设置后，如果需要动态切换，DDR 将有 10000 Cycle 停顿；

2.2.4.3 电源域

Cluster 内多个 CPU core 支持单独掉电，可根据应用需要单独关闭每个 CPU core 的电源，以节省功耗。

2.2.4.4 L2 Cache 存储系统

L2 Cache 存储系统由一个 L2 Cache 和一个内部的监听控制单元组成。在 Cortex-A15 Cluster 内可最多将四个处理器核连接在一起。

L2 Cache 存储系统包括：

- L2 Cache 大小为 512KB；
- Cache line 长度固定为 64 Bytes；
- 二级 Cache 使用物理地址索引；
- 16 路组相联结构；

- 随机替换 Cache 策略；
- 支持 ECC 校验；
- 支持硬件预取；
- 软件可配置的 RAM 访问延时；
- 支持存储器内建自测试；

二级 Cache 的使能和关闭

二级 Cache 通过设置 SCTLR 寄存器的 C bit 使能。

使能 Cache 的顺序如下（必要顺序）:

1. 完成 CPU 的复位；
2. 设置 SCTLR 寄存器的 I-bit 和 C-bit；

注意

Hip04 中二级 Cache 支持 ECC 校验，在配置 C-bit 之前必须使能 ECC 校验。

关闭二级 Cache 的顺序如下（必要顺序）:

1. 无效 C-bit；
2. 清理并无效一级和二级 Cache；

2.2.4.5 定时器

Cortex-A15 Cluster 中，每个 CPU 核都有一个定时器。定时器用来调度事件，可产生中断，基于内部的递增的计数器。包括以下特性：

- 产生定时器时间，输出中断；
- 产生事件流；
- 支持虚拟化扩展；

每个定时器的寄存器通过相应 CPU core 的 CP15 协处理接口访问。定时器的寄存器为 32 位或 64 位宽。

2.2.5 工作方式

2.2.5.1 单 CLUSTER 复位流程

对单 Cluster 复位时，主核流程如下：

步骤 1 配置 GIC，保证系统中不再有中断请求发给当前需要复位的核

步骤 2 查询系统控制器的复位状态控制寄存器（SC_CPU_RESET_STx），确保要复位的核都进入 WFI 状态；

步骤 3 查询总线中相应的 FAB_STx_IDLE_STATE 寄存器和 FAB_ST9_IDLE_STATE 寄存器，确保读写命令以及 dvm cpl 已经完成；

步骤 4 配置系统控制器 CPU 工作模式配置寄存器 3（SC_CPU_CTRL3）的 clust x_acinactm 位，关闭相应 SNOOP 接口；

步骤 5 配置 Snoop Filter 的 FAB_SF_MODE 寄存器将对应 CPU Cluster 使能位关闭；

步骤 6 查询系统控制器中的 CPU 状态寄存器(SC_CPU_ST)，确保 L2 进入 WFI 状态；

步骤 7 配置系统控制器的复位请求控制寄存器（SC_CPU_RESET_REQx），复位相应的 Cluster。

待复位 Cluster 中的主核复位流程如下：

步骤 1 软件需要保证申请的 buffer 资源等资源释放，然后停止所有软件进程。

步骤 2 执行 L1 指令和数据 Cache disable 操作设置（设置 CPU 的 SCTLR 寄存器或 HSCTLR 的 C bit 为 0）。因为其他 3 个核的 L1 cache 已经 disable，这样 Cluster 内的所有 L1 Cache 全部 disable，因此 L2 Cache 将自动 disable。L2 Cache disable 后，该核的所有取指令和数据操作直接发往下一级 Cache（即 L3 Cache），其他 Cluster 的一致性访问正常响应；

步骤 3 按照 by set/way 的方式对 L1 数据 Cache 执行 clean & invalidate 操作。此时 dirty 数据会被同步到 L2 Cache，相应的 L2 duplicate snoop tag 也被清空，这样其他核的一致性访问就不会再查找当前核了；

步骤 4 （可选）按照 by MVA 的方式对该核的私有空间地址执行 clean & invalidate 操作。这样该核残留在 L1、L2 Cache 中且不会被其他核访问的数据将被 flush 到 L3/DDR 中；

步骤 5 配置 L2 Prefetch 控制寄存器，disable L2 Prefetche；

步骤 6 clean&invalidate L2 数据 Cache；

步骤 7 若当前核在 SMP 模式下，将其从 SMP 模式切换到 AMP 模式。这样多核一致性同步操作，如 TLB 的同步将不再发给当前处于 AMP 状态的核；

步骤 8 依次执行 ISB、DSB 同步操作。确保 CP15 的寄存器操作和 cache 维护，TLB 和分支预测部件的维护操作都已经执行完成；

步骤 9 执行 DSB 同步操作；

步骤 10 执行 WFI 指令，进入睡眠模式。

待复位 Cluster 中其他 3 个核复位流程如下：

步骤 11 软件需要保证申请的 buffer 资源等资源释放，然后停止所有软件进程；

步骤 12 执行 L1 指令和数据 Cache disable 操作设置（设置 CPU 的 SCTLR 寄存器或 HSCTLR 的 C bit 为 0）。指令 Cache 被 disable 后，取值操作被发往下一级 Cache；数据 Cache 被 disable 后相对当前核退化为 write back not allocate cache，正常相应其他核的一致性访问；

步骤 13 按照 by set/way 的方式对 L1 数据 Cache 执行 clean & invalidate 操作。此时 dirty 数据会被同步到 L2 Cache，相应的 L2 duplicate snoop tag 也被清空，这样其他核的一致性访问就不会再查找当前核了；

步骤 14 （可选）按照 by MVA 的方式对该核的私有空间地址执行 clean & invalidate 操作。这样该核残留在 L1、L2 Cache 中且不会被其他核访问的数据将被 flush 到 L3/DDR 中；

步骤 15 若当前核在 SMP 模式下，将其从 SMP 模式切换到 AMP 模式。这样多核一致性同步操作，如 TLB 的同步将不再发给当前处于 AMP 状态的核；

步骤 16 依次执行 ISB、DSB 同步操作。确保 CP15 的寄存器操作和 cache 维护，TLB 和分支预测部件的维护操作都已经执行完成；

步骤 17 执行 WFI 指令，进入睡眠模式。

----结束

2.2.6 调试（Debug）

支持 Remote Trace(硬件需要支持将状态或者日志信息存储到 DDR 或者 flash 中)。

2.2.7 注意事项

操作 PMU 额外注意事项如下：

Cache 命中率的统计注意事项：
1、Level 2 data cache access - Write 这个事件很难发生，一个普通的 L1 cache miss 的软件的写操作，在 CPU 内部的操作序列如下：
--CPU 发起 readunique，此操作被统计为 L2 cache 读操作事件
--CPU 写 L1 cache，此操作被统计为 L1 cache 写操作事件
整个过程中没有 L2 cache 写事件发生，只有 L1 满发生替换写 L2 等少数场景才会出现 L2 cache 写统计事件（此时也会统计 L1 cache 写统计事件），所以 L1 cache 的写统计事件就是所有写统计事件的总和，可以作为计算 cache 命中率的分母。
2、Level 2 data cache refill - Write，上述写操作，由于 L2 cache 是 miss 的，引发 L2 cache 的 refill 操作，因此 Level 2 data cache refill - Write 操作会正确统计。

基于以上分析，如果要统计写操作的 cache 命中率，使用如下公式：
1、L1 cache 的写操作 cache 命中率=Level 1data cache refill -Write/Level 1 data cache access - Write
2、L2 cache 的写操作 cache 命中率=Level 2data cache refill - Write/Level 1 data cache access -Write

2.3 TIMER

2.3.1 概述

Timer 模块主要实现定时、计数功能，可以供操作系统用作系统时钟，以及供应用程序使用。

芯片提供 24 个 Dual-Timer 模块，Dual-Timer 0～15 包含两个功能完全相同的 32 bits Timer，Dual-Timer16~23 包含两个功能完全相同的 64 bits Timer，因此共提供 48 个 Timer 供系统使用，所有 Timer 均为公用。

2.3.2 特性

32 bits Dual-Timer 模块具有以下特点：

- Timer 的寄存器配置权限受系统外设安全控制器 TZPC 的控制，Timer 根据 TZPC 的控制可配置为安全或非安全访问，安全 Timer 仅接受安全访问，非安全 Timer 可接受安全访问或非安全访问；
- 2 个带可编程 8 位预分频器的 32 bits/16 bits 减法定时器/计数器；
- 计数时钟可配置，可选为配置系统总线时钟、晶振时钟、晶振 64 分频或者 128 分频的时钟（晶振时钟/64）；
- 支持 3 种计数模式：自由运行模式、周期模式和单次计数模式；
- 有 2 种载入计数初值的方法，分别通过 TIMERx_LOAD（x=0～7）和 TIMERx_BGLOAD 寄存器实现；
- 当前的计数值可随时读取；
- 当计数值减到 0 时会产生中断。

64 bits Dual-Timer 模块具有以下特点：

- 2 个带可编程 8 位预分频器的 64 bits/32 bits 减法定时器/计数器；
- 计数时钟可配置，可选为系统总线时钟或晶振 64 分频的时钟（晶振时钟/64）；
- 支持 3 种计数模式：自由运行模式、周期模式和单次计数模式；
- 有 2 种载入计数初值的方法，分别通过 TIMERx_LOAD（x=0～7）和 TIMERx_BGLOAD 寄存器实现；
- 当前的计数值可随时读取；
- 当计数值减到 0 的当拍会产生中断。

2.3.3 关键术语

如表 2-2 的术语将在本章节表示特殊的含义。

表2-2 Timer 关键术语描述

术语	描述
Dual-Timer	双 Timer，在系统中 Timer 是两个一组，共享一个寄存器接口。

2.3.4 功能描述

2.3.4.1 典型应用

2.3.4.1.1 TICK

16/32 bits Timer 可用于给 VxWorks 操作系统提供 TICK 时钟，控制操作系统运行快慢。一般要求是：1 秒内触发 60~100 次中断，推荐采用 10ms 触发一次中断。

说明

1、 操作系统内部对任务的调度管理是在 TICK 节拍下进行（可以把 TICK 就当作操作系统的工作时钟）。

2、 TICK 跑的越快，那么操作系统就运行得越快，任务调度、上下文切换越频繁，操作系统实时性越高，但对 CPU 的处理能力要求越高！

3、 TICK 跑得慢，对 CPU 处理能力要求不高，但操作系统实时性降低，需要考虑能否满足业务对实时性的要求。

4、 硬件定时器精度可到 us，甚至 ns 级。

2.3.4.1.2 波特率控制

16/32 bits Timer 作为硬件定时器，可用于主从串口通信，波特率为 187500bps。

说明

1、 当主串口（Master）向从串口（Slave）发送命令的同时，启动该 Timer 进行计时。

2、 在 10us 内，如果 Master 没有收到 Slave 的反馈，则认为该次传输失败，Master 将重传，并重置 Timer。

3、 软件会记录重传次数，超过设置阈值，则认为该接口失效，转交接口异常处理。

2.3.4.1.3 流量控制

Timer 作为硬件定时器，可用于以太网口防大流量冲击场景。

2.3.4.1.4 计数范围和计数精度

Timer 计数范围和计数精度如表 2-3 所示。

表2-3 Timer 计数范围和进度表

计数时钟	计数位宽	计数精度	计数范围
100MHz	16 bits Timer	10ns	0 ~ 655us
	32 bits Timer		0 ~ 42s
	64 bits Timer		0 ~ 5852 年（一年按照 365 天计算）

2.3.4.2 功能原理

32 bits Timer 基于一个 32 bits/16 bits（可配置）减法计数器。64 bits Timer 基于一个 64 bits/32 bits（可配置）减法计数器。计数器的值在每个计数时钟的上升沿减 1。当计数值递减到零，Timer 将产生一个中断。

Timer 有以下 3 种计数模式：

- 自由运行模式

  定时器持续计数，当计数值减到 0 时又自动回转到其最大值，并继续计数。当计数长度为 32 bits 时，最大值为 0xFFFF_FFFF。当计数长度为 16 bits 时，最大值为 0xFFFF。在自由模式下，也可以载入计数值，并立即从载入值减计数，但计到 0 时回转到其最大值。

- 周期模式

  定时器持续计数，当计数值减到 0 时从 TIMERx_BGLOAD 寄存器中再次载入初值并继续计数。

- 单次计数模式

  向定时器中载入计数初值。当定时器的计数值减到 0 时就停止计数，直到重新被载入新值且定时器处于使能状态，才再次开始计数。

每个 Timer 具有一个预分频计数器，可将工作时钟在 Timer 内部再次进行 1 分频、16 分频或 256 分频。进一步提高计数时钟频率的选择灵活性。

对定时器载入计数初值的方法如下：

- 通过写 TIMERx_LOAD 寄存器可对定时器载入计数初值。当定时器处于工作状态时，如果向 TIMERx_LOAD 寄存器写入值，会导致定时器立刻从新值开始重新计数。适用于所有计数模式。
- 通过写 TIMERx_BGLOAD 寄存器可以设定周期计数模式的计数周期。写该寄存器不会立刻影响定时器的当前计数，定时器会继续计数直到计数值减到 0。然后载入 TIMERx_BGLOAD 寄存器中的新值开始计数。

2.3.5 工作方式

2.3.5.1 打开时钟

上电时 TIMER 工作时钟默认为打开状态。在不使用 TIMER 的情况下，可以关闭 TIMER 工作时钟，以降低芯片功耗。

打开门控时钟具体配置步骤如下：

1 读 kernel 系统控制器寄存器 SC_TOP_CLK_ST[0]，获取当前时钟门控信息。

2 如果 SC_TOP_CLK_ST[0]为低，进入步骤 3；SC_TOP_CLK_ST[0]为高，表示 TIMER 时钟已经打开，直接跳过下面的步骤结束操作。

3 配置 kernel 系统控制寄存器 SC_TOP_CLK_EN[0]为高，打开 TIMER 模块的时钟门控。

----结束

2.3.5.2 关闭时钟

关闭门控时钟具体配置步骤如下：

1 读 kernel 系统控制器寄存器 SC_TOP_CLK_ST[0]，获取当前时钟门控信息。

2 如果 SC_TOP_CLK_ST[0]为高，进入步骤 3；SC_TOP_CLK_ST[0]为低，表示 TIMER 时钟已经关闭，直接跳过下面的步骤结束操作。

3 配置 kernel 系统控制寄存器 SC_TOP_CLK_DIS[0]为高，关闭 TIMER 模块的时钟门控。

----结束

2.3.5.3 时钟配置

说明

Timerx 表示 Timer0、Timer1、Timer2、Timer3、Timer4、Timer5、Timer6、Timer7、Timer8 或 Timer9。

Timerx 有四种计数时钟可选择。

2.3.5.3.1 选择系统总线时钟 8 分频进行计数

在 Hip04 中总线时钟来自于 PLL0 时钟输出，在不同的配置下会工作在不同的频率。举例：如果 PLL0 的工作时钟配置为 1.5GHz，那么 Timer 的计数值为 187.5MHz。

具体配置步骤如下（以 timer0~3 为例，其他 timer 配置类似，不再累赘）：

1 系统 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0ov 配置为 1，则计数时钟输入为系统总线时钟的 8 分频。

2 初始化 Timer，开始计数。

----结束

2.3.5.3.2 选择系统的晶振输入频率进行计数

计数值直接为系统的晶振时钟输入，在 Hip04 中，晶振可能选择为 25MHz 的晶振输入或者 50MHz 的晶振输入。

具体配置步骤如下（以 timer0~3 为例，其他 timer 配置类似，不再累赘）

1 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0ov 配置为 0；

2 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0sel 配置为 1，则计数时钟输入为系统的晶振输入时钟。

3 初始化 Timer，开始计数。

----结束

2.3.5.3.3 选择系统的晶振输入频率的 64 分频进行计数

计数值直接为系统的晶振时钟输入，在 Hip04 中，晶振可能选择为 25MHz 的晶振输入或者 50MHz 的晶振输入。

具体配置步骤如下（以 timer0~3 为例，其他 timer 配置类似，不再累赘）

1 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0ov 配置为 0；

2 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0sel 配置为 0；

3 kernel 系统控制寄存器中 SC_CLK_SEL[8]配置为 1，则计数时钟输入为系统晶振输入时钟的 64 分频。

4 初始化 Timer，开始计数。

----结束

2.3.5.3.4 选择系统的晶振输入频率的 128 分频进行计数

计数值直接为系统的晶振时钟输入，在 Hip04 中，晶振可能选择为 25MHz 的晶振输入或者 50MHz 的晶振输入。

具体配置步骤如下（以 timer0~3 为例，其他 timer 配置类似，不再累赘）

1 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0ov 配置为 0；

2 kernel 系统控制寄存器中 SC_CTRL 寄存器 timeren0sel 配置为 0；

3 kernel 系统控制寄存器中 SC_CLK_SEL[8]配置为 0，则计数时钟输入为系统晶振输入时钟的 128 分频。

4 初始化 Timer，开始计数。

----结束

2.3.5.4 初始化

1 写 TIMERx_LOAD 寄存器，为 Timer 载入计数初值。

2 当需要 Timer 工作在周期计数模式下、且计数周期与载入 Timer 的计数初值不同时，写 TIMERx_BGLOAD 寄存器，设置 Timer 的计数周期。否则跳过此步骤，进入步骤 3。

3 写系统控制器 SC_CTRL 寄存器，配置 Timer 的时钟使能信号的参考时钟。

4 写寄存器 TIMERx_CONTROL，配置 Timer 的计数模式、计数器长度、预分频因子及中断屏蔽，同时启动 Timer 计数。

----结束

注意

对于 64 位的 timer，计时器通过 TIMERx_LOAD 寄存器载入计数初值时，需要先写高 32 比特的寄存器，再写低 32 比特的寄存器。

2.3.5.5 中断设置

Timer 主要用于产生定时中断，因此 Timer 的中断处理任务主要是激活等待定时中断的进程。

以软件响应 timer0 中断为例，其操作步骤如下：

步骤 1 向 TIMER0_INTCLR（物理地址 0x2000900c）写入任何值，清除 Timer0 的中断。

步骤 2 激活等待该中断的进程，使其继续执行。

----结束

2.4 WatchDog

2.4.1 概述

WatchDog（看门狗）用于系统异常情况下，一定时间内发出复位信号，以复位相对应的CPU 核。

2.4.2 特性

WatchDog 具有以下特点：

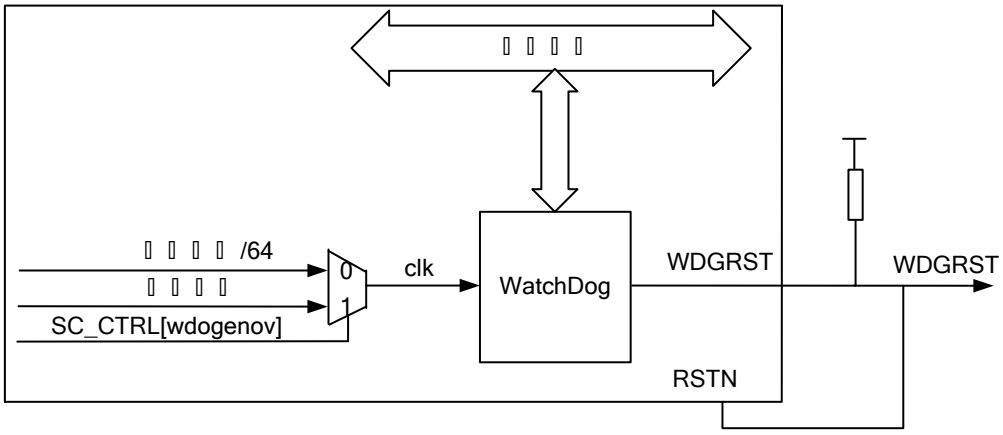
- 内部具有 16 个 32 bits 减法计数器，计数时钟源可配置；
- 支持超时时间间隔（即计数初值）可配置；
- 计数器支持 Free-Running 递减模式；
- 支持锁定寄存器来防止寄存器被误改；
- 支持时钟使能，可控制计数时间间隔；
- 支持超时中断发生，若第一次中断未被响应，第二次超时则产生复位信号；
- 支持中断屏蔽功能。

2.4.3 功能描述

2.4.3.1 典型应用

WatchDog 应用框图如图 2-2 所示。

图2-2 WatchDog 应用框图



WatchDog 可使用晶振时钟（1MHz～75MHz）的 64 分频时钟或总线时钟进行计数，当WatchDog 超时，将产生复位信号直接复位包括芯片在内的相关系统。

2.4.3.2 功能原理

WatchDog 的运行基于 1 个 32bit 减法计数器，计数初值由寄存器 WDG_LOAD 载入。在 WatchDog 时钟使能情况下，计数器的值在每个计数时钟的上升沿减 1。当计数值递减到 0 的当拍，WatchDog 将产生一个中断。在下一个计数时钟上升沿，计数器又从寄存器 WDG_LOAD 中重新载入计数初值，开始递减计数。

如果计数器第二次计数递减到 0 的当拍，CPU 还没有清除 WatchDog 中断，则 WatchDog 将发出复位信号 WDG_RST，计数器停止计数。

根据实际应用需要，可通过配置 WDG_CONTROL 使能或者禁止 WatchDog 产生中断和复位信号：

- 当禁止产生中断时，计数器将停止计数。
- 当重新开启中断时，WatchDog 将从 WDG_LOAD 的设定值开始计数，而不是从计数器上次停止时的计数值开始计数。在中断到来之前，可以重新载入初值。

WatchDog 的计数时钟可以选择晶振时钟 64 分频时钟或者总线时钟，便于选择不同的计数时间范围。

通过配置 WDG_LOCK 寄存器，可以禁止对 WatchDog 内部寄存器进行写操作：

- 向 WDG_LOCK 写入 0x1ACC_E551，可以打开所有 WatchDog 寄存器的写权限。
- 向 WDG_LOCK 寄存器写入其他任何值，可以关闭所有 WatchDog 寄存器（WDG_LOCK 寄存器除外）的写权限。

该特性保护 WatchDog 的寄存器不被软件错误地修改，从而使得在异常情况下，WatchDog 不致被软件错误地中止操作。

- 在调试模式下，WatchDog 自动关闭，以防止干扰正常的调试操作。

2.4.4 工作方式

2.4.4.1 软复位

WDG 不支持系统软复位。

2.4.4.2 计数时钟频率配置

系统支持 2 种 WatchDog 计数时钟：主晶振的 64 分频时钟和总线时钟，通过配置 SC_CTRL[wdogenov]进行选择。

WatchDog 计数时间为：

$$T_{WDG} = Value_{WDG_LOAD} \times (1/f_{clk})$$

说明

- T_{WDG} 表示 WatchDog 计数时间
- $Value_{WDG_LOAD}$ 表示 WatchDog 计数初值
- f_{clk} 表示 WatchDog 计数时钟频率。

WatchDog 在不同时钟下的计数时间范围值如下：

- 当选择晶振 64 分频时钟（例如 187.25MHz/64）时，计数时间范围为 0～549s。

-

2.4.4.3 系统初始化配置

系统上电复位后 WatchDog 计数器处于停止计数状态，在系统初始化过程中需要将 WatchDog 初始化并启动其运行。

WatchDog 的初始化过程如下：

步骤 1 配置寄存器 WDG_LOAD，设定计数初值。

步骤 2 配置寄存器 WDG_CONTROL[inten]为 1，使能中断并启动 WatchDog 计数。

步骤 3 向寄存器 WDG_LOCK 写入 0x1ACC_E551 以外的任何值，给 WatchDog 上锁。

----结束

2.4.4.4 中断处理过程

收到 WatchDog 发出的中断后，应及时清除其中断状态，并使其载入计数初值重新开始计数。

WatchDog 中断处理的过程如下所示：

步骤 1 向锁定寄存器 WDG_LOCK 写 0x1ACC_E551，为 WatchDog 开锁。

步骤 2 向寄存器 WDG_INTCLR 写任意值，清除 WatchDog 的中断状态，同时也使 WatchDog 自动载入计数初值重新开始计数。

步骤 3 向寄存器 WDG_LOCK 写入 0x1ACC_E551 以外的任何值，给 WatchDog 上锁。

----结束

2.4.5 关闭 WatchDog

在系统进入 SLEEP 模式之前，必须关闭 WatchDog。向 WDG_CONTROL[inten]写 0，关闭 WatchDog；向该位写 1，打开 WatchDog。

2.4.6 注意事项

使用 WatchDog 的注意事项如下：

- 一旦 Watchdog 的复位信号输出 WDGRST 有效后，就会一直保持有效，直到系统使用 PRESETn 和 WDGRSTn 复位信号将 Watchdog 的内部寄存器清零，WDGRST 的复位才会取消。因此系统的复位生成电路在接收到 WDGRST 信号后应同时发出 PRESETn 和 WDGRSTn 两个复位信号将 Watchdog 复位，否则系统无法退出复位状态。另外，WatchDog 被复位的同时，系统也完成了复位，此时系统初始化程序应该重新对 Watchdog 进行初始化。
- WDG_CONTROL[inten]不仅控制中断屏蔽，也控制 WatchDog 计数器的启动/停止计数。WatchDog 发出中断时，将 INTEN 位清零只会屏蔽将要发生的 WatchDog 中断，但不会使现有的中断状态清零。当再次将 INTEN 置位时，虽然 WatchDog 的计数器载入初值重新开始计数，但 Watchdog 的中断输出 WDOGINT 会重新有效。所以应注意，清除中断只能通过写 WDG_INTCLR 寄存器来完成。

- 下面 4 种情况下 WatchDog 重新载入计数初值：
  - 写 WDG_LOAD 寄存器时
  - WatchDog 的计数值第一次减到 0 时
  - 写 WDG_INTCLR 寄存器时
  - 写寄存器 WDG_CONTROL，使其 INTEN 位由无效转为有效时
- Watchdog 内部为解决 CPU 发出的载入计数初值、清除中断等控制信号从 PCLK 快时钟域过渡到 WDOGCLK 慢时钟域的问题，对控制信号作了锁存，使其保持有效直到被 WDOGCLK 采样到。这样，即使在 CPU 访问完 WatchDog 的配置寄存器后关断 PCLK 时钟（保留 WDOGCLK），WatchDog 也能完成指定的操作。

如果在 WatchDog 的 WDOGCLK 上升沿或时钟使能 WDOGCLKEN 到来之前，多次写 WDG_LOAD 寄存器，则 WatchDog 的计数器实际载入的值为最后一次写入 WDG_LOAD 寄存器中的值。

2.5 L3 Cache

2.5.1 概述

L3 Cache（Level 3 Memory System Cache）是系统级片上三级高速缓存，该 Cache 被 CPU 和各加速器、IO 共用，用以临时缓存在一段时间内会被多次使用的数据，这可以有效提高处理器及其他加速模块对外部存储器的访问带宽和访问效率。

2.5.2 特性

L3 Cache 具有以下特性：

- Cache 容量为 8MB；
- 写数据位宽为 128 bits、读数据位宽为 256bits 的总线接口；
- 2 组读写命令合并，读写数据位宽各为 128 bits 的 DDRC 接口；
  - 与两组 DDRC 相连，Cache Miss 或 Non-Cacheable 操作将访问 DDR 获取数据，在频率为 800MHz 时，可获取最大 204.8 Gbps 的读写带宽；
  - 支持按地址交织访问两组 DDRC 接口；
- Cache 判定（包括 Cacheable、Read/Write Allocate）可灵活编程；
  - 可基于总线 Cacheable 信息进行 Cache 判定；
  - 可基于固定地址范围进行 Cache 判定；
  - 可全局强制 Cacheable、Write Allocate 或 Read Allocate；
- Cache Line 大小固定为 64 Bytes（16 Words/512 bits）；
- 基于物理地址的 37 bits 地址结构；
- 可编程的三种替换算法；
  - 随机替换算法；
  - PLRU 替换算法；
  - Weighted LRU 替换算法；
- 支持 Format C Lockdown（按 Way Lockdown），被 Lock 的 Cache 容量可当做片上 SRAM 使用，Lockdown 哪些 Way 可编程；
- 支持关键字优先；
- 固定为 Write-Back 策略；
- 支持原子性 Cache 维护（Maintain）操作；
  - 包括 Invalid、Clean、SYNC 三种操作；
  - 支持全局和按物理地址两种操作模式进行 Invalid 和 Clean；
- 支持 Cache Disable 功能；
- 支持基于 Bank 对 Cache 读写操作数量、读写命中数量、读写操作延迟等性能信息进行统计；
- Cache 内 SRAM 支持 SECDED ECC，ECC 错误信息可统计或上报，只支持上报最后一次 ECC 出错地址；
- 对于单 bit ECC 出错，支持将纠错后的数据回写至 Cache 的功能；
- 包含在 L3 Cache 内部，和 L3 Cache 共享与 CPU、IO、ACC 互联的总线接口；

- 与 L3 Cache 共用同一个流水线；
- 容量为独立 1MB；
- 可作为普通 RAM 使用；

2.5.3 关键术语

表2-4 L3 Cache 关键术语描述

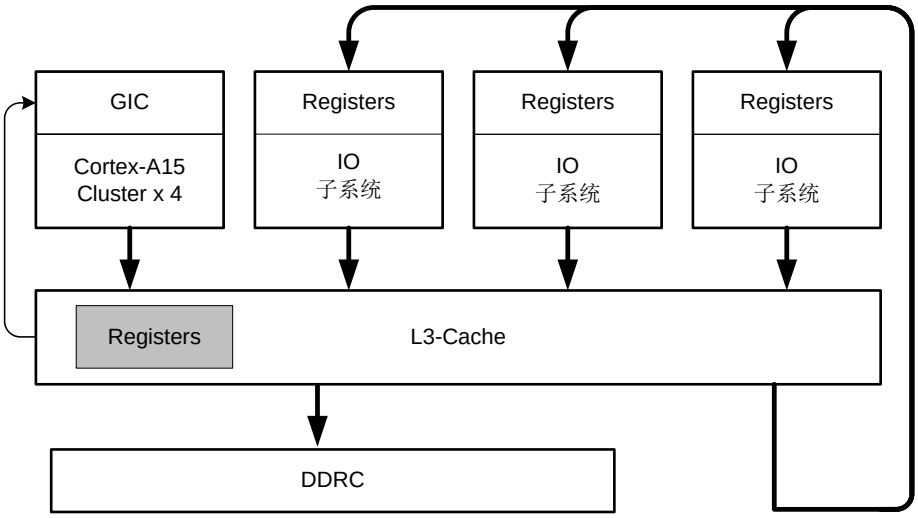
术语	描述
L3 Cache	第三层缓存，相对于 CPU 的 L1 Cache 和 L2 Cache 而定义。
RA	Read Allocate，指示读操作如果缺失，则需要产生 Allocate 操作。
WA	Write Allocate，指示写操作如果缺失，则需要产生 Allocate 操作。
SECDED	单 bit 纠错，双 bit 检错。
RR	Round Robin，轮盘赌算法。
DFA	DFA 节点每个输入都已经混合了多个设备的不同 QoS 传输，调度算法是实时监控每拍的传输 QoS，基于优先级进行调度，相同优先级以 RR 调度，未获得仲裁的传输，每拍 QoS 固定加 1，直到获得仲裁为止。
WRR	Weighted Round Robin，带优先级的轮盘赌算法。
LRU	Least Recently Used，最近最少使用算法。
PLRU	Pesudo Least Recently Used，伪最近最少使用算法。

2.5.4 功能描述

2.5.4.1 应用框图

L3 Cache 的应用如**错误！未找到引用源。**所示。

图2-3 L3 Cache 应用图



各 CPU Cluster、ACC、IO 在读写 DDR 空间时，会先进行 CPU L1/L2 Cache 的 Snoop 查询，如果 Snoop 命中，则读写通过 Snoop 完成；如果 Snoop 未命中，则将对 L3 Cache 进行读写；如果 L3 Cache 命中，则读写在 L3 Cache 中完成；如果读写缺失，则 L3 Cache 通过 DDRC 读取 DDR 的数据。

L3 Cache 的内部流水线与 CPU 以同步方式工作，工作频率与 CPU 相同。

在 CPU 存储系统中，L1 Cache 的速度是最快的，其次是 L2 Cache 和 L3 Cache，而其后的存储访问的速度会急剧下降（DRAM 的结构决定了其随机访问的效率无法得到提升）。

2.5.4.2 相关模块

在系统中，L3 Cache 模块会和下面多个模块进行交互，分别描述如下：

- CPU Cluster/ACC/IO ：CPU Cluster/ACC/IO 均通过写操作 128 bits、读操作 256bits 的总线接口读写 L3 Cache。
- DDRC ：L3 Cache 通过 128 bits 自定义的 DDRC 专用总线接口执行 DDR 内存空间的读写操作，操作包括 CPU/Cluster/ACC/IO 的 Non-Cacheable 操作和 Cacheable 操作但 TAG 查询未命中的操作，操作均基于 Cache Line 为单位。
- GIC ：L3 Cache 在 ECC 出错、统计溢出、访问异常等场景下，会产生中断信号给 GIC 模块，通过 GIC 将中断发送给对 L3 Cache 具有控制权的 CPU。

2.5.4.3 强制写 Allocate

强制写 Allocate 操作可以控制 Cache 的写 Allocate 策略。默认情况下，L3 Cache 是否发生写 Allocate 操作由 AW(R)CACHE 信号的 WA 位决定，但通过该功能，可以使 Cache 强制判定 WA 的值为 1’b1 或 1’b0，即强制写操作发生 Allocate 操作或者强制写操作不发生 Allocate 操作。该功能可以保证某些特殊的应用场景下由用户设定资源的利用。

该功能由 Auxiliary Control 寄存器的[9:8]位控制，该位为 2’b00 时，写 Allocate 策略由 AW(R)CACHE 信号的 WA 位决定；该位为 2’b01 时，写 Allocate 将必然不会发生，所有的写 Miss 操作都将通过 WB 写入 DDR；该位为 2’b10 时，写 Allocate 将必然发生，所有的写 Miss 操作都将发生写 Allocate 操作；该位的 2’b11 为无效。

2.5.4.4 强制读 Allocate

强制读 Allocate 操作可以控制 Cache 的读 Allocate 策略。默认情况下，L3 Cache 是否发生读 Allocate 操作由 AW(R)CACHE 信号的 RA 位决定，但通过该功能，可以使 Cache 强制判定 RA 的值为 1’b1 或 1’b0，即强制读操作发生 Allocate 操作或者强制读操作不发生 Allocate 操作。该功能可以保证某些特殊的应用场景下由用户设定资源的利用。

该功能由 Auxiliary Control 寄存器的[11:10]位控制，该位为 2’b00 时，读 Allocate 策略由 AW(R)CACHE 信号的 RA 位决定；该位为 2’b01 时，读 Allocate 将必然不会发生，所有的读 Miss 操作都将通过 LFB 从 DDRC 读并且不进行 L3 Cache Data Ram 操作；该位为 2’b10 时，读 Allocate 将必然发生，所有的读 Miss 操作都将发生读 Allocate 操作；该位的 2’b11 为无效。

2.5.4.5 Full Line Allocate

在写请求 Miss，需要 Allocate 时，如果写入的数据为一条完整的数据 Line，那么从 DDRC 读入一条 Line 和写入数据进行合并就没有必要。此时 L3 Cache 不会发生 Merge 操作，而是将这条完整数据 Line 写入 L3 Cache。

2.5.4.6 Clean/Invalidate 操作

Cache 的 Clean 和 Invalidate 功能都是通过软件实现的。Clean 操作将 Cache 的脏数据 Line 更新到主存中，使主存中存储最新的数据，随后将 Cache 中该 Line 的 Dirty 指示置零但 Valid 保持有效；Invalidate 操作将 Cache 的数据 Line 直接丢弃，随后将 Cache 中该 Line 的 Dirty 和 Valid 指示都置零。

L3 Cache 支持按地址 Clean／Invalidate 和全局 Clean／Invalidate。

按地址 Clean／Invalidate 指对特定物理地址进行 Clean／Invalidate 操作，通过写 L3 Cache 的 Maintenance 寄存器：L3_MAINT_CTRL、L3_MAINT_ADDR 来完成。每次寄存器操作需要给交织后的物理地址，并指定当前地址所在的 Bank。

全局 Clean／Invalidate 指对 L3 Cache 所有 Bank 里的所有 Line 进行 Clean／Invalidate 操作，通过写 L3 Cache 的 Maintenance 寄存器： L3_MAINT_CTRL 来完成。

配置 Maintenance 寄存器分为安全和非安全操作，全局 Invalidate 操作不论是安全还是非安全的配置，都能对所有的 Line 进行 Invalidate 操作，一般用于上电后的初始化流程。全局 Clean 操作，安全的配置操作只能对安全的地址进行，非安全的配置操作只能对非安全的地址进行。全局 Clean+Invalidate 操作时，Invalidate 会对所有 cacheline 执行，Clean 只对安全属性相同者执行，因此如果 L3 Cache 中既有安全数据又有非安全数据且都不可丢失，需要禁用此操作，否则会造成数据丢失；当系统中只有安全数据或者非安全数据时，需要以相同的安全属性配置此操作。

2.5.4.7 Read Invalidate 操作

Read Invalidate 操作用于将数据从 L3 Cache 里读走并且随后 Invalidate 掉。如果确知数据在被读取后就不会或很少被再使用，可以使用本操作来及时清出不必要的数据，提高 Cache 使用效率。

如果总线传过来的 USER 信息指示该操作为 Read Invalidate 操作，对于读 Miss，L3 Cache 只将 DDR 数据返回给总线，不进行 Allocate；对于读 Hit，读出 Data Ram 数据返回给总线，并将该 Cache line invalidate 掉。 若该 Cache line 为 Dirty，则给总线传递 PassDirty 标志，指示该 Cache line 为 Dirty 数据，让其维护数据一致性。

对于总线传过来的 ReadOnce、ReadClean 和 ReadNotSharedDirty 操作，若同时 USER 信息指示该操作为 Read Invalidate 操作，则不执行 Read Invalidate 动作，因为 ACE 协议中对于上述几种不允许返回 PassDirty 标志。

2.5.4.8 Atomic Sync

Clean 和 Invalidate 操作均不能保证在其操作执行时，L3 Cahce 中所有的命令已经完成且数据已经刷出 Cache。因此，为了保证操作的正确性，需要执行额外的 Sync 操作。

Sync 操作通过读 L3 Cache 的同步寄存器 L3_SYNC 进行。该操作执行时将阻塞 L3 Cache 的 CPU Cluster/ACC/IO 接口，从 Sync 操作执行开始，后续 CPU 请求将无法进入 L3 Cache，然后该操作将一直等待 CPU 所有未完成的操作全部完成，且 L3 Cache 中所有 Pipeline、Buffer 全部清空，然后再返回 CPU Sync 操作的应答。

2.5.4.9 Cache Disable

L3 Cache 的 Disable 功能可以将 Cache 功能禁止掉。此时 CPU Cluster/ACC/IO 端口的传输都不触发 Tag 比较，直接送往 DDRC。

在 L3 Cache 配置为 Disable 后，Atomic RAM 依然可以正常访问，Exclusive Monitor 也可正常工作。

说明

Disable 时，L3 Cache 的寄存器配置及地址交织功能保持有效。

2.5.4.10 LOCKDOWN

L3 Cache 使用 Format C Lockdown 算法，该算法及相关的 A、B Format 算法可参见参考资料《ARM Architecture Reference Manual》，Format C Lockdown 为基于 Way 的 Lockdown 方式。该功能限制了 Cache Allocate 时的替换算法，用以保护 L3 Cache 中载入的数据或指令不被替换出去。

当 Cache 发生 Lookup 查找时，Index 地址指示了其对应的 8 个数据 Line 的位置。当 Lookup 发生 Miss，并且 Line Allocate 发生时，这 8 个数据 Line 的位置都可能是将来存放读入的数据 Line 的位置，而 Format C Lockdown 正是用来限制其选择存放位置的算法。

被 Lockdown 后 Cache 使用的路数减少，通过 Clean 和 Invalidate 操作，被 Lockdown 的数据可以排出回到主存。

2.5.4.11 RAM ECC 校验

L3 Cache 支持所有 RAM，包括 Atomic RAM 的 ECC 校验，该功能提供高速 Memory 访问检错纠错能力。ECC 校验能通过系统控制器控制 l2_ecc_en 信号使能。对 Data RAM，SECDED 基于每 Line 每 64 bits 进行；对 Tag RAM，SECDED 基于每 Tag 进行；对 Atomic RAM，SECDED 基于每 64 bits 进行。

在 CPU Cluster/ACC/IO 发出读或写传输时，如果有 1bit 校验错误，则 ECC 逻辑将自动纠正错误；如果错误为 1bit 以上，ECC 逻辑将无法纠正，此时 L3 Cache 通过 Interrupt 信号通知系统。

2.5.4.12 Monitor 统计功能

L3 Cache 提供额外的 Monitor 统计功能，以统计 L3 Cache 相关性能数据。

Montior 对每所有统计信号一共提供四个 32 比特位宽的统计寄存器，四个统计寄存器均可编程为任一统计事件。计数器为加法计数器，在系统写入计数值（默认为全零），并使能 Monitor 后计数，当计数到全 F 时将产生一个中断通知 CPU。

L3 Cache Monitor 按照 Bank 对事件进行统计，每 Bank 内部统计的事件共 25 个。

表2-5 L3 Cache 内部统计事件功能列表

L3 Cache Event	Description
event_secc_dram	DATA RAM ECC 出错并被纠正。
event_secc_tram	TAG RAM ECC 出错并被纠正。
event_read_hit	读命中
event_crd_req	Cacheable 读请求。
event_ncrd_req	Non-Cacheable 读请求。
event_singlerd_req	Single 读请求。
event_burstrd_req	Burst 读请求。
event_write_hit	写命中。
event_cwr_req	Cacheable 写请求。
event_ncwr_req	Non-Cacheable 写请求。
event_singlewr_req	Single 写请求。
event_burstwr_req	Burst 写请求。
event_alloc_read	读 Allocate 请求。
event_alloc_write	写 Allocate 请求。
event_evict_op	Evict 操作。
event_ex_op	Exclusive 操作。
event_pipe_stall_tag	TAG RAM 更新导致流水线阻塞。
event_pipe_stall_buffer	Buffer 相关导致流水线阻塞。
event_pipe_stall_order	Order 相关导致流水线阻塞。
event_pipe_stall_resouce	申请不到 Buffer 资源导致流水线阻塞。

event_secc_aram	Atomic RAM ECC 出错并被纠正。
event_w_aram	Atomic RAM 普通写请求。
event_aw_aram	Atomic RAM 原子写请求。
event_r_aram	Atomic RAM 读请求。
event_ex_aram	Atomic RAM 的 Exclusive 操作。

2.5.5 工作方式

2.5.5.1 初始化

L3 Cache 复位后为 Disable 状态，总线的所有操作都不查 Tag，直接发送往 DDRC 完成。

因为 Data RAM 和 Tag RAM 在初始状态为未知数据，因此 L3 Cache 在使能之前需要将 Tag RAM 和 Data RAM 初始化，即使用 Invalidate 操作将 L3 Cache 所有 Valid 位都置零。

对于复位后的 Invalidate 工作，推荐使用全局 Invalidate 方式进行。

对于 L3 Cache，具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_CTRL 设置为全局 Invalidate 模式，并启动。

步骤 2 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束，进入下一步骤。

步骤 3 根据需要配置 L3 Cache 的其他特性，通过配置 L3 Cache 辅助控制寄存器 L3_AUCTRL 完成。

步骤 4 配置寄存器 L3_CTRL bit[0]为 1，使能 L3 Cache。

步骤 5 对 L3 中 Atomic RAM 全部地址空间进行 64bit 对齐的全零写操作。

----结束

L3 Cache 初始化结束。

2.5.5.2 Clean 及 Invalidate 操作

2.5.5.2.1 全局 Clean

全局 Clean 的具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_CTRL 配置为全局 Clean 模式，并启动。

步骤 2 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束。

----结束

2.5.5.2.2 全局 Invalidate

全局 Invalidate 的具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_CTRL 配置为全局 Invalidate 模式，并启动。

步骤 2 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束。

----结束

2.5.5.2.3 全局 Clean+Invalidate

全局 Clean+Invalidate 的具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_CTRL 配置为全局 Clean+Invalidate 模式，并启动。

步骤 2 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束。

----结束

2.5.5.2.4 按地址 Clean

按地址 Clean 的具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_ADDR，配置需要执行 Maintain 操作的物理地址低 32 位。

步骤 2 写寄存器 L3_MAINT_CTRL，配置需要执行 Maintain 操作的物理地址高位，以及配置为按地址 Clean 模式，并启动。

步骤 3 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束。

----结束

注意

按地址 Maintain 操作所需要配置的是交织后的物理地址，并同时配置该地址位于哪个 Bank。

2.5.5.2.5 按地址 Invalidate

按地址 Invalidate 的具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_ADDR，配置需要执行 Maintain 操作的物理地址低 32 位。

步骤 2 写寄存器 L3_MAINT_CTRL，配置需要执行 Maintain 操作的物理地址高位，以及配置为按地址 Invalidate 模式，并启动。

步骤 3 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束。

----结束

2.5.5.2.6 按地址 Clean+Invalidate

按地址 Clean+Invalidate 的具体配置步骤如下：

步骤 1 写寄存器 L3_MAINT_ADDR，配置需要执行 Maintain 操作的物理地址低 32 位。

步骤 2 写寄存器 L3_MAINT_CTRL，配置需要执行 Maintain 操作的物理地址高位，以及配置为按地址 Clean+Invalidate 模式，并启动。

步骤 3 查询 L3_MAINT_CTRL 寄存器最低位（域 mnt_start），判断 Maintain 操作是否结束。若为 0，则表示 Maintain 操作已经结束。

----结束

注意

按地址 Maintain 操作所需要配置的是交织后的物理地址，并同时配置该地址位于哪个 Bank。

2.5.5.3 Lockdown 操作

Lockdown 操作可将某一条 Way 或多条 Way 对应的存储空间设置为 Internal Memory 使用。下面以将第 4、5 条 Way 设定为 Internal Memory 为例，其具体的配置步骤如下：

步骤 1 对 L3 Cache 的所有 Line 进行 Clean 操作。可以采用全局 Clean 方式。

步骤 2 对 L3 Cache 的所有 Line 进行 Invalidate 操作。可以采用全局 Invalidate 方式。

步骤 3 设定 CPU 的页表，将需要设置为 Internal Memory 的地址区域设定为 L1、L2 Non-cacheable 但 L3 Cacheable，Allocate。

步骤 4 配置寄存器 L3_LOCKDOWN[way]为 0xCF，即锁定 Way0～3 和 Way6～7。

步骤 5 通过执行读操作将对应地址的数据载入 L3 Cache。因为数据在 L1、L2 为 Non-cacheable，因此 L1、L2 不会存储该数据；而数据在 L3 是 Cacheable，因此数据被载入填充满 Way4 和 Way5。

步骤 6 配置寄存器 L3_LOCKDOWN[way]为 0x30，锁定 Way4～5，将其他 Way 解锁。

----结束

此时 L3 Cache 成为 Way0～3 和 Way6～7 有效的六路组相联 Cache。Ways 4、5 成为具有连续物理地址的 Internal Memory。

2.5.6 调试（Debug）

本模块提供了以下方式用于内部状态查询，以方便系统调试（Debug）。

- 本模块提供给了 Debug Bus 接口，该接口可配置和可读取 RAM 寄存器配置信息及统计信息。

- L3 Cache 内部的所有寄存器，除了 L3_SYNC 和 L3_STATUS 以外，均可以通过 Debug Bus 接口进行读写访问。

2.6 中断控制模块（GIC）

2.6.1 概述

GIC 是配合 Cortex A15 高效实现全局中断管理的模块，完成外部中断，内部中断及软件中断的选择，路由至 16 CPU Cores 之一处理，支持安全扩展和虚拟化扩展。

2.6.2 特性

GIC 具有以下特性：

- 支持三种中断：软中断 SGIs、私有外设中断 PPIs 和共享外设中断 SPIs；
- SPIs 支持电平敏感和边沿触发方式中断，每个中断源的触发方式可单独配置；
- 为每个 CPU Core 提供一个物理中断 FIQ 和 IRQ 接口；
- 支持虚拟化扩展，只支持物理 IRQ 中断虚拟化。通过配置产生虚拟中断 Virtual FIQ 和 Virtual IRQ，同时为每个 CPU Core 增加一个虚拟中断 VFIQ 和 VIRQ 接口；
- 支持中断路由，物理中断查询目标处理器寄存器路由至相应 CPU Core；虚拟中断的路由目标由 Hyperisor 根据 VMID 等信息配置目标处理器寄存器，路由至相应 CPU Core。
- 支持中断单独使能；
- 支持中断状态查询；
- 支持中断屏蔽，设置 Mask Priority，中断的优先级大于 Mask Priority 的才能上报。
- 支持中断嵌套，对物理中断，中断嵌套的深度由 Group Priority 决定，只有 Group Priority 较大中断才能引发中断嵌套，Group Priority 的级数可配；
- 为每个核提供六个内部器件的 PPI、一个内部虚拟 maintenance PPI 中断、一个 BypassFIQ 和一个 BypassIRQ 输入，各核之间 PPI 中断相互独立；
- 支持 32 个 SPIs 中断输入作为 N-N 模式中断；
- 支持 TrustZone 扩展，输入中断可按照 Secure/Non-Secure 分组；
- 对于物理中断，所有 Secure 中断的优先级高于 Non-secure 中断。在 Secure 模式下，优先级等级有 32 等级；在 Non-secure 模式下，优先级等级有 16 个等级；中断优先级的值越小说明该中断优先级越大。对于虚拟中断，其优先级等级固定为 32 等级。
- 支持 SPIs 中断锁定，支持 32 个 LSPIs，LSPI 只能是安全中断，锁定后其相关配置寄存器不能被改写。
- 支持中断唤醒事件，预留 nFIQOUT，nIRQOUT 直接输出端口，在 GIC 不使能的情况下也能将中断作为唤醒事件上报。

2.6.3 关键术语

如**错误！未找到引用源。**的术语将在本章节表示特殊的含义。

GIC 关键术语描述

术语	描述
SGI	Software Generated Interrupt，软中断。

术语	描述
PPI	Private Peripheral Interrupt，私有外设中断。
SPI	Shared Peripheral Interrupt，共享外设中断。
VM	Virtual Machine，虚拟机
VMID	Virtual Machine Identity，虚拟机编号
GIC	Generic Interrupt Controller，中断控制器
ISR	Interrupt Service Routine，中断服务程序
LSPI	Lockable SPIs，锁定的 SPI
EOI	End of Interrupt，中断结束

2.6.4 功能描述

2.6.4.1 应用框图

GIC 的应用如图 2-4 所示，外围设备诸如 UART、DMAC 等通过 GIC 的输入端口 IRQS 将中断（SPIs）送给 GIC，GIC 留有内部中断（PPIs）的输入端口，SGIs 由软件通过配置寄存器产生，每个中断对应特定的中断 ID。在系统初始化之后，每种外设的中断，内部中断以及软件中断的相关寄存器被配置好，包括是否使能，各个中断的优先级大小，目标 CPU Core，Mask Priority 以及 Group Priority 的大小都已经确定。当同时有多个中断上报至 GIC 时，GIC 将这些中断的状态由 Inactive 转换为 Pending 状态，通过比较将优先级最高的中断以 FIQ 或者 IRQ 的形式送至相应的处理器接口，CPU Core 在接收到中断之后需要发出读请求去确认中断源，然后将中断状态由 Pending 状态转换为 Active 状态，此时 CPU Core 就可以执行相应的中断服务程序了。在中断处理完毕之后，CPU Core 需要配置 GIC 的寄存器，完成 Priority Drop（降低刚刚处理完的中断的优先级为最小值）和 Deactive（使 GIC 输出至 CPU Core 的中断接口 nFIQ 或 nIRQ 无效）过程。从中断确认到中断完成，CPUCore 与 GIC 的通信通过总线完成。

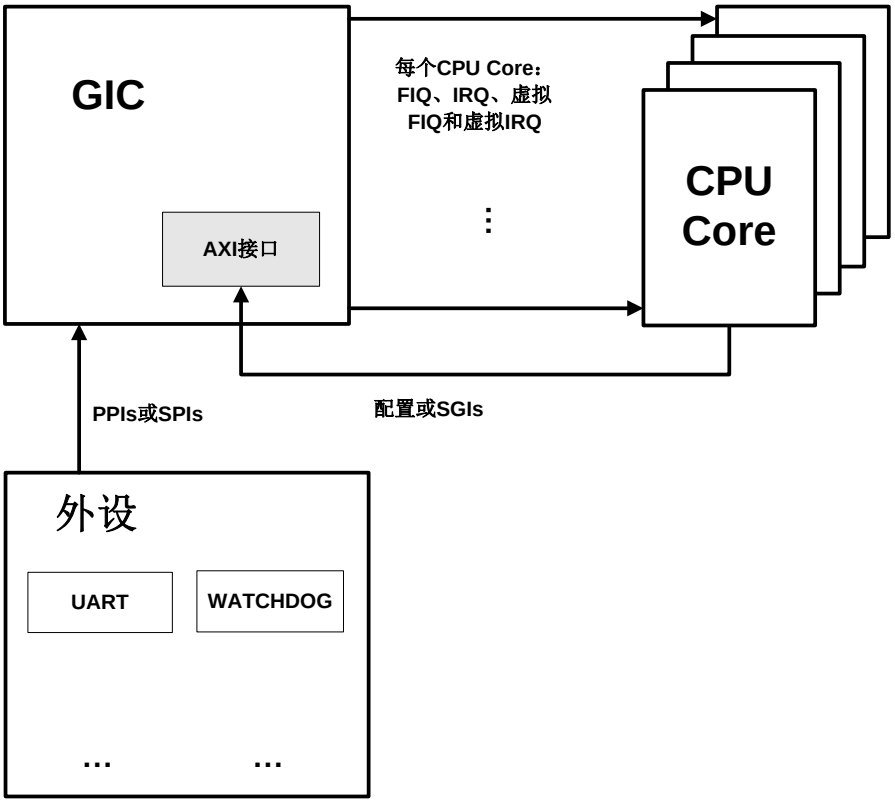


图2-4 GIC 应用图

GIC 应用了安全扩展。GIC 把中断分组，分为安全中断（Group0）和非安全中断（Group1），安全中断以 FIQ 的形式路由至 CPU Core，非安全中断以 IRQ 的形式路由至 CPU Core。中断向量表也分成两份，安全的中断向量表置于安全存储区域，并且指向安全的 ISA；非安全的中断向量表和 ISA 置于常规的存储区域，这样可以避免某些恶意程序修改安全的中断向量表。中断的安全属性在系统初始化时配置寄存器 GICD_IGROUPRn 时已经确定。

GIC 应用了虚拟化扩展，如图 2-5 所示，非安全物理中断（物理 IRQ）发送给 CPU Core 后，触发某个 CPU Core（能够处理 Hypervisor 中断的 CPU Core）进入 Hyp 模式，运行 Hypervisor 的中断处理程序。在中断处理程序中，查询该物理中断对应的 VMID 等信息，配置 GIC 寄存器产生虚拟中断，并路由到相应 VM 的逻辑 CPU Core 上。虚拟中断分发器也可以将虚拟中断分组为 Group0 和 Group1，但是此分组和安全属性没有关系，只是确定虚拟中断以何种形式发送给 CPU Core。

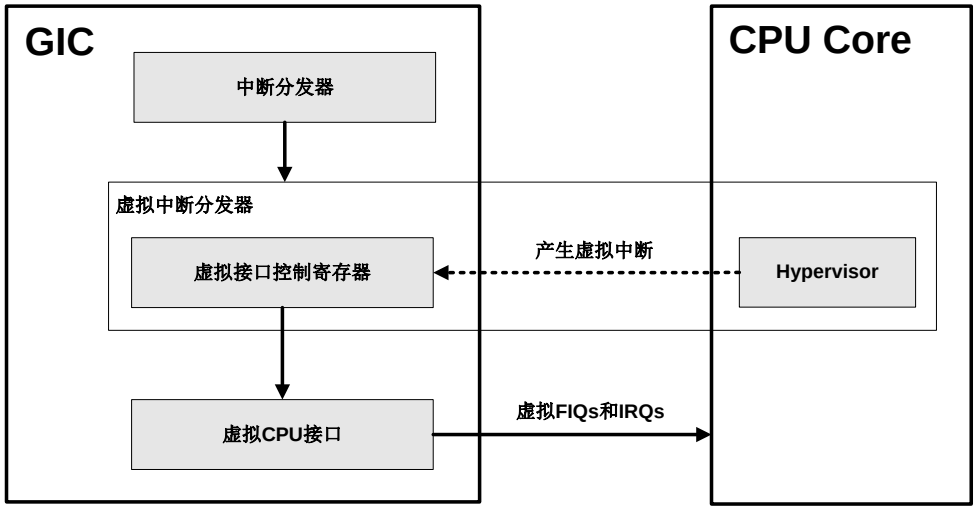


图2-5 虚拟中断分发

2.6.4.2 相关模块

在系统中，GIC 模块会和下面多个模块进行交互，分别描述如下：

- CPU Core：GIC 和 CPU Core 直接连接的信号包括 nFIQ，nIRQ，如果应用虚拟化扩展，增加了 Virtual nFIQ、Virtual nIRQ 信号，这些信号用于将中断上报给 CPU Core。另外，GIC 与 CPU Core 通过总线进行中断的配置、查询与确认等。
- 系统控制器：GIC 提供 CFGSDISABLE 输入端口，用来保护 32 个中断 ID 为 32～63 的安全 SPI 的内部寄存器不被非正常修改。此端口的输入值来自系统控制器。
- 外设单元：外设单元通过 IRQS 输入信号上报中断给 GIC。

2.6.4.3 中断 ID 的分配

表2-6 中断 ID 的分配表

ID 区间	中断名称
ID480～ID511	N-N 模式的 SPIs
ID32 及其以上	1-N 模式的 SPIs
ID31	Bypass IRQ
ID30	非安全物理 Timer
ID29	安全物理 Timer
ID28	Bypass FIQ
ID27	虚拟 Timer

ID26	Hypervisor Timer
ID25	虚拟 maintenance
ID16～ID24	保留
ID0~ID15	SGIs

2.6.4.4 物理中断路由

当物理中断发生时，GIC 的物理中断分发器根据软件配置将该中断分发给相应的 CPU Core。

GIC 接收到有效中断后，输出最高优先级的中断到相应的 CPU Core；GIC 将输入的中断划分成 Group 0 和 Group 1，由中断分组寄存器 GICD_IGROUPRn 配置。不同分组的中断被路由至不同的 CPU Core，以及以 FIQ 或 IRQ 的方式分发到 CPU Core。

各 CPU Core 的 PPIs 相互独立且只能分发到对应的 CPU Core。可以配置每个 SPIs 路由到特定的 CPU Cores。

2.6.4.5 虚拟中断路由

在虚拟化扩展中，当 Hypervisor 收到中断时，首先通过查表判断该中断是给 VM 的，还是给 Hyperisor 自身的（比如 Maintenance Interrupt）。如果中断是发给 VM 处理的，则 Hypervisor 将物理中断分发给 Hypervisor 中断处理程序所在的 CPU Core，触发该 CPU Core 进入 Hyp 模式，执行 Hypervisor 中断处理程序。在该程序中，软件查询该物理中断的 VMID 等相关信息，根据该信息配置 GIC 的虚拟中断控制寄存器（包括 GICH_HCR、GICH_VMCR、GICH_LRn），产生虚拟中断。在分发给虚拟 CPU Interface 之前，Hypervisor 发出 EOI 请求，写物理中断寄存器 GICC_AEOIR，完成 Drop Priority，这样在 VM 还没有处理完虚拟中断，可以继续接收新的物理中断。虚拟中断通过虚拟 CPU Interface 上报至相应 VM 的虚拟 CPU Core。

2.6.4.6 中断状态

Distributor 分别为每个中断维护一个中断状态机，标识当前中断 ID 对应的中断状态。中断具有 4 种状态，分别为 Pending、Inactive、Active、Actve and Pending，各含义如下：

Pending：表示产生对应中断 ID 的中断，但是此中断并未被处理。对 PPIs 和 SPIs 而言，当中断输入请求有效时，即相应中断进入 Pending 状态，或者写 1 至相关寄存器 GICD_ISPENDRn 对应位增加中断的 Pending 状态；对 SGIs 而言，当配置 GICD_SGIR 表示有 SGIs 产生时，相应中断进入 Pending 状态。

Active：当中断上报给处理器，处理器读寄存器 GICC_IAR 确认该中断的中断源，中断即由 Pending 状态转换为 Active 状态，一直维持到中断处理完毕，处理器去写清寄存器 GICC_EOIR；或者写 1 至寄存器 GICD_ISACTIVERn 对应位增加 Acive 状态。

Actvie and Pending：表示处理器正在处理该中断，并且该中断的中断源又发出了一个中断给 GIC，当前正在处理的中断状态由 Active 转换为 Active and Pending 状态，因为在 GIC 中还存在另一个同源的 Pending 状态的中断。对于电平敏感的中断，在处理器确认该中断时，其上报至 GIC 的中断输入信号仍然有效，直到执行到中断服务程序中清中断

命令，才能使输入信号无效，从上报至 GIC 的中断输入信号无效到中断处理结束这段时间内，如果上报至 GIC 的中断输入信号重新有效，则正在被处理的中断状态转化为 Actvie and Pending 状态；对于边沿触发的中断，从中断确认后到中断处理完毕这段时间内，又有同源的中断脉冲，则也转换为 Actvie and Pending 状态。

Inactive：当中断不包含 Active 或者 Pending 状态时，即为无效状态。

2.6.4.7 中断触发方式

中断触发方式包含电平敏感方式和边沿触发方式。可由寄存器 GICD_ICFGRn 配置，对于 SGIs，固定为边沿触发方式；对于 PPIs，固定为电平敏感方式；对于 SPIs，是配置的，复位后为电平敏感方式。

这两种触发方式的中断，GIC 处理时有很大的区别，主要包括：

1、 清除中断源输入信号的方式。边沿触发方式不需要专门清除，电平敏感方式的中断在 CPU Core 执行中断服务程序过程中清除。

2、 产生 Pending 状态（中断状态为 Pending 或者 Active and Pending）的逻辑。边沿触发方式中断产生 Pending 状态由锁存写入 GICD_ISPENDRn 的值或者锁存输入至 GIC 的中断有效脉冲信号两种途径得来；电平敏感方式中断产生 Pending 状态的途径有锁存写入 GICD_ISPENDRn 的值或者由输入至 GIC 的中断有效电平产生，电平值不锁存。

3、 清除 Pending 状态的逻辑。

4、 对 N-N 模式的中断广播支持不同。边沿触发方式中断可以支持 N-N 模式的中断广播，电平敏感方式中断不支持 N-N 模式的中断广播。

2.6.4.8 中断优先级

中断优先级用寄存器 GICD_IPRIORITYRn 表示，此寄存器可以按字节访问，每 8bit 代表一个中断的优先级。在 Secure 模式，GIC 支持 5～8bits 中断优先级域，表示 32～256 个优先级等级；而在 Non-secure 模式，则只支持 4～7bits 中断优先级域。所有安全中断的优先级高于所有的非安全中断，即 Group0 中断的优先级高于 Group1 中断。在同一分组中，优先级的值越小，表示该中断的优先级越大，从多个 Pending 状态的中断中查找出优先级最高的中断。

如果一个中断 INT0 要能上报至 CPU Core，则 INT0 的优先级在所有和 INT0 目标处理器一致的 Pending 状态的中断的优先级中是最大的，并且 INT0 的优先级需要大于 Mask Priority，当 CPU Core 开始处理 INT0 时，INT0 的优先级即为 Running Priority。

2.6.4.9 中断嵌套

GIC 支持中断嵌套，中断嵌套的深度由 Group Priority 的级数决定。8bit 中断优先级域 Priority{7,0}分为两部分，Group priority 和 Subpriority，格式如下，Group Priority 的位数为 3，表示 Group Priority 的级数为 8 级，中断嵌套的深度为 8。Group Priority 的位数由寄存器 GICC_BPR、GICC_ABPR、GICV_BPR 可配。

Group Priority			Subpriority				
7	6	5	4	3	2	1	0

图2-6 GIC 应用图

相同 Group Priority 的中断不能嵌套，当最新确认的中断的 Group Priority 大于当前 CPU 响应中断的 Group priority 时，才可以产生中断嵌套。

2.6.4.10 寄存器属性

由于 GIC 中应用安全扩展，安全世界中的数据需要和非安全世界中数据需要隔离，因此寄存器有多个属性：

Banked： 存在安全和非安全寄存器的副本，而且不同寄存器副本中域的分配可以不相同，安全访问只能访问安全副本，非安全访问只能访问非安全副本。

Secure：具有这属性的寄存器只能被安全访问请求访问，非安全访问为 RAZ/WI。

Common：可以被安全和非安全访问请求访问。

在 GIC 中的 Banked 属性的寄存器包括：GICD_CTLR、GICC_CTLR、GICC_BPR、GICC_IAR、GICC_EOIR、GICC_HPPIR。

Secure 属性的寄存器包括：GICD_IGROUPRn、GICC_ABPR、GICC_AIAR、GICC_AEOIR、GICC_AHPPIR、GICC_NSAPRn。

另外，GIC 中还有另一种形式的 Banked 寄存器，在 16 核系统中，对 SGIs 与 PPIs 而言，同一个中断 ID 因为中断源的不同对应 16 个中断，识别这些中断需要中断 ID 与 CPUID，而与 SGIs 和 PPIs 相关的寄存器则需要具有 16 个副本对应。这些寄存器包括：GICD_SPENDSGIRn、GICD_CPENDSGIRn、GICD_ITARGETSRn（n 值为 0～15）、GICD_IPRIORITYRn（n 值为 0～15）、GICD_ISENABLER0、GICD_ICENABLER0、GICD_ISACTIVER0、GICD_ICACTIVER0。

2.6.4.11 中断锁定

GIC 支持中断 ID 号为 32～62 的安全中断被锁定，锁定由外部输入信号 CFGDDISABLE 控制，高有效。LSPIs 的下列的寄存器的相关域将不容许软件修改：

- GICD_CTLR.EnableGrp0。
- GICC_CTLR 安全备份的 EOImodeS、IRQBypDisGrp0、FIQBypDisGrp0、CBPR、FIQEn、AckCtl、EnableGrp0 等域。
- GICD_GROUPRn、GICD_ISENABLERn、GICD_ICENABLERn、GICD_ISPENDRn、GICD_ICPENDRn、GICD_ISACTIVERn、GICD_ICACTIVERn、GICD_IPRIORITYRn、GICD_ITARGETSRn、 GICD_ICFGRn中Group0中断对应的相关域。
- GICC_PMR_S。注意：GICC_PMR_NS仍然可以被改写。

在系统 boot 时，系统读 GICD_TYPER.LSPI 域获得 LSPIs 的数目，配置 LSPIs 相关的寄存器，然后将 CFGDDISABLE 置高。因此，安全启动的顺序为当全局复位撤销时，紧接着运行安全配置程序。

利用中断锁定可以实现不可屏蔽中断（NMI），如果一个中断需要是 NMI，则配置时该中断的 GICD_GROUPRn 需将它配置成 Group0 中断，在初始化完成后，将 CFGDDISABLE 置高，锁定该中断的配置。

注：此处所说的不可屏蔽中断，仅仅是指在中断经过 GIC 时可以安全顺利的上报。SOC 系统中，一个不可屏蔽中断还有可能在 I/O 复用端口，以及在 CPU Core 内部被屏蔽，而这两点也是需要特别注意的地方。

2.6.4.12 SPIs 支持 N-N 模式

GIC 支持 32 个 SPIs 作为 N-N 模式中断，占用 32 个外部中断输入端口，分配的中断 ID 为 480～511，用同一个中断 ID 和不同的 CPUID 表示 16 个中断，需要将该 16 个 SPIs 对应的寄存器（参考 1.1.4.10 章节）bank 16 个副本。由于电平敏感的中断不支持 N-N 模式中断，电平方式输入的 SPIs 有鉴沿逻辑。

注意

N-N 中断只能通过外部中断输入触发。外部中断输入管脚与 GPIO 管脚复用（具体复用关系参考 SI Requirement 和 IO Config nManager），不能通过配置 GPIO 触发 N-N 中断。

2.6.5 工作方式

2.6.5.1 初始化

GIC 复位后使用时初始化：

步骤1　配置中断分发器。配置寄存器 GICD_IGROUPRn 将物理中断分组为安全中断和非安全中断；配置寄存器 GICH_HCR 将虚拟中断分组；配置寄存器 GICD_ICFGRn 初始化 SPIs 为电平敏感或者是边沿触发；配置寄存器 GICD_IPRIORITYRn 设置中断的优先级；配置寄存器 GICD_ITARGETSRn 设置 SPIs 的目标处理器。

步骤2　配置处理器接口。配置寄存器 GICC_PMR 设置屏蔽优先级；配置寄存器 GICC_BPR、GICC_ABPR 设置安全中断和非安全中断的 Group Priority。

步骤3　使能中断分发器。配置寄存器 GICD_ISENABLERn 使能单个中断；配置寄存器 GICD_CTLR.EnableGrp1、 GICD_CTLR.EnableGrp1 域使能 Group1 和 Group0 中断。

步骤4　使能处理器接口。配置寄存器 GICC_CTLR 使能处理器接口。

步骤5　关键寄存器位配置：GICV_CTLR.AckCtl==1’b0，GICC_CTLR.AckCtl==1’b0。

----结束

2.6.5.2 软复位

不存在单独软复位。

2.6.5.3 安全模式下处理物理中断操作

步骤1　GIC 的 Distributor 根据配置将安全物理中断 INT0 分组为 Group0。

步骤2　通过 Cpu_interface 将 INT0 以 FIQ 的形式分发给相应的 CPU Core。

步骤3　CPU Core 接收到 FIQ 请求，确认 INT0 中断源，得到响应后进入中断服务程序。

步骤4　如果此时在 Cpu_interface 中存在 Group Priority 大于 INT0 的 Pending 状态的安全中断 INT1，则会发生中断嵌套，保存当前 Active 状态中断的寄存器值，开始响应 INT1。

步骤5　当 INT1 处理完毕之后，发出 EOI 请求，完成 INT1 的 Prority Drop；恢复 INT0 的寄存器，继续完成 INT0 的处理，发出 EOI 请求，并 Deactive FIQ 输出信号。

----结束

2.6.5.4 处理虚拟中断操作（虚拟中断对应的物理中断为 SPIs 或者 PPIs）

步骤1　GIC 的 Distributor 根据配置将该物理中断分组，非安全中断分组为 Group1。

步骤2　通过 Cpu_interface 将优先级最高的 Group1 中断以 IRQ 的形式分发给相应的 CPU Core。

步骤3　物理中断 IRQ 触发该 CPU Core 进入 Hyp 模式，并执行 Hypervisor 的中断处理程序，在该中断处理程序中，查询中断源等信息，配置 GIC 的虚拟中断控制寄存器，将该物理中断映射为虚拟中断(GICH_LRn.HW==1，即虚拟中断对应的物理中断为 SPIs 或者 PPIs)，并且 Hyperisor 发出一个 EOI 请求，写物理中断 IRQ 的寄存器 GICC_EOIR，完成 Priority Drop，但并未清除该物理中断。

步骤4　GIC 的虚拟 Distributor 将该虚拟中断分组为 Group0 和 Group1。

步骤5　虚拟 Cpu_interface 将 Group1 虚拟中断以虚拟 IRQ 形式，Group0 虚拟中断以虚拟 FIQ 的形式路由到相应的 CPU Core，进入对应的中断服务程序处理该中断。

步骤6　当虚拟中断完成以后，VM 发出 EOI 请求时，写寄存器 GICV_EOIR 或者 GICV_AEOIR 和寄存器 GICC_EOIR，自动完成虚拟中断和相应物理中断的 Priority Drop 和 Deactive（前提是 GICV_CTLR.EOImod==0，若其值不是 0，则只完成 Priority Drop；VM 再写寄存器 GICV_DIR，完成虚拟中断的 Deactive。对物理中断同样处理）。

----结束

2.6.5.5 处理虚拟中断操作（虚拟中断对应的物理中断为 SGIs）

步骤1　GIC 的 Distributor 根据配置将该物理中断分组，非安全中断分为 Group1。

步骤2　通过 Cpu_interface 将优先级最高的 Group1 中断以 IRQ 的形式分发给相应的 CPU Core。

步骤3　物理中断 IRQ 触发该 CPU Core 进入 Hyp 模式，并执行 Hypervisor 的中断处理程序，在该中断处理程序中，查询中断源等信息，配置 GIC 的虚拟中断控制寄存器，将该物理中断映射为虚拟中断(GICH_LRn.HW==0，即虚拟中断对应的物理中断为 SGIs)。并且 Hyperisor 发出一个 EOI 请求，写物理中断 IRQ 的寄存器 GICC_EOIR，完成 Priority Drop，但并未清除该物理中断。

步骤4　GIC 的虚拟 Distributor 将该虚拟中断分组为 Group0 和 Group1。

步骤5　虚拟 Cpu_interface 将 Group1 虚拟中断以虚拟 IRQ 形式，Group0 虚拟中断以虚拟 FIQ 的形式路由到相应的 CPU Core，进入对应的中断服务程序处理该中断。

步骤6　当虚拟中断完成以后，VM 发出 EOI 请求时，写 GICV_EOIR 或者 GICV_AEOIR 寄存器，完成虚拟中断的 Priority Drop 和 Deactive（前提是 GICV_CTLR.EOImod==0，若其值不是 0，则只完成 Priority Drop。VM 再写寄存器 GICV_DIR，完成虚拟中断的 Deactive）。

步骤7　虚拟 Distributor 产生 EOI maintenance 中断给 Hyperisor，Hyperisor 执行中断服务程序，去写 GICC_DIR 寄存器将该虚拟中断对应的物理中断也做 Deactive 操作。

----结束

2.6.6 性能

2.6.6.1 延迟（Latency）

中断产生到中断处理完毕这一过程的延迟取决于：

- 中断的优先级以及内部 Pending 状态中断的个数
- CPU 通过总线确认中断源，总线上带来的延迟
- 该中断是否被嵌套
- 对于虚拟中断，需要考虑 Hypervisor 响应物理中断，配置虚拟中断相关寄存器的延迟

典型的一种场景下，不存在中断优先级比较和中断嵌套的情况下，从中断上报给 GIC 到 CPU Core 的 nIRQ 管脚接收到中断需要 4 个 cycles。

2.6.7 调试（Debug）

通过 CPU 查询 GIC 内部中断状态寄存器 GICC_APRn、GICC_NSAPRn、GICV_APRn 寄存器等提供 Debug 信息，通过这些寄存器可以查询到中断当前的状态（Pending、Active、Inactive 等），中断的优先级，GIC 的 Running Priority 等信息。

2.7 系统存储管理单元（SMMU）

2.7.1 概述

系统存储管理单元（SMMU）是 ARM 架构下实现虚拟化扩展（Virtualization Extensions）所提供的一个重要组件。该结构为虚拟化具体的实现提供了一种灵活的框架，其可以应用于 I/O 及加速引擎的虚拟化，并可通过 LPAE（Large Physical Address Extension）实现对 40 位物理地址空间的访问。

系统存储管理单元（SMMU）是在多个 Guest OS 被一个 Hypervisor 管理的虚拟化系统中使用的，其主要支持中间物理地址（IPA）到物理地址（PA）的地址转换，中间物理地址到物理地址的转换是被 Hypervisor 控制的。

系统存储管理单元（SMMU）可以同时处理来自多个设备的 Transactions，并且根据这些 Transactions 的 Stream ID 来使用不同的 Stage2 的页表进行地址转换。

系统存储管理单元（SMMU）是通过在系统地址空间内可 Memroy Map 的寄存器来配置的，SMMU 的配置地址空间的大小由其支持的上下文场景数目决定。

2.7.2 特性

SMMU 支持如下特性：

- 支持 Stage2 地址转换——中间物理地址（IPA）到物理地址（PA）的地址转换；
- 支持多个 VM 的存储空间相互之间隔离访问；
- Stage2 地址转换支持最大 1TBytes 的物理地址空间；
- SMMU 支持同时存在多个活跃的上下文场景进行地址转换；
- 页表大小端可配；
- 支持 Hit Under Miss；
- 支持 Hit Under Previous Context Fault；
- 支持通过 DVM 对 SMMU 内 TLB 进行同步操作；
- 可以配置成 Transaction 透传模式，这种情况下 SMMU 不对 Transaction 的地址进行任何翻译；
- 支持地址转换的错误处理；
- 支持 TrustZone 扩展
- 支持调试和性能监控功能；

2.7.3 关键术语

如表 2-7 的术语将在本章节表示特殊的含义。

表2-7 SMMU 关键术语描述

术语	描述
Virtualization	虚拟化。
Context	上下文，使用不同页表的 Transaction 称其有不同上下文。

术语	描述
Transaction	总线上的数据传输。

2.7.4 功能描述

2.7.4.1 参考文档

ARM 提供的文档：《IHI0062B_system_mmu_architecture.pdf》。

2.7.4.2 应用框图

SMMU 的应用如图 2-7 所示。SMMU 将系统中的 Masters 发出的 32bits IPA 地址转换成 40bits PA。

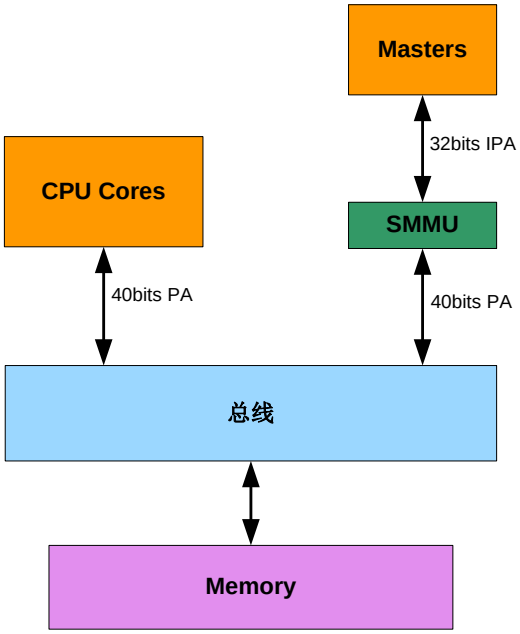


图2-7 SMMU 应用图

SMMU 为每个 VM 维护一个 Stage2 页表，根据 Masters 发出读写操作的 VM 属性（VMID），选取相应的页表进行地址转换。SMMU 的地址转换过程如图 2-8 所示。

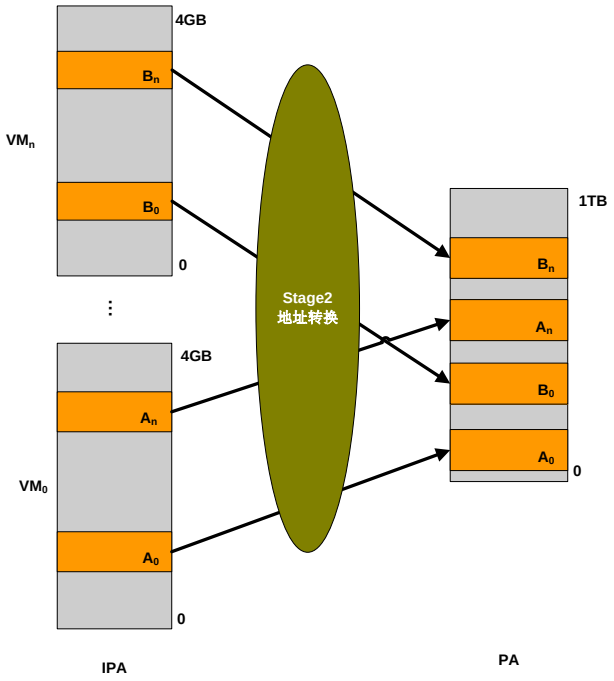


图2-8 SMMU 地址转换

每个 VM 的 IPA 地址范围均为 0~4GB，经过 Stage2 的地址转换后，被映射到 0~1TB 的物理地址空间，互相完全隔离。

2.7.4.3 数据结构

SMMU 和 A15 使用相同的二级页表，相同的 VMID 和 IPA 对于 A15 和连接到 SMMU 上的 IO 及加速器看到的是相同的物理地址。

SMMU 进行地址转换过程中用到的主要寄存器的数据结构如下。

- SMT（Stream Mapping Table）数据结构：

SMT 的每一行为一个 Entry，每个 Entry 叫做一个 Stream Mapping Register Group（Group 数目可以为 2 or 4 or 8 or 16 0r 24 or 32），其由 Stream Match Register 和 Stream to Context Register 组成，如下表（n 表示 group 的编号，可以通过查询 SMMU_IDR0.NUMSMRG 得到）。

SMMU_SMRn　　SMMU_S2CRn

- Stream Match Register 数据结构：

SMMU_SMRn (Stream Match Register)

31	30	16	15	14	0
VALID	MASK		RESERVED	ID	

VALID　　0：在查询SMT时该Entry是有效的；

　　1：在查询SMT时该Entry是无效的；

MASK　　MASK的某一位为1时，进行Stream ID匹配的相应位被忽略掉。

RESERVED 保留。

ID 用以匹配的Stream ID，若输入的Stream ID与该ID相同，则同一Entry的SMMU_S2CRn指示的Translation Context被用于地址转换。

- Stream to Context Register 数据结构：

SMMU_S2CRn (Stream to Context Register)

31	28	27	26	25	24	23	22	21	20	19	18
IMPLEMENTATION DEFINED		INSTCFG		PRIVCFG		WACFG		RACFG		NSCFG	

17	16	15	12	11	10	9	8	7	0
TYPE		MemAttr		MTCFG	RESERVED	SHCFG		CBNDX	

INSTCFG、PRIVCFG 、WACFG、RACFG、NSCFG、MemAttr、MTCFG、SHCFG为进行地址转换的memroy属性。

TYPE 00：指出用于进行地址转换的Translation Context Bank。

CBNDX 在TYPE=00时指出Context Bank的索引。

Stream to Context Register 里面的 Memroy 属性可以替换随 Transaction 自带的 Memroy 属性，也可以被 Translation Context Bank 的 Memroy 属性修改。

一个 Translation Context Bank 包括的主要信息有页表基地址、Page Table Walk 时使用的 memroy 属性以及页表属性的重新映射，其包括如下一些寄存器：

Translation Context Bank 是以表的形式配置的，Translation Context Bank Table 的每一个 Entry 占有一个页大小的地址空间。

Translation Context Bank Table 的 Entry 没有复位初始值，在撤离复位之后软件需要在使能 SMMU 之前先将 Translation Context Bank Table 内的每个 Entry 都进行初始化。

每一个 Context Bank 都有一个对应的 SMMU_CBARn（Context Bank Attribute Register），SMMU 内的 Context Bank Attribute Register 必须和 Translation Context Bank 数目相等，Context Bank Attribute Register 可以看做是 Translation Context Bank 的一部分，Context Bank Attribute Register 可以指定 VMID 给使用该 Translation Context Bank 的 Transaction，Context Bank Attribute Register 是以表的形式配置的。

Context Bank Attribute Register 的 Entry 没有复位初始值，在撤离复位之后软件需要在使用之前先将 Context Bank Attribute Register 内的每个 Entry 都进行初始化。

详细数据结构可参考 SMMU 寄存器描述。

2.7.4.4 Stage2 地址转换功能

SMMU 提供 Stage2 地址转换功能，其通过输入的特定 VMID 以及针对不同 VM 所建立的 Stage2 页表来实现 32 位 IPA 到 40 位 PA 的转换，在建立页表时保证不同 VM 的 Stage2 页表的 PA 域没有交叠的部分即可以实现多个 VM 的存储空间相互之间隔离访问。

SMMU 根据 VMID 索引到不同 Transaction 所属上下文的页表基地址寄存器，SMMU 内部提供 TLB 以减少对主存内页表的频繁访问。

2.7.4.5 页表大小端配置功能

SMMU 页表的大小端可以通过寄存器 SMMU_CBn_SCTLR[E] 来配置，配置为 0 时表示小端模式，配置为 1 时表示大端模式。

2.7.4.6 Hit under miss 功能

SMMU 在查找 TLB 时 Miss 的情况下后续的查找仍然可以 Hit，Hit Under Miss 的 Outstanding 深度取决于 Write Buffer 的深度，SMMU 的 Write Buffer 的深度为 16，其可以保持长度为 2 的 8 个 Transaction，当 Write Buffer 内被 Miss 的 Transaction 填满后 Hit Under Miss 不再有效。

在查找 TLB Miss 时 SMMU 会产生 Page Table Walk 自动从内存将相应的页表的 Entry 导入 TLB。

2.7.4.7 Hit Under Previous Context Fault 功能

SMMU 在进行地址转换的过程中可能会出现各种错误，Hit Under Previous Context Fault 功能允许 SMMU 在遇到错误的情况下依然可以支持对后续操作的地址转换。

SMMU 产生的错误可能是由相应的 Translation Context Bank 错误或全局（Global）错误引起的，错误状态信息可以被记录在一些错误状态寄存器中。

SMMU 在遇到错误的情况下可以产生错误报告或错误中断。

错误报告：

SMMU 可以通过返回 error 给发起产生错误的 Transaction 的发起者来报告错误。

错误中断：

SMMU 可以通过输出管脚上报中断给管理程序。

2.7.4.8 TLB 同步功能

SMMU 内的 TLB 是主存中页表的一个小的副本，当主存中的页表被修改时就需要对 TLB 进行同步操作以保证 TLB 的内容与主存中页表的内容是一致的。

SMMU 通过将其内部 TLB 无效化（TLB Invalidate）来实现 TLB 的同步，TLB 被无效化后 SMMU 会通过 Page Table Walk 自动从内存将页表内更新的 Entry 导入 TLB。

SMMU 有两种方式来实现 TLB Invalidate。

一种是通过广播（Broadcast）的方式，该方式通过专用的 DVM 通道将 TLB Invalidate 命令广播给所有挂接在总线上的 CPU 以及 SMMU 来实现的，该方式在 TLB 同步时可以减少软件开销，Hip04 采用该方式；另外一种是通过配置特定的 TLB Invalidate 寄存器来实现。

2.7.4.9 透传模式功能

SMMU 可以通过配置寄存器 SMMU_CR0[CLIENTPD]来实现是否对进入 SMMU 的 Transaction 进行透传（Bypass）不进行任何地址转换，该寄存器配置为 1 时对进入 SMMU 的 Transaction 进行透传（Bypass），配置为 0 时需要对 Transaction 进行地址转换，默认情况下该寄存器为 1。

2.7.4.10 支持 TrustZone 扩展

SMMU 内部寄存器分为安全寄存器和非安全寄存器两类，其中安全寄存器只能在安全世界下访问，非安全寄存器只能在非安全世界下访问。SMMU 寄存器描述中对于哪些寄存器是安全寄存器在成员说明一栏中进行了说明，在进行寄存器配置时需关注。

SMMU 对于输入的安全 Transaction 不进行地址转换，直接进行 Bypass，对于输入的非安全 Transaction 根据页表进行 Stage2 地址转换。

安全 Transaction 可以根据安全寄存器输出为安全和非安全 Transaction，非安全 Transaction 只能输出为非安全 Transaction。

2.7.4.11 性能监控功能

SMMU 提供一系列性能监控计数器对其内部性能相关的事件进行统计，包括 TLB 替换的次数，处理读操作和写操作的次数等，并且可以只对某一个 VM 的事件进行统计。

2.7.5 工作方式

2.7.5.1 Stage2 地址转换操作

步骤 1 Hypervisor 针对各个 VM 建立不同的 Stage2 页表，根据页表的存储位置配置 Context Bank 的页表基地址寄存器 SMMU_CBn_TTBR0 (n 表示不同 Context Bank 的编号，n 最大为 7，即可以支持 8 个 VM 的上下文)；

步骤 2 配置页表控制寄存器；

- SMMU_CBn_TTBCR[EAE]位为 1，选择 LPAE 模式，在 Stage2 地址转换时该位不可配置为 0；
- SMMU_CBn_TTBCR[SL0]位应根据实际需求配置为从 Leve l 或 Leve 2 开始取页表并进行地址转换；
- SMMU_CBn_TTBCR[T0SZ]位及 SMMU_CBn_TTBCR[S]位应根据输入 SMMU 的地址位宽进行合理配置，默认输入 SMMU 的地址为 32 位；

步骤 3 配置 SMT 寄存器组；

- 配置 SMMU_SMRn[ID]为{0x0,VMID}，SMMU_SMRn[MASK]为全 0，表明用 VMID 来匹配该 SMT Entry 且匹配时不忽略 VMID 任何一位；
- 配置 SMMU_SMRn[VALID]为 1，使能 SMT 的该 Entry；
- 配置 SMMU_S2CRn[TYPE]为 0b00，SMMU_S2CRn[CBNDX]为 0~15（n 为几则配为几），表明用该 SMMU_S2CRn 寄存器指示的 Context Bank 来作为相应 VM 的上下文场景；

步骤 4 配置 Context Bank 系统控制寄存器 SMMU_CBn_SCTLR[M]为 1，使能该 Context Bank；

步骤 5 配置寄存器 SMMU_CR0[CLIENTPD]为 0，表明需要对 Transaction 进行地址转换；

步骤 6 配置连接到 SMMU 的 Master（如 ROHC）发相应的操作，SMMU 即可对 Master 发出的操作进行 Stage2 地址转换。

2.7.5.2 Hit Under Previous Context Fault 操作

步骤 1 配置 SMMU_CR0[GCFGFIE]和 SMMU_CR0[GFIE]为 1，配置 SMMU_CR0[GCFGRE]和 SMMU_CR0[GFRE]为 1，使能全局错误中断和全局错误报告；

步骤 2 配置 SMMU_CBn_SCTLR[CFIE]为 1，配置 SMMU_CBn_SCTLR[CFRE]为 1，使能相应 Context Bank 的错误中断和错误报告；

步骤 3 配置 SMMU_CBARn[TYPE]为 0b00，表明进行 Stage2 地址转换，在 SMMU_CBARn[IRPTDX]配置当相应的 Context Bank 发生中断时的中断号；

步骤 4 配置 SMMU_CBn_SCTLR[CFCFG]为 1，产生错误的 Transaction 将被阻塞住；

步骤 5 配置 SMMU_CBn_SCTLR[HUPCF]为 1，使能 Hit Under Previous Context Fault 模式。

2.7.5.3 TLB 同步操作

步骤 1 CPU 执行 DVM 指令，从而通过 AR 通道发出 TLB Invalidate DVM Operation；；

步骤 2 Interconnect 通过 AC 通道将 TLB Invalidate 广播；

步骤 3 所有参与的组件在收到 Interconnect 广播的消息后通过 CR 通道进行应答，表示其收到该操作；

步骤 4 Interconnect 收集完所有的 CR 通道的应答后通过 R 通道返回 response 给 CPU。

步骤 5 CPU 通过 AR 通道发出 DVM Sync 操作；

步骤 6 Interconnect 通过 AC 通道将 DVM Sync 广播；

步骤 7 所有参与的组件在收到 Interconnect 广播的消息后通过 CR 通道进行应答；

步骤 8 Interconnect 收集完所有的 CR 通道的应答后通过 R 通道返回 response 给发出 DVM Sync 操作的 CPU；

步骤 9 所有参与的组件在彻底完成 TLB Invalidate DVM Operation 后 CPU 通过 AR 通道发出 DVM Complete 操作；

步骤 10 Interconnect 收到 DVM Complete 后立即通过 R 通道返回 response；

步骤 11 当 Interconnect 收到所有组件发出的 DVM Complete 后其通过 AC 通道发出 DVM Complete 给发出 DVM Sync 操作的 CPU；

步骤 12 CPU 收到 DVM Complete 后通过 CR 通道进行应答，表明其已收到 DVM Complete。

2.7.6 性能

2.7.6.1 吞吐率（Throughput）

SMMU 吞吐率与 TLB 命中率，Hit 延时以及 Miss 延时有关，具体精确数据要根据特定的测试程序来得到。

2.7.6.2 延迟（Latency）

SMMU 内的 TLB hit 时的延时为 2cycles，若 TLB 是用 RAM 实现的，则 hit 时的延时为 4cycles；SMMU 内的 TLB miss 时的延时取决于 interconnect 和 Memory 系统的延时。

ARM 提供了两种测试场景的延时和开销数据：

1 Mali 图形测试：

TLB 命中率	Hit 延时	Miss 延时	Hit 开销	Miss 开销	总开销
95.6%	2cycles	平均 59cycles	1.3%	2%	3.3%

2 8 通道 DMA 测试：

TLB 命中率	Burst 长度为 1（最差情况）		Burst 长度为 16	Burst 长度为 16 时软件开销
	Hit 开销	总开销	总开销	
99%	2.5%	2.67%	1.5%	36%

2.7.7 调试（Debug）

2.7.7.1 TLB 查询

SMMU 支持对 TLB 的查询以利于 Debug，该 Debug 功能是通过 Debug 读指针寄存器（Debug Read Pointer Register）和 Debug 读数据寄存器（Debug Read Data Register）来实现的。

当有读操作访问 Debug 读数据寄存器时，Debug 读指针寄存器的值自动加 1，读数据通过总线接口返回 TLB 的 1 个 word，Debug 读指针寄存器的地址若超出 TLB 的范围，则通过总线接口访问 Debug 读数据寄存器将会返回 Error，该 Debug 读指针寄存器可写。

若 SMMU 正在进行地址转换时对 Debug 读数据寄存器进行读操作则返回的数据有可能是 TLB 更新之前或之后的数据。

2.7.8 注意事项

控制程序需要保证 SMMU 的状态正在被修改的时候没有会被状态修改影响的 Transactions 正在处理，不被 SMMU 状态修改影响的 Transactions 可以同时执行。

SMMU 不支持 Write Data Interleaving，写数据和写地址的顺序须一致。

SMMU 不支持 Lock 操作，输入 SMMU 的 Lock 访问会输出为一个 Normal 访问。

2.8 DDR 控制器（DDRC）

2.8.1 概述

DDRC（DDR2/3 DDR SDRAM Controller）实现对 DDR2/3 DRAM 的存取控制，是整个芯片的数据存储中心，Hip04 包括两组 DDRC，每个 DDRC 中有两组 DMC 可以实现共享命令总线的双通道， DDRC 完成系统内部总线操作到 DDR 的时序转换。

2.8.2 功能描述

DDRC 为整个系统的主存控制器，共有两组 DDRC 分别连接两组 L3 Cache master 独立访问两组 DDR 主存。不同组的 DDRC 访问根据 L3_cache 的地址映射方式决定。

每个 DDRC 里面包含了 2 组 DRAM 控制器（DMC）可以实现共享命令总线的双通道 32Bits DDR 或者单通道 32Bits/64BitsDDR 访问。DDRC 支持的器件容量空间以及地址映射

DDRC 支持外接 64Bits 器件或 32Bits 器件，支持多 rank 地址映射，每个 DDRC 最大支持 4 个 Rank，每个 Rank 最小容量支持 128MB（32BitsDDR），256MB（64BitsDDR）最大容量支持 8GB，所以每个 DDRC 最大支持 4X8GB 的地址空间，所以 2 个 DDRC 最大能够支持 64GB 的地址空间。

2.9 包解析引擎（PPE）

2.9.1 概述

包解析引擎（PPE）是控制 GE 网口收发报文的模块，预申请缓存并管理，对接收报文进行解析产生调度信息和解析结果给其他加速模块和软件使用，按照报文发送描述符对报文进行处理并通过网口发送出去。

2.9.2 特性

PPE 具有以下特性：

- 支持 10 个通道的网口 GE；
- 支持以太网 V2、802.3snap、802.1Q、Q-in-Q 封装的报文识别；
- 支持 IPv4、IPv6 报文的识别；
- 支持 TCP、UDP、GRE、SCTP、AH、ESP、ICMP、IGMP 报文的识别；
- 支持 DHCP、BOOTP、IP PM、GTPU、UDP 封装报文、扩展头报文/上行反压报文/传输到 L2 的 License 反压报文的识别；
- 支持分片包的识别；
- 接收方向支持已知报文有效性校验加速、畸形报文检查；
- 发送方向支持软件指定报文的 IPv4 头、TCP、UDP 的 Checksum 计算（对于被截断无完整 Checksum 域的报文不计算）；
- 存放报文支持 SKIP 区（SKIP1 最大配置为 15 个 CacheLine，SKIP2 最大配置为 7 个 CacheLine），支持在 CacheLine 对齐地址的后面空闲 0~63 个字节存放报文内容；
- 支持通过链表的方式对接收报文进行分离存放；
- 支持软件通过报文发送描述符配置发送报文的读操作是否为 Read invalid 和 Read Allocate；
- 支持反压 CPU Station 功能：
  - 12 个通道的接收 FIFO 可以单独设定反压、解反压水线，各通道是否反压可配；
  - 12 个通道只要有一个通道接收 FIFO 达到反压水线就反压 CPU Station，所有通道接收 FIFO 低于解反压水线才取消反压 CPU Station；
  - 支持连续反压时间值可配，连续反压 CPU Station 超时后，强制结束反压；
- 支持按 Group 存放报文；
  - 接收有 32 个 FIFO 用于存放向配置的 Pool 申请的 Buffer 或由软件分配的 Buffer（软件分配缓存时只支持 1 个核分配缓存，且需要第一个缓存分配完再分配第二个缓存）；
  - 软件可以将这 32 个 FIFO 申请到的 Buffer 分配给不同 Group 的报文；
- 支持 34 位地址访问 DDR：
  - 接收方向可以基于通道配置访问 DDR 时的地址[33:32]；
  - 发送方向可以基于发送描述符配置访问 DDR 时的地址[33:32]；
- 发送支持 CPU 独立推包寄存器（16 个），并提供推送是否成功状态查询；
- 访问模块的寄存器仅支持小端模式；

- 支持模块访问 DDR 空间大小端可配；
- 支持基于 Group 的接收报文统计；
- 支持一个可配置的本地计时功能：
  - 支持片外输入时钟计时（1—65MHz），计时时钟周期值支持寄存器配置；
  - 支持由片外主 RTC 输入的秒同步信号进行本地时间同步；
  - 支持片外输入时钟的频率误差校正；
  - 支持使用片中系统时钟提高计时精度；
  - 支持本地 1pps 输出管脚观测；
  - 支持当前时间寄存器可读；
- 支持中断管理：
  - 支持配置中断屏蔽寄存器屏蔽中断；
  - 支持从中断状态寄存器获取中断状态；
  - 支持中断清除寄存器清除中断状态；
  - 支持写工作描述符失败中断；
  - 支持申请接收缓存失败中断，连续申请一定次数（可配置）均失败后，停止申请缓存，并上报中断，同时启动定时器，当定时器计数超时后重新申请，或者在等待期间中断被清除后可重新申请缓存；
  - 支持内部 FIFO 出现又空又满异常情况的中断上报；
  - 支持读写总线错误中断上报；
  - 支持收包中断，当接收报文个数达到阈值或未达到阈值但超时，上报中断；
  - 支持接收丢帧中断；
  - 支持发送丢帧中断；
  - 发送配置 FIFO 中数据个数由高于等于满水线到低于满水线时中断；
  - MEM 读写奇偶校验错误中断状态；
  - 支持片外秒同步信号中断上报。
- 提供智能心跳功能，实时监控内部核心状态机运行，如果状态机超时无跳转，则表示心跳停止，将上报中断；
- 提供总线访问延迟统计功能，在设定滑动窗口内将实时监控总线访问延迟，统计传输次数、总延迟及单次传输最大延迟；
- 各种统计不会影响 PPE 的性能。

2.9.3 关键术语

如表 2-8 的术语将在本章节表示特殊的含义。

表2-8 PPE 关键术语描述

术语	描述
PPE	Packet Parse Engine，包解析引擎模块。

术语	描述
Group	包描述符中一个域，每个包都属于一个 Group，每个处理单元可以处理特定的 Group 包，通过此域将包分配到不同处理单元进行处理，达到各处理单元负载均衡的效果。
Pool	资源池，缓存空闲 buffer 空间的指针。
接收解析描述符	接收报文的解析结果和存储在 DDR 中的位置等信息。
工作描述符	写给 POE 的简化的接收解析描述符。
报文发送描述符	发送报文存储在 DDR 中的位置和报文的处理方式等信息。

2.9.4 功能描述

2.9.4.1 数据结构

报文的接收解析描述符用于硬件的加速，其结构如下表所示：

PPE 有 3 个数据结构：PPE 解析描述符、工作描述符、PPE 发送描述符。

警告

当禁止 PPE 的虚拟化功能时 ，PPE 解析得到的报文 VMID 字段为 0，而在禁止 PPE 的虚拟化功能时，若向 PPE 发送报文时，软件需要将对应报文描述符的 VMID 配置为 0

2.9.4.2 接收报文的存储方式

当一个 Buffer 存放不下报文时会使用多 Buffer 存放，如图 2-9 所示是一个多 Buffer 存储的示意图

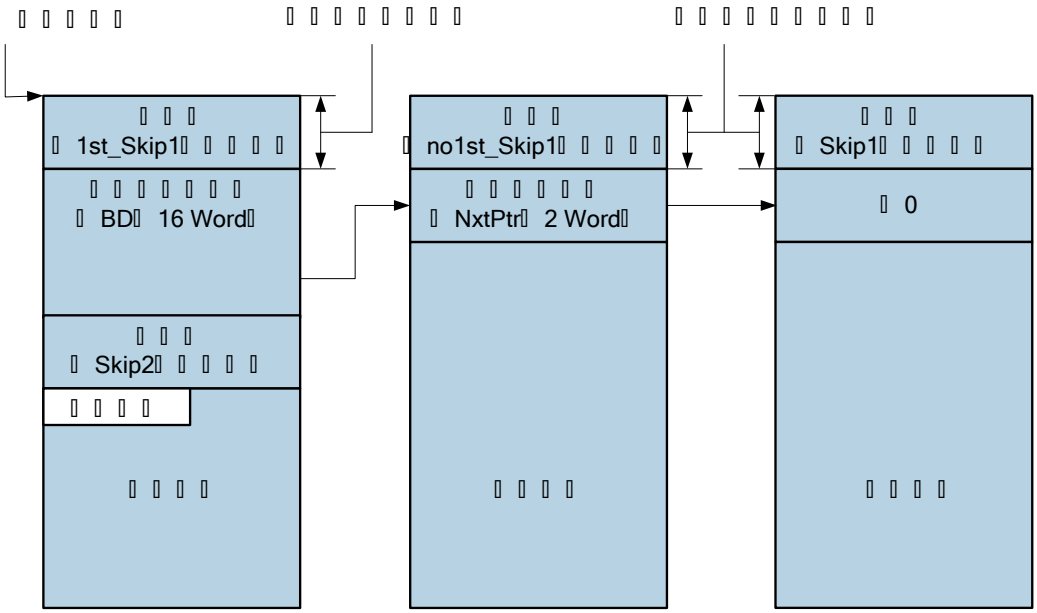


图2-9 接收报文在 DDR 中的存储方式示意图

2.9.4.3 支持的发送报文格式

发送报文格式支持单缓存和多缓存格式。前者的数据组织如图 2-10 所示，后者的数据组织如图 2-11 所示。

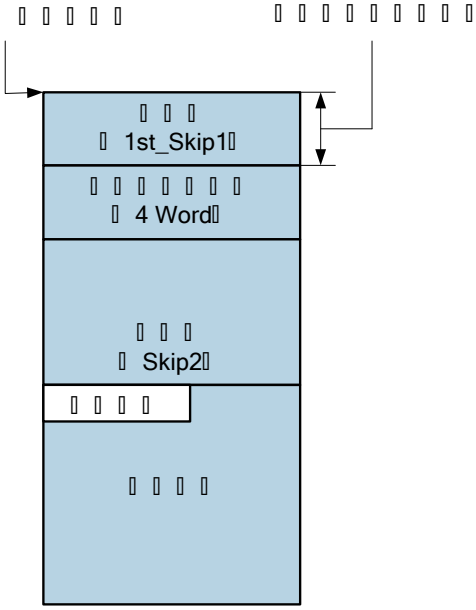


图2-10 单缓存发送报文格式

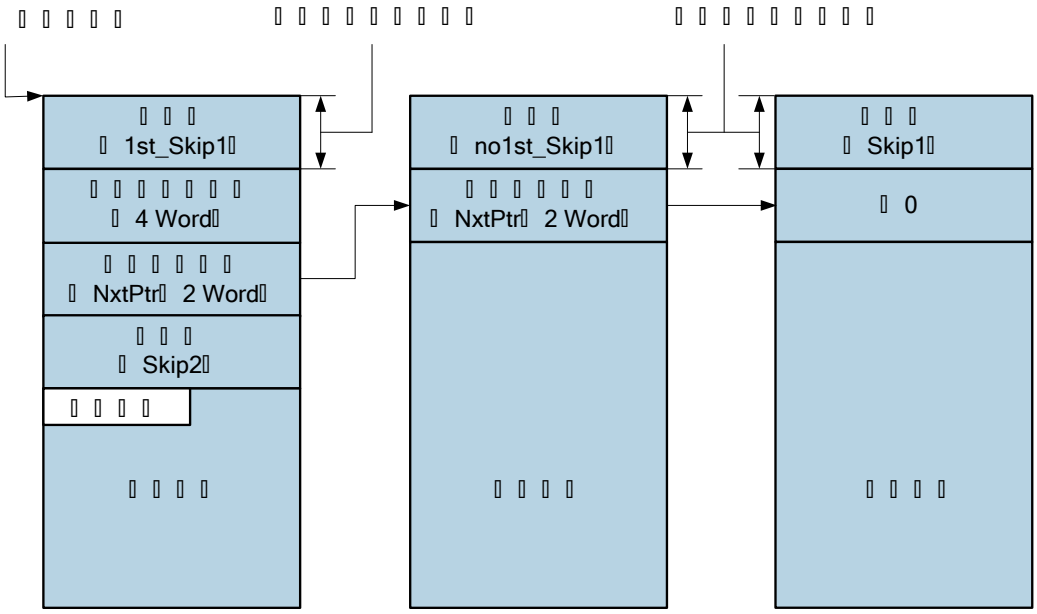


图2-11 多缓存发送报文格式

2.9.4.4 配置限制

以下是所用端口统一配置，其他是单端口可配置：

- 接收报文的缓存申请和分配给哪些 GRP；

2.9.5 工作方式

2.9.5.1 初始化

PPE 复位后使用时初始化：

步骤 1 将分配给 PPE 存放报文的 Pool 基地址配置到需要主动申请 Buffer 的缓存 FIFO 上，使得需要主动申请 Buffer 的缓存 FIFO 向一个指定的 Pool 申请 Buffer；

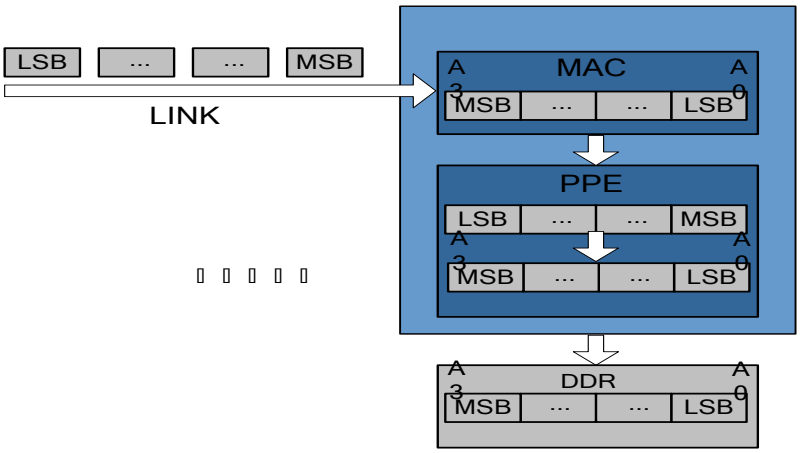
步骤 2 将各个缓存 FIFO 配置上 VMID 和 Group，使得每个接收报文按照其 VMID 和 Group 使用对应的缓存 FIFO 中的 Buffer；

步骤 3 配置各个缓存 FIFO、接收数据 FIFO、发送数据 FIFO 的水线；

步骤 4 配置各个网口通道解析报文的方式；

----结束

2.9.5.2 大小端配置


LSB
...
...
MSB
LINK
MAC
A3
A0
MSB
...
...
LSB
PPE
LSB
...
...
MSB
A3
A0
MSB
...
...
LSB
DDR
A3
A0
MSB
...
...
LSB

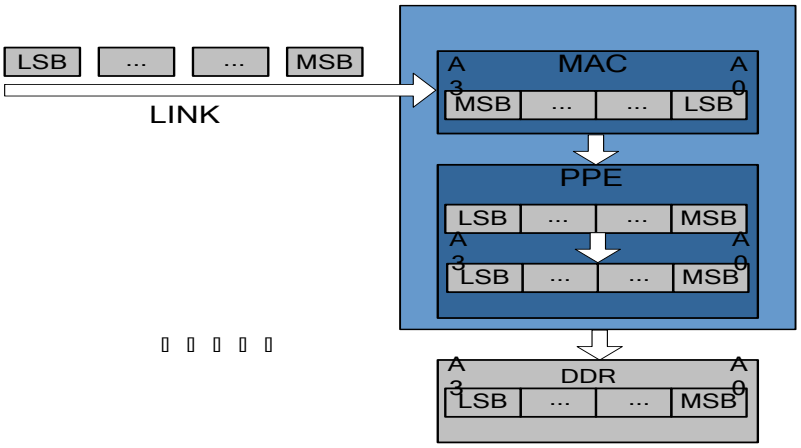

LSB
...
...
MSB
LINK
MAC
A3
A0
MSB
...
...
LSB
PPE
LSB
...
...
MSB
A3
A0
LSB
...
...
MSB
DDR
A3
A0
LSB
...
...
MSB

图2-12 大小端模式下接收报文的处理方式

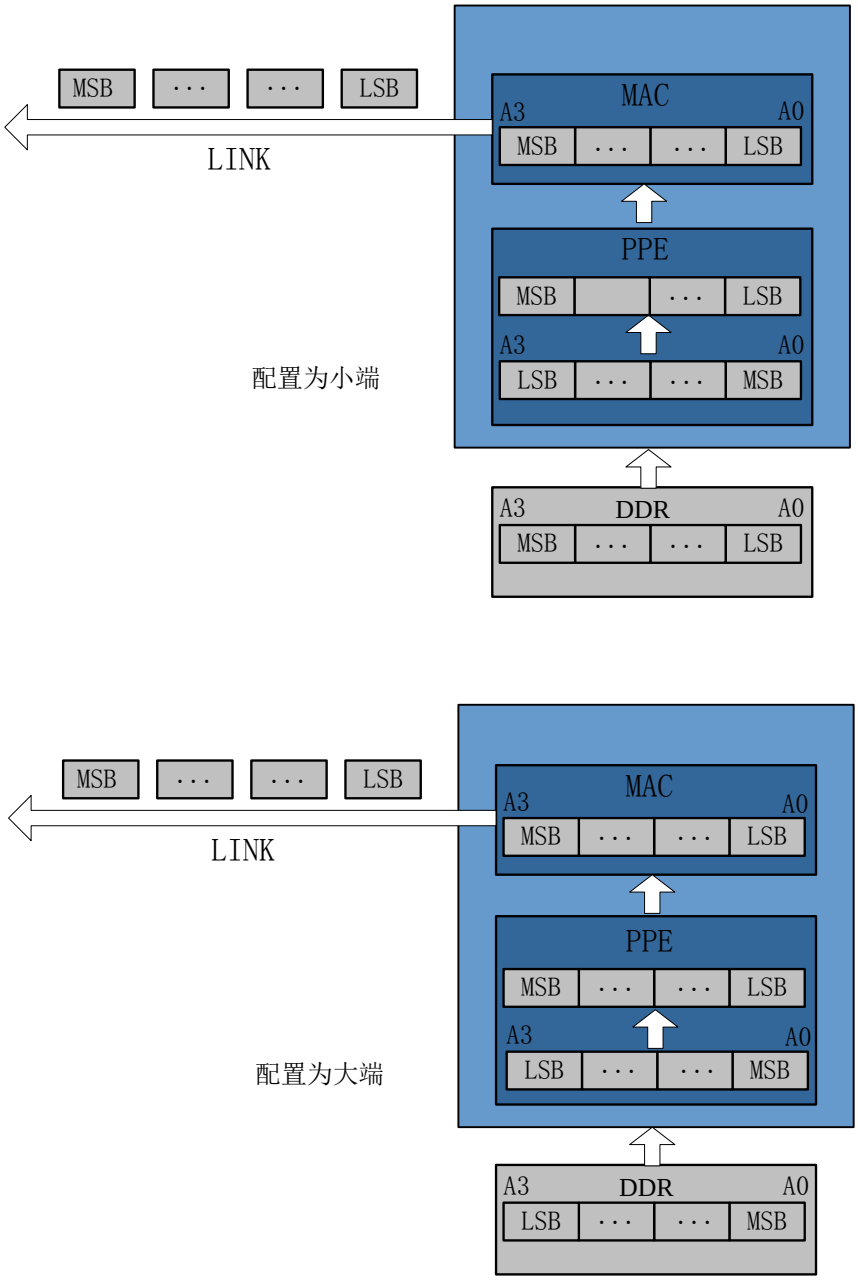


图2-13 大小端模式下发送报文的处理方式

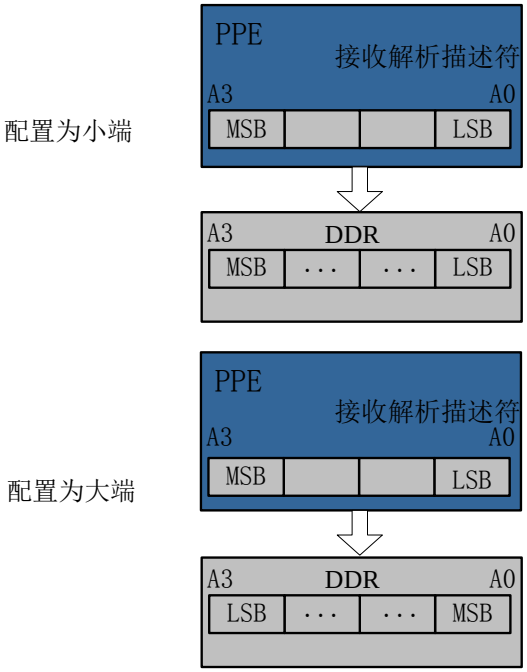


图2-14 大小端模式下接收解析描述的处理方式

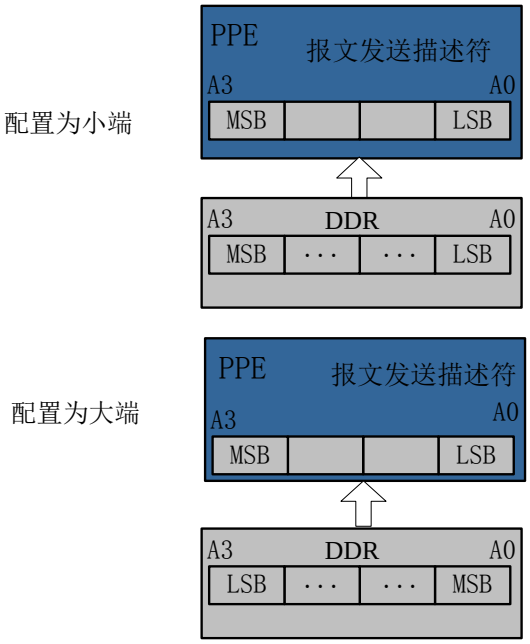


图2-15 大小端模式下报文发送描述的处理方式

2.9.6 性能

2.9.6.1 吞吐率（Throughput）

PPE 总共可以接 11 个网口通道。

Hip04 中接了 4 个 10G 的 GE、4 个 1G/2.5G 的 GE、2 个 1G 的 GE、1 个 100M 的 GE，最大收发吞吐率分别是 52.1Gbps，78Mpps（64 字节小包）。

2.9.6.2 延迟（Latency）

报文在 PPE 中是线速处理的，所以报文在 PPE 中的延时取决于以下 3 方面：

- 发送数据 FIFO、接收数据 FIFO 的水线；
- 读写总线的延时；
- 下游是否拥塞及响应时间。

2.9.7 调试（Debug）

2.9.7.1 发送通道查询

当发送通道发生异常时可以检查下面几方面：

- 发送状态机处于什么状态，根据状态进一步确认原因；
- 发送报文截断统计寄存器是否有统计，确认总线性能是否足够；

2.9.7.2 接收通道查询

当接收通道发生异常可以检查下面几方面：

- 接收状态机处于什么状态，根据状态进一步确认原因；
- 本地缓存 FIFO 是否有 Buffer；
- 接收报文截断、丢包统计寄存器是否有统计，确认总线性能是否足够；

2.10 GE 网口（GE）

2.10.1 概述

Hip04 支持 9 个 GE Serdes 业务网口，一个 MII 和 RGMII 复用的网口。整个 GE 提供网络报文收发、数据链路层协议处理、差错检测与校正、报文类型统计和流控等功能。

2.10.2 特性

GE 具有以下特性：

- 提供 9 个 10M/100M/1G/2.5Gbps SerDes 接口；
- 提供 1 个 MII 和 RGMII 复用的网口，可以配成 MII 或者 RGMII 使用；
- RGMII 接口发送方向支持 Normal 模式时序和 Delay 模式时序，接收方向只支持 Normal 模式；
- 支持半/全双工从用户侧至 PHY 侧的帧发送功能；
  - 全双工支持上述所有接口；
  - 半双工支持 10/100Mbps RGMII/SGMII 接口；
  - 支持流控请求响应功能；
  - 支持流控帧发送功能；
- 支持符合 802.3 协议的帧发送处理功能；
- 支持半/全双工从 PHY 侧至用户侧的帧接收功能；
  - 全双工支持上述所有接口；
  - 半双工支持 10/100Mbps MII/RGMII/Serdes 接口；
  - 支持识别流控帧，并将流控请求通知发送通道；
- 支持符合 802.3 协议的帧接收处理功能；
- 提供帧统计计数信息和状态信息；
  - 支持流控帧、VLAN 帧、广播帧、单播帧、组播帧、数据帧等收发数量统计信息；
  - 支持特殊帧或者错误帧收发统计信息，主要包括超长帧（对于超长帧会上报二层错）、超短帧、CRC 错误帧、过滤帧等；
  - 支持收发帧长统计，并对各个长度范围内收发帧进行个数统计；
- 支持流控帧功能；
- 最大支持 64KB Jumbo 帧，最小接收 64 字节包长，小于 64 字节报文直接过滤；
- 支持对接收报文进行目的 MAC 地址过滤，目的 MAC 和本地 MAC 地址不同的报文直接丢弃：
- 支持 SGMII 协议定义的自协商功能（即可以通过 Serdes 链路读取 PHY 的工作状态，但 GE 的速率和双工模式的改变需要软件参与）；
- 接收支持对所有的报文申请时间戳；
- 时间戳的精度误差不大于 15ns；
- 各种统计不会影响 GE 的性能。

2.10.3 关键术语

如表 2-9 的术语将在本章节表示特殊的含义。

表2-9 GE 关键术语描述

术语	描述
GE	Gigabit Ethernet，千兆以太网
MAC	Media Access Control，媒体接入控制
MII	Medium Independent Interface，介质独立接口
RGMII	Reduced Gigabit Medium Independent Interface，精简千兆介质独立接口
SGMII	Serial Gigabit Media Independent Interface，串行千兆介质独立接口
PPE	Packet Parse Engine，包解析引擎模块
FCS	Frame Check Sequence，帧校验序列
VLAN	Virtual Local Area Network，虚拟局域网
SFD	Start Frame Delimiter，帧开始分隔符

2.10.4 功能描述

2.10.4.1 应用框图

GE 的应用如图 2-16 所示。

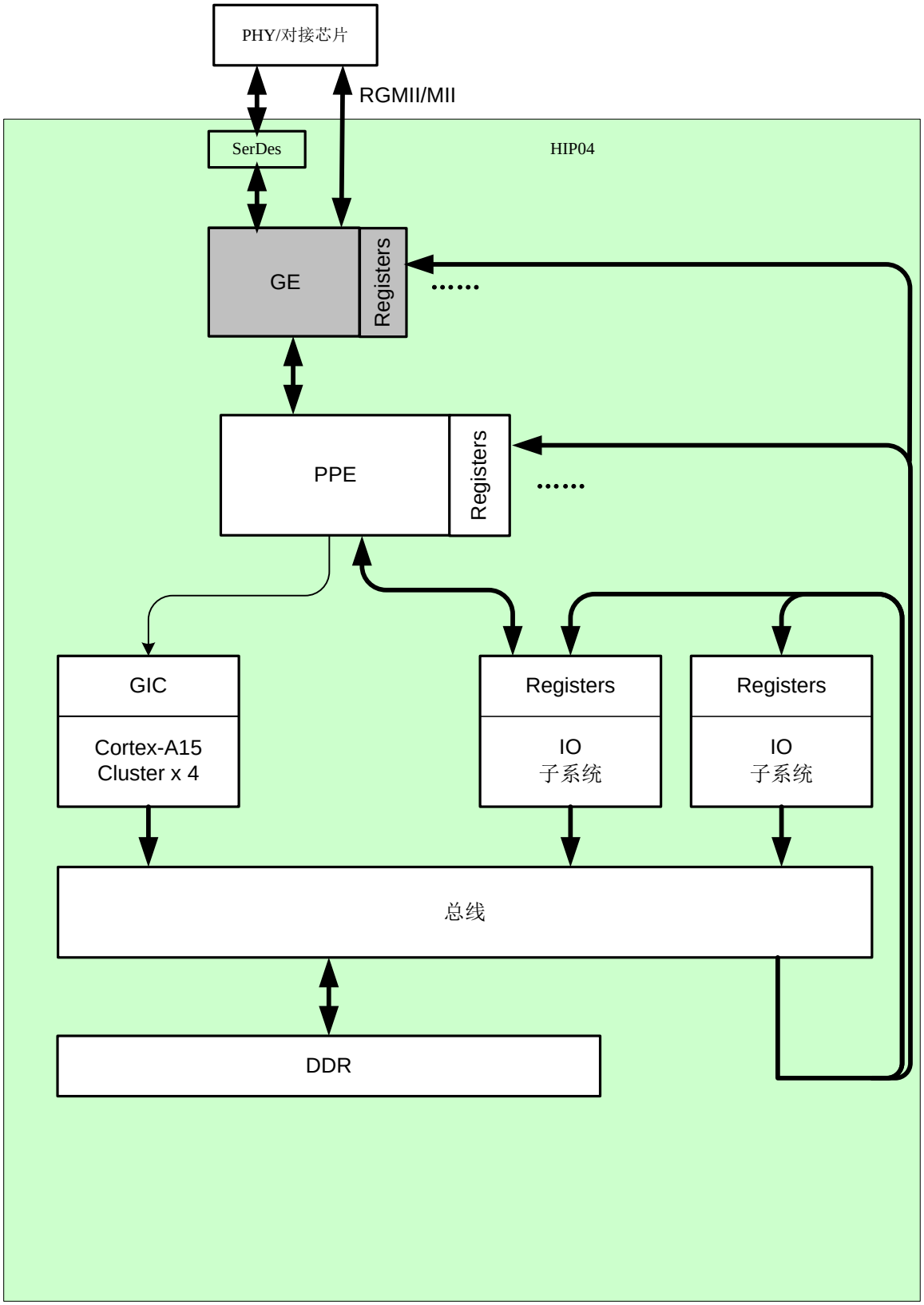

PHY/对接芯片
RGMII/MII
SerDes
HIP04
GE
Registers
.....
PPE
Registers
.....
GIC
Cortex-A15
Cluster x 4
Registers
IO
子系统
Registers
IO
子系统
总线
DDR

图2-16 GE 应用图

GE 位于 PPE 和 PHY/对接芯片之间，按照 802.3 协议完成应用侧和链路侧的数据交换。

发送方向 GE 接收 PPE 发送过来的报文，将报文添加前导码、计算并添加 FCS（可选）、添加 PAD（报文小于 64Bytes 时可选）后通过 RGMII/MII 接口或 SerDes 接口发送给 PHY/对接芯片，完成应用侧到链路侧的数据转换。

接收方向 GE 通过 RGMII/MII 接口或 SerDes 接口接收 PHY/对接芯片发送过来的报文，将报文剥离前导码、校验 FCS 的正确性、剥离 FCS（可选）、剥离 PAD（仅对 802.3 snap 帧）、检测报文链路层错误后将报文和报文正确性检测结果传递给 PPE，完成链路侧和应用侧的数据转换。

2.10.4.2 相关模块

在系统中，GE 模块会和下面多个模块进行交互，分别描述如下：

- PPE：通过自定义的接口完成应用侧和链路侧的数据交换；
- SerDes：完成收发报文的串并转换。

2.10.4.3 流控功能

当芯片的接收缓存不够，PPE 可以动态指示 GE 发送流控帧（GE 发送是否响应流控帧可配，发送流控帧的间隙和流控帧的时间域值可配置），通知对端控制流量。

当接收到流控帧时，发送方向会按照流控帧内的时间值暂时停止发送报文，缓解对端压力。

2.10.4.4 MAC 地址过滤功能

接收方向支持 36 个可配的单播 MAC 地址过滤，对于 MAC 地址不匹配的报文进行过滤。

接收方向支持 8 个可配的带掩码的多播 MAC 地址过滤，对于和掩码的 MAC 地址不匹配的报文进行过滤。

2.10.5 工作方式

2.10.5.1 管脚复用配置

GMAC9 的 RGMII 管脚和 GPIO 复用，GMAC0 使用 RGMII 接口时需要配置 IO CONFIG 第 12 个寄存器 IOMG011[iomg011]（基地址 0x2000D000，偏移地址为 0x0030，Bit[1:0]）为 2’b01（默认值是 2’b00）

GMAC10 的 RGMII 管脚和 GPIO 复用，GMAC1 使用 RGMII 接口时需要配置 IO CONFIG 第 13 个寄存器 IOMG011[iomg011]（基地址 0x2000D000，偏移地址为 0x0034，Bit[1:0]）为 2’b01（默认值是 2’b00）。

2.10.5.2 初始化

步骤 1 配置端口通道使能寄存器 PORT_EN [tx_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0044，Bit[2]）为 0，关闭发送通道。

配置端口通道使能寄存器 PORT_EN [rx_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0044，Bit[1]）为 0，关闭接收通道。

步骤 2 配置全双工半双工模式寄存器 DUPLEX_TYPE [duplex_sel]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0008，Bit[0]）为对接 PHY 的双工模式。

步骤 3 配置端口模式寄存器 PORT_MODE [port_mode]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0040，Bit[3:0]）为对接 PHY 的工作模式。

步骤 4 配置端口模式寄存器 PORT_MODE [rgmii_1000m_delay]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0040，Bit[4]）选择 RGMII 的发送方向输出时钟是否 delay（若是 SGMII 模式则不需要该步骤，RGMII 模式是否 delay 取决于 PHY 是否配置了 delay 模式，若 PHY 配置为 delay 模式，则 rgmii_1000m_delay 配置为 0，否则配置为 1）。

步骤 5 配置端口模式改变使能寄存器 MODE_CHANGE_EN [mode_change_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x01B4，Bit[0]）为 1。

步骤 6 配置 MAC_CORE 的应用侧时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0140，Bit[9]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0140，Bit[8]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0140，Bit[7]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0140，Bit[6]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0140，Bit[5]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0140，Bit[4]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0140，Bit[3]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0140，Bit[2]。

步骤 7 配置 MAC 的工作时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[18]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[19]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[20]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[21]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0044，Bit[22]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0044，Bit[23]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0044，Bit[24]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0044，Bit[25]。

步骤 8 配置 MAC 的发送方向 SERDES 的时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[4]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[5]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0048，Bit[6]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0048，Bit[7]；

- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[8]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[9]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[10]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[11]。

步骤 9　配置 MAC 的接收方向 SERDES 时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[28]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[29]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[30]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[31]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[0]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[1]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[2]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[3]。

步骤 10　配置 MAC 的 RGMII 接收方向时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[22]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[23]。

步骤 11　配置 MAC_CORE 的应用侧时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0140，Bit[9]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0140，Bit[8]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0140，Bit[7]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0140，Bit[6]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0140，Bit[5]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0140，Bit[4]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0140，Bit[3]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0140，Bit[2]。

步骤 12　配置 MAC 的工作时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[18]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[19]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[20]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[21]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0044，Bit[22]；

- GMAC5：基地址 0x20000000，偏移地址为 0x0044，Bit[23]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0044，Bit[24]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0044，Bit[25]。

步骤 13 配置 MAC 的发送方向 SERDES 的时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[4]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[5]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0048，Bit[6]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0048，Bit[7]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[8]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[9]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[10]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[11]。

步骤 14 配置 MAC 的接收方向 SERDES 时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[28]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[29]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[30]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[31]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[0]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[1]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[2]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[3]。

步骤 15 配置 MAC 的 RGMII 接收方向时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[22]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[23]。

步骤 16 配置端口通道使能寄存器 PORT_EN [tx_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0044，Bit[2]）为 1，打开发送通道。

配置端口通道使能寄存器 PORT_EN [rx_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0044，Bit[1]）为 1，打开接收通道。

----结束

2.10.5.3 软复位

芯片上电后，Serdes 锁定前的不稳定时钟可能会使 GMAC 进入异常状态，所以 Serdes 锁定后请务必按照软复位下 GMAC；改变 GMAC 模式请按照下面初始化章节操作。

步骤 1 配置 MAC 的总线接口软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[12]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[13]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0048，Bit[14]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0048，Bit[15]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[16]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[17]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[18]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[19]。

步骤 2 配置 MAC 的工作时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[18]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[19]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[20]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[21]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0044，Bit[22]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0044，Bit[23]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0044，Bit[24]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0044，Bit[25]。

步骤 3 配置 MAC_CORE 的配置寄存器软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0140，Bit[19]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0140，Bit[18]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0140，Bit[17]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0140，Bit[16]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0140，Bit[15]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0140，Bit[14]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0140，Bit[13]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0140，Bit[12]。

步骤 4 配置 MAC 的发送方向 SERDES 的时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[4]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[5]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0048，Bit[6]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0048，Bit[7]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[8]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[9]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[10]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[11]。

步骤 5　配置 MAC 的接收方向 SERDES 时钟软复位寄存器为 1

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[28]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[29]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[30]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[31]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[0]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[1]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[2]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[3]。

步骤 6　配置 MAC 的 RGMII 接收方向时钟软复位寄存器为 1。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[22]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[23]。

步骤 7　配置 MAC 的总线接口软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[12]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[13]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0048，Bit[14]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0048，Bit[15]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[16]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[17]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[18]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[19]。

步骤 8　配置 MAC 的工作时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[18]；

- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[19]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[20]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[21]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0044，Bit[22]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0044，Bit[23]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0044，Bit[24]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0044，Bit[25]。

步骤 9 配置 MAC_CORE 的配置寄存器软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0140，Bit[19]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0140，Bit[18]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0140，Bit[17]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0140，Bit[16]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0140，Bit[15]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0140，Bit[14]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0140，Bit[13]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0140，Bit[12]。

步骤 10 配置 MAC 的发送方向 SERDES 的时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[4]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[5]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0048，Bit[6]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0048，Bit[7]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[8]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[9]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[10]；
- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[11]。

步骤 11 配置 MAC 的接收方向 SERDES 时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0044，Bit[28]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0044，Bit[29]；
- GMAC2：基地址 0x20000000，偏移地址为 0x0044，Bit[30]；
- GMAC3：基地址 0x20000000，偏移地址为 0x0044，Bit[31]；
- GMAC4：基地址 0x20000000，偏移地址为 0x0048，Bit[0]；
- GMAC5：基地址 0x20000000，偏移地址为 0x0048，Bit[1]；
- GMAC6：基地址 0x20000000，偏移地址为 0x0048，Bit[2]；

- GMAC7：基地址 0x20000000，偏移地址为 0x0048，Bit[3]。

步骤 12 配置 MAC 的 RGMII 接收方向时钟软复位寄存器为 0。

寄存器具体地址如下：

- GMAC0：基地址 0x20000000，偏移地址为 0x0048，Bit[22]；
- GMAC1：基地址 0x20000000，偏移地址为 0x0048，Bit[23]。

----结束

2.10.5.4 自协商

Hi1381V100 支持 802.3 协议规定的 1000Base-X 自协商，SGMII 协议规定的自协商功能（包括 SGMII 自协商 PHY 模式、SGMII 自协商 MAC 模式）。

注意

自协商只会读取 PHY 的状态到本地寄存器，不会改变 MAC 的状态，本地 MAC 的状态仅受端口模式寄存器 PORT_MODE[port_mode]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x0040，Bit[3:0]）控制。

1000Base-X 自协商

步骤 1 配置自协商本地配置寄存器 TX_LOCAL_PAGE（基地址 0x?0000000+10000*ch_num，偏移地址为 0x005C）为 32’h1A0。

步骤 2 配置自协商链接时间寄存器 SIXTEEN_BIT_CNTR [sixteen_bit_cntr]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x01CC，Bit[15:0]）为 16’h3F。

步骤 3 配置自协商链接时间更新寄存器 LD_LINK_COUNTER [ld_link_counter]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x01D0，Bit[1]）为 1。

步骤 4 配置发送控制寄存器 TRANSMIT_CONTROL [an_enable]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x0060，Bit[5]）为 1。

步骤 5 读取 SGMII 自协商状态寄存器 AN_NEG_STATE（基地址 0x?0000000+10000*ch_num，偏移地址为 0x0058）的值。当该寄存器的 Bit[21]为 1 时，表示自协商完成，此时 Bit[15:0]表示 PHY 的状态，具体含义参考寄存器含义。

----结束

SGMII 自协商 PHY 模式

使用该功能时，对接设备无条件服从本端配置速率和双工模式：

步骤 6 配置 SGMII 自协商本地配置寄存器 TX_LOCAL_PAGE（基地址 0x?0000000+10000*ch_num，偏移地址为 0x005C）为需要设置的双工模式和速率（bit0 必须配置为 1）。

步骤 7 配置自协商链接时间寄存器 SIXTEEN_BIT_CNTR [sixteen_bit_cntr]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x01CC，Bit[15:0]）为 16’h3F。

步骤 8 配置自协商链接时间更新寄存器 LD_LINK_COUNTER [ld_link_counter]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x01D0，Bit[1]）为 1。

步骤 9 配置发送控制寄存器 TRANSMIT_CONTROL [an_enable]（基地址 0x?0000000+10000*ch_num，偏移地址为 0x0060，Bit[5]）为 1。

步骤 10 读取 SGMII 自协商状态寄存器 AN_NEG_STATE（基地址 0x?0000000+10000*ch_num，偏移地址为 0x0058）的值。当该寄存器的 Bit[21]为 1 时，表示自协商完成。

----结束

SGMII 自协商 MAC 模式

使用该功能时，无须 MDIO 就能获取 PHY 的工作状态，软件可以读取自协商结果再配置 GMAC 的端口模式(基地址 0x?0000000+10000*ch_num，偏移地址为 0x0040，Bit[3:0])，使得 GMAC 和 PHY 可以正常通信。具体配置如下：

步骤 11 配置 SGMII 自协商本地配置寄存器 TX_LOCAL_PAGE（基地址 0x30000000+1000*ch_num，偏移地址为 0x005C）为 32’h1。

步骤 12 配置自协商链接时间寄存器 SIXTEEN_BIT_CNTR [sixteen_bit_cntr]（基地址 0x30000000+1000*ch_num，偏移地址为 0x01CC，Bit[15:0]）为 16’h3F。

步骤 13 配置自协商链接时间更新寄存器 LD_LINK_COUNTER [ld_link_counter]（基地址 0x30000000+1000*ch_num，偏移地址为 0x01D0，Bit[1]）为 1。

步骤 14 配置发送控制寄存器 TRANSMIT_CONTROL [an_enable]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0060，Bit[5]）为 1。

步骤 15 读取 SGMII 自协商状态寄存器 AN_NEG_STATE（基地址 0x30000000+1000*ch_num，偏移地址为 0x0058）的值。当该寄存器的 Bit[21]为 1 时，表示自协商完成，此时 Bit[15]表示 PHY 的线路侧链接状态，Bit[12]表示 PHY 的双工模式，Bit[11：10]表示 PHY 的工作速率。

----结束

2.10.5.5 短帧过滤

MAC CORE 的接收通道可以通过配置相应的寄存器选择是否透传短帧。如果该功能被使能，接收到的短帧不会被丢弃，并且被统计进接收有效帧字节统计寄存器。

警告

以下必须配置为短帧透传禁止。

配置短帧透传使能寄存器 RE_CV_CONTROL[runt_pkt_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x01E0，Bit[4]）为 0，短帧透传禁止。

注意

- 当同时使能短帧过滤和 PAD 剥离功能时，对于接收到的 802.3 封装短包（小于 64 字节）将无法过滤。
- 对于接收到的 802.3 封装短包（小于 64 字节，且 CRC 等均正确），MAC 不会因为其报文长度小于 64 字节而上报 2 层解析错误。

2.10.5.6 流控帧过滤

MAC CORE 的接收通道可以通过配置相应的寄存器对流控帧进行过滤。首先对接收到的流控帧 DA 域进行检查，如果该流控帧的 DA 域与 01-80-C2-00-00-01 完全匹配，或者与流控帧目的地址寄存器值完全匹配，则该流控帧 MAC 地址匹配。继续判断该流控帧的长度/类型域是否为 88-08-00-01，如果匹配在流控帧过滤使能的情况下则丢弃该流控帧，并进行接收方向过滤掉的帧统计和接收方向过滤掉的字节统计。流控帧虽然被丢弃但流控时间被存储，发送方向将按照流控时间停止发送报文流。

注意

接收流控帧过滤使能时，当接收到流控帧该帧将被丢弃并统计进接收方向丢弃的帧和字节统计寄存器。全双工模式下相应流控帧使能，发送方向才能按照流控时间停止发送报文，并在 CRC 检测有效且 CRC 正确、帧长大于等于 64byte 的情况下被统计进接收到的流控帧统计寄存器。

流控帧过滤配置如下：

步骤 1 配置流控帧目的地址低 32 位寄存器 FD_FC_ADDR_LOW（基地址 0x30000000+1000*ch_num，偏移地址为 0x0020，Bit[31:0]），配置控帧目的地址高 16 位寄存器 FD_FC_ADDR_HIGH（基地址 0x30000000+1000*ch_num，偏移地址为 0x0024，Bit[15:0]），配置流控帧目的地址。

步骤 2 配置接收方向相应流帧使能寄存器 PAUSE_EN[rx_fdfc]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0048，Bit[0]）为 1，全双工模式下响应流控帧使能。

步骤 3 配置流控帧过滤使能寄存器 REC_FILT_CONTROL[pause_frm_pass]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0064，Bit[4]）为 0，流控帧过滤使能。

----结束

2.10.5.7 广播帧过滤

MAC CORE 的接收通道可以通过配置相应的寄存器对接收到的广播帧进行过滤，当该功能有效时检查接收帧的 DA 域是否全 1，匹配则该帧将被丢弃，并进行接收方向过滤掉的帧统计和接收方向过滤掉的字节统计。

配置广播帧过滤使能寄存器 REC_FILT_CONTROL[bc_drop_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0064，Bit[2]）为 1，广播帧过滤使能。

2.10.5.8 多播帧过滤

MAC CORE 的接收通道可以通过配置相应的寄存器对接收到的多播帧进行过滤，当该功能有效时检查接收帧的 DA 域与 2 个多播帧地址匹配寄存器的配置值是否相等，检查时会根据地址屏蔽寄存器配置值将相应地址 bit 位屏蔽掉（即屏蔽掉的地址 bit 不进行比较），如果不匹配，则该帧将被丢弃，并进行接收方向过滤掉的帧统计和接收方向过滤掉的字节统计。

注意

检查多播帧 DA 域才有使用带掩码 MAC 地址的需求，因此带掩码的 MAC 地址用于多播帧 MAC 地址匹配。

多播帧过滤配置如下：

步骤 1 配置本机 MAC 地址 0 寄存器 1STATION_ADDR_LOW_0（基地址 0x30000000+1000*ch_num，偏移地址为 0x0200，Bit[31:0]），配置本机 MAC 地址 0 寄存器 2STATION_ADDR_HIGH_0（基地址 0x30000000+1000*ch_num，偏移地址为 0x0204，Bit[15:0]），第 8 位为 1，其它位根据需求可选；配置本机 MAC 地址 1 寄存器 2STATION_ADDR_LOW_1（基地址 0x30000000+1000*ch_num，偏移地址为 0x0208，Bit[31:0]），配置本机 MAC 地址 1 寄存器 2STATION_ADDR_HIGH_1（基地址 0x30000000+1000*ch_num，偏移地址为 0x020C，Bit[15:0]），第 8 位为 1，其它位根据需求可选。

步骤 2 配置本机 MAC 地址 0 寄存器 1 的掩码 STATION_ADDR_MSK_LOW_0（基地址 0x30000000+1000*ch_num，偏移地址为 0x0230，Bit[31:0]）；配置本机 MAC 地址 0 寄存器 2 的掩码 STATION_ADDR_MSK_HIGH_0（基地址 0x30000000+1000*ch_num，偏移地址为 0x0234，Bit[15:0]）；这两个寄存器与步骤 1 中所配的前两个寄存器按位与的结果构成一个带掩码的 MAC 地址。配置本机 MAC 地址 1 寄存器 1 的掩码 STATION_ADDR_MSK_LOW_1（基地址 0x30000000+1000*ch_num，偏移地址为 0x0238，Bit[31:0]）；配置本机 MAC 地址 1 寄存器 2 的掩码 STATION_ADDR_MSK_HIGH_1（基地址 0x30000000+1000*ch_num，偏移地址为 0x023C，Bit[15:0]），第 8 位为 1，其它位根据需求可选；这两个寄存器与步骤 1 中所配的后两个寄存器按位与的结果构成一个带掩码的 MAC 地址。

步骤 3 配置多播帧过滤使能寄存器 REC_FILT_CONTROL[mc_drop_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0064，Bit[1]）为 1，多播帧过滤使能。

----结束

2.10.5.9 单播帧过滤

MAC CORE 的接收通道可以通过配置相应的寄存器对接收到的单播帧进行过滤，当该功能有效时检查接收帧的 DA 域是否与 36 个 MAC 源地址中至少一个完全匹配，否则该帧将被丢弃，并进行接收方向过滤掉的帧统计和接收方向过滤掉的字节统计。

单播帧过滤配置如下：

步骤 1 配置本机 MAC 地址 2 寄存器 1STATION_ADDR_LOW_2（基地址 0x30000000+1000*ch_num，偏移地址为 0x0210，Bit[31:0]），配置本机 MAC 地址 2 寄存器 2STATION_ADDR_HIGH_2（基地址 0x30000000+1000*ch_num，偏移地址为 0x0214，Bit[15:0]），第 8 位为 0，其它位根据需求可选；这两个寄存器的组合值为一个本机 MAC 源地址。配置本机 MAC 地址 3 寄存器 2STATION_ADDR_LOW_3（基地址 0x30000000+1000*ch_num，偏移地址为 0x0218，Bit[31:0]），配置本机 MAC 地址 3 寄存器 2STATION_ADDR_HIGH_3（基地址 0x30000000+1000*ch_num，偏移地址为 0x021C，Bit[15:0]），第 8 位为 0，其它位根据需求可选；这两个寄存器的组合值为一个本机 MAC 源地址。配置本机 MAC 地址 4 寄存器 1STATION_ADDR_LOW_4（基地址 0x30000000+1000*ch_num，偏移地址为 0x0220，Bit[31:0]），配置本机 MAC 地址 4 寄存器 2STATION_ADDR_HIGH_4（基地址 0x30000000+1000*ch_num，偏移地址为 0x0224，Bit[15:0]），第 8 位为 0，其它位根据需求可选；这两个寄存器的组合值为一个本机 MAC 源地址。配置本机 MAC 地址 5 寄存器 2STATION_ADDR_LOW_5（基地址 0x30000000+1000*ch_num，偏移地址为 0x0228，Bit[31:0]），配置本机 MAC 地址 5 寄存器 2STATION_ADDR_HIGH_5（基地址 0x30000000+1000*ch_num，偏移地址为 0x022C，Bit[15:0]），第 8 位为 0，其它位根据需求可选；这两个寄存器的组合值为一个本机 MAC 源地址。

步骤 2 配置单播帧过滤使能寄存器 REC_FILT_CONTROL[uc_drop_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x0064，Bit[0]）为 1，单播帧过滤使能。

----结束

2.10.5.10 PAD 剥离

802.3 帧在封装数据时，封装完的报文不足 64 字节会填充 PAD，Hip04 支持这种 PAD 的剥离。

注意

VLAN 封装的帧及以太网帧无法剥离 PAD；

剥离 PAD 时连 CRC 一起剥离，无法只剥离 PAD 不剥离 CRC

步骤 1 配置接收控制寄存器 RECV_CONTROL [strip_pad_en]（基地址 0x30000000+1000*ch_num，偏移地址为 0x01E0，Bit[3]）为 1

----结束

2.10.5.11 发送截断功能

发送的数据的最大长度可以通过寄存器配置，如果发送的数据长度大于配置值，发送超过该长度报文则截断数据，并置为 CRC 错：

- 配置最大帧长寄存器 MAX_FRM_SIZE(基地址 0x30000000+ 1000* ch_num，偏移地址为 0x003C，Bit [15:0])值为 0x05EE，设置发送数据帧的最大长度为 0x05EE 个字节。

表2-10 发送方向截断报文及统计计数相关说明（1000M，MAX_FRM_SIZE 配置值为 N）

报文类型	CPU 发送的报文字节长度 M(无 CRC)	MAC 发送的报文字节长度（有 CRC）	OCTETS_ TRANSMITTED _BAD 统计值
VLAN 帧	N+4 < M < N+10	N+9	M+4
	N+10 =< M		N+14
非 VLAN 帧	N<M< N+6	N+5	M+4
	N+6 =< M		N+10

注：OCTETS_TRANSMITTED_BAD(基地址 0x30000000+ 1000* ch_num，偏移地址为 0x0104，Bit [14:0])

表2-11 发送方向截断报文及统计计数相关说明（10/100M，MAX_FRM_SIZE 配置值为 N）

报文类型	CPU 发送的报文字节长度 M(无 CRC)	MAC 发送的报文字节长度（有 CRC）	OCTETS_TRANSMITTED _BAD 统计值
VLAN 帧	M＝N+5	N+9	N+9
	M＝N+6 或 N+7		N+10
	M＝N+8 或 N+9		N+11
	M＝N+10 或 N+11		N+12
	M＝N+12 或 N+13		N+13
	N+13 < M		N+14
非 VLAN 帧	M＝N+1	N+5	N+5
	M＝N+2 或 N+3		N+6
	M＝N+4 或 N+5		N+7
	M＝N+6 或 N+7		N+8
	M＝N+8 或 N+9		N+9
	N+9 < M		N+10

注：OCTETS_TRANSMITTED_BAD(基地址 0x30000000+ 1000* ch_num，偏移地址为 0x0104，Bit [14:0])

2.11 USB 2.0 Host

2.11.1 概述

USB 2.0 Host 模块集成 USB2.0 Host 控制器和 USB2.0 PHY。USB2.0 Host 在 USB 系统中处于中心地位并且对 USB 系统及其连接的 USB Device 有着特殊的责任,USB2.0 Host 控制着所有的 USB 通信，USB2.0 Device 只有在 USB2.0 Host 的允许才有权力进行 USB 通信，USB2.0 Host 同时也监测整个 USB 通信系统的状态。

2.11.2 特性

USB2.0 Host 具有以下特性：

- 完全兼容 USB 2.0；
- 完全符合 OHCI Rev 1.0a、EHCI Rev 1.0；
- 可以支持 High-speed、Full-speed、Low-speed 三种设备；
- 支持低功耗的解决方案；
- 支持 Control Transfer、Bulk Transfer、Isochronous Transfer、Interrupt Transfer 四种基本数据传输类型；
- 支持 2 个 USB 端口；
- 可以通过连接 USB Hub，连接最多 127 个设备；
- 支持 USB2.0 规定的测试包模式；
- USB2.0 Host PHY 支持 BIST 和 环回测试模式；
- 支持异常中断上报功能；
- 支持内部总线错误探测，并将其存在相应寄存器中；
- 支持可配置的 word 内的 BE-8 和小端模式。

2.11.3 接口信号

USB2.0 Host 接口信号如表 2-12 所示。

表2-12 USB2.0 Host 接口信号描述

管脚	内部信号名称	方向	描述
USB_HOST_DP	DP	I/O	USB 端口的 D+数据线
USB_HOST_DM	DM	I/O	USB 端口的 D–数据线
USB_HOST_REXT	REXT	I/O	外接电阻接口
USB_HOST_OVRCUR	USB_OVRCUR	I	USB 端口的端口过流指示信号
USB_HOST_PWREN	USB_PWREN	O	USB 端口电源控制输出管脚

管脚	内部信号名称	方向	描述
USB_HOST_VDD33	VDD33	I/O	3.3V 模拟电源，电压范围为 3.3✐7%V。
USB_HOST_VSS33	VSS33	I/O	模拟地。

2.11.4 功能描述

2.11.4.1 应用框图

USB 2.0 Host 典型应用框如图 2-17 所示。U 盘通过 USB 线连接到 USB2.0 Host 端口，可以用于版本升级。

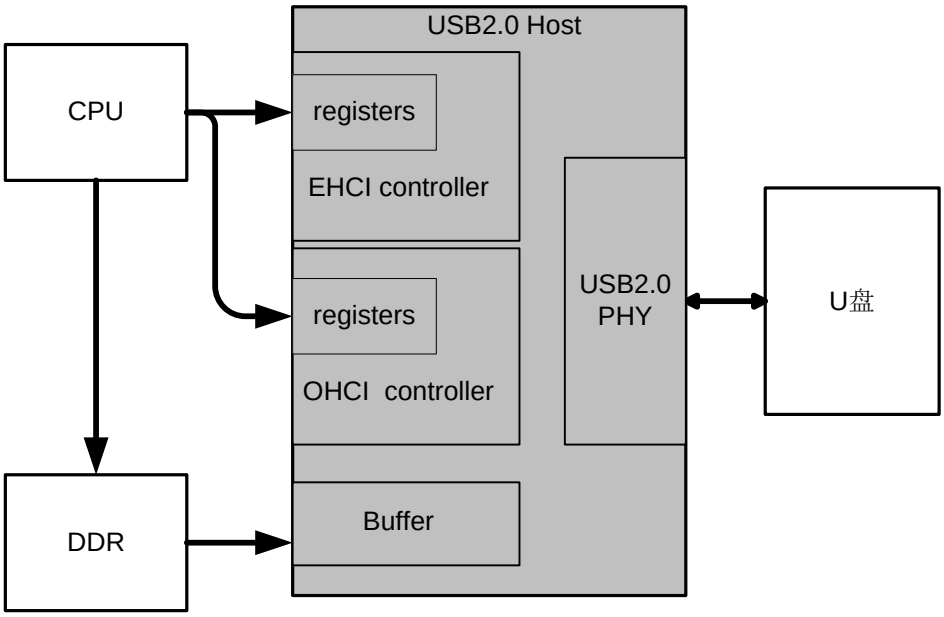


图2-17 USB 2.0 HOST 典型应用框图

USB2.0 Host 内部有 EHCI Controller 和 OHCI Controller，分别完成对 EHCI 协议和 OHCI 协议的支持。当对接设备是 Full-speed 或者 Low-speed 设备时，使用 OHCI controller；如果对接的设备是 High-speed 设备时，则使用 EHCI controller。

当 U 盘插到 USB 口后，USB2.0 Host 先与 U 盘进行速度协商握手，如果 U 盘支持 High-speed，则内部路由到 EHCI Controller；如果 U 盘支持 Full-speed 或者 Low-speed，则内部路由到 OHCI Controller。然后 USB2.0 Host 向 CPU 发出设备 Connect 中断，然后 CPU 在 DDR 构建相应的描述符链表，并将基地址等信息写入到 USB2.0 Host 的寄存器中。USB2.0 Host 将 DDR 中的描述符链表取到内部 Buffer 中，然后对其进行解析，如果描述符表明是需要向 U 盘发数据，则需要向 DDR 中取需要发送的数据，再对 U 盘发起

OUT 传输；如果描述符表明是需要从 U 盘中获取数据，则 USB2.0 Host 先向 U 盘发起 IN 操作，然后将回读到数据存入 Buffer，再写到描述符内指针指向的 DDR 空间内。

2.11.4.2 相关模块

在系统中，USB2.0 Host 模块会和下面多个模块进行交互，分别描述如下：

- DDR：一方面，USB2.0 Host 的链表描述符以及要发送的数据由 CPU 储存在 DDR 中，USB2.0 Host 向 DDR 中读取链表描述符后进行解析，从而进行相应的 USB 传输；另一方面，USB2.0 Host 接收到的数据，也会存储在 DDR 中；
- CPU：一方面，CPU 可以对 USB2.0 Host 进行配置，并监视其状态；另一方面，CPU 模块负责构建链表，并将链表储存到 DDR 中。

2.11.4.3 USB2.0 Host 支持的传输功能

USB 2.0 Host 支持以下 4 种标准的传输方式：

2.11.4.3.1 Control Transfer（控制传输）

主要用于 USB Host 与 USB Device 端点 0 之间的传输，某些特定型号的 USB 设备的控制传输可能用到其他的端点。控制传输是双向传输，数据量通常较小，可以传输 8Bytes、16Bytes、32Bytes 或 64Bytes 的数据，依赖于设备和传输速度。

控制传输用于支持客户程序与功能设备之间配置/命令/状态等类型的通信。

USB 设备在枚举过程中使用的就是 Control TransferTransfer，所以任何 USB 设备都需要支持 Control Transfer.

2.11.4.3.2 Bulk Transfer（批量传输）

主要用于没有带宽和间隔时间要求的情况下发送和接收大量的数据，这种类型的设备适合于传输非常慢和大量被延迟的传输，可以等到所有其他类型的数据的传送完成之后再传送和接收数据。它的特点是以错误检测和重传的方式保证 USB Host 与 USB Device 的数据被无差错地发送。

Bulk Transfer 主要用于非周期性的大型突发通信，通常用于可以利用任何可用带宽或者可以延时直到宽带可用的传输。

U 盘、USB 网卡等需要发送和接收大量数据的 USB 设备，在数据传输时都是使用 Bulk Transfer.

2.11.4.3.3 Isochronous Transfer（同步传输）

主要用于时间严格并具有较强容错性的流数据传输，或者用于数据传输速率恒定的即时应用中。同步传输提供了确定的带宽和间隔时间。

Isochronous Transfer 主要用于周期性的连续通信。

USB 摄像头等设备就是使用 Isochronous Transfer.

2.11.4.3.4 Interrupt Transfer（中断传输）

主要用于少量、分散、不可预测的数据的传输。中断传输方式下，定时查询设备是否有中断数据要发送。设备的端点模式器的结构决定了它的查询频率为1ms～255ms。典型的中断方式传输是单向的，并且对于 USB Host 来说只有输入的方式。

USB 鼠标、USB 键盘、USB 游戏柄等设备使用 Interrupt Transfer。

2.11.5 工作方式

2.11.5.1 初始化

USB2.0 Host 复位后使用时初始化：

步骤 1 配置 PHY 中与 USB2.0 眼图质量相关的寄存器；

步骤 2 打开 USB2.0 Host 相关时钟；

步骤 3 加载 USB 相关驱动。

----结束

2.11.5.2 软复位

对接器件容许的情况下，可以任意时间对 USB2.0 Host 工作逻辑进行软复位，不会对系统造成影响。通过配置系统控制器 XXX 为 0bX，可实现对 USB2.0 Host 工作逻辑的单独软复位。

USB 控制器和 PHY 在上电后默认处于复位状态，撤销复位的步骤如下：

步骤 1 至少延时 10us；

步骤 2 USB PHY 端口软复位；

步骤 3 延时 250us 后，撤消 USB 控制器的 port 接口软复位；

步骤 4 撤消 USB 控制器的 root hub 软复位；

步骤 5 撤消 USB 总线侧软复位。

----结束

2.11.5.3 时钟门控

在不使用 USB 2.0 Host 时，可关断 USB 2.0 Host 的时钟，以降低功耗。

关断时钟的步骤如下：

步骤 1 对 USB 控制器和 PHY 进行复位；

步骤 2 关断 USB 2.0 Host 相关时钟。

----结束

打开时钟的步骤如下：

步骤 1 打开 USB 2.0 Host 相关时钟；

步骤 2 撤销 USB 控制器和 PHY 的复位。

----结束

2.11.5.4 挂起和重启

挂起 PHY0（即 SUSPEND）：软件通过 EHCI/OHCI 寄存器设置挂起模式后，PHY 就进入了挂起模式。

重启 PHY（即 RESUME）：软件通过 EHCI/OHCI 寄存器设置退出 SUSPEND 模式后，如果 phy_commononn 为 1，需要至少延时 225us 后，软件才能发起 USB 操作；如果 phy_commononn 为 0，需要至少延时 5us 后，软件才能发起 USB 操作。

说明

挂起 PHY1 的操作与挂起 PHY0 的操作类似，重启 PHY1 的操作与重启 PHY0 的操作类.

2.11.5.5 大小端转换

USB Host 的大小端转换使能时，大端数据 0x1122_3344 将转换为 0x4433_2211 的小端数据。

2.12 USB2.0 Device

2.12.1 概述

USB2.0 Device 模块包括 USB2.0 Device 控制器和 USB2.0 Device PHY，支持 High-speed 和 Full-speed2 种传输速度。最大支持 16 个 IN 和 16 个 OUT 端点，端点类型可配置。

2.12.2 特性

USB2.0 Device 具有以下特性：

- 完全兼容 USB 2.0 协议；
- 支持 High-speed 和 Full-speed；
- 支持 LPM 协议；
- 最大支持 16 个 IN 端点和 16 个 OUT 端点,端点类型可配；
- 端点 0 为 Control Transfer 端点，端点 1~端点 15 类型可配置，可配置为 Control Transfer、Bulk Transfer、Isochronous Transfer、Interrupt Transfer 4 种端点类型；
- 每个端点有独立的发送 FIFO，FIFO 大小动态可配；
- 支持 1 个 USB 端口；
- 内置 DMA，支持 Scatter/Gather；
- 支持 Suspend/Resume 功能；
- 支持传输异常中断上报；
- 支持 FIFO 异常中断上报，并且可以通过寄存器查询 FIFO 的状态；
- USB2.0 Device PHY 支持 BIST 和环回测试；
- 支持可配置的 word 内的 BE-8 和小端模式。

2.12.3 关键术语

如表 2-13 的术语将在本章节表示特殊的含义。

表2-13 USB2.0 Device 关键术语描述

术语	描述
LPM	Link Power Manage， 链路电源管理。定义了多种功耗状态，规定在不同工作模式下对应不同功耗状态之间切换。
IN	Input，输入。指由 USB2.0 Host 发起的，向 USB2.0 Device 中取数据的操作。
OUT	Ouput，输出。指由 USB2.0 Host 发起的，向 USB2.0 Device 中发送数据的操作。
Suspend	挂起。USB 的一种低功耗状态，当 USB 总线没有业务传输时会进入这种状态。

术语	描述
Resume	唤醒。当 USB 接收到唤醒信号时，会从 Suspend 状态进入 Resume 状态，从而撤离低功耗状态。
Scatter/Gather	聚合/分散。一个 USB 的 Transfer 内的不同 transcation 数据允许在不同的地方存储，采用链表的形式将这些分散的存储连在一起。
Full-speed	全速。USB 的传输速率是 12Mbit/s。
High-speed	高速。USB 的传输速率是 480Mbit/s。
Control Transfer	控制传输。详见 2.12.5.3 。
Isochronous Transfer	实时传输。详见 2.12.5.3 。
Interrupt Transfer	中断传输。详见 2.12.5.3 。
Bulk Transfer	批量传输。详见 2.12.5.3 。

2.12.4 接口信号

USB2.0 Device 接口信号如表 2-14 所示。

表2-14 USB2.0 Device 接口信号描述

管脚	内部信号名称	方向	描述
USB_DEVICE_XO	XO	I	晶体接口
USB_DEVICE_XI	XI	I	晶体接口
USB_DEVICE_ID	ID	I	连接器 ID
USB_DEVICE_VBUS	VBUS	I	连接器 VBUS
USB_DEVICE_DP	DP	I/O	D+数据线
USB_DEVICE_DM	DM	I/O	D-数据线
USB_DEVICE_RKELVIN	RKELVIN	I/O	外接电阻连接

2.12.5 功能描述

2.12.5.1 应用框图

USB 2.0 Device 典型应用框如图 2-18 所示。PC 机通过 USB 线连接到 USB2.0 Device 端口，可以 Debug 调试。

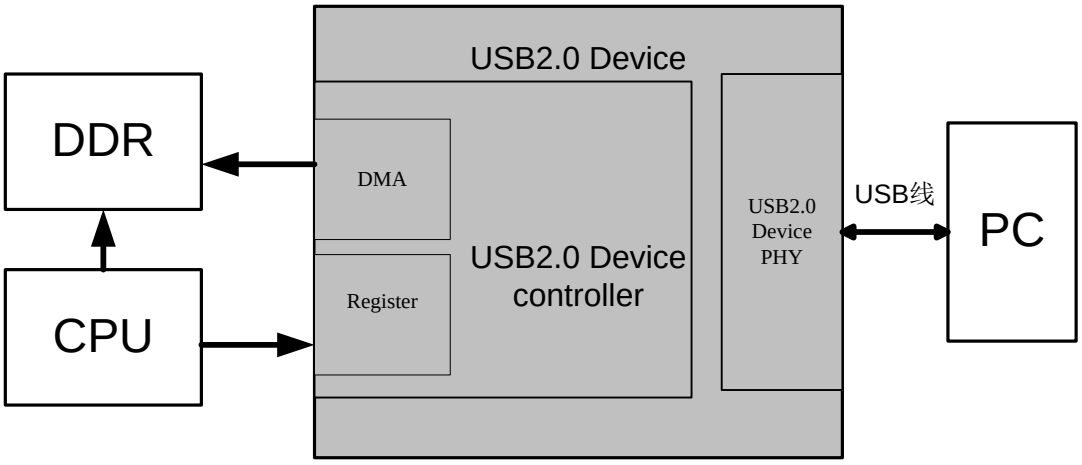


图2-18　USB2.0 Device 典型应用框图

当 USB2.0 Device 通过 USB 线连接到 PC 机上，此时 USB2.0 Device 向 CPU 发起 connect 中断，CPU 在 DDR 内构建相应的描述符，并且将描述符的入口地址写入到 USB2.0 Device 的寄存器中，然后 DMA 根据这个入口地址去 DDR 内取描述符，USB2.0 Device 解析描述符，如果是 PC 要取 USB2.0 Device 的数据，则 DMA 需要去 DDR 中取数据然后进行传输，传输完毕后更新 DDR 中的描述符状态；如果是 PC 向 USB2.0 Device 发送数据，则 DMA 需要将接收到的数据回写到 DDR 中，并且更新 DDR 中的描述符状态。

2.12.5.2 相关模块

在系统中，USB2.0 Device 模块会和下面多个模块进行交互，分别描述如下：

- DDR：USB2.0 Device 的内置 DMA 从 DDR 中读取描述符和数据，从而进行 USB 的相应传输，同时将接收到的数据存储到 DDR 中；
- CPU：CPU 用于配置 USB2.0 Device，同时 CPU 完成 USB 链表的构建，并将构建的链表存在 DDR 中；

2.12.5.3 USB2.0 Device 支持的传输功能

USB 2.0 Device 支持以下 4 种标准的传输方式：

2.12.5.3.1 Control Transfer

主要用于 USB Host 与 USB Device 端点 0 之间的传输，某些特定型号的 USB 设备的控制传输可能用到其他的端点。Control Transfer 是双向传输，数据量通常较小，可以传输 8Byte、16Byte、32Byte 或 64Byte 的数据，依赖于设备和传输速度。

Control Transfer 用于支持客户程序与功能设备之间配置/命令/状态等类型的通信。

USB 设备在枚举过程中使用的就是 Control　Transfer，所以任何 USB 设备都需要支持 control Transfer.

2.12.5.3.2 Bulk Transfer

主要用于没有带宽和间隔时间要求的情况下发送和接收大量的数据，这种类型的设备适合于传输非常慢和大量被延迟的传输，可以等到所有其他类型的数据的传送完成之后再传送和接收数据。它的特点是以错误检测和重传的方式保证 USB Host 与 USB Device 的数据被无差错地发送。

Bulk Transfer 主要用于非周期性的大型突发通信，通常用于可以利用任何可用带宽或者可以延时直到宽带可用的传输。

U 盘、USB 网卡等需要发送和接收大量数据的 USB 设备，在数据传输时都是使用 Bulk Transfer.

2.12.5.3.3 Isochronous Transfer（同步传输）

主要用于时间严格并具有较强容错性的流数据传输，或者用于数据传输速率恒定的即时应用中。同步传输提供了确定的带宽和间隔时间。

Isochronous Transfer 主要用于周期性的连续通信。

USB 摄像头等设备就是使用 Isochronous Transfer.

2.12.5.3.4 Interrupt Transfer（中断传输）

主要用于少量、分散、不可预测的数据的传输。中断传输方式下，定时查询设备是否有中断数据要发送。设备的端点模式器的结构决定了它的查询频率为 1ms～255ms。典型的中断方式传输是单向的，并且对于 USB Host 来说只有输入的方式。

Interrupt Transfer 用于处理服务需要。

USB 鼠标、USB 键盘、USB 游戏柄等设备使用 Interrupt Transfer。

2.12.6 工作方式

2.12.6.1 初始化

USB2.0 Device 复位后使用时初始化：

步骤 1 在 DCFG 寄存器中设置如下域：DescDMA、设备速度、用于 OUT 传输握手的非归零长度握手、周期针间隔；

步骤 2 设置设备门槛控制寄存器；

步骤 3 设置 GINTMSK 寄存器屏蔽如下寄存器：USB reset、Enumeration Done、Early Suspend、USB Suspend、SOF。

步骤 4 等待 GINTSTS.USBReset 中断，如果检测到上报这个中断，则等待 10ms，然后执行 USB 复位相关操作。

步骤 5 等待 GINTSTS.EnumerationDone 中断，如果检测到上报这个中断，则表明 USB 复位已经完成。

----结束

2.12.6.2 软复位

对接器件容许的情况下，可以任意时间对 USB2.0 Device 工作逻辑进行软复位，不会对系统造成影响。通过配置系统控制器 XXX 为 0bX，可实现对 USB2.0 Device 工作逻辑的单独软复位。

2.12.6.3 Suspend/Resume 处理流程

Suspend/Resume 处理流程可以分成时钟 gating 处理流程和掉电处理流程。

时钟 gating 处理流程：

步骤 1 控制器检测到 suspend 状态，上报 suspend 中断

步骤 2 软件设置寄存器 PCGCCTL 的 Stop PHY Clock 位为 1，控制器送给 PHY 的信号 suspend_n 置为 0，PHY 关闭 UTMI 接口时钟；软件设置寄存器 PCGCCTL 的 Gate hclk 位设置为 1，控制器关闭部分模块的总线时钟

步骤 3 控制器处于 suspend 状态

步骤 4 检测到 Resume 状态，控制器送给 PHY 的信号 suspend_n 置为 1，PHY 送出 UTMI 接口时钟，然后控制器产生 Resume 中断上报

步骤 5 软件清除寄存器 PCGCCTL 的 Stop PHY Clock 和 Gate hclk 位

步骤 6 Host 结束 resume 操作

步骤 7 控制器回到正常操作状态

----结束

2.12.6.4 掉电处理流程

步骤 1 控制器检测到 suspend 状态，上报 suspend 中断

步骤 2 软件设置寄存器 PCGCCTL 寄存器的 Power Clamp 位为 1，控制器将穿越不同电源域的信号箝制在固定电平

步骤 3 软件配置 PCGCCTL 寄存器的 Reset to Power-Down Module 位为 1

步骤 4 软件设置寄存器 PCGCCTL 的 Stop PHY Clock 位为 1，控制器送给 PHY 的信号 suspend_n 置为 0，PHY 关闭 UTMI 接口时钟

步骤 5 软件保存寄存器状态，关闭掉电域模块电源

步骤 6 控制器处于部分模块掉电状态

步骤 7 检测到 Resume 状态，控制器送给 PHY 的信号 suspend_n 置为 1，PHY 送出 UTMI 接口时钟，然后控制器产生 Resume 中断上报

步骤 8 软件清除寄存器 PCGCCTL 的 Stop PHY Clock 位

步骤 9 打开掉电域模块电源，等待电源稳定

步骤 10 软件清除寄存器 PCGCCTL 的 Power Clamp 位

步骤 11 软件清除寄存器 PCGCCTL 寄存器的 Reset to Power-Down Module 位

步骤 12 软件恢复寄存器状态，设置寄存器 DCTL 的 Power-on Programming Done 位为 1

步骤 13 控制器回到正常操作状态

----结束

2.12.6.5 大小端转换

USB Host 的大小端转换使能时，大端数据 0x1122_3344 将转换为 0x4433_2211 的小端数据。

2.13 MMC/eMMC/SD/SDIO 存储控制接口（MMC）

2.13.1 概述

MMC/SD/SDIO 控制器（以下简称 MMC）用于处理对 SD/MMC 卡的读写等操作，并通过 SDIO 协议实现对扩展外设（如蓝牙、WiFi 等）的支持。

MMC 支持符合以下协议的设备：

- Secure Digital Memory（SD mem-version 2.00）
- Secure Digital I/O（SDIO-version 2.0）
- MultiMediaCard（MMC-version 4.3）

2.13.2 特性

MMC 具有以下特性：

支持 MMC 卡流数据读写方式；

支持 eMMC 接口；

支持 SDIO 的 Suspend 操作、Resume 操作和 Read Wait 操作；

接口数据位宽支持 1、4、8 bits，可根据对接器件选择；

支持 1 Byte～65535 Byte 的块数据读写操作；

支持 DMA 数据传输方式；

- 包含数据发送方向与接收方向的 2 个 FIFO，FIFO 深度为 256 Words；
- 支持 FIFO 阈值可配，DMA 传输时传输长度可配；
- 支持 FIFO 上溢出与下溢出中断告警，防止数据传输错误；

支持命令与数据的 CRC 生成与校验；

接口时钟频率可编程；

支持低功耗模式关闭 MMC 时钟和接口时钟。

2.13.3 关键术语

如表 2-15 的术语将在本章节表示特殊的含义。

表2-15 MMC 关键术语描述

术语	描述
MMC	MultiMediaCard，西门子和 SanDisk 推出了多媒体卡，其外形比 CF 卡小，从而可实现更小巧的便携式设备。

2.13.4 接口信号

MMC 接口信号如表 2-16 所示。

表2-16 MMC 接口信号描述

管脚	内部信号名称	方向	描述
SDIO_CCMD	SDIO_CCMD	O	SDIO 命令信号。
SDIO_CDATA7~0	SDIO_CDATA7~0	I/O	SDIO 数据信号。
SDIO_CARD_PWR_EN	SDIO_CARD_PWR_EN	O	SDIO 上电使能。
SDIO_CARD_DETECT	SDIO_CARD_DETECT	I	SDIO 卡检测。
SDIO_CWPR	SDIO_CWPR	I	SDIO 写保护。
SDIO_CCLK_OUT	SDIO_CCLK_OUT	O	SDIO 输出时钟。
SDIO_CCMD_ODPULLUP_EN_N	SDIO_CCMD_ODPULLUP_EN_N	O	SDIO 输出上拉使能。

2.13.5 功能描述

2.13.5.1 应用框图

MMC 的应用如图 2-19 所示。

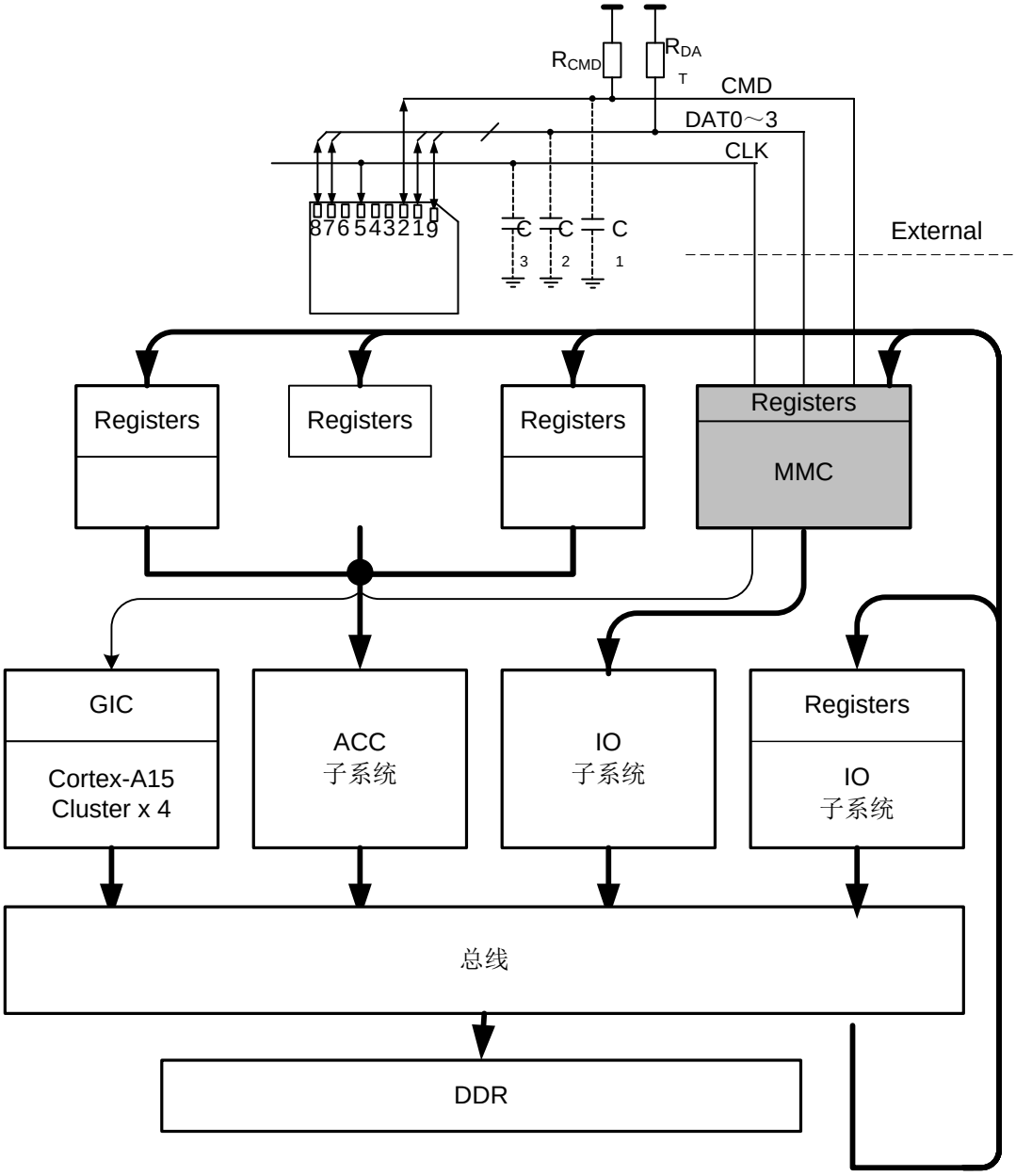


图2-19 MMC 应用框图

2.13.5.2 相关模块

在系统中，MMC 模块会和下面多个模块进行交互，分别描述如下：

- CPU：CPU 用于配置和查询 MMC 的寄存器，并配置 MMC 操作指令，MMC 与卡设备之间的所有交互操作均通过指令完成，包括卡初始化、寄存器读写、状态查询、数据传输等，此外，MMC 如果出现传输错误或超时，通过中断信号上报 CPU 处理；

- DDR：CPU 配置 MMC 搬移数据，MMC 可以主动将数据从 SD 卡搬移到 DDR 或从 DDR 搬移到 SD 卡。

2.13.5.3 数据传输

单块读写指令和多块读写指令为较常用的数据传输方式。通常 SD/MMC 卡数据传输的一个块大小为 512 Bytes，而 SDIO 设备可根据应用自定义。

说明

以块读写指令方式进行数据传输时，传输数据总量必须为块大小的整数倍。

数据传输指令均为短响应指令，并伴随着数据线上的数据传输。指令、响应及数据线上的时序配合关系如图 2-20 和图 2-21 所示。

（1）单块与多块读操作

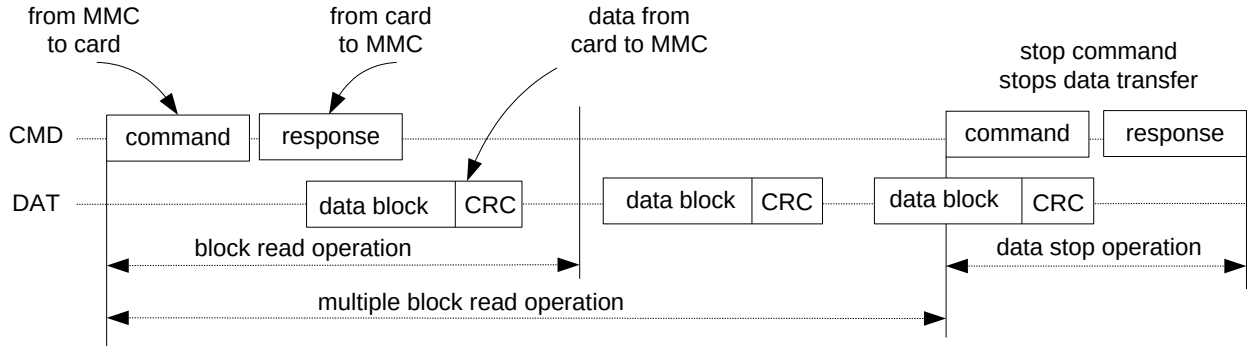


图2-20 单块与多块读操作

MMC 向卡发送单块或多块读指令。在接收响应的过程中，接收以块为单位的数据，其中每块数据中均包含有 CRC 校验位，以保证数据传输的完整性。

单块读指令操作时，MMC 在接收一块数据后完成一次数据传输；多块读指令操作时，MMC 在接收多块数据后，需发送一条停止指令结束本次数据传输（仅 open ended 多块读指令）。

（2）单块与多块写操作

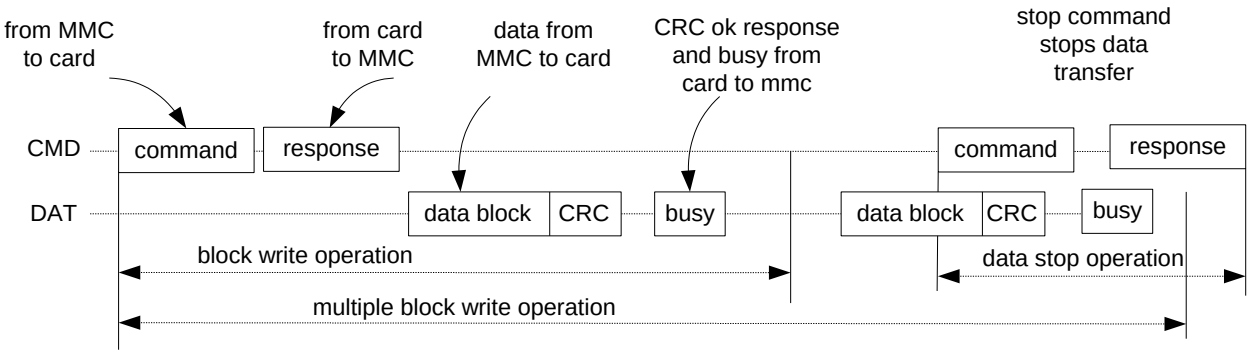


图2-21 单块与多块写操作

MMC 往卡发送单块或多块写指令。在接收到响应后，往卡发送以块为单位的数据，其中每块数据中均包含有 CRC 校验位，卡会对每块数据进行 CRC 校验，并反馈 CRC 状态以确认数据传输的正确性。

单块写指令操作时，MMC 在发送一块数据后完成一次数据传输；多块写指令操作时，MMC 在发送多块数据后，需发送一条停止指令完成本次数据传输（仅 Open Ended 多块读指令）。写操作结束后，卡可能会因为编程 Flash 而处于繁忙状态，MMC 需查询 DAT0 状态，以确认卡脱离繁忙状态后才能对卡进行下一步操作。

（3）数据传输格式

块方式读写中，MMC 与卡之间可采用 1bit 或 4bits 数据线方式进行数据传输。在发送数据传输指令之前，应分别设置 MMC 与卡的数据传输位宽模式（1bit 或 4bits），使它们保持一致。MMC 的数据位宽通过寄存器设置，卡的数据位宽则通过发送相应的指令进行设置。

1bit 和 4bit 模式下的数据传输格式如图 2-22 和图 2-23 所示。

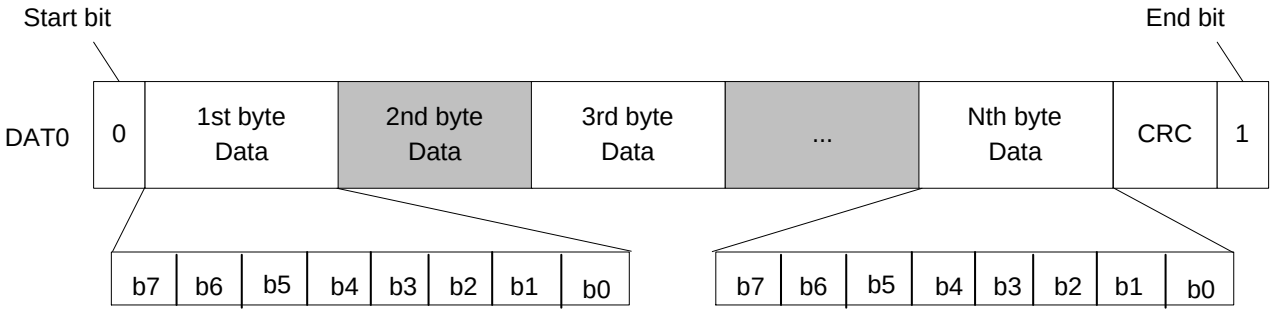


图2-22 1 bit 数据线传输模式下的块数据格式

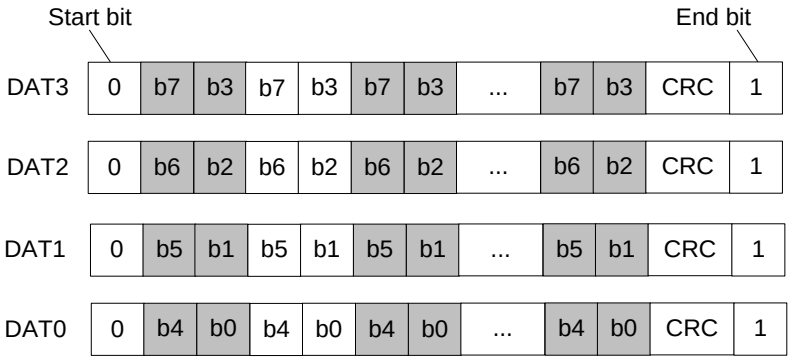


图2-23 4 bits 数据线传输模式下的块数据格式

2.14 Local Bus

2.14.1 概述

Local Bus 可对外提供异步静态存储器接口，可以连接 SRAM（Static Random Access Memory）、ROM（Read Only Memory）、NOR FLASH 等异步静态存储器，用于实现系统启动、数据存储等功能。同时也可以用于连接带异步存储器接口（包括 Intel 时序和 Moto 时序）的控制芯片。

2.15 NAND 存储控制接口（NANDC）

2.15.1 概述

NANDC（NAND FLASH Controller）提供存储控制器接口连接片外 NAND FLASH，从而完成数据的存取。

2.15.2 特性

NANDC 具有以下特性：

- 支持 4 个片选信号和 4 个 Ready/Busy 信号，同时也支持 4 个 NAND FLASH 器件共用 1 个 Ready/Busy 信号；
- 支持异步接口模式 8/16bit Data-Bus NAND FLASH 接口 4
- 支持异步 NAND Flash 接口；
- 支持 NAND Boot 功能，支持 2KB、4KB、8KB 和 16KB Page Szie 的 NAND FLASH 器件，支持从片选 0 对应的 NAND FLASH 启动；
- 支持 ECC 校验使能和关闭，支持纠错使能和关闭；
  - 支持 1bit（Hanming）校验和纠错；
  - 支持 8bit（BCH）/1024byte 校验和纠错；
  - 支持 16bit（BCH）/1024byte 的校验和纠错；
  - 支持 24bit（BCH）/1024byte 的校验纠错；
  - 支持 40bit（BCH）/1024byte 的校验纠错；
- 支持内置 DMA 方式读写 NAND FLASH；
- Boot 模式下支持各个厂家的默认坏块标识检测；
- 支持读写、擦除、编程完成；ECC 校验错误等中断上报；
- 支持不定长数据读写；
- 可以灵活配置控制器发出的命令，从而可以支持各种 NAND FLASH 命令操作（包括 Cache 读写等）；
- 支持读写 NAND FLASH 过程可以被 EBI 打断，以支持存储器共享总线出让；
- 支持交替操作 2 个 NAND FLASH 器件，并行工作以提高效率；
- 支持对 NAND FLASH 写保护功能，写保护地址空间可配置；
- 支持 Lock 和 Lock-down 两种模式。支持 FLASH Lock 使能/去使能、FLASH 整体锁定使能/去使能，默认为 FLASH Lock 使能，FLASH 整体锁定使能。凡是对锁定地址范围内的写操作，NANDC 均上报操作错误中断。

2.15.3 关键术语

如表 2-17 的术语将在本章节表示特殊的含义。

表2-17 NANDC 关键术语描述

术语	描述
NANDC	NAND Controller，NAND FLASH 控制器
ECC	Error Correction Code，纠错码
EBI	External Bus Interface，外部总线接口

2.15.4 接口信号

NANDC 接口信号如表 2-18 所示。

表2-18 NANDC 接口信号描述

管脚	内部信号名称	方向	描述
	NFC_BUS_WIDE	I	从 NAND FLASH 启动时，NAND FLASH 的数据位宽设置。 0：8bit； 1：16bit。
	NFC_ADDR_NUM	I	Boot 模式下，NANDC 发送给 NAND FLASH 的地址周期数. 0：4 个地址周期； 1：5 个地址周期。
	NFC_PAGE_SIZE[1:0]	I	Boot 模式下，外接 FLASH 的 Page size[1:0]： 00：512B； 01：2KB； 10：4KB； 11：8KB； 100：16KB； 101~111：保留。
	NFC_ECC_TYPE[2:0]	I	Boot 时，选择 ECC 模式。 000：不使能； 001：1bit 模式； 010：8bit 模式； 100：24bit 模式； 101：40bit 模式； 011：16bit 模式。 111：保留。

管脚	内部信号名称	方向	描述
	NFC_BLOCK_SIZE [1:0]	I	Boot 时，NAND FLASH 器件的 Block 大小。 00：64 个 page； 01：128 个 page； 10：256 个 page； 11：512 个 page。
	NF_RB0	I	NAND FLASH 0 的 Ready_busy 信号。
	NF_CE0	O	NAND FLASH 0 的片选信号，低电平有效。
	NF_RB1	I	外接 2 个 NAND FLASH 时，该管脚是 NAND FLASH 1 的 Ready_busy 信号。
	NF_CE1	O	外接 2 个 NAND FLASH 时，该管脚是 NAND FLASH 1 的片选信号，低电平有效。
	NF_RB2	I	NAND FLASH 2 的 Ready_busy 信号。
	NF_CE2	O	NAND FLASH 2 的片选信号，低电平有效。
	NF_RB3	I	NAND FLASH 3 的 Ready_busy 信号。
	NF_CE3	O	NAND FLASH 3 的片选信号，低电平有效。
	NF_CLE	O	命令锁存信号。
	NF_ALE	O	地址锁存信号。
	NF_WR_N	O	写使能信号，低电平有效。
	NF_RD_N	O	读使能信号，低电平有效。
	NF_DATA[15:0]	I/O	NAND FLASH 数据地址信号。

2.15.5 功能描述

2.15.5.1 应用框图

NANDC 的应用如图 2-24 所示，NANDC 通过管脚使能，可实现系统 NAND FLASH 启动功能。

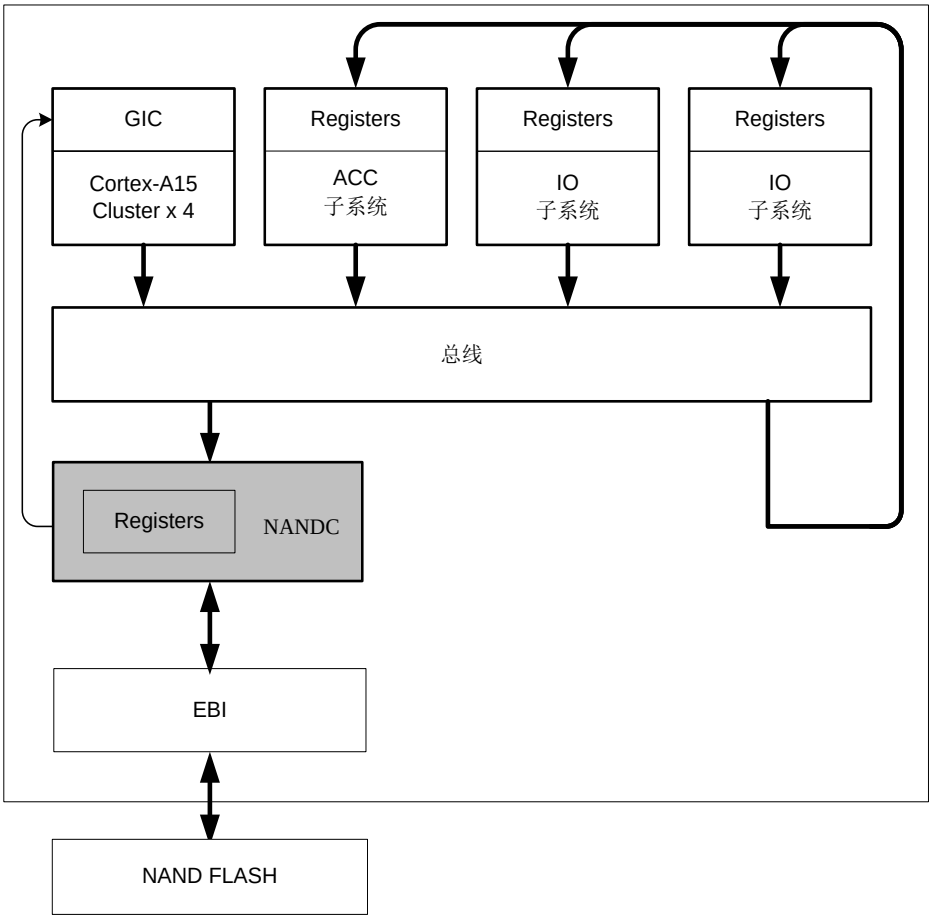


图2-24 NANDC 应用图

NAND FLASH 器件虽然没有一个标准的总线协议，但接口信号基本一致，异步接口需提供片选、读写有效、指令有效、地址有效、Ready/Busy 和数据信号，而同步接口则需要多一个 DQS 信号。

CPU 可以通过总线配置 NANDC 的寄存器配置数据传输到 NAND FLASH 器件，或者读取寄存器获取接收到的 NAND FLASH 器件的数据；由于 NANDC 内置 DMA，为了提高读写效率，CPU 还可以通过总线配置 NANDC 内置的 DMA 自动完成一段数据的发送和接收。

NANDC 在传输完成或传输发生错误时，可以通过中断通知 CPU 进行处理。

2.15.5.2 相关模块

在系统中，NANDC 模块会和下面多个模块进行交互，分别描述如下：

- CPU　　：CPU 通过总线访问 NANDC 的寄存器接口，通过配置操作寄存器、输出模式寄存器以及命令寄存器来发起对 NAND FLASH 器件的读或写。
- DDR　　：因为 NANDC 本身内置 DMA，为了提高读写效率，一般都使用 DMA 进行数据的操作，一次完整的 DMA 操作需读写完成一个 Page 的大小，所以 DMA 会通过总线将数据一次或几次写入 DDR 或者从 DDR 中读出；
- GIC　　：NANDC 在 DMA 传输完成时，或者对 Lock 地址进行写操作，或者 ECC 出错，或者片选信号对应的 Ready/Busy 由低变高等，均会产生中断信号给 GIC 模块，通过 GIC 将中断发送给对 NANDC 具有控制权的 CPU。
- EBI　　：为减少管脚的数量，可以通过 EBI 来实现 NANDC 与其他存储器控制器数据线的复用，例如 NOR FLASH。NANDC 通过 EBI_REQ 信号向 EBI 发出总线占用请求，EBI 则通过 EBI_GNT 信号给出 NANDC 是否成功占用总线的指示标志，当多个控制器同时发出总线占用请求时，EBI 根据特定的仲裁机制，决定哪一个控制器优先级最高。

2.15.5.3 数据结构

NANDC 内部 Buffer 大小为（2048+320）byte。读写 NAND FLASH 数据在 NANDC Buffer 中的数据结构如下所述。

2.15.5.3.1 无 ECC 模式

当不需要 NANDC 进行 ECC 校验时：

对于写操作，NANDC 对于 Buffer 中的数据不做任何处理，透明写入 NAND FLASH 器件。

对于读操作，NANDC 对于从 NAND FLASH 读出的数据不做任何处理，写入内部 Buffer。

读写数据的 Byte 数由寄存器 NFC_DATA_NUM 设定。

支持 2K，4K，8K Page Size。

2.15.5.3.2 1bit ECC 模式：2KB（2048+64）PageSize

驱动软件中的数据结构与 nandcbuffer 中的数据结构，以及 NAND FLASH 中的数据结构完全相同，如下。

2KB（2048+ 64）byte page size NAND FLASH 数据存储结构

BB: bad block ,2byte

DECC:数据区的 ECC 码，12byte

CTRL:留给软件用的控制区域，40byte

2.15.5.3.3 1bit ECC 模式：4KB（4096+128）PageSize

驱动软件中的数据结构 4096byte＋128byte：

DATA(4096)	BB	CTRL	RSV

BB: bad block ,2byte

CTRL:控制区域，54byte

RSV: 保留区域，72byte

在 NAND FLASH 中，数据按照 2048byte+12byte ECC 码的方式交替存放（共 4096byte＋24byte）。

DATA(2048)+DECC(12)	DATA(2036)	BB	DATA(12)	DECC(12)	CTRL(54)	SECC(8)

BB: bad block ,2byte

CTRL:留给软件用的控制区域，54byte

SECC: bad block 标志 和 CTRL 区域的 ECC 码，8byte。

2.15.5.3.4 8bits ECC 模式：2KB（2048＋64）PageSize

驱动软件中的数据结构 2048byte＋64byte：

DATA(2048B)	BB	CTRL	RSV

BB: bad block ,2byte

CTRL：留给软件用的控制区域，34byte

RSV：保留区

当写入 NANDC 的 Buffer 时，数据结构与驱动软件中的数据结构相同。

写入 NAND FLASH 中的数据结构如下，把 2084byte 的软件有效数据切成 2 个 1042byte 的数据块，每个计算出 14byte 的 ECC 码，然后把数据块和 ECC 码交替写入 NAND FLASH。

DATA(1042)+ECC(14)+DATA(992)+BB(2) +DATA(14)+ECC(14)+CTRL(34)

2.15.5.3.5 8bits ECC 模式：4KB（4096+128）PageSize

对 4KB Page Size 的器件，软件可用的 Spare Area 区的大小为 46byte。驱动软件中的数据结构如下：

DATA(4096)	BB	CTRL	RSV

BB: bad block ,2byte

CTRL:留给软件用的控制区域，46byte

RSV: 保留区域，80byte

写到 NAND FLASH 器件内部的数据结构会自动调整为 1036byte 数据+14byte ECC 码的格式交替存放,共 3 组。对于最后一组（1036byte 数据+14byte ECC 码），调换一下坏块标识的位置，以使 BB 正好位于 NAND FLASH 中的 4096 的 byte 地址位置。

DATA(1036)+ECC(14)+DATA(1036)+ECC(14)+DATA(1036)+ECC(14)+DATA(946)+BB(2) +DATA(42)+ECC(14)+CTRL(46)

2.15.5.3.6 24bit ECC/1024byte 模式

该模式兼容 8bit/512byte ECC 的器件和 12bit/512byte ECC 的器件。

2.15.5.3.7 24bit ECC/1024byte 模式：2KB(2048+128) PageSize

驱动软件中的数据结构 2048byte＋32byte：

DATA(2048B)	BB	CTRL

BB: bad block ,2byte

CTRL：留给软件用的控制区域，30byte

写入 NAND FLASH 中的数据的结构如下,BB 在 2048 的位置。

DATA(1040)	ECC(42)	DATA(966)	BB(2)	DATA(42)	ECC(42)	CTRL(30)

2.15.5.3.8 24bit ECC/1024byte 模式：4KB(4096+xxx) PageSize

4K Page Size 时，驱动软件中的数据结构如下：

DATA(4096)	BB	CTRL

BB: bad block ,2byte

CTRL:留给软件用的控制区域，30byte

当写到 NAND FLASH 器件中时，先把上述数据切分成 1024+8 的 4 个数据段，然后对每个数据段计算一个 42byte 的 ECC 码。

写到 NAND FLASH 中时，对于前三个数据段，数据和 ECC 码交替存放。对于最后一个 1024＋8＋42 的数据段，坏块标识插入到数据中间，以便使坏块标识位于 NAND FLASH 的 4096 的 byte 地址位置。

DATA(1032)+ECC(42)+DATA(1032)+ECC(42)+DATA(1032)+ECC(42)+DATA(874)+BB(2)+DATA(126)+ECC(42)+CTRL(30)

2.15.5.3.9 24bit ECC/1024byte 模式：8KB(8192+xxx) PageSize

对于 8K Page Size，驱动软件中的数据结构如下：

DATA(8192)	BB	CTRL

BB: bad block ,2byte

CTRL:留给软件用的控制区域，30byte

当写到 NAND FLASH 器件中时，先把上述数据切分成 1024+4 的 8 个数据段，然后对每个数据段计算一个 42byte 的 ECC 码。对于前 7 个数据段，写到 NAND FLASH 中时，数据段和 ECC 码交替存放。对于最后一个 1024＋4＋42 的数据段，坏块标识插入到数据中间，以便使坏块标识位于 NAND FLASH 的 8192 的 byte 地址位置。

DATA(1028)+ECC(42)+DATA(1028)+ECC(42)+DATA(1028)+ECC(42)+DATA(1028)+ECC(42)
DATA(1028)+ECC(42)+DATA(1028)+ECC(42)+DATA(1028)+ECC(42)+DATA(702)+BB(2)+DATA(294)+ECC(42)+CTRL(30)

2.15.5.3.10 40bit ECC/1024byte 模式：8KB(8192+xxx) PageSize

对于 8K Page Size，驱动软件中的数据结构如下：

DATA(8192)	BB	CTRL	RSV

BB: bad block ,2byte

CTRL:留给软件用的控制区域，30byte

RSV：保留区

当写到 NAND FLASH 器件中时，先把上述数据切分成 1024+4 的 8 个数据段，然后对每个数据段计算一个 70byte 的 ECC 码。对于前 7 个数据段，写到 NAND FLASH 中时，数据段和 ECC 码交替存放。对于最后一个 1024＋4＋70 的数据段，坏块标识插入到数据中间，以便使坏块标识位于 NAND FLASH 的 8192 的 byte 地址位置。

DATA(1028)+ECC(70)+DATA(1028)+ECC(70)+DATA(1028)+ECC(70)+DATA(1028)+ECC(70)+DATA(1028)+ECC(70)
DATA(1028)+ECC(70)+DATA(1028)+ECC(70)+DATA(506)+BB(2) +DATA(490)+ECC(70)+CTRL(30)

每个 Page 的写入需要操作 NANDC4 次。每次把（2048＋8）字节的有效数据写入 NANDC 的 Buffer 中 0 地地址开始的位置，然后启动 NANDC 的只写数据的操作模式；对于每次操作，NANDC 生成的 ECC 码都存放在内部 Buffer 的地址 2048+128 开始的位置。

读一个完整的 Page 也需要操作 NANDC4 次，与写入类似。

2.15.5.3.11 40bit ECC/1024byte 模式：16KB(16384+xxx) PageSize

对于 16K Page Size，驱动软件中的数据结构如下：

DATA(16384)	BB	CTRL	RSV

BB: bad block ,2byte

CTRL:留给软件用的控制区域，30byte

RSV：保留区

当写到 NAND FLASH 器件中时，先把上述数据切分成 1024+2 的 16 个数据段，然后对每个数据段计算一个 70byte 的 ECC 码。对于前 15 个数据段，写到 NAND FLASH 中时，数据段和 ECC 码交替存放。对于最后一个 1024＋2＋70 的数据段，坏块标识插入到数据中间，以便使坏块标识位于 NAND FLASH 的 16384 的 byte 地址位置。

DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+
DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+
DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+ECC(70)+DATA(1026)+
ECC(14)+BB(2)+ECC(56) +DATA(994)+ECC(70)+CTRL(30)

每个 Page 的写入需要操作 NANDC8 次。每次把（2048＋4）字节的有效数据写入 NANDC 的 Buffer 中 0 地地址开始的位置，然后启动 NANDC 的只写数据的操作模式；对于每次操作，NANDC 生成的 ECC 码都存放在内部 Buffer 的地址 2048+128 开始的位置。

读一个完整的 Page 也需要操作 NANDC8 次，与写入类似。

2.15.5.3.12 16bit ECC/1024byte 模式：2KB(2048+64) PageSize

驱动软件中的数据结构 2048byte＋64byte：

DATA(2048B)	BB	CTRL	RSV

BB: bad block ,2byte

CTRL：留给软件用的控制区域，6byte

当写入 nandc 的 buffer 时，数据结构与驱动软件中的数据结构相同。

写入 nandflash 中的数据的结构如下，把 2056byte 的软件有效数据切成 2 个 1028byte 的数据块， 每个计算出 28byte 的 ECC 码，然后把数据块和 ECC 码交替写入 nandflash。

DATA(1028)+ECC(28)+DATA(992)+BB(2)+DATA(28)+ECC(28)+CTRL(6)

2.15.5.3.13 16bit ECC/1024byte 模式：4KB(4096+128) PageSize

对 4KB page size 的器件，软件可用的 spare area 区的大小为 14byte。驱动软件中的数据结构如下：

DATA(4096)	BB	CTRL	RSV

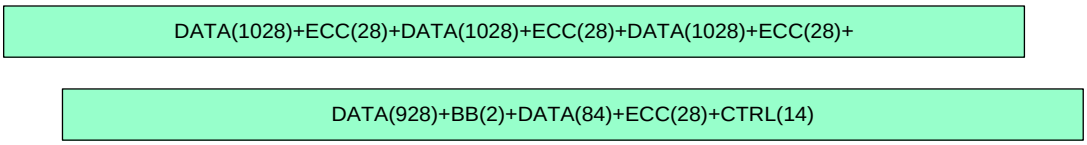
BB: bad block ,2byte

CTRL:留给软件用的控制区域，14byte

RSV: 保留区域，112byte

写到 nandflash 中的数据结构如下。把软件有效数据切成 4 个 1032byte 的数据块，每 1032byte 数据计算一次 ECC。

写到 NAND Flash 器件内部的数据结构会自动调整为 1032byte 数据+28byte ECC 码的格式交替存放,共 3 组。对于最后一组（1032byte 数据+28byte ECC 码），调换一下坏块标识的位置，以使 BB 正好位于 nandflash 中的 4096 的 byte 地址位置。



2.15.5.4 NAND Flash 读写功能

NAND FLASH 器件的数据存储结构一般分为 block 和 page，每个 block 包括若干个 page。对 NAND FLASH 写入数据前，必须先进行擦除操作，擦除以 block 为单位。然后以 page 为单位进行读写。

不同厂家提供的操作 NAND FLASH 的命令会有所不同，应以厂家器件手册为准。

一次典型的读数据操作过程如下：

步骤 1　向 NAND FLASH 发读命令 0x00。

步骤 2　发送读取的起始地址（由页内地址和页地址、Block 地址共同组成，相关信息，请参见相关厂家 NAND FLASH 器件手册）。

步骤 3　发送读确认命令 0x30。此时，NAND FLASH 器件会拉低 RB 信号，表示 NAND FLASH 正在进行内部读操作。过一段时间后（一般约 25μs），RB 变高，此时代表 NAND FLASH 已经准备好数据。

步骤 4　CPU 通过 NF_RD_N 信号，把数据从 NAND FLASH 读出来。

----结束

NANDC 读 NAND FLASH 一个 Page 数据的典型时序如图 2-25 所示。

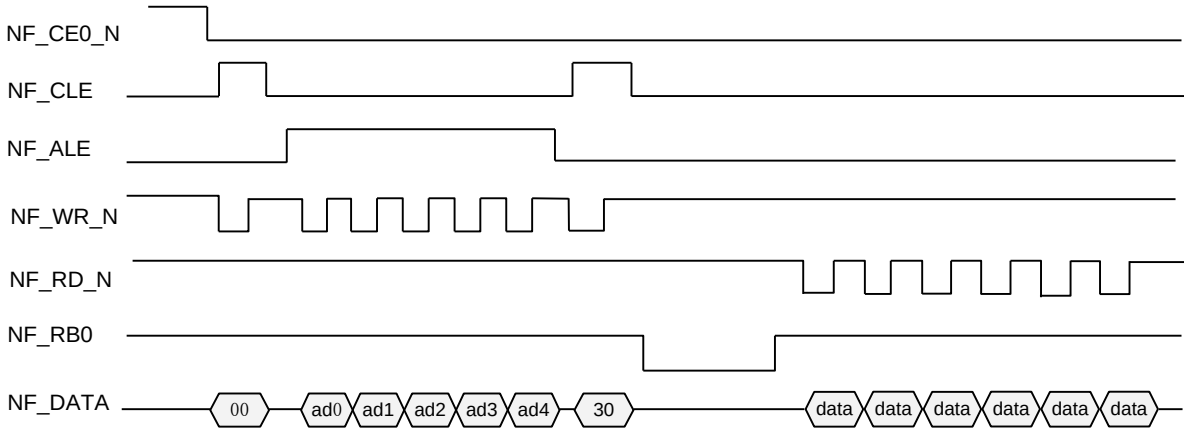


图2-25 读 NAND FLASH 一个 Page 数据的典型时序图

一次典型的编程（写数据）的操作如下：

步骤 5 向 NAND FLASH 发编程命令 0x80。

步骤 6 发送写入数据的起始地址（由页内地址和页地址、block 地址共同组成，需参照对应的 NAND FLASH 器件手册）。

步骤 7 把数据写入 NAND FLASH 的内部缓存。

步骤 8 CPU 再发送编程确认命令 0x10。此时，NAND FLASH 器件会拉低 RB 信号，表示 NAND FLASH 正在进行内部编程操作，维持一段时间后（一般约 200us），RB 变高，此时，代表 NAND FLASH 内部编程已经结束。

步骤 9 CPU 通过发送 0x70 读状态，读出本次编程是否成功的状态数据。

----结束

启动 NANDC 进行编程操作时的时序如图 2-26 所示。

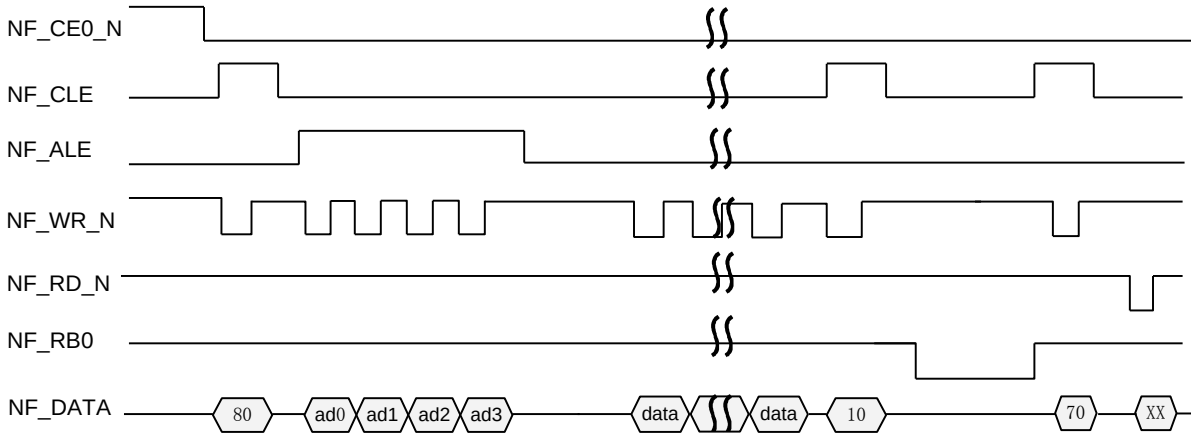


图2-26 启动 NANDC 进行编程操作时的时序图

2.15.6 工作方式

2.15.6.1 初始化

NANDC 复位后使用时初始化：

步骤 1 向 NFC_CON[op_mode]写 1，进入 Normal 模式。根据对接器件的位宽和 Page Size 大小，设置 NFC_CON[bus_width]和 NFC_CON[page_size]。根据外接器件的片选个数和 ready_busy 信号个数，设置 NFC_CON [rb_sel]。写寄存器 NFC_CON [ecc_type]，设定校验和纠错模式。

步骤 2 根据对接器件的时序要求，写寄存器 NFC_PWIDTH。

步骤 3 如果是查询方式，写中断使能寄存器 NFC_INTEN，屏蔽所有中断。如果是中断方式，只需使能 Op_done 中断，其余可屏蔽。

----结束

2.15.6.2 软复位

对接器件容许的情况下，可以任意时间对 NANDC 控制逻辑及 ECC 逻辑进行软复位，不会对系统造成影响。通过配置系统控制器 XXX 为 0bX，可实现对 XXX 工作逻辑的单独软复位。

2.15.6.3 对 NAND FLASH 执行擦除操作

执行擦除操作的步骤如下：

步骤 1 向寄存器 NFC_ADDRL 和 NFC_ADDRL 写入编程 Page 地址，向寄存器 NFC_CMD 写入擦除命令 0x0070_D060。

步骤 2 向寄存器 NFC_OP 写入 0x369，启动 NANDC 对 NAND FLASH 进行擦除操作（假设 NAND FLASH 芯片需要 3 个地址，并且是对片选 0 操作）。

步骤 3 查询方式下，检测 NFC_STATUS [nfc_ready]，如果为 1，进入步骤 4。否则，继续查询；中断方式下，检测 NFC_INTS [op_done]，如果为 1，进入步骤 4。

步骤 4 读寄存器 NFC_STATUS [nf_status]，判断擦除是否成功。

----结束

2.15.6.4 DMA 方式写 NAND FLASH

步骤 1 根据对接器件类型配置好 NFC_CON 的 Page Size、Ecc_type、Bus_wide 参数。向寄存器 NFC_CMD 写入擦除命令 0x0070_1080。设置写 NAND FLASH 的地址寄存器 NFC_ADDRL 和 NFC_ADDRH。

步骤 2 CPU 配置数据在 DDR 中存放的基地址寄存器 NFC_BADDR_D，配置 tag 区在 DDR 中存放的基地址寄存器 NFC_BADDR_OOB，如果是 ECC0 模式，还要配置 NFC_DMA_LEN 寄存器。

步骤 3 CPU 写寄存器 NFC_DMA_CTRL，启动 NANDC 写 NAND FLASH。

步骤 4 CPU 等待 NANDC 发出 DMA 传输完成中断。（如果是查询方式，启动 Dma 操作后，读 NFC_DMA_CTR[dma_start]，如果返回 0，表示 Dma 操作结束。）

----结束

2.15.6.5 DMA 方式读 NAND FLASH

步骤 1 根据对接器件类型配置好 NFC_CON 的 Page Size、Ecc_type、Bus_wide 参数。向寄存器 NFC_CMD 写入擦除命令 0x0000_3000。设置写 NAND FLASH 的地址寄存器 NFC_ADDRL 和 NFC_ADDRH。

步骤 2 CPU 配置读出数据在 DDR 中存放的基地址寄存器 NFC_BADDR_D。配置 DMA 读 NAND FLASH 的逻辑地址寄存器 NFC_RD_LOGIC_ADDR，配置 DMA 读 NAND FLASH 的逻辑长度寄存器 NFC_RD_LOGIC_LEN。

步骤 3 CPU 写寄存器 NFC_DMA_CTRL，启动 NANDC 读 NAND Flash。

步骤 4 CPU 等待 NANDC 发出 DMA 传输完成中断。（如果是查询方式，启动 Dma 操作后，读 NFC_DMA_CTR[dma_start]，如果返回 0，表示 Dma 操作结束。）

----结束

2.15.7 注意事项

NANDC 额外注意事项如下：

- 各个厂家的 NAND FLASH 器件支持的操作命令有所不同，需要根据器件手册合理设置命令寄存器 NFC_CMD。同时，不同容量的 NAND FLASH 器件需要的地址周期数不同，需要参照器件手册设置 NFC_OP 中的 Address_cycles 域。不同器件支持的时序有所不同，需要根据器件手册合理设置读写脉冲宽度寄存器 NFC_PWIDTH 和操作间隔寄存器 NFC_OPIDLE。
- 在配置好相关寄存器和 Buffer 后，再写 NFC_OP 寄存器或者 NFC_DMA_CTRL 寄存器，以启动 NANDC 发起 FLASH 读写操作。之后，不要再写相关的寄存器，否则可能导致 NANDC 或 FLASH 工作不正常。
- 写 NFC_OP 寄存器或者 NFC_DMA_CTRL 寄存器启动读写 NAND FLASH 之后，在 NFC_STATUS [nfc_ready]标志为 0 期间，不要读写 NANDC 的 Buffer。否则，可能返回错误的数据。

2.16 SPI FLASH 接口（SFC）

2.16.1 概述

SFC 逻辑是一个 SPI FLASH 控制器，主要功能完成系统对 SPI FLASH 的访问控制。

2.16.2 特性

SFC 具有以下特性：

- 支持 Standard SPI、Dual-SPI、或 Quad-SPI、Serial Quad I/O 四种接口模式。上电后默认支持 Standard SPI 接口模式，可通过寄存器配置选择支持其他几种接口模式。两个片选所接 FLASH 的接口模式必须相同；
- 支持 Alias 方式从 SPI FLASH 启动；
- 支持 SPI FLASH 的最小容量为 512K bits，最大容量为 128M bits；
- 支持 SPI FLASH 的时钟频率最高为 50MHz；
- 支持 Write Protect 和 Hold 操作。

注意

SFC 模块共有两个片选，Hip04 使用片选 1，片选 0 不可使用。

2.16.3 关键术语

如表 2-19 的术语将在本章节表示特殊的含义。

表2-19 SFC 关键术语描述

术语	描述
SFC	SPI FLASH Controller，SPI FLASH 控制器，主要功能完成系统对 SPI FLASH 的访问控制。

2.16.4 接口信号

SFC 接口信号如表 2-20 所示。

表2-20 SFC 接口信号描述

管脚	内部信号名称	方向	描述
SFC_CSN	SFC_CS1_N	O	SFC 片选 1 信号，低有效。
SFC_CLK	SFC_CLK	O	SFC 时钟信号。

管脚	内部信号名称	方向	描述
SFC_SIO	SFC_SIO	I/O	在 Standard SPI 模式，为数据的输出信号； 在 Dual SPI 模式下，为数据的输入输出信号； 在 Quad SPI 模式下，为数据的输入输出信号。
SFC_SOI	SFC_SOI	I/O	在 Standard SPI 模式，为数据的输入信号； 在 Dual SPI 模式下，为数据的输入输出信号； 在 Quad SPI 模式下，为数据的输入输出信号。
SFC_WP_IO2	SFC_WP_IO2	I/O	在 Standard SPI 模式，作为 write protect 功能，低有效； 在 Dual SPI 模式下，作为 write protect 功能，低有效； 在 Quad SPI 模式下，为数据的输入输出信号。
SFC_HOLD_IO3	SFC_HOLD_IO3	I/O	在 Standard SPI 模式，作为 holdt 功能，低有效； 在 Dual SPI 模式下，作为 holdt 功能，低有效； 在 Quad SPI 模式下，为数据的输入输出信号。

2.16.5 功能描述

2.16.5.1 接口模式描述

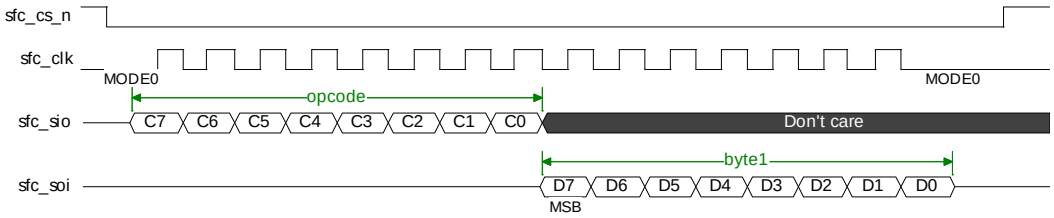
SFC 支持四种接口模式，下面分别进行描述。

- Standard SPI

Standard SPI 接口有 1 bit 数据输入线和 1 bit 数据输出线，接口时序如图 2-27 所示。

在 Standard SPI 接口模式下，写数据操作只有一种为 Page Program；读操作只支持 Read 和 Fast Read 两种操作。

图2-27 SFC Standard SPI 模式接口时序

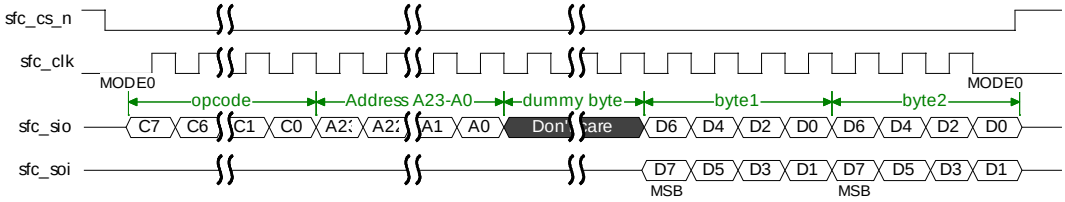


- Dual-SPI

Dual-SPI 接口模式，有 2 bits 双向数据线，接口时序如图 14-2 所示。

在 Dual-SPI 即可模式下，写数据操作只支持 Dual-Input Page Program 操作；读数据操作只支持 Dual-Output Read 操作。

图2-28 SFC Dual-SPI 接口时序

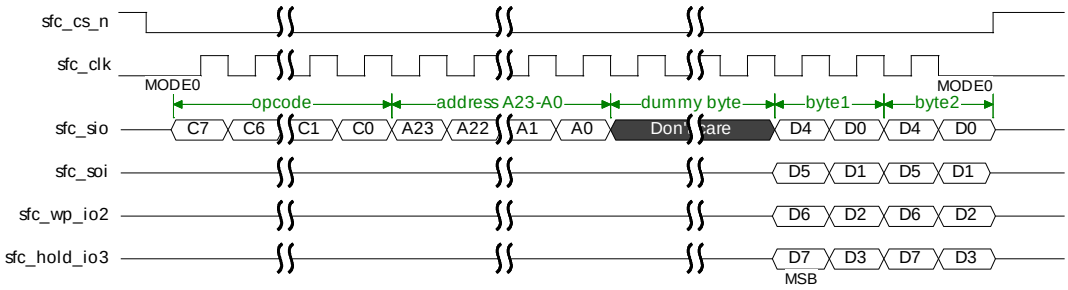


- Quad-SPI

Quad-SPI 接口模式，有 4 bits 双向数据线，接口时序如图 2-28 所示。

在 Quad-SPI 接口模式下，写数据操作只支持 Quad-Input Page Program 操作；读数据操作只支持 Quad Output Read 操作。

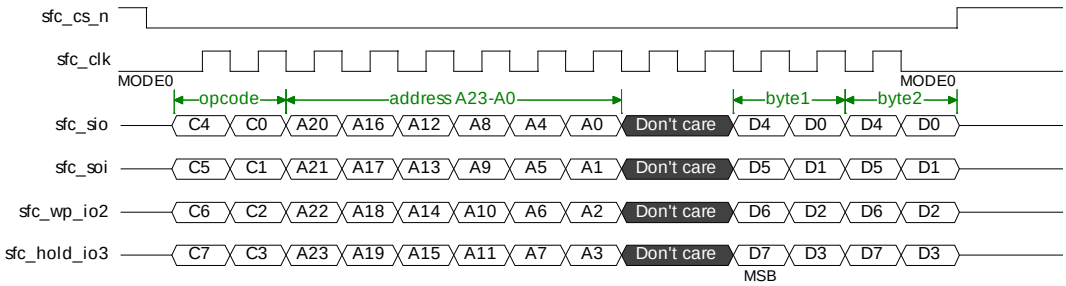
图2-29 SFC Quad-SPI 接口时序



- Serial Quad I/O

Serial Quad I/O 接口模式，有 4 bits 双向数据线，命令、地址、读写数据全部通过这 4 bits 数据线传输。时序如图 2-30 所示。

图2-30 SFC Serial Quad I/O 模式接口时序



2.16.5.2 读写 Memory

2.16.5.2.1 读 Memory

提供两种读 FLASH 的途径：第一种通过总线读，第二种通过设置 SFC 的配置寄存器来读 FLASH。第一种读 FLASH 操作效率高，SFC 模块自动将内部总线的读操作时序映射为 SPI 时序。第二种读 FLASH 操作效率低，需要软件配置寄存器。

对于内部总线的任何读 Memory 操作，SFC 模块都将其转换成 SPI 接口的读操作。上电后默认执行 Normal read 操作（Opcode=0x3），这种读操作对应的 Dummy Byte 个数为 0。上电系统 Boot 起来后，可以通过软件配置读命令的 opcode 和 Dummy Byte 个数来进行其他的读操作，如 Fast Read（iocode=0xB），Dummy Byte 为 1。

2.16.5.2.2 写 Memory

提供两种写 FLASH 的途径：第一种通过内部总线写，第二种通过设置 SFC 的配置寄存器来写 FLASH。第一种写 FLASH 操作效率高，SFC 模块自动将总线的一次写操作转换为 FLASH 侧的 WREN 操作（Write Enable）＋PP 操作（Page Program）＋RDSR（Read Status Register）操作。第二种写 FLASH 操作效率低，需要软件配置寄存器。

在写 Memory 之前要对 FLASH 进行 Erase 操作，这项操作必须要由软件通过配置 SFC 的寄存器来完成。

FLASH 执行 PP 操作需要一定的时间，需要通过读 FLASH 的状态寄存器来判断 PP 操作是否完成才能开始下一次写操作，否则会丢失数据。

PP 操作不能跨 Page boundary，对于跨 Page boundary 的写操作必须拆成 2 次来完成。CPU 通过总线写 FLASH 时，SFC 模块根据地址和 Burst 长度自动判断此次操作是否会跨 Page boundary，如果跨 Page boundary 就拆成两次来处理。

2.16.5.2.3 其他操作

对 FLASH 的其他操作如 Erase、进入 Deep Power Down、读 Device ID 等必须通过寄存器访问来实现。

2.16.6 工作方式

2.16.6.1 软复位

对接器件容许的情况下，可以对 SPI 工作逻辑进行软复位，不会对系统造成影响。通过配置系统控制器 SC_SPI_RESET_REQ 为 0b1，可实现对 SPI 工作逻辑的单独软复位。

SPI 的软复位只能复位 SPI 控制器，不能复位 SPI 中的寄存器，软复位流程按如下操作：

1、软复位 spi 前先 disable spi；

2、对 spi 进行软件复位；

3、重新配置 spi 寄存器；

4、enable spi。

2.17 MDIO 接口（MDIO）

2.17.1 概述

MDIO 是对外部 PHY 芯片的接口控制模块，软件可以通过配置 MDIO 相应寄存器实现对 PHY 芯片内部寄存器的读写，最终实现对 PHY 芯片的控制和管理。例如复位控制、自环控制、速度选择、双工选择、冲突测试、自协商使能、物理层状态获取等等。

2.17.2 特性

MDIO 具有以下特性：

- 支持 2 个 MDIO 模块；
- 支持 802.3 协议 Clause 45 章节描述的功能；
- 支持 MDIO 接口时序，MDIO 时钟频率为 2.5MHz；
- 支持对 PHY 内部寄存器的读写访问，PHY 地址、寄存器地址等可配置。

2.17.3 关键术语

如表 2-21 的术语将在本章节表示特殊的含义。

表2-21 MDIO 关键术语描述

术语	描述
MDIO	It is a series of packets; these packets must be transmitted as received order. An ordered flow can be a series of packets that are in a same channel or have the same packet attributes. It‘s decided by the application.

2.17.4 接口信号

MDIO 接口信号如表 2-22 所示。

表2-22 MDIO 接口信号描述

管脚	内部信号名称	方向	描述
MDIO0_CLK	MDIO0_CLK	O	MDIO 时钟信号。
MDIO0_DATA	MDIO0_DATA	I/O	MDIO 数据信号。
MDIO1_CLK	MDIO1_CLK	O	MDIO 时钟信号。
MDIO1_DATA	MDIO1_DATA	I/O	MDIO 数据信号。

2.17.5 功能描述

2.17.5.1 数据结构

MDIO 接口的数据帧格式如表 2-23 所示。

表2-23 MDIO 接口的数据帧格式

操作方式	数据帧格式							
	PRE	ST	OP	PHY AD	REG AD	TA	DATA	IDLE
READ	1……1	01	10	AAAAA	RRRRR	Z0	DDDDDDDDDDDDDDDD	Z
WRITE	1……1	01	01	AAAAA	RRRRR	10	DDDDDDDDDDDDDDDD	Z

数据帧各域的含义如下：

- PRE：帧同步码，32bit 位宽。

  PRE 是 32 个连续的“1”，用于器件的同步通信。但该域不是必须的，为了提高效率，有些 PHY 器件的 MDIO 操作就没有此域。
- ST：帧开始标志。MAC 输出“01”比特通知 PHY 表示帧操作开始。
- OP：帧操作码，2bit 位宽。
  - 01：表示此帧为一次写操作请求。
  - 10：表示此帧为一次读操作请求。
- PHY AD：PHY 器件地址，5bit 位宽。

  每个 PHY 器件都把自己的地址与这 5bit 进行比较，若匹配则响应后面的操作；若不匹配，则忽略后面的操作。

  5bit 决定了一个 MDIO 接口最多只能支持 32 个 PHY 器件的配置。
- REG AD：PHY 寄存器的地址，5bit 位宽。

  用来选择 PHY 内部的 32 个寄存器中的某个寄存器，PHY 为物理层接口，每个 PHY 接口含有 32 个 16bit 的寄存器。

  一般情况下，不同 PHY 芯片的前 16 个寄存器内容应该一致，后面 16 个是可由用户指定功能的寄存器。
- TA：状态转换域，2bit 位宽。
  - 读操作时，第 1bit 由 MDIO 送出高阻态，第 2bit 由 PHY 给 MDIO 信号置 0。
  - 写操作时，MDIO 信号由 MDIO 模块控制，连续输出“10”两个比特。
- DATA：帧的寄存器数据域，16bit 位宽。
  - 读操作时，为 PHY 送到 MDIO 模块的串行数据。
  - 写操作时，为 MDIO 模块送到 PHY 的串行数据。
- IDLE：帧结束后的空闲状态。

  此时 MDIO 无源驱动，处于高阻状态，但一般用上拉电阻使其处在高电平。

2.17.5.2 接口读时序

MDIO 接口包括 MDC 和 MDIO 2 根信号线。

MDIO 接口读时序如图 2-31 所示。

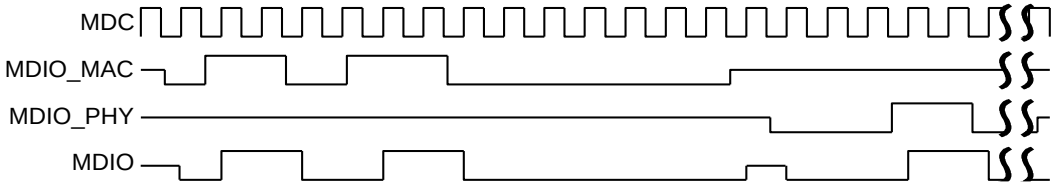


图2-31 MDIO 接口读时序图

2.17.5.3 接口写时序

MDIO 接口写时序如图 2-32 所示。

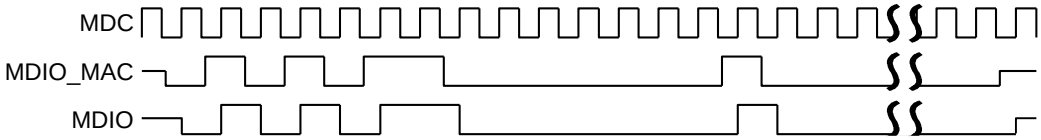


图2-32 M DIO 接口写时序图

说明

- 图 2-31 和图 2-32 中，MDC 为 2.5MHz 时钟，MDIO_MAC 是由 MAC 发出的信号，MDIO_PHY 是由 PHY 发出的信号，MDIO 是两侧信号在三态总线上的状态。
- 无论是 MAC 还是 PHY，都是在 MDC 的下降沿发出数据，对端在 MDCL 的上升沿接收数据，确保数据的稳定接收窗口。

2.18 UART 接口（UART）

2.18.1 概述

UART（Universal Asynchronous Receiver/Transmitter）是一个异步串行的通信接口，主要执行串并转换功能，将芯片发送到外围设备的数据进行并串转换，同时将来自外围设备的串行数据进行串并转换。该接口模块遵循 RS232 串行通信协议。

UART 内部包括 32 Bytes 的接收 FIFO 和发送 FIFO，CPU 可通过访问片内的状态寄存器查询数据传输的错误信息和器件的工作状态，可通过访问片内中断控制寄存器调整系统中断来满足用户的要求。

Hip04 提供了 3 个 UART：UART0、UART1 和 UART2，每个 UART 包括独立的地址和中断线。

2.18.2 特性

UART 具有以下特点：

- 支持 8 bits 宽、深度为 32 的发送 FIFO 和 10 bits 宽、深度为 32 的接收 FIFO，支持 FIFO 禁止或使能；
- 支持 5/6/7/8 bits 可编程数据位；
- 支持 1/1.5/2 bits 可编程停止位；
- 支持奇、偶校验方式，支持无校验位，支持校验位固定为软件设定值；
- 支持 DMA 数据搬运，也支持以中断方式和查询方式进行数据搬运；
- 支持传送速率可编程，传送速率范围为 100Mbps/（16×N）（0<N<65536，N 为整数）；
- 支持一个组合中断连接到中断控制器，中断类型包括：接收 FIFO 中断、发送 FIFO 中断、接收超时中断、错误中断；
- 支持初始中断状态查询和屏蔽后中断状态查询；
- 支持关断 UART 接口时钟以节省功耗；
- UART 需要支持主从模式，9 bits UART。

由于工作频率固定，在支持某些标准的波特率时会有误差，表 2-24 给出以 100MHz 为工作时钟的误差率，当速率越高时，误差将会越小。

表2-24 UART 波特率误差

波特率（bps）	分频比	误差率
9600	977	0.04%
56k	167	0.25%
187.5k	50	0.00%

2.18.3 关键术语

如表 2-25 的术语将在本章节表示特殊的含义。

表2-25 UART 关键术语描述

术语	描述
UART	Universal Asynchronous Receiver/Transmitter，异步串行的通信接口，主要执行串并转换功能，将芯片发送到外围设备的数据进行并串转换，同时将来自外围设备的串行数据进行串并转换。该接口模块遵循 RS232 串行通信协议。

2.18.4 接口信号

UART 接口信号如表 2-26 所示。

表2-26 UART 接口信号描述

管脚	内部信号名称	方向	描述
URXD0	UART0_RXD	I	UART0 接收信号。
UTXD0	UART0_TXD	O	UART0 发送信号。
URXD1	UART1_RXD	I	UART1 接收信号。
UTXD1	UART1_TXD	O	UART1 发送信号。
URXD2	UART2_RXD	I	UART2 接收信号。
UTXD2	UART2_TXD	O	UART2 发送信号。

2.18.5 功能描述

2.18.5.1 应用框图

UART 的应用如图 2-33 所示。

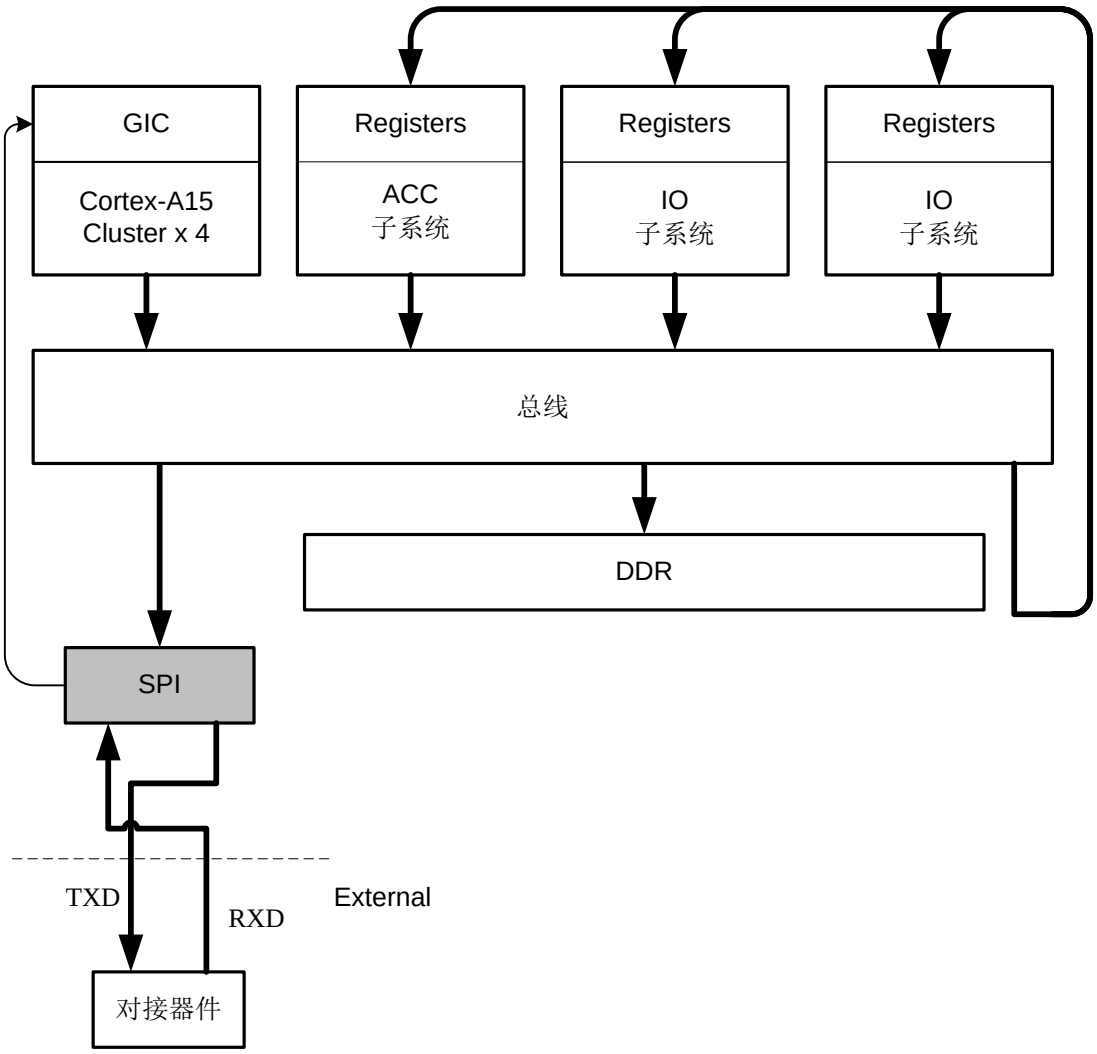


图2-33 UART 应用框图

UART 是一种异步双向串行总线，提供了一种简单有效的数据传输方式，只需要两根数据线互相对接即可。

CPU 可通过总线配置 UART 的寄存器配置数据传输到对端，或者读取寄存器获取接收到的对端数据；对于较大量的数据，CPU 还可以通过总线配置 DMA 自动完成一段数据的发送和接收。

UART 在传输完成或传输发生错误时，可以通过中断通知 CPU 进行处理。

2.18.5.2 相关模块

在系统中，UART 模块会和下面多个模块进行交互，分别描述如下：

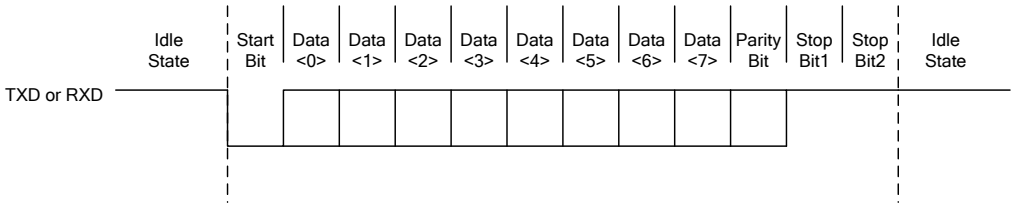
- CPU ：CPU 通过总线访问 UART 的寄存器接口，操作属性必须为 Device 或 Strongly-Ordered。CPU 首先需要完成 UART 的传输模式和速率配置，对于少量或即时性要求高的数据，CPU 可以直接通过 FIFO 接口读写完成数据传输。

- DMAC ：如果 UART 传输的数据量较大，且即时性并不强，可以通过 DMAC 完成数据的发送和接受，这需要 CPU 配置 DMAC 的一个独立通道，将为该通道与 UART 及传输的数据区域绑定，DMAC 在总线空闲时，将自动完成数据的发送和接收；
- GIC ：UART 在传输时如果发生错误，例如奇偶校验错误、数据帧断裂等，以及在非 DMAC 搬移模式时，如果接收 FIFO 高于水线，或发送 FIFO 低于水线，均会产生中断信号给 GIC 模块，通过 GIC 将中断发送给对 UART 具有控制权的 CPU。

2.18.5.3 数据结构

UART 使用 RS232 串行传输协议的帧结构完成数据传输。一次帧传输主要包括起始信号、数据、校验位和结束信号，如图 2-34 所示。数据帧从 UART 的 TXD 端输出，从 UART 的 RXD 端输入。

图2-34 UART 帧格式



说明

- 起始信号（Start bit）：一个数据帧开始的标志，UART 协议规定 TXD 信号出现一个低电平就表示一个数据帧的开始。在 UART 不传输数据时，应该保持高电平。
- 数据信号（Data bit）：数据位宽可以根据不同的应用要求进行调整，可以配置成 5 bits、6 bits、7 bits 或 8 bits 数据位宽。
- 校验位（Parity bit）：校验位是 1bit 纠错信号，UART 的校验位有奇校验、偶校验和固定校验位，同时支持校验位的使能和禁止，详细描述请见 UARTLCR_H 寄存器。
- 结束信号（Stop bit）：结束信号即数据帧的停止位，支持 1 bit 和 2 bits 停止位两种配置。数据帧的结束信号是把 TXD 拉成高电平。

2.18.5.4 同步功能

说明

UARTx 表示 UART0，UART1 或 UART2。

异步串行通信的关键是要保证通信双方收发同步。由于没有随路时钟线，所以双方必须约定一个固定的频率，用于同步收发数据，这样才能保证数据传送的正确性。

UART 的同步就是寻找起始位来确定一帧数据的开始。UART 的同步过程如下：

1 UART 产生一个 16 倍于波特率的总线时钟 CLK_X（设时钟周期为 T）。

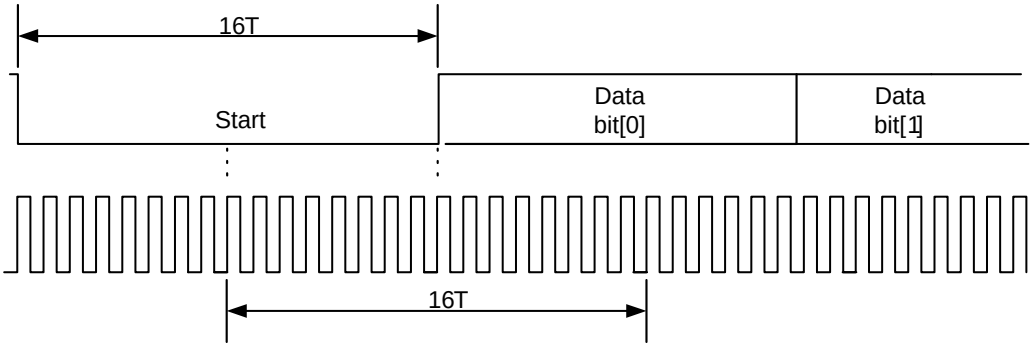
2 当 CLK_X 检测到串行输入的下降沿后，启动周期计数。

3 在第 8 个 CLK_X 周期采样数据，若为低电平，表明检测到了起始位。

4 每隔 16 个 CLK_X 周期便对数据进行采样，直到一帧数据结束。

数据采样过程如图 2-35 所示。

图2-35 UART 数据采样过程



通过配置寄存器 UARTx_DLL 和 UARTx_DLH（x=0 或 1），可设置 UART 工作的波特率，波特率计算公式如下：

当前波特率=总线时钟频率/（16×分频因子）。

分频因子由高位和低位两部分组成，分别对应寄存器 UARTx_DLL 和 UARTx_DLH。

例如当内部总线时钟为 125MHz 时，如果配置 UARTx_DLL 为 0x000000_44，UARTx_DLH 为 0x0000_0000，按照波特率计算公式，则表示当前的波特率为 125/（68×16）=115200bps。

UART 波特率配置的典型值有：9,600bps、14,400bps、19,200bps、38,400bps、57,600bps、76,800bps、115,200bps、230,400bps、460,800bps。

2.18.5.5 主从串口通信（UART 9 bits 通信）

说明

UARTx 表示 UART0、UART1 和 UART2。

网络应用中，可以通过 PEN、SP、EPS 实现 UART 9 bit 通信功能。

UARTx_LCR 寄存器 PEN 为奇偶校验使能，SP 为 Stick Parity 使能，EPS 为偶校验使能，具体功能如表 2-29 和表 2-30 所示。

表2-27 UART 发送方向奇偶校验功能

PEN	SP	EPS	功能描述
0	0	0	不发送 Parity bit。
0	0	1	不发送 Parity bit。
0	1	0	不发送 Parity bit。
0	1	1	不发送 Parity bit。
1	0	0	奇偶校验：奇校验使能，发送带有奇校验位；接收检测奇校验。
1	0	1	奇偶校验：偶校验使能，发送带有偶校验位；接收检测偶校验。
1	1	0	发送帧带有奇偶校验位，发送的 Parity bit = 1。

PEN	SP	EPS	功能描述
1	1	1	发送帧带有奇偶校验位，发送的 Parity bit = 0。

表2-28 UART 接收方向奇偶校验功能

PEN	SP	EPS	功能描述
0	0	0	没有 Parity bit，不检测奇偶校验。
0	0	1	没有 Parity bit，不检测奇偶校验。
0	1	0	没有 Parity bit，不检测奇偶校验。
0	1	1	没有 Parity bit，不检测奇偶校验。
1	0	0	奇偶校验：奇校验使能，接收检测奇校验。
1	0	1	奇偶校验：偶校验使能，接收检测偶校验。
1	1	0	预期接收到的 Parity bit =1；如果为 0 则上报 Parity error。
1	1	1	接收到的 Parity bit = 0；如果为 1 则上报 Parity error。

以采用 UART 主从串口方式进行多机通信为例，波特率为 187500bps，1 个起始位、8 个数据位、1 个控制位、1 个停止位，一共 11 位，可以从节点通过第 9 位区分是地址还是数据。

说明

由于 Windows 上的超级终端没有 187500bps 这个频点，所以在 FPGA 测试中需要考虑对接场景测试。

2.18.6 工作方式

2.18.6.1 中断或查询方式下的数据传输

说明

UARTx 表示 UART0、UART1 和 UART2。

2.18.6.1.1 初始化

具体配置步骤如下：

步骤 1 配置线性控制寄存器 UARTx_LCR 的 bit[7]为 1，使 UARTx_DLL 与 UARTx_DLH 可访问。

步骤 2 写相应的配置值到低位分频因子寄存器 UART_DLL 和高位分频因子寄存器 UARTx_DLH，配置传输速率。

步骤 3 配置寄存器 UARTx_LCR bit[6:0]，设定 UART 的工作模式。

步骤 4 配置 FIFO 控制寄存器 UART_FCR 使能或者关闭 FIFO，如果使能 FIFO，设定相应的发送及接收 FIFO 水线值，并配置配置寄存器 UARTx_LCR 的 bit[3]为 1，使能 DMA 功能。

步骤 5 中断方式下配置中断使能寄存器 UARTx_IEL 相应位为 1，使能相应中断；查询方式下配置中断使能寄存器 UARTx_IEL 相应位为 0，禁止中断。

----结束

2.18.6.1.2 数据发送

具体配置步骤如下：

步骤 1 将待发送数据写入寄存器发送数据缓存寄存器 UARTx_THR，启动数据发送。

步骤 2 查询方式下，进行连续数据发送时，读取发送 FIFO 水线寄存器 UART_TFL 和 UART 状态寄存器 UARTx_USR 检测发送 FIFO 状态，根据发送 FIFO 的状态决定是否向发送 FIFO 中发送数据；中断方式下，则检测相应中断状态，决定是否向发送 FIFO 中发送数据。

步骤 3 读取 UART_USR bit[0]，直到该值为 1。

----结束

2.18.6.1.3 数据接收

具体配置步骤如下：

步骤 1 查询方式下，读取接收 FIFO 水线寄存 UARTx_RFL 和 UART 状态寄存器 UARTx_USR 检测接收 FIFO 状态，根据接收 FIFO 的状态决定是否读取接收 FIFO 中的数据；中断方式下，根据相应中断状态位检测决定是否读取接收 FIFO 中的数据。

步骤 2 读取寄存器接收数据缓存寄存器 UARTx_RBR，获取接收的数据。

----结束

2.18.6.2 DMA 方式下的数据传输

说明

UARTx 表示 UART0、UART1 和 UART2。

2.18.6.2.1 初始化

具体配置步骤如下：

步骤 1 配置线性控制寄存器 UARTx_LCR bit[7]为 1，使 UARTx_DLL 与 UARTx_DLH 可访问。

步骤 2 写相应的配置值到低位分频因子寄存 UARTx_DLL 和高位分频因子寄存器 UARTx_DLH，配置传输速率。

步骤 3 配置寄存器 UARTx_LCR bit[6:0]，设定 UART 的工作模式。

步骤 4 配置寄存器 UARTx_FCR，使能或者关闭 FIFO。如果使能 FIFO，还需要设定相应的发送及接收 FIFO 水线值。

步骤 5 配置配置寄存器 UARTx_LCR bit[3]为 1，使能 DMA 功能。

----结束

2.18.6.2.2 数据发送

具体配置步骤如下：

步骤 1 配置 DMA 数据通道，包括数据传输源和目的地址、数据传输个数、传输类型等参数，其中数据传输的目的地址为发送数据缓存寄存器 UARTx_THR。

步骤 2 通过 DMA 中断上报，判断数据是否发送完成。

----结束

2.18.6.2.3 数据接收

具体配置步骤如下：

步骤 1 配置 DMA 数据通道，包括数据传输源和目的地址、数据传输个数、传输类型等参数，其中数据传输的源地址为接收数据缓存寄存器 UARTx_RBR。

步骤 2 通过 DMA 中断上报，判断数据是否接收完成。

----结束

2.18.6.3 主从串口通信（UART 9 bits 通信）

说明

UARTx 表示 UART0、UART1 和 UART2。

网络应用中，通常通过 PEN、SP、EPS 实现 UART 9 bit 通信功能。

UARTx_LCR 寄存器 PEN 为奇偶校验使能，SP 为 Stick Parity 使能，EPS 为偶校验使能，具体功能如表 2-29 和表 2-30 所示。

表2-29 UART 发送方向奇偶校验功能

PEN	SP	EPS	功能描述
0	0	0	不发送 Parity bit。
0	0	1	不发送 Parity bit。
0	1	0	不发送 Parity bit。
0	1	1	不发送 Parity bit。
1	0	0	奇偶校验：奇校验使能，发送带有奇校验位；接收检测奇校验。
1	0	1	奇偶校验：偶校验使能，发送带有偶校验位；接收检测偶校验。

PEN	SP	EPS	功能描述
1	1	0	发送帧带有奇偶校验位，发送的 Parity bit = 1。
1	1	1	发送帧带有奇偶校验位，发送的 Parity bit = 0。

表2-30 UART 接收方向奇偶校验功能

PEN	SP	EPS	功能描述
0	0	0	没有 Parity bit，不检测奇偶校验。
0	0	1	没有 Parity bit，不检测奇偶校验。
0	1	0	没有 Parity bit，不检测奇偶校验。
0	1	1	没有 Parity bit，不检测奇偶校验。
1	0	0	奇偶校验：奇校验使能，接收检测奇校验。
1	0	1	奇偶校验：偶校验使能，接收检测偶校验。
1	1	0	预期接收到的 Parity bit =1；如果为 0 则上报 Parity error。
1	1	1	接收到的 Parity bit = 0；如果为 1 则上报 Parity error。

假设接收方向采用奇校验，那么接收方向识别地址和数据的步骤如下：

步骤 2 主节点发送地址时，将发送数据设置为奇校验；

步骤 3 接收端收到数据后，将上报校验正确，接收端则识别为地址；

步骤 4 主节点发送数据时，将发送数据设置为偶校验；

步骤 5 接收端收到数据后，将上报校验错误，接收端则识别为数据。

----结束

在采用 UART 主从串口方式进行多机通信时，波特率为 187500bps，1 个起始位、8 个数据位、1 个控制位、1 个停止位，一共 11 位，从节点通过第 9 位区分是地址还是数据。

说明

由于 Windows 上的超级终端没有 187500bps 这个频点，所以在 FPGA 测试中需要考虑对接场景测试。

2.19 I2C 接口（I2C）

2.19.1 概述

I2C（The Inter-Integrated Circuit）控制器实现标准 I2C 主设备功能，兼容 Philips I2C 总线协议，可完成对 I2C 总线上的从设备的数据发送和接收。主要用于音视频 A/D、D/A 等外部 I2C 器件的控制。

Hip04 提供了 2 个 I2C：I2C0 和 I2C1，每个 I2C 包括独立的地址和中断线。

2.19.2 特性

I2C 控制器具有以下特点：

- 提供 2 个 I2C；
- 支持标准 I2C 总线协议；
- 支持主设备和从设备操作；
- 支持 7 bits 和 10 bits 从设备地址；
- 支持可编程时钟，可实现通讯速率控制；
- 支持 DMA 数据搬运，也支持以中断方式和查询方式进行数据搬运；
- I2C_DELAY 功能可将 I2C_SDA 信号输出延迟约 300ns。

2.19.3 关键术语

如表 2-31 的术语将在本章节表示特殊的含义。

表2-31 I2C 关键术语描述

术语	描述
I2C	Inter-Integrated Circuit，I2C 总线是由 PHILIPS 公司开发的两线式串行总线，用于连接微控制器及其外围设备。是微电子通信控制领域广泛采用的一种总线标准。它是同步通信的一种特殊形式，具有接口线少，控制方式简单，器件封装形式小，通信速率较高等优点。

2.19.4 接口信号

I2C 控制器接口信号如表 2-32 所示。

表2-32 I2C 控制器接口信号描述

管脚	内部信号名称	方向	描述
SDA0	I2C0_SDA	I/O	I2C0 双向数据信号。

管脚	内部信号名称	方向	描述
SCL0	I2C0_SCL	I/O	I2C0 输出时钟信号。
SDA1	I2C1_SDA	I/O	I2C1 双向数据信号。
SCL1	I2C1_SCL	I/O	I2C1 输出时钟信号。

2.19.5 功能描述

2.19.5.1 应用框图

I2C 应用框图如图 2-36 所示。

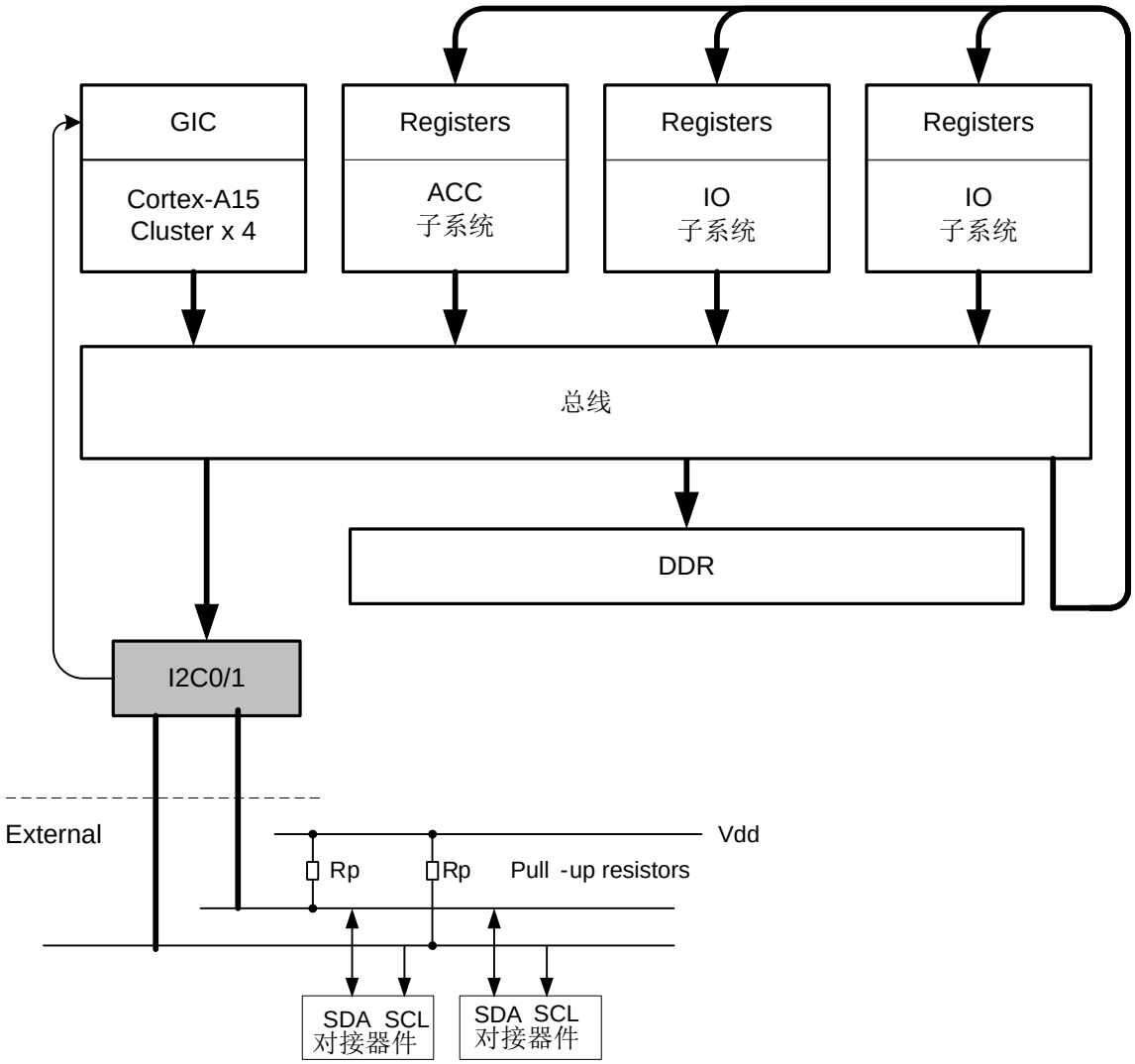


图2-36 I2C 应用框图

I2C 总线是一种双线、双向串行总线，它提供了一种简单有效的数据传输方式，可以简化设备间的连接，适合多设备间的短距离少量数据传输。I2C 总线的灵活性使得系统开发和设备扩展变得很容易。

CPU 可通过总线配置 I2C 的寄存器配置数据传输到对端，或者读取寄存器获取接收到的对端数据；对于较大量的数据，CPU 还可以通过总线配置 DMA 自动完成一段数据的发送和接收。

I2C 在传输完成或传输发生错误时，可以通过中断通知 CPU 进行处理。

2.19.5.2 相关模块

在系统中，I2C 模块会和下面多个模块进行交互，分别描述如下：

- CPU ：CPU 通过总线访问 I2C 的寄存器接口，操作属性必须为 Device 或 Strongly-Ordered。CPU 首先需要完成 I2C 的传输模式和速率配置，对于少量或即时性要求高的数据，CPU 可以直接通过 FIFO 接口读写完成数据传输。
- DMAC ：如果 I2C 传输的数据量较大，且即时性并不强，可以通过 DMAC 完成数据的发送和接受，这需要 CPU 配置 DMAC 的一个独立通道，将为该通道与 I2C 及传输的数据区域绑定，DMAC 在总线空闲时，将自动完成数据的发送和接收；
- GIC ：I2C 在传输时如果发生错误，例如传输冲突等，以及在非 DMAC 搬移模式时，如果接收 FIFO 高于水线，或发送 FIFO 低于水线，均会产生中断信号给 GIC 模块，通过 GIC 将中断发送给对 I2C 具有控制权的 CPU。

2.19.5.3 数据结构

一次典型的主模式 I2C 数据传输主要包括起始信号、从地址发送、数据传送、结束信号等部分。其时序如图 2-37 所示，数据帧格式如图 2-38 所示。

图2-37 I2C 数据传输时序图

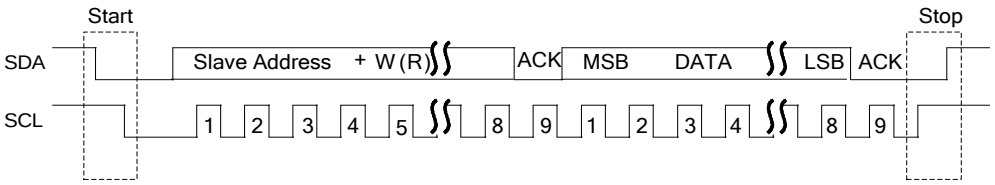
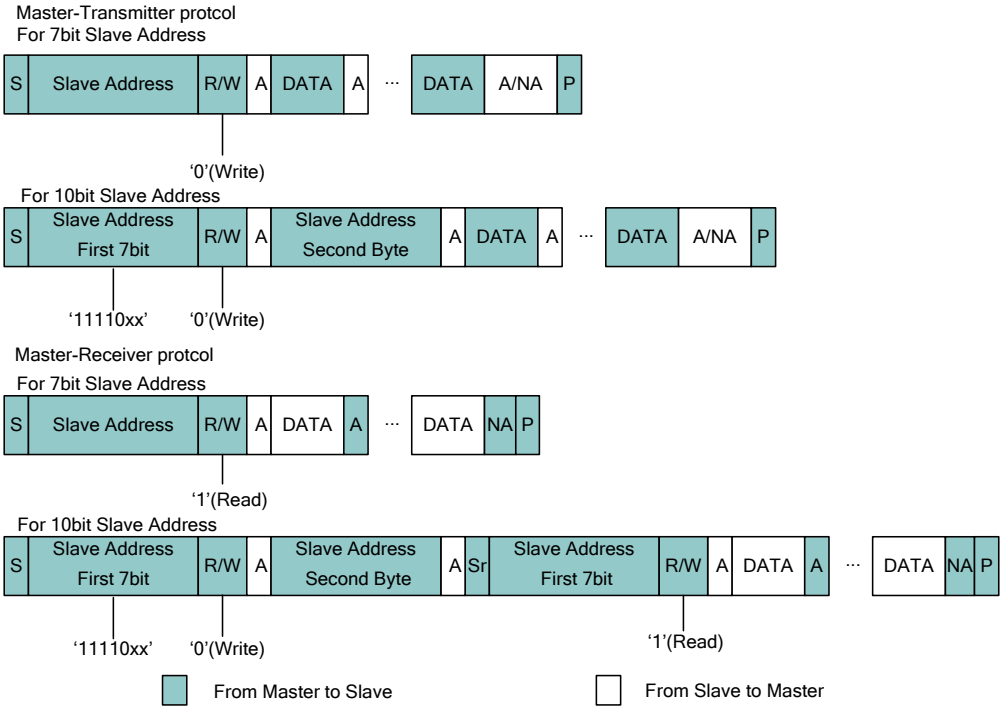


图2-38 I2C 数据传输帧格式图



说明

- A：Acknowledge（SDA Low）
- NA：No Acknowledge（SDA High）
- S：Start Condition
- Sr：Repeated Start Condition
- P：Stop Condition

I2C 标准协议规定的起始信号（Start），由总线上的主设备发出，用来唤醒所有从设备并指示数据传送开始的特殊信号。

从地址发送（Slave Address + W/R）是指主设备在发出起始信号后所发出的第一个数据包，包括 7bit 从地址位（如果是 10bit 从地址，则需发送两个数据包）和 1bit 读/写数据。各从设备依靠该地址信息来确认是否需要传送或接收数据。当从地址被成功接收后，相应从设备会在第 9 个时钟周期（SCL）将 SDA 拉低，从而反馈给主设备一个应答信号（ACK），然后就可以开始进行后续的数据传输。

数据传输（DATA）是指主设备在成功接收到从地址响应信号后按照读、写命令进行相应的数据接收或发送。在数据传输过程中 SDA 只能在 SCL 为低电平时变化，而在 SCL 高电平时保持；同时接收设备在每接收到 1 个字节后需要给发送设备发出应答信号（ACK）。如果发送设备没有成功接收到应答信号，就会终止数据传输或者重新开始发送。

结束信号（Stop）是主设备在当前数据传输完成并且没有新的数据传输需要发起时，所发出的 I2C 标准协议规定的表示数据传输结束的特殊信号。当主设备发出结束信号后从设备必须释放总线。

2.19.6 工作方式

2.19.6.1 软复位

对接器件容许的情况下，可以任意时间对 XXX 工作逻辑进行软复位，不会对系统造成影响。通过配置系统控制器 XXX 为 0bX，可实现对 XXX 工作逻辑的单独软复位。

2.19.6.2 时钟配置

说明

I2Cx 表示 I2C0 或 I2C1。

I2C Master 模式下需要通过配置 I2Cx_SS_SCL_HCNT、I2Cx_SS_SCL_LCNT、I2Cx_FS_SCL_HCNT、I2Cx_FS_SCL_LCNT 等寄存器可以设置标准模式和快速模式下 I2C 总线 SCL 信号的高低电平宽度，即相对于 I2C 工作时钟的周期数，其配置值的计算公式分别为：$*_HCNT = T_{scl_h} \times F_{I2C} - 8$ 和 $*_LCNT = T_{scl_l} \times F_{I2C} - 1$。

说明

- Tscl_h 为 SCL 高电平宽度，单位为 I2C 工作时钟周期。
- Tscl_l 为 SCL 低电平宽度，单位 μs。
- FI2C 为 I2C 工作时钟，单位 MHz。
- I2C Slave 模式下无需配置 I2Cx_SS_SCL_HCNT、I2Cx_SS_SCL_LCNT、I2Cx_FS_SCL_HCNT 和 I2Cx_FS_SCL_LCNT 寄存器。

I2Cx_SS_SCL_HCNT、I2Cx_SS_SCL_LCNT、I2Cx_FS_SCL_HCNT 和 I2Cx_FS_SCL_LCNT 典型配置值如表 2-33、表 2-34、表 2-35 和表 2-36 所示。

表2-33 I2Cx_SS_SCL_HCNT 典型配置值

I2C 总线速率（kbps）	I2C 工作时钟（MHz）	SCL 高电平宽度（μs）	I2Cx_SS_SCL_HCNT（Cycle）
100	100	4	392

表2-34 I2Cx_SS_SCL_LCNT 典型配置值

I2C 总线速率（kbps）	I2C 工作时钟（MHz）	SCL 低电平宽度（μs）	I2Cx_SS_SCL_LCNT（Cycle）
100	135	4.7	469

表2-35 I2Cx_FS_SCL_HCNT 典型配置值

I2C 总线速率（kbps）	I2C 工作时钟（MHz）	SCL 高电平宽度（μs）	I2Cx_FS_SCL_HCNT（Cycle）
400	100	0.6	52

表2-36 I2Cx_FS_SCL_LCNT 典型配置值

I2C 总线速率（kbps）	I2C 工作时钟（MHz）	SCL 低电平宽度（μs）	I2Cx_FS_SCL_LCNT（Cycle）
400	100	1.3	129

2.19.6.3 I2Cx_DELAY

I2C 控制器额外具有 I2Cx_DELAY 功能，该功能用以将 I2Cx_SDA 信号输出延迟 300ns，以满足某些特定的时序需要。

I2Cx_DELAY 模块可单独使能，不使能时该功能被旁路。通过配置为 1，可实现对 I2C0 的 I2Cx_DELAY 功能的旁路。

2.19.6.4 中断或查询方式下的数据传输

说明

I2Cx 表示 I2C0 或 I2C1。

2.19.6.4.1 初始化

具体配置步骤如下：

步骤 1 设定超时标志，检测 I2Cx_STATUS bit[0]（判断 I2C 控制器是否空闲），如果是 1，则延时等待，继续检测，直到 I2Cx_STATUS[0]变为 0 或者超时，向 I2Cx_ENABLE bit[0]写 0，使 I2C 处于关闭状态。

步骤 2 写相应的配置值到 I2Cx_CON，配置传输速率模式等参数。

步骤 3 将对接器件的从设备地址写入 I2Cx_TAR[9:0]。

步骤 4 配置 I2Cx_SS_SCL_HCNT、I2Cx_SS_SCL_LCNT 等寄存器，设定相应的 I2C 总线时钟周期。

步骤 5 配置 I2Cx_RX_TL、I2Cx_TX_TL，设定相应的发送及接收 FIFO 阈值。

步骤 6 如果驱动程序采用中断方式则需设定 I2Cx_INTR_MASK，使能相应中断信号；采用查询方式时应禁止产生相应中断信号。

步骤 7 向 I2Cx_ENABLE bit[0]写 1，使能 I2C，完成初始化配置。

----结束

2.19.6.4.2 数据发送

具体配置步骤如下：

步骤 1 将发送数据写入 I2Cx_DATA_CMD，启动数据发送。

步骤 2 查询方式下，进行连续数据发送时通过读取 I2Cx_STATUS 和 I2Cx_TXFLR 检测发送 FIFO 状态；中断方式下，根据相应中断状态位检测。在数据传送完成之前，既要保证

发送 FIFO 中的数据没有溢出（否则会造成数据丢失），同时也要保证发送 FIFO 中始终有数据（否则 I2C 会认为该次传输已全部完成而发出结束信号）。

步骤 3 通过检测 I2Cx_STATUS bit[2]是否为 1，判断 I2C 是否完成全部数据发送。

----结束

2.19.6.4.3 数据接收

具体配置步骤如下：

步骤 1 将读命令（0x100）写入 I2Cx_DATA_CMD，启动数据接收。

步骤 2 进行连续数据接收时，首先要发送次数与接收数据数目相同的读命令（0x100）到 I2Cx_DATA_CMD（例如要接收 3 个数据，则需发送 3 次读命令到 I2Cx_DATA_CMD 寄存器）。在数据接收完成之前，既要保证发送 FIFO 中的数据（即读命令 0x100）没有溢出，又保证发送 FIFO 非空（否则 I2C 会认为没有新的数据需要接收而结束），另外在连续数据接收过程中还需要检测接收 FIFO 的状态（如果采用中断方式，则根据相应中断状态位检测），避免接收 FIFO 溢出。

步骤 3 通过检测 I2Cx_STATUS bit[4]是否为 1，判断 I2C 是否完成全部数据接收。

----结束

2.19.6.5 DMA 方式下的数据传输

说明

I2Cx 表示 I2C0 或 I2C1。

2.19.6.5.1 初始化

具体配置步骤如下：

步骤 1 设定超时标志，检测 I2Cx_STATUS[0]（判断 I2C 控制器是否空闲），如果是 1，则延时等待，继续检测，直到 I2Cx_STATUS bit[0]变为 0 或者超时，然后向 I2Cx_ENABLE bit[0]写 0，使 I2C 处于关闭状态。

步骤 2 写相应的配置值到 I2Cx_CON，配置主模式以及传输速率模式等参数。

步骤 3 将对接器件的从设备地址写入 I2Cx_TAR bit[9:0]。

步骤 4 配置 I2Cx_SS_SCL_HCNT、I2Cx_SS_SCL_LCNT 等寄存器，设定相应的 I2C 总线时钟周期。

步骤 5 配置 I2Cx_DMA_TDLR、I2Cx_DMA_RDLR，设置 DMA 发送 FIFO 及接收 FIFO 阈值。

步骤 6 向 I2Cx_ENABLE[0]写 1，使能 I2C。

----结束

2.19.6.5.2 数据发送

具体配置步骤如下：

步骤 1 配置 DMA 数据通道，包括数据传输源和目的地址、数据传输个数、传输类型等参数。

步骤 2 向 I2Cx_DMA_CR bit[1]写 0，使能 I2C 的 DMA 发送功能。

步骤 3 读 I2Cx_INTR_STAT 寄存器，查询中断状态，判断数据是否发送完成。如果完成则向 I2Cx_DMA_CR bit[1]写 0，禁止 I2C 的 DMA 发送功能。

----结束

2.19.6.5.3 数据接收

数据接收包括数据发送和数据接收两步，首先将读命令发送给 I2C 控制器，然后从接收 FIFO 中读取接收数据，具体配置步骤如下：

步骤 1 配置 DMA 数据通道，包括数据传输源（传输源内容包括读命令 0x100 等）和目的地址、数据接收区地址、数据传输个数、传输类型等参数。

步骤 2 配置 I2Cx_DMA_CR 为 0x3，使能 I2C 的 DMA 发送和接收功能（因数据接收时要先发送 0x100 到 I2Cx_DATA_CMD 寄存器，所以要使能 DMA 发送功能）。

步骤 3 读 I2Cx_INTR_STAT 寄存器，查询中断状态，判断数据是否接收完成。如果完成则向 I2Cx_DMA_CR bit[1]写 0，禁止 I2C 的 DMA 发送功能。

----结束

2.20 GPIO 接口（GPIO）

2.20.1 概述

GPIO（General Purpose Input/Output）为通用输入输出接口。Hip04 共有 4 组 GPIO 接口：GPIO0、GPIO1、GPIO2 和 GPIO3。每组分别提供 32 个可编程的输入输出管脚，除了 GPIO0 的部分管脚，其他 GPIO 均和功能管脚复用。

每个管脚可以配置为输入或者输出。这些管脚用于生成或采集特定应用的输出或输入信号。作为输入管脚时，可作用为中断源；作为输出管脚时，每个 GPIO 都可以独立地清 0 或置 1。

2.20.2 特性

GPIO 具有以下特点：

- 提供四组 GPIO 接口，每组 GPIO 接口有独立的地址和中断线；
- 支持对串行及低于 32 bits 的并行数据总线进行模拟；
- 每根信号有各自的数据寄存器和数据流方向寄存器；
- 上电复位时默认为输入模式；
- 支持初始中断状态查询和屏蔽后中断状态查询；
- 当有多个中断同时发生的时候，将会统一汇集成一个中断进行上；
- GPIO_INTTYPE_LEVEL（中断类型寄存器）和 GPIO_INT_POLARITY（中断触发极性控制寄存器）两个寄存器共同决定了中断源的特性和中断触发类别；
- 当 GPIO 作为输入管脚时，可作为中断源，每个 GPIO 管脚都具有独立的中断控制；
- 当 GPIO 作为输出管脚时，每个 GPIO 管脚支持独立使能或关闭；
- 当 GPIO 作为输入管脚时，GPIO 支持滤毛刺功能，可滤除宽度小于晶振时钟 64 分频（晶振时钟/64）的脉冲。

2.20.3 关键术语

表2-37 GPIO 关键术语描述

术语	描述
GPIO	General Purpose Input/Output，通用输入输出接口。当微控制器或芯片组没有足够的 I/O 端口，或当系统需要采用远端串行通信或控制时，GPIO 产品能够提供额外的控制和监视功能。

2.20.4 功能描述

2.20.4.1 应用框图

GPIO 应用框图如图 2-39 所示。

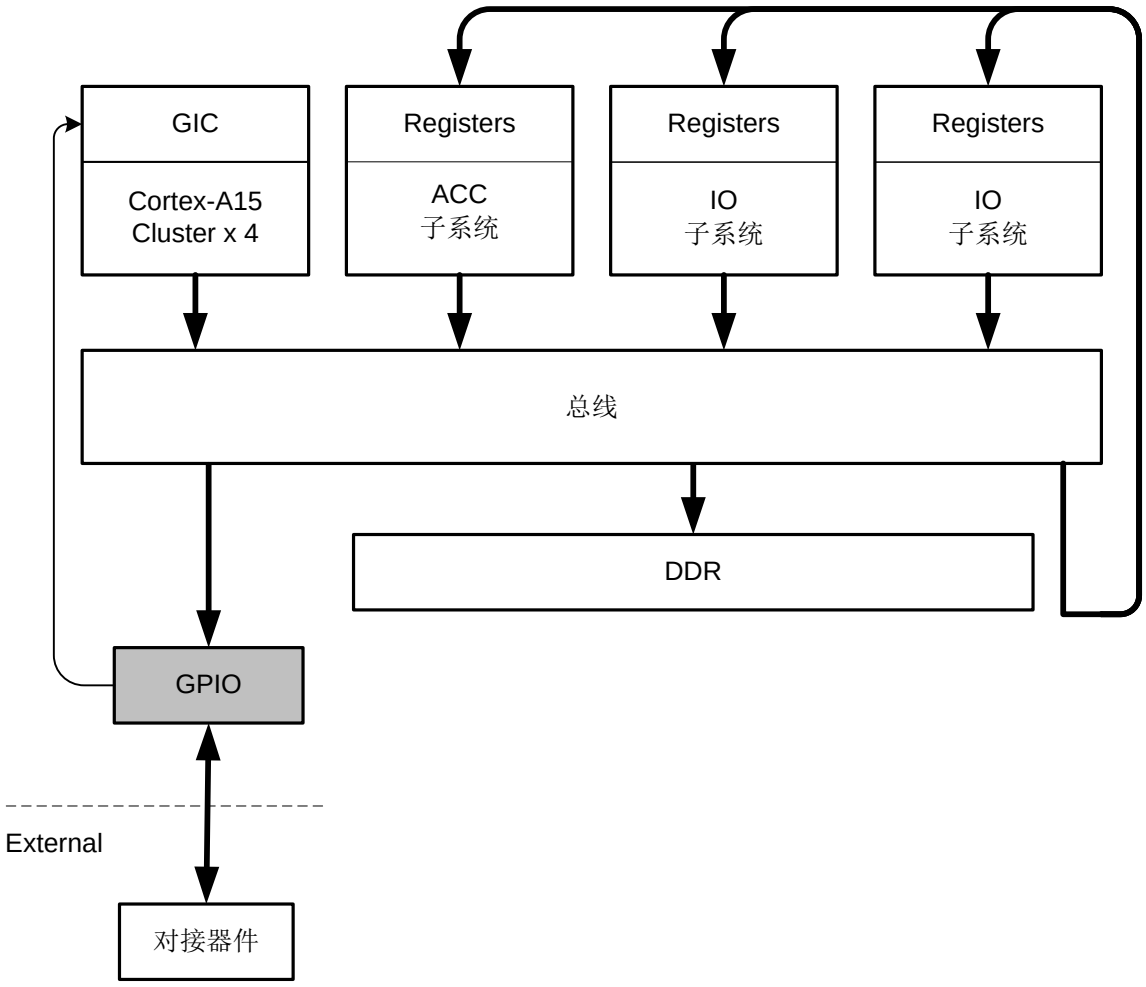


图2-39 GPIO 应用框图

GPIO 通用输入输出接口。当芯片没有足够的 I/O 端口，或当系统需要采用远端串行通信或控制时，CPU 可配置 GPIO 提供额外的控制和监视功能。

2.20.4.2 相关模块

在系统中，GPIO 模块会和下面多个模块进行交互，分别描述如下：

- CPU ：CPU 通过总线访问 GPIO 的寄存器接口，操作属性必须为 Device 或 Strongly-Ordered。CPU 首先需要配置 GPIO 的输入或输出特性，如果是输入，则给出输出的数据，如果是输入，则读取输入数据。
- GIC ：GPIO 可配置为中断信号，在对外信号发送电平或脉冲变化时，将触发 GPIO 中断产生中断信号给 GIC 模块，通过 GIC 将中断发送给对 GPIO 具有控制权的 CPU。

2.20.4.3 功能原理

每组 GPIO 提供 32 个可编程的输入输出管脚。每个管脚可以配置为输入或者输出。这些管脚用于生成或采集特定应用的输出或输入信号。

作为输入管脚时，可作为中断源；作为输出管脚时，每个 GPIO 都可以独立地清 0 或置 1。GPIO 可以根据电平或跳变值产生可屏蔽的中断。GPIOINTR 信号给中断控制器一个指示，表示有中断发生。用户可以配置中断为电平触发或边沿触发。

作为输入管脚时，GPIO 还支持滤毛刺功能，在芯片使用 50MHz 晶振的情况下，可滤除脉冲宽度小于 1μs 的毛刺。

2.20.4.4 GPIO 中断

GPIO 的编号和中断号如表 2-38。

表2-38 GPIO 编号和中断号

GPIO Macro	GPIO 编号	Combo 中断号
GPI00	GPI00~GPI031	421
GPI01	GPI032~GPI063	422
GPI02	GPI064~GPI095	423
GPI03	GPI096~GPI0113	424

其中 GPIO16~GPIO31 与外部中断输入信号复用，可以从这 16 个外部中断中选择 2 个作为 Lock Down SPI 中断（选择信号来自系统控制器），对应的中断号是 32 和 33；可以从这 16 个外部中断中选择 4 个作为 N-N 中断（选择信号来自系统控制器），对应的中断号是 480~483。

2.20.5 工作方式

2.20.5.1 初始复位

说明

GPIOx 表示 GPIO0、GPIO1、GPIO2 或 GPIO3。

上电复位时，所有管脚默认为输入。

复位信号有效时，GPIO 有如下反应：

- 所有的寄存器被清零
- 所有管脚都被配置为输入
- 原始中断都被清零
- 清除寄存器 GPIOx_INTEN 中相应的 bit，使中断无效
- 中断被设为电平触发的中断

2.20.5.2 输入输出配置

说明

GPIOx 表示 GPIO0、GPIO1、GPIO2 或 GPIO3。

GPIO 的软件控制工作模式即为通用输入输出模式，在该模式下，数据的输入输出方向和输出数据均由软件控制，管脚的输入信号也通过软件读取 GPIOx_EXT_PORT bit[7:0]来获得。

配置步骤如下：

步骤 1 选择要使用的 GPIO 管脚，配置 IOCFG 的相应的管脚复用控制寄存器，选择 GPIO 管脚。

步骤 2 配置寄存器 GPIOx_SWPORT_DDR 设置 GPIO 管脚方向选择 GPIO 是输入还是输出。

- 当 GPIO 用于输入时，外部信号通过 GPIO 管脚送进来，可通过寄存器 GPIOx_EXT_PORT 查看输入信号值。
- 当 GPIO 用于输出时，通过写寄存器 GPIOx_SWPORT_DR，GPIO 会将写进此寄存器的值通过 GPIO 输出。需要注意的是，此时若 GPIO 中断处于使能状态，则当输出信号满足触发条件时会产生中断。

----结束

2.20.5.3 中断处理

说明

GPIOx 表示 GPIO0、GPIO1、GPIO2 或 GPIO3。

注意

- 每个 GPIO 的中断设置 32 个寄存器控制。当有一个或多个 GPIO 管脚产生了中断，一个中断输出就会送到中断控制器。对于边沿触发的中断，软件必须清除该中断以使能更深的中断；对于电平触发的中断，外部中断源应该保持该电平直到处理器识别到该中断。
- 设置 GPIO 中断为电平触发时，如果设置该中断为异步中断，则该中断仍然可以在 GPIO 时钟关闭时传递中断到 VIC，如果设置该中断为同步中断，则该中断在 GPIO 时钟关闭时无法传递到 VIC。

配置步骤如下：

步骤 1 选择要使用的 GPIO 管脚，配置该管脚为输入（配置方法请参见 GPIOx_SWPORT_DDR）。

步骤 2 配置 GPIOx_INTTYPE_LEVEL 对应 bit，选择边沿触发或电平触发。

步骤 3 如果选择电平触发，配置 GPIOx_INT_POLARITY 对应 bit，选择高电平或低电平触发。否则跳过此步。

步骤 4 如果选择电平触发，配置 GPIOx_LS_SYNC（中断同步使能寄存器）对应 bit，选择该中断与总线时钟同步或异步。否则跳过此步。

步骤 5 如果选择了边沿触发，配置 GPIOx_INT_POLARITY（中断触发极性控制寄存器）对应 bit，选择上升沿或下降沿触发。否则跳过此步。

步骤 6 保证 GPIO 数据线在这些操作的过程中保持稳定。

步骤 7 通过往寄存器 GPIOx_PORTA_EOI（中断清除寄存器）对应 bit 写 1，清中断。

步骤 8 配置 GPIOx_INMASK（中断屏蔽寄存器），打开对应 bit 的中断屏蔽

步骤 9 配置 GPIOx_INTEN（中断使能寄存器）使能该 bit 中断。

步骤 10 发生中断时，查询 GPIOx_INTSTATUS（屏蔽后中断状态寄存器），获取中断发生的对应 bit。

步骤 11 中断处理完成。往寄存器 GPIOx_PORTA_EOI（中断清除寄存器）对应 bit 写 1，清除中断。

----结束

2.20.5.4 毛刺滤除

说明

GPIOx 表示 GPIO0、GPIO1、GPIO2 或 GPIO3。

配置步骤如下：

步骤 1 选择要使用的 GPIO 管脚，并配置为输入（配置方法请参见 GPIO0_SWPORT_DDR）。

步骤 2 配置 GPIOx_DEBOUNCE 对应 bit 为 1，使能毛刺滤除功能。

----结束

2.21 JTAG 接口（JTAG）

2.21.1 概述

JTAG 模块用来辅助生产测试、协助硬件调试和程序调试等等。包括：在芯片制造阶段后期配合 ATE 等设备对芯片本身进行制造测试，排除诸如 RAM 制造失效等问题；在单板加工完成阶段，通过 JTAG Studio 对单板互联和设备基本功能进行测试，排除单板加工中的焊接错误和设备失效；在软件控制下使 ARM 进入 DEBUG 模式进行程序调试。

2.21.2 特性

JTAG 具有以下特性：

- 支持 IEEE1149.6 标准；
- 兼容 1149.1 标准；
- 支持芯片边界扫描和 CPU Debug 调试，功能选择由管脚控制；
- 芯片边界扫描支持标准 BSDL 标准，提供 ECID_ACCESS、PRELOAD/SAMPLE、EXTEST, BYPASS、EXTEST_PULSE、EXTEST_TRAIN、HIGHZ、CLAMP 和 IDCODE 等标准指令；
- 芯片边界扫描支持特定的 DFT 调试指令，可用作部分功能调试使用；
- CPU Debug 调试支持 ARM 特有 JTAG 指令。

3 存储与中断映射

3.1 存储映射

3.1.1 概述

Hip04 内部使用的总线寻址模式为 40 bits，可寻址的地址空间为 1TB。

Hip04 使用的 Cortex A15 是 32 bits 寻址 CPU，其 32->40 bits 地址扩展是基于 MMU 页表扩展的，详细内容详见 ARM 的文档《Large Physical Address Extensions Specification》。为保证 MMU 未使能前系统可用，关键组件在初始化时地址映射集中在 32 bits 译码的 4GB 空间内。

注意

在 Cortex A15 的 MMU 中，支持 2 级页表转换，其中操作系统使用的是 VA（Virtual Address），VA 通过一级页表翻译为临时的 IPA（Intermediate Physical Address），然后 IPA 通过二级页表翻译为最终的 PA（Physical Address）。这里的存储映射是指最终的 PA 地址。

在外设的 SMMU 中，仅支持 1 级页表转换，其中处理的是 IPA（Intermediate Physical Address）对 PA （Physical Address）的转换，因此 CPU 配置外设寄存器时，需要使用 IPA 进行配置，外设发出的 IPA 再由 SMMU 转换为 PA。

CPU MMU 翻译后的 PA 地址与外设 SMMU 翻译后的 PA 地址应该是一一对应完全相同的，即为本章节描述的存储地址。

3.1.2 特性

存储映射主要特性如下：

- Hip04 内部使用的总线寻址模式为 40 bits，可寻址的地址空间为 1TB；
- Hip04 支持以下 4 种 Boot 方式，不同的 Boot 模式下，0x00000000 地址被默认重映射（Remap）到相应的启动空间：
  - 使用 Local Bus 外接的 NOR FLASH Boot；

- 使用 NANDC 外接的 NAND FLASH Boot；
- 使用 SFC 外接的 SPI FLASH Boot；
- 重映射（Remap）地址可独立或统一撤离，通过寄存器配置，重映射（Remap）的地址可任意配置，主核可配置从核从任意地址启动；

3.1.3 关键术语

如表 3-1 的术语将在本章节表示特殊的含义。

表3-1 Memory Map 关键术语描述

术语	描述
PeriSub	Peripheral Subsys，一系列外设和小存储空间的总和，并包括了系统控制及 DAW 寄存器配置本身，因为这些设备和存储占用的空间均较小，应用单纯，所以归为一类，统一处理。
DID	Destination ID，存储及外设设备，在系统中的目的 ID 分配。
Remap	地址重映射，CPU 复位时，默认从 0x00_0000_0000 地址取第一条指令，而该地址通常是 DDR 空间，为保证 CPU 能够正确从指定的 FLASH 空间取指，系统会设定 0x00_0000_0000 上电默认被重映射到 FLASH 空间地址，通过寄存器配置撤离。

3.1.3.2 Boot 模式

Hip04 支持从外接的 NOR FLASH、NAND FLASH、SPI FLASH、PCIe、sRIO 启动，也支持从片上 Boot ROM 启动。

此时需要设置芯片外部管脚 BOOT_SEL0~2 和 BOOT_CFG0~4，用于选择 Boot 的存储器，真值表如表 3-2 所示。

表3-2 Boot 模式真值表

BOOT SEL2	BOOT SEL1	BOOT SEL0	启动模式	启动位宽	地址重映射
0	0	0	内部 ROM 启动	32	
0	0	1	NOR FLASH 启动	16	
0	1	0	NAND FLASH 启动	8	
0	1	1	SPI FLASH 启动	8	
1	0	0	PCIe/sRIO 启动	-	

表3-3 Nandc Boot 模式真值表

boot_cfg4～0	page size	ecc_type	block_size	addr_num	bus_wide	启动模式
00000	000	001	00	0	0	512Byte page size,1bit ecc,32page/block，4 个地址周期
00001	001	001	00	1	0	2k page size,1bit ecc,64page/block, 5 个地址周期
00010	011	101	10	1	0	8k page size,40bit ecc,256page/block
00011	001	010	00	1	0	2k page size,4bit ecc,64page/block，5 个地址周期
00100	010	100	10	1	0	4k page size,24bit ecc,256page/block
00101	001	100	00	1	0	2k page size,24bit ecc,64page/block
00110	001	001	00	0	0	2k page size,1bit ecc,64page/block，4 个地址周期
00111	011	100	10	1	0	8k page size,24bit ecc,256page/block
01000	010	010	01	1	0	4k page size,4bit ecc,128page/block
01001	010	010	00	1	0	4k page size,4bit ecc,64page/block
01010	001	010	00	0	0	2k page size,4bit ecc,64page/block，4 个地址周期
01011	010	100	01	1	0	4k page size,24bit ecc,128page/block
01100	100	101	01	1	0	16k page size,40bit ecc,128page/block
01101	011	100	01	1	0	8k page size,24bit ecc,128page/block
01110	011	101	00	1	0	8k page size,40bit ecc,64page/block

01111	100	101	11	1	0	16k page size,40bit ecc,512page/block
10000	011	100	00	1	0	8k page size,24bit ecc,64page/block
10001	010	100	00	1	0	4k page size,24bit ecc,64page/block
10010	011	000	01	1	0	clear nand,8k page size,128 page/block
10011	010	001	00	1	0	4k page size,1bit ecc,64page/block
10100	100	101	10	1	0	16k page size,40bit ecc,256page/block
10101	001	010	01	1	0	2k page size,4bit ecc,128page/block
10110	001	110	01	1	0	2k page size,8bit ecc,128page/block
10111	100	000	01	1	0	clear nand,16k page size,128 page/block
11000	100	000	10	1	0	clear nand,16k page size,256 page/block
11001	001	100	01	1	0	2k page size,24bit ecc,128page/block
11010	001	110	00	1	0	2k page size,8bit ecc,64page/block
11011	011	000	10	1	0	clear nand,8k page size,256 page/block
11100	100	000	11	1	0	clear nand,16k page size,512 page/block
11101	011	101	01	1	0	8k page size,40bit ecc,128page/block
11110	010	110	00	1	0	4k page size,8bit ecc,64page/block
11111	010	110	01	1	0	4k page size,8bit ecc,128page/block

表3-4 Nor Boot 模式真值表

BOOT CFG1	BOOT CFG0	地址偏移	启动位宽	地址重映射
0	0	地址不偏移	8	
0	1	地址偏移	8	
1	0	地址不偏移	16	
1	1	地址偏移	16	

3.2 中断映射

3.2.1 概述

Hip04 使用统一的 GIC 来管理芯片上外设和芯片外中断的使能、分发和优先级排序。GIC 的控制寄存器空间是 Memory-Map 的，在系统 PeriSub Map 中，可被任意 CPU 或 IO 访问，GIC 的具体配置详见其模块详述章节。

Hip04 支持 SGI、PPI、SPI 三种模式中断，本章节主要描述各种中断在系统的分配和应用。

3.2.2 特性

3.2.2.1 Software Generated Interrupt (SGI)

软件中断由软件写 GIC 内软中断寄存器产生，可以分配到多个 CPU Core，可以利用 SGI 进行核间通信。

3.2.2.2 Private Peripheral Interrupt (PPI)

私有外设中断来自于 CPU 内部的功能模块。包括一个 Hypervisor Timer、一个 Virtual Timer、一个 Secure Physical Timer、一个 Non-secure Physical Timer，及特定低功耗的 FIQ 和 IRQ 应用，如表 3-5 所示。

表3-5 PPI 中断映射

PPI 编号	中断 ID 号	中断源	描述
PPI0	ID28	Legacy nFIQ	CPU nFIQ 中断，用以低功耗睡眠唤醒。
PPI1	ID29	Secure Physical Timer	CPU 内安全物理 Timer 中断。
PPI2	ID30	Non-secure Physical Timer	CPU 内非安全物理 Timer 中断。
PPI3	ID31	Legacy nIRQ	CPU nIRQ 中断，用以低功耗睡眠唤醒。
PPI4	ID27	Virtual Timer	CPU 内虚拟 Timer 中断。
PPI5	ID26	Hypervisor Timer	CPU 内 Hypervisor Timer 中断。
PPI6	ID25	Virtual Maintenance Interrupt	Hypervisor CPU 控制产生的虚拟化维护中断。

3.2.2.3 Shared Peripheral Interrupt (SPI)

说明

在上一版多核处理器中，描述 SPI 中断是单独作为一个编号开始，在使用中断向量时，还需要在提供的 SPI 中断号基础上增加 32 后访问。在 HIP04 中，按照 ARM GIC 架构文档的方式描述，在处理中断时，可以直接按照中断向量表中的编号进行中断处理。

共享外设中断来源于片上外设和芯片外部中断输入。Hip04 共支持 480 个 SPI 中断源，详见下节的中断向量表。SPI 的中断类型可以分为 1-N 和 N-N 模式，每个 N-N 模式的 SPI 中断可配置为高电平或上升沿触发，N-N 模式的 SPI 中断只能是上升沿触发，具体根据外设的输出模式配置。SPI 的中断号从 ID32 开始。

3.2.3 关键术语

如表 3-6 的术语将在本章节表示特殊的含义。

表3-6 中断映射关键术语描述

术语	描述
SGI	Software Generated Interrupt，软件写 GIC 内寄存器产生的软中断，主要用于核间通信。
PPI	Private Peripheral Interrupt，私有外设中断，来自于 CPU 内部独享的功能模块。
SPI	Shared Peripheral Interrupt，共享外设中断，来自于片内外设和片外中断输入。

3.2.4 中断向量表

Hip04 中断基于功能进行分类，分类如表 3-7 所示。

表3-7 Hip04 中断分类表

中断编号	中断功能分类
0～15	SGI。
16～31	PPI。
32~63	Lock Down SPI 中断
64~479	普通 SPI 中断
480~511	N-N 模型 SPI 中断

Hip04 详细中断向量表如表 3-8 所示。

表3-8 Hip04 详细中断向量表

中断编号	中断源	描述
0～15	SGI x16	16 个 SGI 软件中断，用作 CPU 核间交互。
16～24	-	保留。
25	PPI6	Virtual Maintenance Interrupt。
26	PPI5	Hypervisor Timer。
27	PPI4	Virtual Timer。
28	PPI0	Legacy nFIQ。
29	PPI1	Secure Physical Timer。
30	PPI2	Non-secure Physical Timer。
31	PPI3	Legacy nIRQ。
32	Lock Down SPI0	Lock Down SPI 0，可以用于板级输入的非可屏蔽中断，中断源从板上输入的 16 个中断源中选择，由系统控制器中的域段控制选择。
33	Lock Down SPI1	Lock Down SPI 0，可以用于板级输入的非可屏蔽中断，中断源从板上输入的 16 个中断源中选择，由系统控制器中的域段控制选择。
34~63	-	保留。
64~79	CPU CommTx x16	CPU Core 0~15 各自 Debug 的 Angel 接收信息中断，正常使用时保留。
80~95	CPU CommRx x16	CPU Core 0~15 各自 Debug 的 Angel 发送信息中断，正常使用时保留。
96~111	CPU PMU x16	CPU Core 0~15 各自的 PMU 统计计数中断。
112~127	CPU CTI x16	CPU Core 0~15 各自的 CTI(Cross Trigger Interface)中断。
128~143	WatchDog x16	WatchDog 0~15 对 CPU Core 0~15 各自计数溢出中断（系统提供 16 个 WatchDog，每个核一个）。
144~255	-	保留
256~279	Timer INT x24	片上 Timer 中断
280~295	Externel INT X16	外部输入中断，和 GPIO 复用
296	1PPS Timestamp INT	来自网口秒中断

中断编号	中断源	描述
297	Power Managemant INT	来自功耗管理模块的中断
298	PM Bus INT	功耗管理总线中断
299~306	L3 Cache INT	L3 Cache 各 Bank 中断
307	IO SMMU NS INT	IO 子系统 SMMU 非安全中断
308	IO SMMU S INT	IO 子系统 SMMU 安全中断
309	NP SMMU NS INT	NP 子系统 SMMU 非安全中断
310	NP SMMU S INT	NP 子系统 SMMU 安全中断
311	ETH SMMU NS INT	ETH 子系统 SMMU 非安全中断
312	ETH SMMU S INT	ETH 子系统 SMMU 安全中断
313	SDS SMMU NS INT	Serdes 子系统 SMMU 非安全中断
314	SDS SMMU S INT	Serdes 子系统 SMMU 安全中断
315	PTS SMMU NS INT	PTS 子系统 SMMU 非安全中断
316	PTS SMMU S INT	PTS子系统SMMU安全中断
317	ALG SMMU NS INT	ALG 子系统 SMMU 非安全中断
318	ALG SMMU S INT	ALG子系统SMMU安全中断
319	-	保留
320~323	CPU AXI INT x4	CPU Cluseter 0~3 各自的总线接口访问错误中断。
324~327	CPU INT ERR x4	CPU Cluser 0~3 各自的内部错误中断，包括 L2 Cache ECC 2bits 错误等。
328~337	-	保留
338	CPU Clock Switch INT	CPU 频率切换异常中断
339~346	-	保留
347~354	GE Debug INT x8	GMAC0~GMAC7 中断
355~360	-	保留
361	SYNC ETH0 INT	同步以太网逻辑 0 中断
362	SYNC ETH1 INT	同步以太网逻辑 1 中断

中断编号	中断源	描述
363~369	PCIe0 INT x7	PCIe 接口 0 中断
370~376	PCIe1 INT x7	PCIe 接口 1 中断
377~383	PCIe2 INT x7	PCIe 接口 2 中断
384~400	-	保留
401	USB Host EHCI INT	USB Host EHCI INT
402	USB Host OHCI INT	USB Host OHCI INT
403	USB Device INT	USB Device 中断
404	SATA INT	SATA 接口中断
405	DDR0 INT	DDR 接口 0 中断
406	DDR1 INT	DDR 接口 1 中断
407~408	-	保留
409	MMC INT	MMC 接口中断
410	SPI Flash INT	SPI Flash 接口中断
411	NAND Flash INT	NAND Flash 接口中断
412	Local Bus INT	Local Bus 接口中断。
413~415	UART INT x3	UART 接口中断
416~417	I2C INT x2	I2C 接口中断
418~419	MDIO INT x2	MDIO 控制的网口 PHY 链路状态发生变化中断。
420	SPI INT	SPI 的 Comb 中断。
421~425	GPIO INT x5	GPIO 0~4 Macro 各自的 Comb 中断。 其中 GPIO_INT0 对应于 GPIO0~31； 其中 GPIO_INT1 对应于 GPIO32~63； 其中 GPIO_INT2 对应于 GPIO64~95； 其中 GPIO_INT3 对应于 GPIO96~127； 其中GPIO_INT4对应于GPIO128~159；
426~479	-	保留
480	N-N Model INT0	外部输入 N-N 模型中断 0，由系统控制器中 SC_GIC_CTRL 寄存器中的域段选择片外中断源

中断编号	中断源	描述
481	N-N Model INT1	外部输入 N-N 模型中断 1，由系统控制器中 SC_GIC_CTRL 寄存器中的域段选择片外中断源
482	N-N Model INT2	外部输入N-N模型中断2，由系统控制器中 SC_GIC_CTRL寄存器中的域段选择片外中断源
483	N-N Model INT3	外部输入 N-N 模型中断 3，由系统控制器中 SC_GIC_CTRL 寄存器中的域段选择片外中断源
484~511	-	保留

3.2.5 安全扩展

Hip04 支持对中断进行安全管理：

- 中断进入 GIC 后，将被区分为两个 Group，Group0 为安全中断，Group1 为非安全中断；
- Group0 只能触发 FIQ，FIQ 只会送到 CPU Core 0，触发 CPU Core 0 进入安全模式处理；
- Group1 只能触发 IRQ，IRQ 可发送给任意 CPU Core，如果是虚拟化系统，IRQ 可发送给 Hypervisor 的中断分发 CPU Core，然后再产生 VIRQ 给特定 CPU Core；
- 中断的 Group 划分，只能被安全模式下的 CPU Core0 配置。

3.2.6 1-N 中断分发

Hip04 支持中断 1-N 分发：

- 一个中断可分配给多个 CPU Core；
- 当中断发生时，同时上报到多个 CPU Core；
- 多个 CPU Core 都可以识别到这个中断，但只有一个 CPU Core 能够获得中断处理权限（最先获取的一个）并处理中断，其他 CPU Core 并不会获得中断任务。

3.2.7 N-N 中断分发

Hip04 支持中断 N-N 分发：

- 一个中断分配给多个 CPU Core；
- 当中断发生时，同时上报到多个 CPU Core；
- 多个 CPU Core 都可以识别到这个中断，并且每个 CPU Core 都各自独立处理中断，互不影响；
- 对应 ID 480~ID 511 为 N-N 的 SPI 中断。

3.2.8 使用 SGI 实现核间消息通信

软件可写中断分发器寄存器 ICDSGIR 向其他 CPU Core 发送 SGI 软件中断，软件中断可分发到多个 CPU Core，并可配置如下分发模式：

- 支持基于 16 bits 的 TargetList（即 CPU Core 掩码）对多个 CPU Cores 发送中断；
- 支持对本核发送中断；
- 支持对本核外各个核发送中断。

SGI 中断处理为 N-N 模式，每个核都需要独立响应并处理，并独立清除。

通过 SGI 可以实现如下两种核间消息通信：

- 每个 CPU Core 可产生 ID0-ID15 共 16 个软件中断，可分配到多个 CPU Core。可利用软件中断进行 CPU Core 核间通信，利用 ID 号共可传递 16 种既定信息，在核间通信不多的应用场景里，可利用此特性代替系统的 Mailbox；
- 每个 CPU Core 可固定某个 ID，基于该 ID 实现对其他核的中断广播，以此实现 16x16 的核间软件中断矩阵。

3.2.9 中断优先级

Hip04 支持每个中断的优先级设置，可配置 32 级优先级。利用中断优先级设置，可支持中断嵌套。

中断优先级数值越小，优先级越高。当两个中断具有同级优先级，以 ID 小者优先。

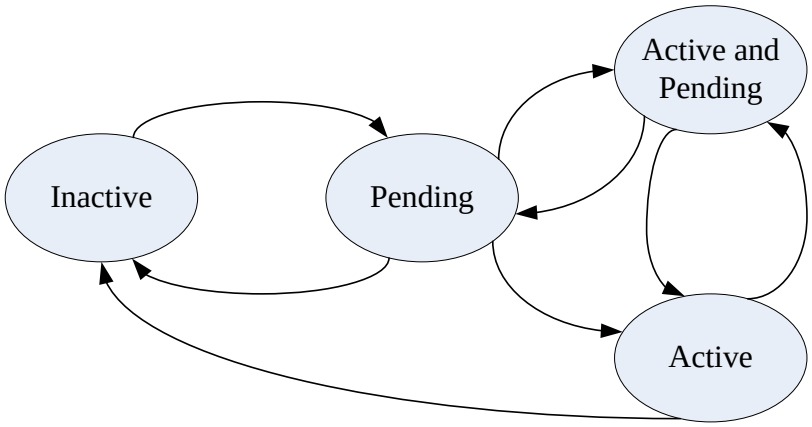
支持将比目前正在响应的中断优先级高的中断发送到 CPU Core，即支持中断嵌套。

支持中断优先级屏蔽，在 CPU 接口内可配置一个中断优先级阈值，若中断分发器发送给 CPU 接口的中断高于这个阈值，CPU 就把这个中断上报给 CPU Core，否则就屏蔽这个中断。

3.2.10 中断状态

每个中断有 Inactive、Pending、Active、Active and Pending 的四种状态，如图 3-1 所示。

图3-1 中断状态转换图



上图说明了中断的状态转换，A-E 表示中断的转换路径。

1. 当中断没有发生时，处于 Inactive 状态；
2. 中断发生后，中断控制器将中断上报给 CPU Core，在被响应之前处于 Pending 状态；
3. CPU 响应读取中断号，表示已响应中断，中断状态转换为 Active；
4. CPU 处理完中断后，回写 EOIR，通知此中断已响应完毕，中断状态转换为 Inactive；
5. 如果 CPU Core 正在处理中断，中断再次发生。中断进入 Active and Pending 状态。
6. 如果中断处于 Pending 状态，CPU 读取中断号的同时，中断再次发生，中断进入 Active and Pending 状态。

3.2.11 特殊中断号

每个中断都有唯一的中断号。CPU Core 利用此中断号识别中断源，进入相对应的中断服务程序。

- SGI 的中断号为 ID0-ID15；
- PPI 的中断号为 ID16-ID31；
- SPI 的中断号为 ID32-ID479；

除了以上中断号外，还有两个特殊的中断号：1022 和 1023，这两个特殊中断号称作超级中断。

中断控制器在下列情况下返回 1022 中断号:

- CPU 在安全模式下读取中断号；
- 最高优先级的未决中断为非安全中断；
- GIC 的 ICCICR 寄存器的 AckCtl 位设置为 0；
- 中断优先级足够高，可以上报给 CPU Core。

注意

1022 号中断通知安全模式运行的软件有个优先级足够高的非安全中断上报给了 CPU Core，并且这个中断必须软件在非安全模式下处理。在这种情况下，软件可直接切换到非安全模式调用中断处理程序，以减小中断响应延时。

而当没有优先级足够高的未处理中断时，CPU Core 读取中断号，返回 1023 中断号。

3.2.12 电平触发和沿触发

Hip04 中断配置为高电平触发或者是上升沿触发，上电默认为高电平触发。

片上外设均为高电平触发，可以保持默认配置。而芯片的外部中断 EXTINT 以 ID352-ID359 接入 GIC。GIC 可以配置其为高电平中断或上升沿中断，此外，在系统寄存器内可以配置将外部中断源 EXTINT 作取反操作，即 EXTINT 对四种中断形式均支持。

4 时钟与复位

4.1 时钟

4.1.1 概述

Hip04 时钟管理模块支持对芯片时钟输入、时钟生成和控制进行统一的管理。

4.1.2 特性

时钟管理模块对芯片时钟输入、时钟生成和控制进行统一的管理，包括：

- 各 PLL 控制
- 时钟分频和控制
- 生成各模块的工作时钟
- 时钟门控

4.1.3 关键术语

如表 4-1 的术语将在本章节表示特殊的含义。

表4-1 关键术语描述

术语	描述
Ordered Flow	It is a series of packets; these packets must be transmitted as received order. An ordered flow can be a series of packets that are in a same channel or have the same packet attributes. It‘s decided by the application.
PM	Power Manager，功耗管理模块
ACC	Accelerate module，加速模块

4.1.4 时钟控制

控制信号由系统控制器和功耗管理模块产生，因此时钟生成、门控均通过配置系统控制器进行，部分门控还需要通过 PM 控制。

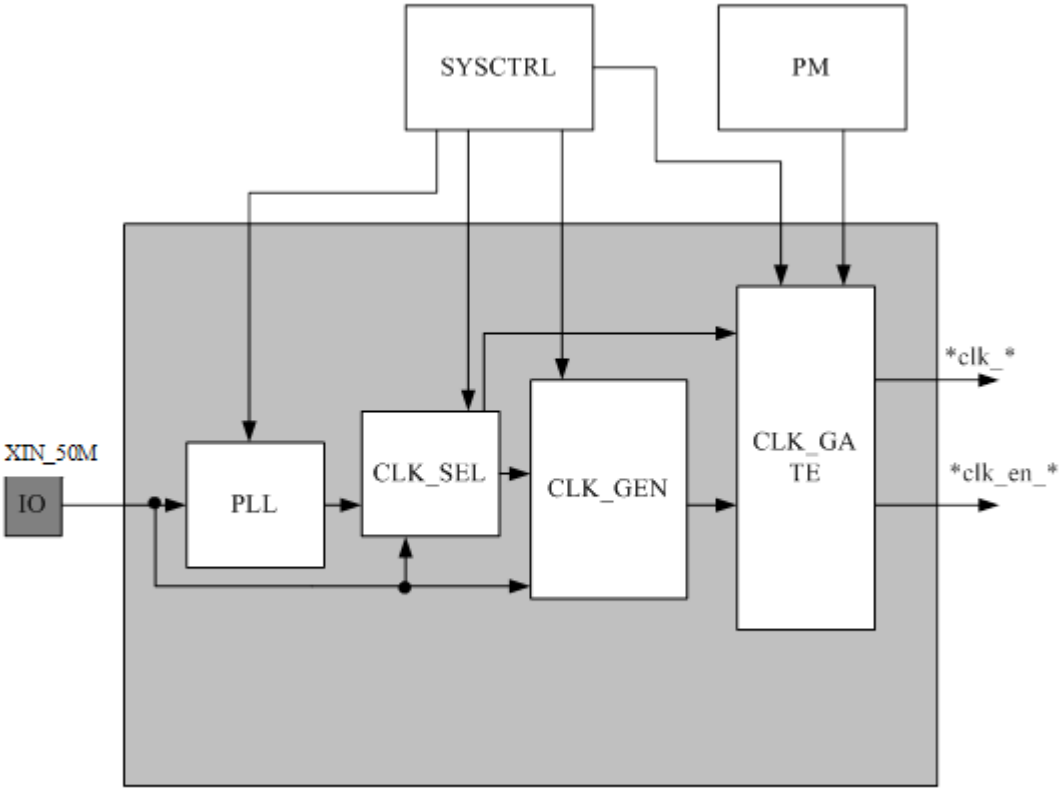


图4-1 系统时钟管理模块框图

时钟管理模块输入有以下几部分组成：

- 源自芯片管脚时钟输入 XIN_50M，此输入为片内五个 PLL 提供参考时钟，根据应用场景，频率为 50MHz
- 源自芯片管脚的 Serdes 参考时钟输入。
- 源自系统控制器的时钟控制信号，包括五个 PLL 的频率配置，时钟之间的频率切换配置，各模块时钟门控配置等。
- 源自功耗管理模块的时钟门控配置信号。

时钟管理模块输出以下信号：

- 系统各模块工作时钟信号
- 各接口模块接口时钟信号
- 相同时钟域中倍频时钟间的时钟使能信号

4.1.5 时钟结构

Slow 模式，是系统的大部分模块都停止工作，只有 cpu，总线，系统控制器等少数模块在工作的模式，这时系统的工作时钟来源于 50M 晶振；

normal 模式，是系统的正常工作模式，系统时钟来源于 PLL 正常输出时钟。

4.1.6 PLL 功能描述

4.1.6.1 PLL 控制信号

PLL 控制信号如表 4-2 所示。

表4-2 PLL 控制信号列表

PLL 信号名	说明
FREF	参考时钟（1MHz～800MHz），fractional mode 最小频率是 10MHz
REFDIV	参考分频值（1～63），分频后时钟频率>1MHz 且<40MHz，fractional mode 下>10MHz
FBDIV	整数反馈分频值
FRAC	小数反馈分频值
POSTDIV1	VCO 之后第一级分频值，1~7
POSTDIV2	VCO 之后第二级分频值，1～7（输出值是 FVCO/POSTDIV1/POSTDIV2）
PD	全局 power down 信号，高有效
BYPASS	参考时钟 bypass，有效时，输出参考时钟
DSMPD	
DACPD	
FOUT4PHASEPD	四相位输出时钟的 power down。同时对 FOUT2，FOUT3，FOUT4 有效
FOUTPOSTDIVPD	VCO 之后的分频器 power down，同时对四相位输出时钟，FOUT2，FOUT3，FOUT4 有效
FOUTVCOPD	VCO 之后的 buffer power down

4.1.6.2 PLL 频率配置公式

PLL 的输出频率配置公式如下：

FOUTVCO=FREF /REFDIV*(FBDIV+FRAC)

FOUTPOSTDIV=FOUTVCO/POSTDIV1/POSTDIV2

FOUT1PH0= FOUTPOSTDIV/2

FOUT2= FOUT1PH0/2

FOUT3= FOUT1PH0/3

FOUT4= FOUT1PH0/4

4.1.7 时钟配置

芯片内集成四个相同的 PLL，PLL 内部输出时钟频率关系如图 4-2 所示。

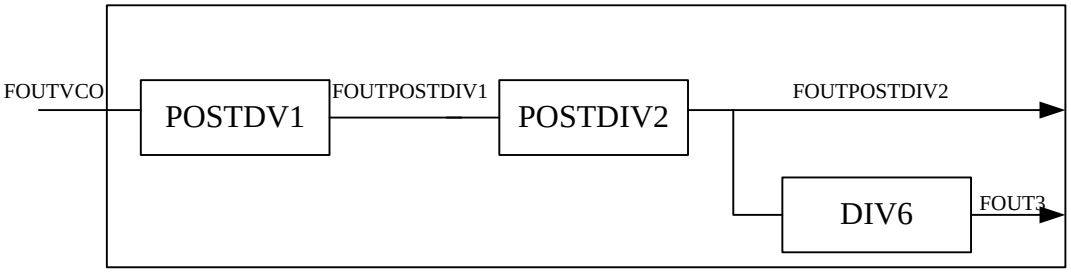


图4-2 PLL 内部输出时钟频率关系示意图

4.1.7.2 PLL 时钟配置

PLL 产生不同频率的时钟，需要对 PLL 的输入进行不同的配置。以 PLL0 产生 1500MHz 的时钟配置为例，推荐配置步骤如下：

步骤 1 PLL 上电并输入 PLL 的参考时钟，待上电稳定后即表示完成；

步骤 2 在 slow 模式，CPU 写系统控制器的 PLL 配置寄存器 SC_PLLFCTRL0（偏移地址 0x100），通过 PLL 配置寄存器来配置 PLL，产生 1500MHz 的时钟频率。由 1.1.7.3 章节的 PLL 配置公式，确定配置参数如下：

- 配置寄存器 SC_PLLFCTRL0[5:0]为 6’b00_0010，使 FREFDIV=2；
- 配置寄存器 SC_PLLFCTRL0[17:6]为 12’b0000_0011_1100，使 FBDIV=60；
- 配置寄存器[18:23] 为 6’b001001，使 POSTDIV1 为 1，POSTDIV2 为 1；

步骤 3 配置 PLL 的 power down 信号：

- 配置寄存器 SC_PLLFCTRL0[25]为 1’b1；

步骤 4 等待超过 1us 后，将 Power down 信号 PD 置为低电平：

- 配置寄存器 SC_PLLFCTRL0[25]为 0；

  ----配置结束

PLL2 的配置可通过系统控制器寄存器 SC_PERCTRL4 实现，步骤同上。

PLL3 的配置可通过系统控制器寄存器 SC_PERCTRL6 实现，步骤同上。

PLL4 的配置可通过系统控制器寄存器 SC_PERCTRL8 实现，步骤同上。

各 PLL2 配置参数值，请参考《Hip04 SysCtrl 寄存器描述》

4.1.8 系统时钟选择配置

4.1.8.1 各 SUBSYS 时钟选择配置

PLL替换在上电复位撤离之后，slow模式下软件首先静态配置不同subsys选择不同的PLL，完成PLL选择配置后，动态切换各个subsys的PLL选择后的时钟动态切换电路（起到去毛刺的作用）。不同的subsys配置流程如下：

4.1.8.1.1 IO_SUBSYS 时钟选择配置

步骤 1 上电复位撤离之后，默认进入 slow 模式，首先根据系统需求选择不同 PLL 代替 PLL1，软件配置寄存器：

默认 SC_PLL_CLK_BYPASS[15:14]为 2’b00，上电后配置 SC_PLL_CLK_BYPASS[15:14]为 2’b11，选择 PLL4。

步骤 2 当其他 subsys PLL 选择配置完成，配置寄存器 SC_PLL_CLK_BYPASS[13]为 1’b1，使时钟输出为 PLL 时钟;

步骤 3 在完成以上配置后，软件可以配置寄存器 SC_CLK_SEL0[8]为低（默认为高，低频模式），使 scheduler 的频率变为高频模式：

选择 PLL4（1.25GHz），scheduler 高频模式为 1.25GHz（默认低频模式 625MHz）;

4.1.8.1.2 ETH_SUBSYS 时钟选择配置

步骤 1 上电复位撤离之后，默认进入 slow 模式，首先根据系统需求选择不同 PLL 代替 PLL1，软件配置寄存器：

默认 SC_PLL_CLK_BYPASS[12:11]为 2’b00，上电后配置 SC_PLL_CLK_BYPASS[12:11]为 2’b11，选择 PLL4。

步骤 2 当其他 subsys PLL 选择配置完成，配置寄存器 SC_PLL_CLK_BYPASS[10]为 1’b1，使时钟输出为 PLL 时钟;

步骤 3 在完成以上配置后，软件可以配置寄存器 SC_CLK_SEL0[10]为低（默认为高，低频模式），使 scheduler 的频率变为高频模式：

选择 PLL4（1.25GHz），scheduler 高频模式为 1.25GHz（默认低频模式 625MHz）;

4.1.8.2 CPU_CLUSTER 和总线时钟频率配置

因为 Hip04 内部 DDR 时钟不支持动态切换频率，所以 CPU_CLUSTER 和总线时钟频率切换只允许在 slow 模式下进行。

- Kernel 子系统中四个 cluster0～3 的时钟是单独可配的，默认四个 cluster 的频率为最高运行在高频。可以通过

  配置寄存器 SC_CLK_SEL0[0]为高（默认为低），使 cluster0 的频率降为低频；

  配置寄存器 SC_CLK_SEL0[1]为高（默认为低），使 cluster1 的频率降为低频；

  配置寄存器 SC_CLK_SEL0[2]为高（默认为低），使 cluster2 的频率降为低频；

  配置寄存器 SC_CLK_SEL0[3]为高（默认为低），使 cluster3 的频率降为低频；

  注意：*只有在总线降为低频的时候，四个 cluster 才可以降频到低频。*

- 总线总线和 L3 cache 是同频的，默认频率为最高运行在高频。可以通过配置寄存器 SC_CLK_SEL0[16]为高，使总线总线和 L3 cache 的频率降为低频。

  CPU 的频率时钟要高于等于总线频率，初始上电 CPU 和总线同时工作在频率高频。所以如果调整任意一个 CPU 的频率，从高频降到低频，那么都需要先将总线频率降为低频，然后才能将 CPU 的频率从高频降到低频。

注意

- **软件切换时钟频率，每次只能在 cluster0，cluster1，cluster2，cluster3，总线之中选择其中一个切换，不支持同时切换。如果同时切换，有可能会出现挂死。**
- 软件配置不允许出现，各 cluster 频率低于总线总线频率的情况，这中情况会导致系统挂死。

4.1.8.3 PLL 旁路配置

- **PLL0 时钟域**

PLL0 输出时钟允许内部和外部 bypass，当内部或者外部 bypass 有效时，外部晶振时

钟代替 PLL0 输出时钟，成为系统内部时钟。配置寄存器 SC_PLLFCTRL1[24]（偏移地址为 0x100）可以使能或禁止内部 bypass 功能，高有效，默认为低。配置寄存器 SC_PLL_CLK_BYPASS[0] （偏移地址 0x0180）使能或禁止外部 bypass 功能，低有效，默认为的低。

步骤 1 配置寄存器 SC_PLLFCTRL1[24]为 1’b1 表示 PLL0 内部 bypass 有效，此时 PLL0 输入参考时钟作为 PLL0 输出时钟；配置寄存器 SC_PLLFCTRL1[24]为 1’b0 表示 PLL0 内部 bypass 无效，此时 PLL0 输出内部生成的时钟。

步骤 2 配置寄存器 SC_PLL_CLK_BYPASS[0]为 1’b1 表示 PLL0 外部 bypass 无效，此时 PLL0 输出时钟作为下级时钟产生电路的输入时钟；配置寄存器 SC_PLL_CLK_BYPASS[0]为 1’b0 表示 PLL0 外部 bypass 有效，此时 PLL0 输入参考时钟作为下级时钟产生电路的输入时钟。

步骤 3 各 PLL 静态 bypass 之后，软件可以配置 PLL0 时钟先进入 normal 模式；首先配置寄存器 SC_PLLCTRL[27:3]，设置等待 PLL0 切换时间；然后配置寄存器 SC_CTRL[2:0]为 3‘b100，PLL0 时钟域在等待若干时间后（该时间由 SC_PLLCTRL[27:3]配置得到），进入 normal 模式，寄存器[2:0]bit 默认值为 3’b010。

● PLL2 时钟域

PLL2 同样支持输出时钟在内部和外部的 bypass。当内部或者外部 bypass 有效时，晶振时钟代替 PLL2 的输出时钟。配置寄存器 SC_PLLFCTRL4[24]（偏移地址为 0x110）可以使能或者禁止内部 bypass 功能，高有效，默认为低。配置寄存器 SC_PLL_CLK_BYPASS[2]（偏移地址 0x0180）可以使能或者禁止外部 bypass 功能，低有效，默认为低。

步骤 4 配置寄存器 SC_PLLFCTRL4[24]为 1’b1 表示 PLL2 内部 bypass 有效，此时 PLL2 的输入参考时钟作为 PLL2 的输出时钟；配置寄存器 SC_PLLFCTRL4[24]为 1’b0 表示 PLL2 内部 bypass 无效，此时 PLL2 输出内部生成的时钟。

步骤 5 配置寄存器 SC_PLL_CLK_BYPASS[2]为 1’b1 表示 PLL2 外部 bypass 无效，此时 PLL2 的输出时钟作为下级时钟产生电路的输入时钟；配置寄存器 SC_PLL_CLK_BYPASS[2]为 1’b0 表示 PLL2 外部 bypass 有效，此时 PLL2 的输入参考时钟作为下级时钟产生电路的输入时钟。

步骤 6 PLL0 时钟域进入 normal 模式之后，软件可以配置寄存器 SC_PLL_CLK_BYPASS[16]为 1’b1，选择 PLL2 输出时钟，使 PLL2 时钟域由 slow 模式切换进入 normal 模式； 寄存器默认值为 1’b0，输出晶振时钟。

● PLL3 时钟域

PLL3 同样支持输出时钟在内部和外部的 bypass。当内部或者外部 bypass 有效时，晶振时钟代替 PLL3 的输出时钟。配置寄存器 SC_PLLFCTRL6[24]（偏移地址为 0x118）可以使能或者禁止内部 bypass 功能，高有效，默认为高。配置寄存器 SC_PLL_CLK_BYPASS[3]（偏移地址 0x0180）可以使能或者禁止外部 bypass 功能，低有效，默认为低。

步骤 7 配置寄存器 SC_PLLFCTRL6[24]为 1’b1 表示 PLL3 内部 bypass 有效，此时 PLL3 的输入参考时钟作为 PLL2 的输出时钟；配置寄存器 SC_PLLFCTRL6[24]为 1’b0 表示 PLL3 内部 bypass 无效，此时 PLL3 输出内部生成的时钟。

步骤 8 配置寄存器 SC_PLL_CLK_BYPASS[3]为 1’b1 表示 PLL3 外部 bypass 无效，此时 PLL3 的输出时钟作为下级时钟产生电路的输入时钟；配置寄存器 SC_PLL_CLK_BYPASS[3]为 1’b0 表示 PLL3 外部 bypass 有效，此时 PLL3 的输入参考时钟作为下级时钟产生电路的输入时钟。

步骤 9 PLL0 时钟域进入 normal 模式之后，软件可以配置寄存器 SC_PLL_CLK_BYPASS[17]为 1’b1，选择 PLL3 输出时钟，使 PLL3 时钟域由 slow 模式切换进入 normal 模式； 寄存器默认值为 1’b0，输出晶振时钟。

● PLL4 时钟域

PLL4 同样支持输出时钟在内部和外部的 bypass。当内部或者外部 bypass 有效时，晶振时钟代替 PLL4 的输出时钟。配置寄存器 SCPLLFCTRL8[24]（偏移地址为 0x120）可以使能或者禁止内部 bypass 功能，高有效，默认为高。配置寄存器 SC_PLL_CLK_BYPASS[4]（偏移地址 0x0180）可以使能或者禁止外部 bypass 功能，低有效，默认为低。

步骤 10 配置寄存器 SCPLLFCTRL8[24]为 1’b1 表示 PLL4 内部 bypass 有效，此时 PLL4 的输入参考时钟作为 PLL4 的输出时钟；配置寄存器 SCPLLFCTRL8[24]为 1’b0 表示 PLL4 内部 bypass 无效，此时 PLL4 输出内部生成的时钟。

步骤 11 配置寄存器 SC_PLL_CLK_BYPASS[4]为 1’b1 表示 PLL4 外部 bypass 无效，此时 PLL4 的输出时钟作为下级时钟产生电路的输入时钟；配置寄存器 SC_PLL_CLK_BYPASS[4]为 1’b0 表示 PLL4 外部 bypass 有效，此时 PLL4 的输入参考时钟作为下级时钟产生电路的输入时钟。

步骤 12 PLL0 时钟域进入 normal 模式之后，软件可以配置寄存器 SC_PLL_CLK_BYPASS[18]为 1’b1，选择 PLL4 输出时钟，使 PLL4 时钟域由 slow 模式切换进入 normal 模式； 寄存器默认值为 1’b0，输出晶振时钟。

4.1.9 低功耗的时钟管理

4.1.9.1 低功耗时钟复位流程

各掉电模块在掉电之前，PM 发出复位请求，对掉电模块进行模块复位。经过若干个 PM 工作周期，PM 发出时钟门控信号，关断时钟。然后掉电模块掉电；等待上电，复位信号保持有效。PM 打开时钟，经过若干个 PM 工作周期撤离。

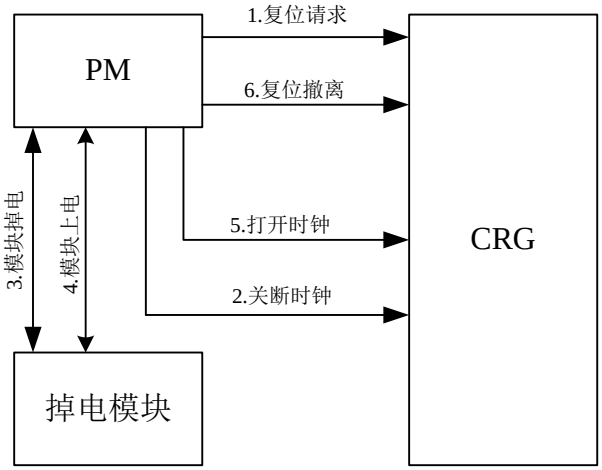


图4-3 模块掉电流程图

4.1.10 时钟门控

HIP04 为了节省功耗，对于可以门控的模块全部加入 ICG 门控单元。

4.2 复位

4.2.1 概述

复位管理模块对整个芯片的复位、各功能模块的复位进行统一管理，生成芯片内部各功能模块的复位信号。

4.2.2 特性

复位管理模块的主要特性如下：

- 上电复位的管理和控制。滤掉 300ns 的复位毛刺，并扩展到 256 个晶振周期的宽度。
- 系统软复位、功能模块单独软复位。系统软复位请求需要扩展到上电复位等宽的脉冲宽度。
- 复位信号同步到各模块对应时钟域。
- 所有控制通过系统控制器寄存器和功耗管理模块进行配置。

4.2.3 关键术语

表4-3 关键术语描述

术语	描述
Ordered Flow	It is a series of packets; these packets must be transmitted as received order. An ordered flow can be a series of packets that are in a same channel or have the same packet attributes. It‘s decided by the application.

4.2.4 复位接口信号

芯片复位接口如图 4-4 所示。

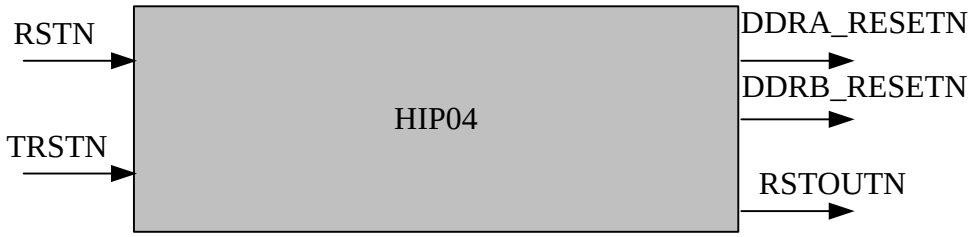


图4-4 芯片复位接口

表4-4 芯片复位接口信号描述

名称	方向	描述	对应管脚
RSTN	I	系统复位输入，低电平有效	RSTN

名称	方向	描述	对应管脚
TRSTN	I	JTAG 复位输入，低电平有效。	TRSTN
RSTOUTN	O	系统复位输出，低电平有效	RSTOUTN
DDRA_RESETN	O	DDR3 复位信号，低有效。	DDRA_RESETN
DDRB_RESETN	O	DDR3 复位信号，低有效。	DDRB_RESETN

4.2.5 复位控制

所有控制信号均由系统控制器和功耗管理模块产生，因此所有时钟生成、管理均通过配置系统控制器和功耗管理模块进行。

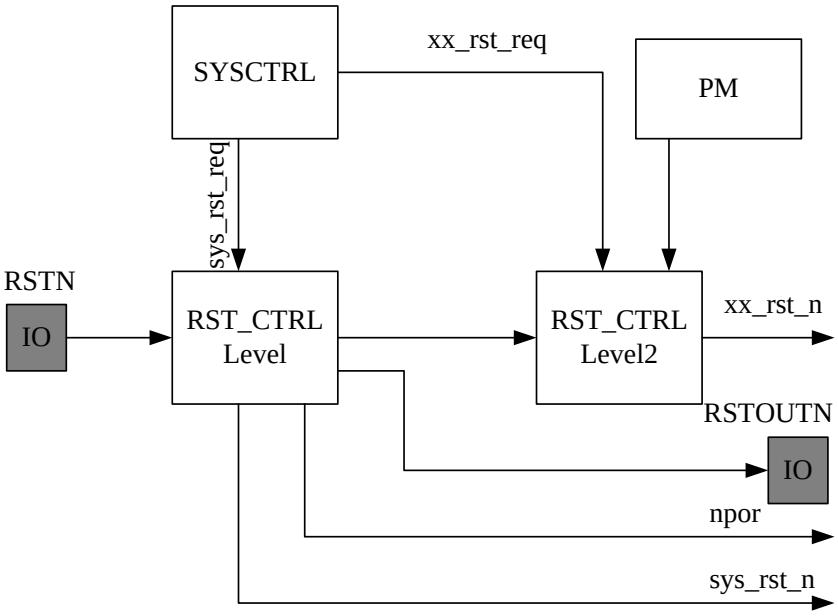


图4-5 系统复位信号控制框图

RSTN：上电复位和硬复位输入，来自芯片管脚。
RSTOUTN：系统复位输出，复位外围器件。
sys_rst_req：全局软复位请求信号，源自系统控制器。
xx_rst_req：各模块单独软复位请求信号，源自系统控制器。
xx_rst_n、rst_comb_n、rst_por_n：输出的三类复位信号，具体描述如表 4-5 所示。

表4-5 三类复位输出描述

复位信号类型	产生方式	用途
rst_por_n	上电复位信号 RSTN 经过去抖动和同步扩展等处理后得到 rst_por_n。	用于对系统控制器的复位。

复位信号类型	产生方式	用途
rst_comb_n	由上电复位和系统软复位逻辑组合生成系统复位信号 rst_comb_n。	用于对没有单独软复位控制但是允许全局软复位的模块进行复位。
xx_rst_n	系统复位信号 rst_comb_n 与模块单独软复位控制进行逻辑组合的各 IP 模块复位信号。	用于对允许单独软复位的模块进行复位；上电复位、全局软复位、单独软复位控制都会导致该模块复位。

4.2.6 复位配置

4.2.6.1 上电复位

RSTN 是上电复位和硬复位输入 IO，完成上电复位过程必须同时满足以下条件：

- 上电复位 IO 输入一个低电平脉冲。
- 晶振输入管脚 XIN_50M 输入的时钟稳定正常。
- 输入的上电复位信号低电平维持时间 256 个晶振时钟周期。
- 芯片 IO 和 core 电源稳定。

上电复位时序图如图 4-6 所示。

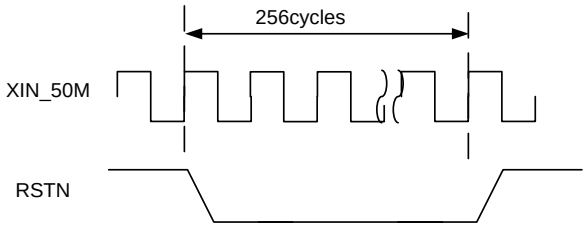


图4-6 上电复位时钟图

4.2.6.2 系统复位

系统复位有三种实现途径：

- 上电复位和硬复位，两者作用相同
- 系统软复位，通过写系统控制器寄存器 SC_SYSSTAT 写特定值 0x5555_5555 即产生系统软复位，PLL0 时钟域自动切换到 SLOW 模式。扩展宽度满足系统内任何模块的复位宽度要求。
- 软件控制各个模块单独软复位，从而实现全系统工作模块复位。

当发生上电复位和硬复位时，片内所有可复位逻辑全部回到初始状态。

当发生系统软复位时，除了系统控制器、时钟复位生成模块、CPU 部分 CLK domain 逻辑（debug，PTM）之外，其他所有系统模块、接口模块、硬件加速引擎、总线、对外接口等都被复位。

当软件控制各模块的单独软复位来实现系统复位，软件保证复位顺序和复位撤离顺序的正确性。

4.2.6.3 系统软复位与系统工作模式切换流程

在发起系统软复位之前，先通过配置相应寄存器，让 pll2，pll3，pll4 进入 slow 模式，然后发起系统软复位，此时 pll0 会自动进入 slow 模式。具体配置流程：

步骤 1 配置系统控制的寄存器 SC_PLL_CLK_BYPASS【18：16】（偏移地址为 0x180）为 3’b000, 此时 pll2,pll3,pll4 会进入 slow 模式；

步骤 2 配置系统控制器 SC_SYSRST（偏移地址为 0x0004）为 32’h55555555,发起系统软复位请求，此时 pll0 会自动进入 slow 模式

系统软复位撤离以后，配置系统控制器的 SC_CTRL【2：0】为 3‘b100，使得 pll0 进入 normal 模式，再配置 SC_PLL_CLK_BYPASS【18：16】为 3’b111，使得 pll2,pll3,pll4 进入 normal 模式。

4.2.6.4 CPU 软复位

下面说明 CortexA15MP 的各个复位信号之间的组合关系：

表4-6 CPU 软复位

复位事件	nCPUPORESET	nCORERESET	nCXRESET（单独电压域时采用）	nDBGRESET	nPRESETDBG	nL2RESET	nMBISTRESET(DFT 确定需求)
上电、硬复位	✓	✓	✓	✓	✓	✓	
系统软复位	✓	✓	✓		✓	✓	
cpupo_srst_req	✓					所有 core 都复位，就复位	
Core_srst_req		✓				所有 core 都复位，就复位	
Cx_srst_req(应该不用)			✓				
Dbg_srst_req				✓			

Pdbg_srst_req					✓		
L2_srst_req(不确定有无)	✓	✓	✓			✓	

Cpu 包含 core 逻辑和 dbg 逻辑两部分，四个 cpu 组成一个 cluster。每个 cpu 配备一个 wdog 在 cpu 外，因此 wdog 的溢出复位使用要小心，需要 wdog 在溢出前报告 cpu，cpu 停止业务，然后才能复位。

同时 cluster0 的 cpu0 相应的 wdog0 溢出复位，会输出系统复位，复位芯片周围器件。

注意

- 当 CLUSTER 的 4 个核全部复位时，L2 CACHE 也被复位；当 CLUSTER 的任何一个核撤离复位时，L2 CACHE 同时撤离复位。
- 系统复位撤离时，要保证 cpu core 最后撤离。

4.2.6.5 各功能模块软复位

各功能模块都有自己的软复位请求信号，有系统控制器发出模块软复位请求信号，从而实现各模块的复位功能。软复位撤离顺序由软件来保证。

各模块的软复位请求信号具体信息请参考在附录《Hip04 SysCtrl 寄存器描述》文档。

注意

- **系统软复位请求发出后，必须等待至少 128 个系统控制器时钟周期，电路才自动完成复位撤销。**
- 各模块单独软复位不会自动撤消，即配置模块复位请求寄存器处于复位状态，必须再配置模块复位去请求寄存器，该模块复位才会撤消。撤离顺序由软件保证。
- 部分模块软复位请求初始处于复位状态，需要软件撤销软复位请求，才可以正常工作
- L3CACHE 模块软复位需要在 slow 模式下撤销软复位请求。

4.2.7 低功耗的复位管理

各掉电模块，有功耗管理模块发出模块复位信号，从而实现各掉电模块的复位功能。复位撤离顺序由软件来保证。具体配置方法请参见功耗管理模块的寄存器配置表单。

5 系统初始化

5.1 概述

本章内容提供一种典型的系统初始化流程作为参考，实际产品的初始化流程可能会有不同，重点描述整个系统的初始化流程，具体模块初始化流程参考模块章节描述。重点描述与芯片相关的硬件初始化，软件系统的初始化不作为重点。

5.2 特性

系统初始化包括以下特性：

- BOOT 初始化流程

5.3 关键术语

如表 5-1 的术语将在本章节表示特殊的含义。

表5-1 系统初始化关键术语描述

术语	描述
小系统	满足操作系统运行的最小系统
BootLoader	引导加载程序，是系统加电后运行的第一段软件代码，主要用于引导并启动操作系统

5.4 BOOT 启动流程

5.4.1 概述

ARM 系统解除复位后是从 0 地址开始取指的，因此 Hip04 中 BOOT 根据系统解除复位后 0 地址的映射媒介分为两种启动方式：

- 0 地址映射到 NORFLASH 的 NORFLASH BOOT
- 0 地址映射到 NANDFLASH 的 NANDFLASH BOOT

5.4.2 NORFLASH BOOT

使用 NORFLASHBOOT，系统需要通过外部管脚配置启动模式为 SMI 控制器外接的存储器进行启动以及配置外接 NORFLASH 的位宽。正确配置后，系统解除复位后会把 NORFLASH 的前 256Mbyte 映射到系统 0-0xFFF_FFFF 这段地址空间。

步骤 1 NORFLASH BOOT 启动流程如下：

步骤 2 系统解除复位，CPU 从 0 地址开始取指

步骤 3 根据系统需要配置大小端。使用 SETEND 指令配置 CPSR 的 E bit，SENEND BE 为大端，SETEND LE 为小端。配置 CP15 SCTLR 寄存器的 EE bit，1 为大端，0 为小端

步骤 4 关闭 Dcache，MMU，Icache。分别配置 CP15 SCTLR 寄存器的 C，M，I bit 为 0.

步骤 5 控制主核继续运行，其他核进入 WFI 状态，等待主核唤醒。

步骤 6 DAW 初始化，按照实际业务的映射关系配置 DAW,配置 NORFLASH 对应的 localbus 片选为新的 NORFLASH 地址（注意：要暂时保留默认的 localbus 的窗口配置，直到新的 localbus 窗口生效）

步骤 7 CPU 跳转到 NORFLASH 的物理地址，配置总线 寄存器 remap_mode bit 为 1，清除系统 remap。

步骤 8 修改 ARM 寄存器 SP,配置临时栈指针在 SRAM 中

步骤 9 CP15 SCTLR 寄存器的 I bit 为 1，使能 Icache

步骤 10 初始化 PLL。PLL 初始化流程描述见 FS 时钟和复位章节

步骤 11 初始化 DDR。DDR 初始化流程描述见 FS DDRC 章节

步骤 12 初始化一些必要的系统配置，如 IO 复用，总线时序等

步骤 13 修改 ARM 寄存器 SP,配置临时栈指针在 DDR 中

步骤 14 将 UBOOT 从 NORFLASH 拷贝到 DDR 中

步骤 15 初始化 UBOOT 划分的各个数据段

步骤 16 跳转到 DDR 中继续运行

步骤 17 初始化 NORFLASH,探测 NORFLASH ID，根据 ID 得到 NORFLASH 大小等信息

步骤 18 在 NORFLASH 中设置环境变量，分为两种情况：1 NORFLASH 中没有有效的环境变量，在 NORFLASH 中写入默认的环境变量 2 NORFLASH 中存在有效的环境变量，保持不变

步骤 19 初始化 USB，I2C 等其他外设。这一部分不是必须，根据实际应用场景决定初始化哪些外设。

步骤 20 初始化调试网口。调试网口初始化流程描述见 FS DDRC 章节

步骤 21 根据环境变量设置选择自动加载操作系统或者进入 shell

END

说明

1 Cortex A15 cpu 在复位启动后会自动 invalidate cache，TLB 等 ram。

2 如果需要提高性能，可以在步骤 9 后使能 MMU 和 Dcache

3 UBOOT 中没有用动中断

5.4.3 NANDFLASH BOOT

使用 NORFLASHBOOT，系统需要通过外部管脚配置启动模式为 NANDFLASH 启动以及配置外接 NANDFLASH 的页大小，命令个数，ecc 模式，数据位宽等信息。正确配置后，系统解除复位后会把 NANDFLASH 的前 128KByte 映射到系统 0x0～0-0x1_FFFF 这段地址空间

NANDFLASHBOOT 跟 NORFLASHBOOT 流程基本相似：

NANDFLASH BOOT 启动流程如下：

步骤 1 系统解除复位，CPU 从 0 地址开始取指

步骤 2 根据系统需要配置大小端。使用 SETEND 指令配置 CPSR 的 E bit，SENEND BE 为大端，SETEND LE 为小端。配置 CP15 SCTLR 寄存器的 EE bit，1 为大端，0 为小端

步骤 3 关闭 Dcache，MMU，Icache。分别配置 CP15 SCTLR 寄存器的 C，M，I bit 为 0.

步骤 4 控制主核继续运行，其他核进入 WFI 状态，等待主核唤醒。

步骤 5 DAW 初始化，按照实际业务的映射关系配置 DAW

步骤 6 CPU 跳转到 NANDFLASH 的物理地址，配置总线 寄存器 remap_mode bit 为 1，清除系统 remap。

步骤 7 修改 ARM 寄存器 SP,配置临时栈指针在 SRAM 中

步骤 8 CP15 SCTLR 寄存器的 I bit 为 1，使能 Icache

步骤 9 初始化 PLL。PLL 初始化流程描述见 FS 时钟和复位章节

步骤 10 初始化 DDR。DDR 初始化流程描述见 FS DDRC 章节

步骤 11 初始化一些必要的系统配置，如 IO 复用，总线时序等

步骤 12 修改 ARM 寄存器 SP,配置临时栈指针在 DDR 中

步骤 13 将 UBOOT 前 128K 从 NANDFLASH 拷贝到 DDR 中

步骤 14 初始化 UBOOT 划分的各个数据段

步骤 15 跳转到 DDR 中继续运行

步骤 16 配置 NANDC 寄存器的 op_mode bit 为 1，将 NANDFLASH 的模式切换为 NORMAL 模式，初始化 NANDFLASH，然后将 UBOOT 超过 128K 的部分从 NANDFLASH 拷贝到 DDR 中

步骤 17 探测 NANDFLASH ID，根据 ID 得到 NANDFLASH 大小等信息

步骤 18 在 NANDFLASH 中设置环境变量，分为两种情况：1 NANDFLASH 中没有有效的环境变量，在 NANDFLASH 中写入默认的环境变量 2 NANDFLASH 中存在有效的环境变量，保持不变

步骤 19 初始化 USB，I2C 等其他外设。这一部分不是必须，根据实际应用场景决定初始化哪些外设。

步骤 20 初始化调试网口。调试网口初始化流程描述见 FS DDRC 章节

步骤 21 根据环境变量设置选择自动加载操作系统或者进入 shell

END

说明

NANDFLASH 只映射了 128K 的空间到 0 地址，并且有 NORMAL 和 BOOT 两种模式（具体细节见 FS NANDC 章节），因此步骤 12-15 与 NORFLASH 略有不同

6 系统功耗

6.1 概述

Hip04 基于 TSMC 28nm 0.9V HPM 工艺。内部集成了 16 个 Cortex-A15 处理器核；提供了 3 级 Cache 结构和基于硬件的数据一致性解决方案；此外还集成了通用的处理器接口。

本章节主要描述 Hip04 系统功耗，包括功耗组成，功耗的的分布，主要应用业务场景下的功耗值，以及功耗控制方。

6.2 特性

Hip04 的系统功耗主要特性如下：

- 多层次低功耗技术；
  - **系统级**
    - 硬件支持多核处理器、IO 和加速引擎之间的全系统一致性，可避免通过 memory copy 实现一致性带来的额外功耗；
    - 各个加速引擎中每一级逻辑的级数选择和设计都基于性能评估结果进行，以降低静态和动态功耗
    - 集中的低功耗管理单元 PM，通过片上 MTCMOS 控制各个模块的 Power Gating；
    - 系统级的 DFS；
    - 支持在不同的产品形态下，通过静态配置 power gating 部分模块以降低系统功耗；
    - Clock Gating 支持按每个模块的粒度可关闭；

6.3 关键术语

如表 6-1 的术语将在本章节表示特殊的含义。

表6-1 系统功耗关键术语描述

术语	描述
PM	Power Manager，片上功耗管理模块。PM 模块管理芯片的 Power Gating 和 AVS。
DFS	Dynamic Frequency Scaling，动态频率调整。一种通过动态改变频率来降低功耗的技术。
DVS	Dynamic Voltage Scaling，动态电压调节。一种通过动态改变电压降低功耗的技术。
AVS	Adaptive Voltage Scaling，自适应电压调节，更智能、更精细的 DVS 技术。Hip04 主要支持两种 AVS 技术：静态工艺补偿和动态温度补偿。
Power Gating	门控电源技术。一种通过关闭电路电源降低功耗的技术。
Clock Gating	门控时钟技术。一种通过关闭电路时钟降低功耗的技术。
HPM	片上集成的硬件速度监控单元（HPM，Hardware Performance Monitor）。
PMU	Power Manage unit，外部电源管理芯片（Voltage Regulator）。
WFI	Wait for Interrupt，cortex-A15 等待中断唤醒状态。
CP	Circuit Probe，即 Wafer 测试。
FT	Final Test，即芯片测试。
Isolation Cell	隔离逻辑单元，用于 Power Gating 时钳位被 Power Down 的模块的输出到固定电平。
Level Shifter	电平转移器。接口物理电平不一样，但是逻辑电平一样。
PWM	Pulse Width Modulation，脉冲宽度调节。一种电源控制技术，其脉冲宽度可以调节。

6.4 功耗分布

表 6-2 列出了 Hip04 所有重要的电源，供电电压和参考功耗值(芯片估计值)。系统中实际使用的电源电压与表中的值可能有细微的出入。

表6-2 Hip04 电源域

电源名称	电源用途	供电电压（V）	参考功耗（mW）
VDD_CORE	Core 电源	0.9	21000
VDD_MEM	High Density Memory 电源	0.9	2000
VDD_IO18	1.8V IO 电源	1.8	308.55
USB DVDD	USB PHY 数字电源	0.9	16.7373
USB_VDD180	USB PHY 模拟电源	1.8	97.362
USB_VDD330	USB PHY 模拟电源	3.3	48.807
AVDDH_PLL	PLL 模拟电源	1.8	34.56
AVDDL_PLL	PLL 模拟电源	0.9	21.6
TSENSOR_VDD	Tsensor 模拟电源	1.8	1.4148
PA_VDD1 (Macro1-Vsemi 4lane)	Serdes 模拟电源	0.9	159.48
PA_VDD2 (Macro1-Vsemi 4lane)	Serdes 模拟电源	1.8	153.36
PA_VDD1 (Macro2-Vsemi 4lane)	Serdes 模拟电源	0.9	159.48
PA_VDD2 (Macro2-Vsemi 4lane)	Serdes 模拟电源	1.8	153.36
RVDD (Macro0-athenaV101 8lane)	Receiver 模拟 0.9V 电源	0.9	274.5
RVDD (Macro0-athenaV101 8lane)	Transmit 模拟电源	0.9	183.5
AVDD (Macro0-athenaV101 8lane)	模拟 1.8V 电源	1.8	67.68
RVDD (Macro3-athenaV101 8lane)	Receiver 模拟 0.9V 电源	0.9	274.5
RVDD (Macro3-athenaV101 8lane)	Transmit 模拟电源	0.9	183.5
AVDD (Macro3-athenaV101 8lane)	模拟 1.8V 电源	1.8	67.68
VDDQ	DDR PHY 模拟电源	1.35/1.5	2028
VREF_DDRA	DDRA 接口 IO 参考电压	0.75/0.675	0.00423
VREF_DDRB	DDRB 接口 IO 参考电压	0.75/0.675	0.00423
DDRPLL_VDD	DDRPHY 内部 PLL 的电压	1.8	444.84
VDD_GEPHY	10 GE PHY 的 MDC/MDIO	1.8/1.2	12

说明

以上数据仅为 Hip04 估算值，环境温度 105C，CPU 运行频率 1.5G，具体数据需等芯片返回后测试数据为准。

VDD_CORE 电源对 Hip04 所有数字电路、非 High-Density SRAM 和 High-Density 的外围电路供电，可以通过片上 AVS 动态温度补偿调整 VDD_CORE 的输出电流。芯片上的 High-Density SRAM 的 Bit Cell 的供电由 VDD_MEM 提供。TsensorV500 IP 由 1.8V 模拟电压 TSENSOR_VDD（±5%精度，纹波小于 20mv）供电。

Hip04 系统的电源供电图如图 6-1 所示。AVS_PMU 电源对应表表 6-2 Core 电源，AVS 静态工艺补偿和 AVS 动态温度补偿可以一次和多次调整 Core 电源，调压范围为 0.75~1.1V。Hip04 集成了两个 Vsemi Serdes IP，每个 IP 需要提供两路模拟电源，PA_VDD1 和 PAVDD2，

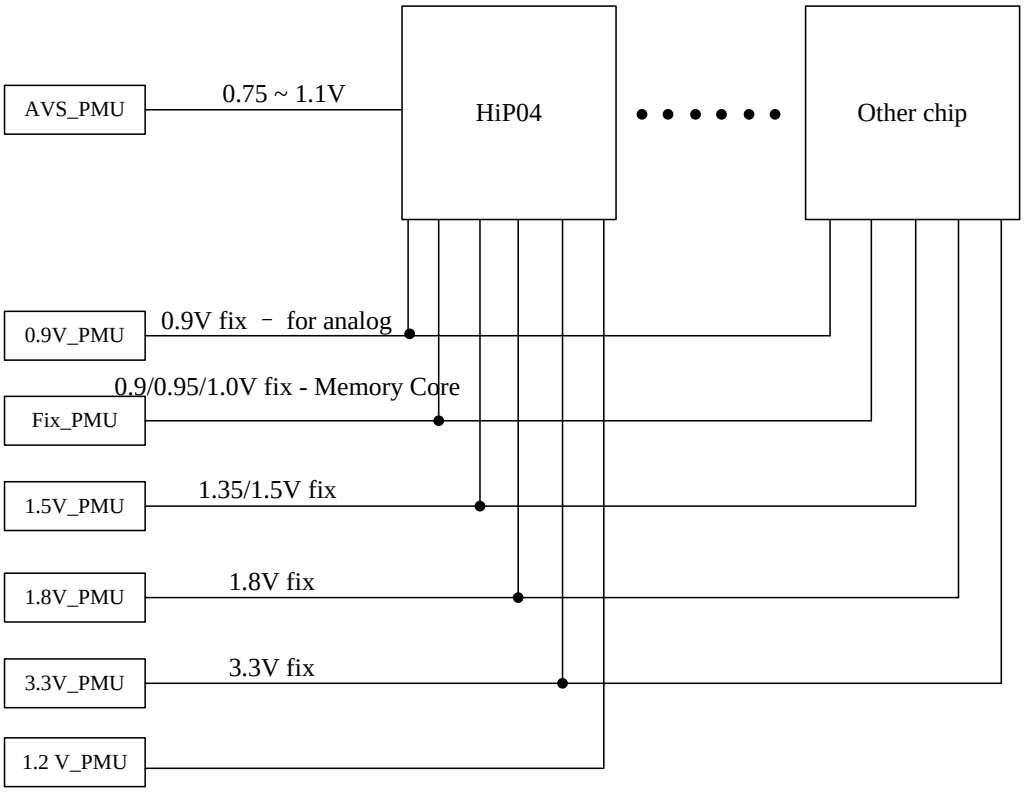


图6-1 Hip04 电源

6.5 功耗组成和计算

Hip04 的功耗主要分为静态功耗和动态功耗。

目前估计 16 核 Cortex-A15 和其他片上逻辑在常温下的功耗是 17.8W。

目前估计常温下 Hip04 的静态功耗为 0.5W，而 105 ˚C 结温下的泄漏功耗为 6W。

6.6 功耗控制

功耗控制的目标就是硬件模块、外设在处于激活状态下所需要的最大工作电流只需要能够完成所分配的任务，在非激活状态尽可能降低工作电流的大小。系统应用场景、内在的系统状态和操作模式的识别和定义需要仔细的评估，确定能够提供给用户的系统性能。通过对整个系统的功耗分析能够确认系统所能够运行的环境，如封装类型，散热形式等。而这些分析也能够帮助如下领域的评估、选择和架构设计：

- 工艺类型；
- 功耗操作模式；
- 低功耗设计架构和技术的选择；
- 时钟结构。
- 在芯片级和系统级使用多个功耗分析工具来评估芯片和系统的功耗。

下面章节介绍 Hip04 采用的低功耗技术。

6.6.1 Power Supply domain

表 6-3 给出了 Hip04 使用的电源域以及电压值，所有的电源电压都是通过片外的电源管理芯片供电。芯片的数字逻辑的供电电源为 VDD_CORE，电压为 0.9V；芯片采用的 Dual-Rail SRAM，其外围逻辑和存储单元的电源分别为 VDD_MEM 和 VDD_CORE，VDD_MEM 的供电电压值为 0.9V。需要注意的是由于 AVS 技术的使用，表中的值与实际芯片使用的电压可能不一致。Hip04 不支持对片外电源管理芯片的关断控制，通过 AVS 动态温度补偿能够动态改变片外电源管理芯片对 Hip04 输出电压。

表6-3 芯片的供电电压域

电源名称	电源用途	供电电压	注释
VDD_CORE	Core 电源	0.9V	芯片逻辑的主要电源，CPU Cluster 0,1
VDD_MEM	High DensityMemory 电源	1.0V	Dual-Rail Memory Bit Cell 电源
VDD_IO18	1.8V IO 电源	1.8V	I/O 管脚电源
USB DVDD	USB PHY 数字电源	0.9V	USB PHY 数字电源
USB_VDD180	USB PHY 模拟电源	1.8V	USB PHY 模拟电源
USB_VDD330	USB PHY 模拟电源	3.3V	USB PHY 模拟电源
AVDDH_PLL	PLL 模拟电源	1.8V	PLL 模拟电源
AVDDL_PLL	PLL 模拟电源	0.9V	PLL 模拟电源
AVDD_TS	Tsensor 模拟电源	1.8V	Tsensor 模拟电源

QL0_AVDD	Serdes 模拟电源	0.9V	Serdes 模拟电源
QL0_C_AVDDH	Serdes 模拟电源	1.8V	Serdes 模拟电源
QL1_AVDD	Serdes 模拟电源	0.9V	Serdes 模拟电源
QL1_C_AVDDH	Serdes 模拟电源	1.8V	Serdes 模拟电源
QL2_AVDD	Serdes 模拟电源	0.9V	Serdes 模拟电源
QL2_C_AVDDH	Serdes 模拟电源	1.8V	Serdes 模拟电源
QL3_AVDD	Serdes 模拟电源	0.9V	Serdes 模拟电源
QL3_C_AVDDH	Serdes 模拟电源	1.8V	Serdes 模拟电源
VDD_DDR	DDR PHY 模拟电源	1.35/1.5V	DDR PHY 模拟电源
VDD_GEPHY	10 GE PHY 的 MDC/MDIO	1.2V	10 GE PHY 的 MDC/MDIO

6.6.2 Clock Gating & DFS

模块或总线在没有任何业务需要处理的场景下可以关闭其时钟。另外在轻业务场景下可以通过动态调整模块的时钟来降低动态功耗。

芯片采用多级 clock gating 结构。在子系统的 CRG 中每个模块 clock gating 的配置可以通过硬件完成，也可以通过软件完成。如果硬件开销比较小的情况下，优先采用硬件自动完成 clock gating，这样能够以更细的时间粒度更有效地降低功耗。软件控制的 clock gating 由于软件处理通常带来比较大的延时。

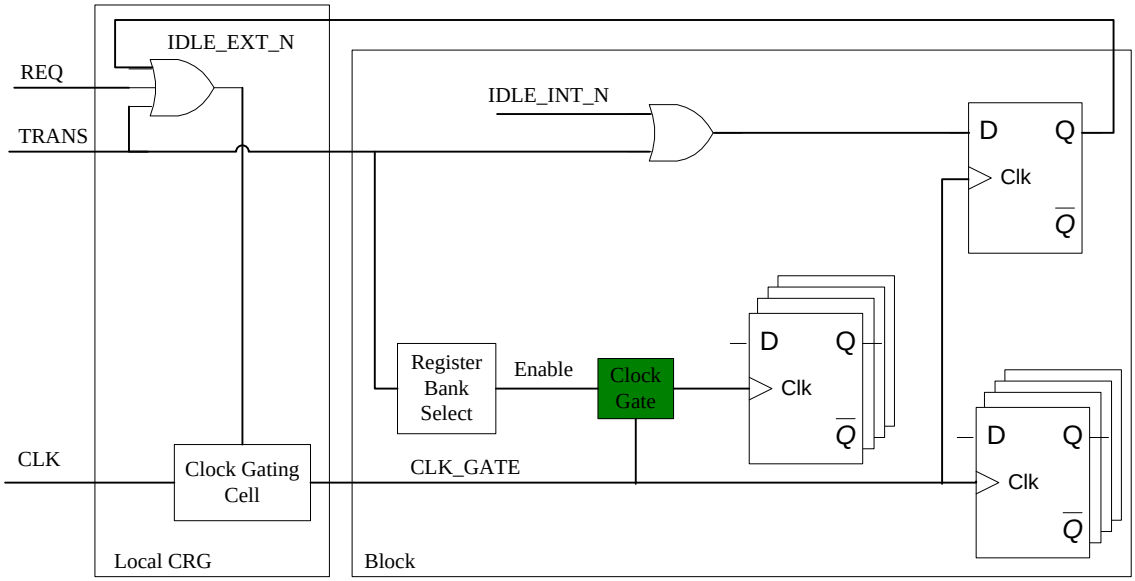


图6-2 多级 clock gating

6.6.2.1.2 Cortex-A15

Cortex-A15 通过执行 WFI、WFE 指令分别进入 WFI 和 WFE 模式。WFI 和 WFE 模式是处理器的一种低功耗状态，处理器的大部分时钟被关闭，但处理器保持上电状态。处理器的静态功耗被大大降低。只有一小部分用于把处理器从 WFI 模式唤醒的逻辑的时钟保持打开。

6.6.2.1.3 DDRC

DDRC 支持 DDR clock gating 功能. 此功能可以在 DDR 访问闲时降低 DDR 器件的功耗。使能此功能 DDRC 会在内部命令排空后开始计数，如果等到系统配置计数值到达后仍然没有新的访问进入 DDRC，DDRC 会将 DDR 颗粒的 CKE 拉低，进入 DDR power down 模式。当新的命令到来或者自动刷新时间到达，DDRC 会将 CKE 拉高退出低功耗模式，进行读写或者刷新访问访问。

6.6.2.1.4 DFS

Hip04 工作模式分为两种：Slow 模式和 Normal 模式。

在 Slow 模式下，系统的大部分模块都停止工作，只有 CPU，总线和系统控制器等少数几个模块工作，系统地工作时钟来源于 25M 晶振。Normal 模式是系统的正常工作模式，系统时钟来源于 PLL 正常输出时钟。

在 Normal 模式下 CPU，总线和加速引擎模块的时钟可以根据实际的业务场景动态地调整，降低系统功耗。详细的时钟频率配置参看 CRG Functional Specificatioin。

6.6.3 Power Gating

Hip04 通过片上电源开关 MT-CMOS 关闭处于空闲模式模块的电源来降低芯片的静态功耗：

- 加速引擎支持两种 Power Gating 模式：静态 Power Gating 和动态 Power Gating；
- 支持 L3 data SRAM、各个加速引擎中集成的的 SRAM 的 Power Gating。

6.6.3.1 加速引擎/接口 Power Gating

Hip04 加速引擎支持两种 Power Gating，静态 Power Gating 和动态 Power Gating。

6.6.4 pd_core 子电源域

如上节所述，为了降低静态功耗，芯片的一些模块支持 power gating。因此 VDD_CORE 电源域被划分成多个子电源域，0 列出了 Hip04 各个 pd_core 子电源域。通过片上电源开关逻辑 MTCMOS 以及 PM 的控制，可以独立的关断任何一个子电源域。唯一的一个例外是 pd_AO 子电源域，在 pd_core 上电之后一直保持有电状态。CPU（Cortex-A15）控制 PM，PM 位于 pd_AO 子电源域。子电源域的下/上电都是由 PM 控制，下/上电过程 PM 可以对 CRG 发出关/开时钟，复位的请求。另外 PM 也需要控制隔离单元逻辑，防止 power down 模块的 floating 输出信号对下游逻辑的影响。软件控制静态 power gating 模块的低功耗握手、复位、时钟、隔离单元和 MTCMOS 的使能；动态 power Gating 下通过 PM 模块内部的状态机，控制 Powern Down 和 Power Up 的时钟、复位、隔离单元和 MTCMOS 的使能信号的产生。pd_core 的子电源域

Sub power domain	说明	模块
pd_AO	always-on 电源域	其余逻辑：CPU Cluster 0，CPU Cluster1 ，CRG 、SysCtrl、PM 等

6.6.5 AVS

Hip04 AVS 分为两种：静态工艺补偿和动态温度补偿，通过微调电压来降低动态功耗。

AVS 静态工艺补偿，即支持为不同工艺角的芯片选择不同的默认工作电压来降低芯片的功耗。调整的精度受限于外部电源管理芯片每步调压的精度（例如，如果选用 12.5mv）。静态补偿主要是通过片上集成的 HPM(Hardware Performance Monitor)和片上 efuse 完成。

除了 AVS 静态工艺补偿，Hip04 也支持运行过程中的动态温度补偿，即保证当前工作频率不变的情况下自动微调工作电压到功耗最低的电平。电源电压微调的过程对业务软件透明。AVS 温度补偿也需要使用 HPM 的延迟信息来查找芯片当前合适的工作电压（参考电压）。

6.6.5.1 HPM

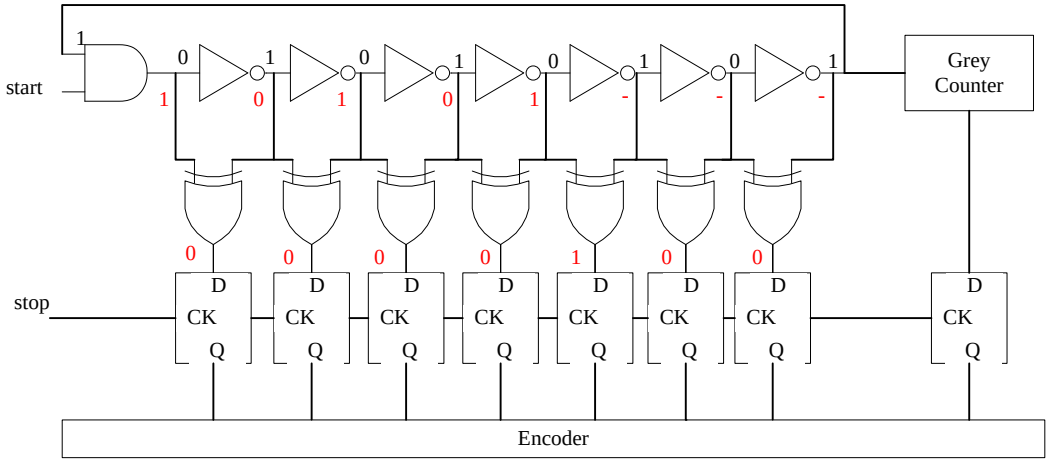


图6-3 HPM Oscillator 电路

上述电路使用一个反相器环路，在 start 信号有效之后，随着时间流逝，环路上的节点会逐个反向，并自激振荡；stop 信号用来锁存环路上的数据。各节点的初始电平在图中用黑色字体标记；假如 stop 信号有效时信号传递到第 4 个反相器，各节点的当前电平在图中用红色字体标记。那么 Encoder 模块可以在第 5 个锁存器上得到一个高电平。如果延时时间大于环路延时，则使用 GrayCounter 来统计。这样，结合 Counter 计数和环路上获取的高电平信号的位置即可得到总的延时数据。

延时测量电路中反相器链的延时受到电压、温度和工艺等因素的影响。同时，这个延时与芯片关键路径的延时存在一定的比例关系。

start 和 stop 信号的间隔时间，一般是工作时钟周期的整数倍，则延时测量的测量数据可以反映芯片关键路径的延时。假定在某个标准环境下 HPM 测试数据是 RCC（通过测试获取），如果实际工作时测量到的数据是 PC。AVS 通过调节电压来保证 PC 与 RCC 互相匹配。如果 PC 小于 RCC，表明当前的电路延时无法满足工作频率的要求，需要提升电压；如果 PC 大于 RCC，表明当前的电路延时超额满足工作频率的要求，则电压可下降。

HPM 测量电路有几个主要参数：反相器延时、反相器级数、计数器位数、PC 编码方式。

6.6.5.2 eFuse

在生产测试中确定的 Process Class 将存储在采用 one-hot 编码的 efuse 中。芯片上电后 efuse 映射到 CPU 可以访问的 32-bit 的状态寄存器中。最低位确定芯片的 Process Class。如果有多个位被设置，则最低位决定 Process Class。

表 6-4 给出了芯片不同 Process Class 对应的缺省电压 (Intersil ISL62882C, VID [6:5]= “01”)。

表6-4 Process Class-efuse

Bit	描述
[0]	Process Class 28
[1]	Process Class 27
[2]	Process Class 26
[3]	Process Class 25
[4]	Process Class 24
[5]	Process Class 23
[6]	Process Class 22
[7]	Process Class 21
[8]	Process Class 20
[9]	Process Class 19
[10]	Process Class 18
[11]	Process Class 17
[12]	Process Class 16
[13]	Process Class 15
[14]	Process Class 14
[15]	Process Class 13
[16]	Process Class 12
[17]	Process Class 11
[18]	Process Class 10
[19]	Process Class 9
[20]	Process Class 8
[21]	Process Class 7

[22]	Process Class 6
[23]	Process Class 5
[24]	Process Class 4
[25]	Process Class 3
[26]	Process Class 2
[27]	Process Class 1
[28]	Process Class 0
31:[29]	Reserved

PMU VID [VID4:VID0]	电压值（v)	Process Class
11100	0.7500	28
11011	0.7625	27
11010	0.7750	26
11001	0.7875	25
11000	0.8000	24
10111	0.8125	23
10110	0.8250	22
10101	0.8375	21
10100	0.8500	20
10011	0.8625	19
10010	0.8750	18
10001	0.8875	17
10000	0.9000	16
01111	0.9125	15
01110	0.9250	14
01101	0.9375	13
01100	0.9500	12
01011	0.9625	11
01010	0.9750	10
01001	0.9875	9
01000	1.0000	8

PMU VID [VID4:VID0]	电压值 (v)	Process Class
00111	1.0125	7
00110	1.0250	6
00101	1.0375	5
00100	1.0500	4
00011	1.0625	3
00010	1.0750	2
00001	1.0875	1
00000	1.1000	0

6.6.5.3 AVS 静态工艺补偿

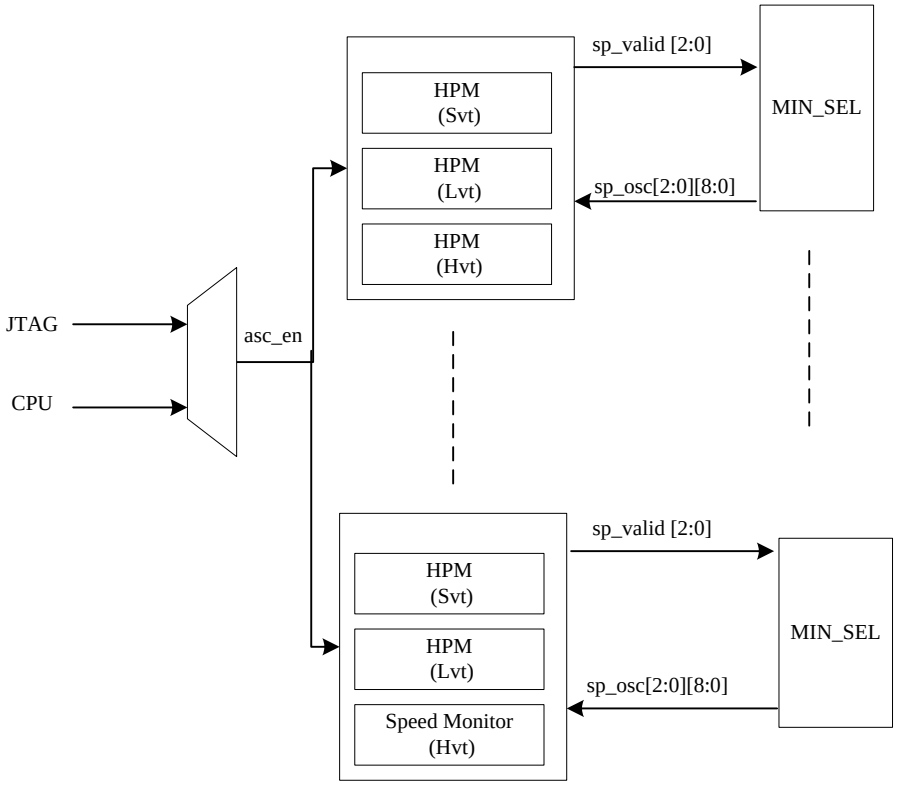


图6-4 HIP04 AVS 静态工艺补偿电路

如图 6-4 所示，Hip04 采用两种方式启动静态工艺补偿的 HPM：JTAG 方式和软件方式。

AVS 静态补偿的测试应用流程如所示，解释如下：

2 Power Measurements

3 需要在不同电压下测量芯片的静态和动态功耗，需要使用不同场景下的循环测试图形。通过 power-voltage 曲线找到 AVS 静态工艺补偿的功耗影响。

4 Frequency Measurement of Speed Monitor：在不同电压下测量 HPM 的计数值。通过一个功能测试图形（CPU 和/或 JTAG 方式）初始化 HPM，输出有效的 HPM 的计数器的值；

5 PM 比较这些 HPM 有效的值后选择最差的作为参考值，并存储起来；

6 Tester 读取这个参考值;

7 FuseProcess：通过 Process Class 电压查找表（表 6-4 Process Class 电压查找表）找到对应的电压值，对应于 PMU 的 VID 配置，通过 Fuse 过程把配置值烧入片上 eFuse 中;

8 VID Configuration：在系统正常 boot 时，eFuse 的值会通过接口来配置片外的 PMU 芯片得到工作电压。其应用框图如图 6-5 所示。

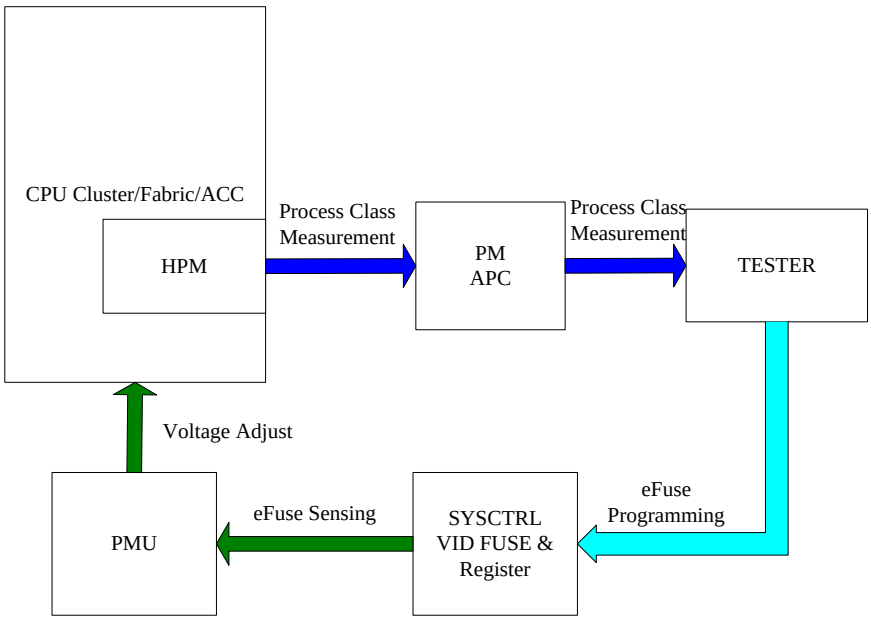


图6-5 AVS 静态补偿应用框图

6.6.5.3.2 AVS 静态工艺补偿生产测试流程

AVS 测试流程分为两个步骤：CP 和 FT。

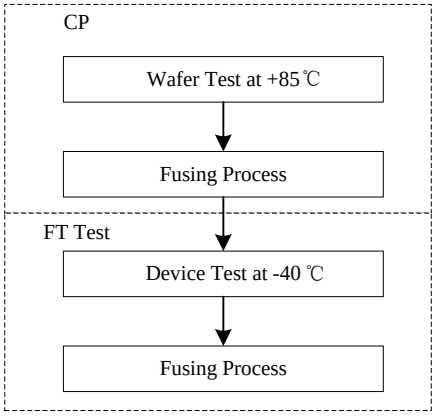


图6-6 AVS 静态工艺补偿测试

AVS 生产测试需要按照海思的测试序列完成。由于 Test Coverge 和质量需求，在 HPM 频率测量必须执行基于扫描的延迟测试。

CP Test 流程如下：

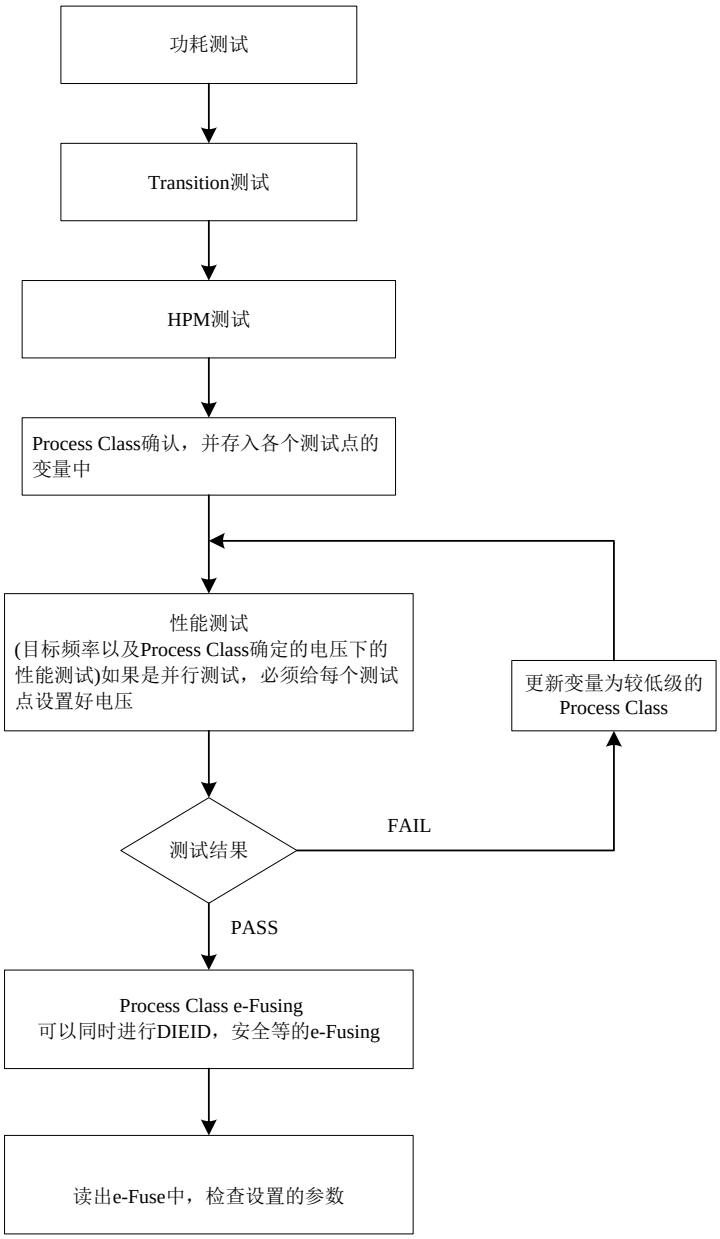


图6-7 CP 流程

FT 流程如下：

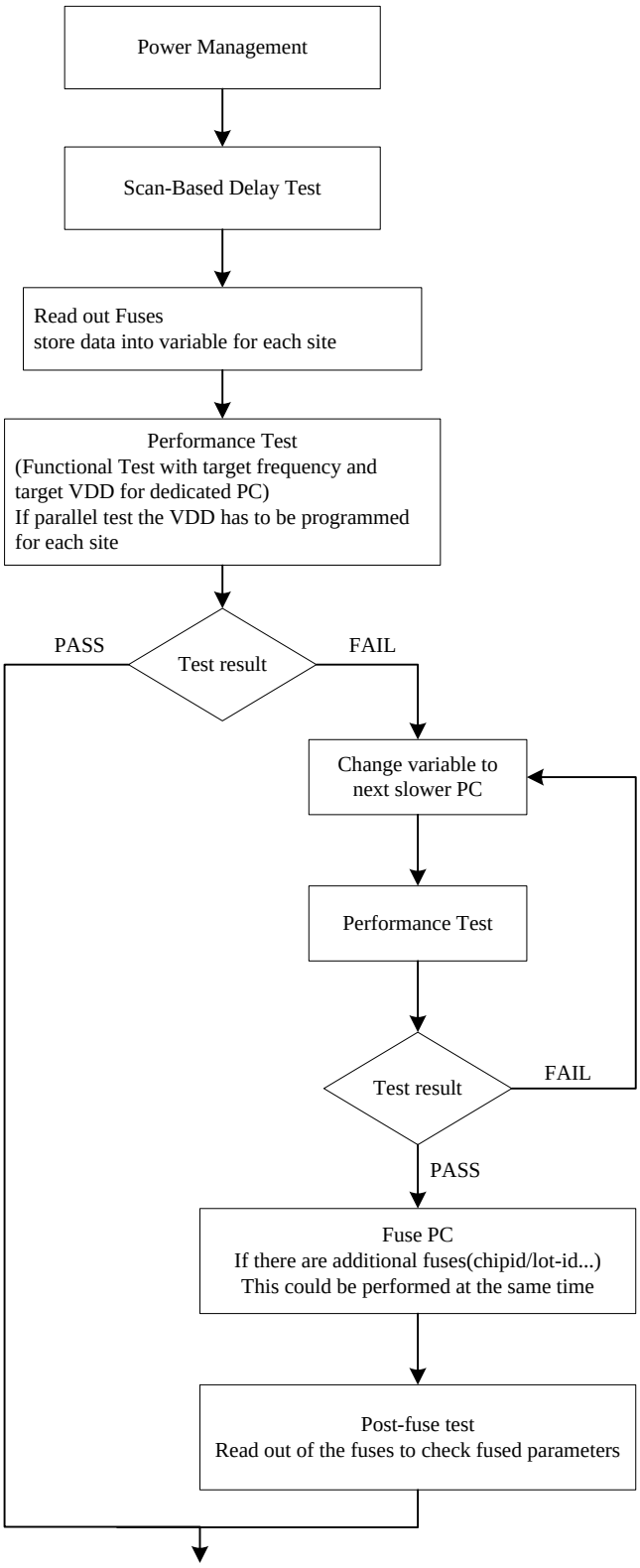


图6-8 FT 流程

在 BE 测试下通过下调芯片的 PC，进而上调芯片的实际工作电压，能够把“坏”片子重新归类为好片子，提高良品率。

详细测试流程参看：

Hip04 AVS0 测试流程.doc

6.6.5.4 AVS 动态温度补偿

AVS 动态温度补偿同样根据芯片上集成的 HPM 来反馈芯片的工作条件。在一定的工作场景下启动 AVS 动态温度补偿，HPM 输出有效的原始性能码，根据配置的参考码来确定芯片可工作的最小电压，并自动编程外部 PMU 接口，使 PMU 输出可满足芯片工作的最小电压，达到动态温度补偿的目的。

图6-9 AVS 动态温度补偿总体框图

6.6.5.4.2 AVS 温度补偿工作流程

AVS 温度补偿步骤：

步骤 1 在业务正常运行中，CPU 配置寄存器使能 HPM；

步骤 2 HPM 达到稳定的输出后，PM 选 HPM 有效的值中的最小值；

步骤 3 通过与参考值的比较，PM 选择正确的电压调整值来配置外部 PMU；

步骤 4 电压调整稳定后，通过 EPFF_WARN 的检测电压调整是否满足当前工作状态，如果告警，上调电压。

----结束

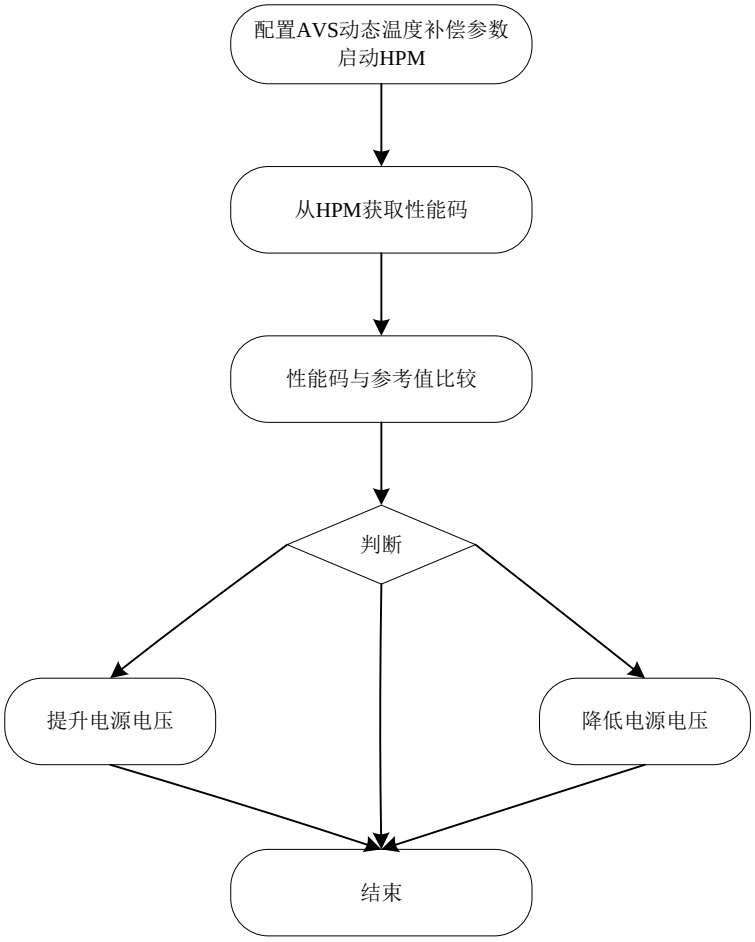


图6-10 AVS 温度补偿工作流程

6.6.6 低功耗控制结构

Hip04 低功耗控制结构主要包括 CPU 子系统、Speed Monitor、PM 和 CRG 等组成。CRG（Hip04 采用分布式 CRG 实现方案，为了简化图中只画出一个 CRG）提供 PM、CPU、ACC 子系统的时钟、复位逻辑，并支持模块的 clock gating；Speed Monitor 提供 CPU 和 ACC 子系统的延迟信息；PM 主要用于 Cortex-A15 单、多核 power gating、子系统模块 power gating 和 AVS 静态、温度补偿；CPU 子系统用来配置和访问 PM 模块。

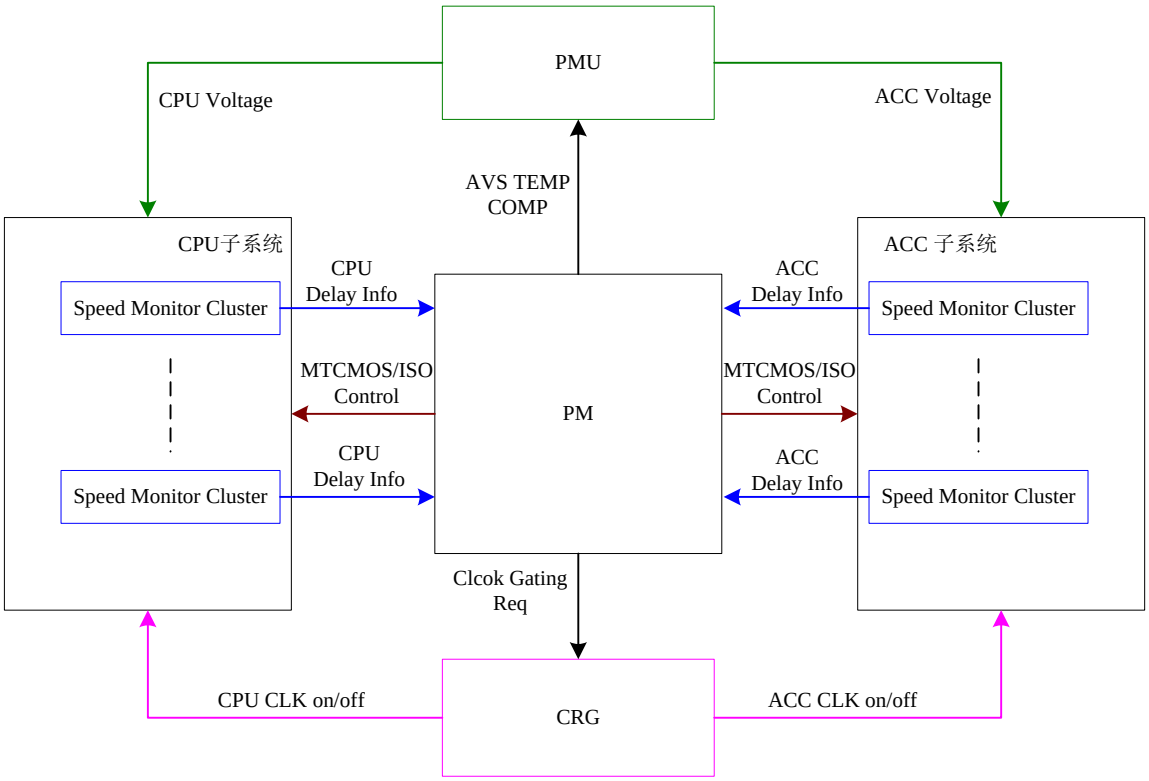


图6-11 低功耗控制结构框图

6.6.7 Power Down 调试

在本节中调试逻辑指的不仅仅是调试逻辑本身，还包括其他的一些 trace 逻辑和 trace Memory。

在应用模式下如果不需要调试逻辑，必须尽可能降低调试逻辑的静态功耗。如果调试逻辑的漏电太大而难以接受的情况下应该使用独立的调试逻辑电源域。如果调试逻辑能够采用专门的电源域，那么模式下模块被 power down 的时候调试逻辑能够保持 powered up。在应用模式下这个调试逻辑可以独立于 powered up 的模块而 power down。调试逻辑使用专用的电源域的最明显的好处就是在所有的应用模式下避免调试逻辑的漏电。但是这也带来两个问题。在很多情况下调试逻辑都是与被调试的模块紧耦合的，因此调试逻辑都是集成在同一个子系统中，能够更好的优化时序并降低面积开销。另外引入一个调试电源域需要在子系统中增加一个额外的 power island，并且在子系统和调试逻辑之间需要双向 isolation 逻辑来防止无法预测的泄漏电流。模块和调试逻辑之间的 isolation 逻辑必须是双向的，因为必须考虑两种关键场景：调试模式下仅仅模块 power down 或者应用模式下仅仅调试逻辑 power down。这些都需要额外的面积、设计和验证开销。

所有 power down 的场景必须是可调试、可跟踪的，特别是模块在开始或者结束 power down 转换时模块的行为。为了调试系统在 Power Down 场景下的行为，片上调试和 Trace 功能必须保持上电状态。

6.6.8 PMU 接口

芯片提供两种与外部 PMU 芯片的接口：IMVP6.5 和 PMbus1.1。支持 IMVP6.5 接口的 PMU 芯片包括 Intersil ™ SL62882 系列。PMbus 的对接芯片包括 TI TPS40400，Intersil ZL6105 和 Huawei IPower。电源管理芯片的配置接口如图 1-15 所示。

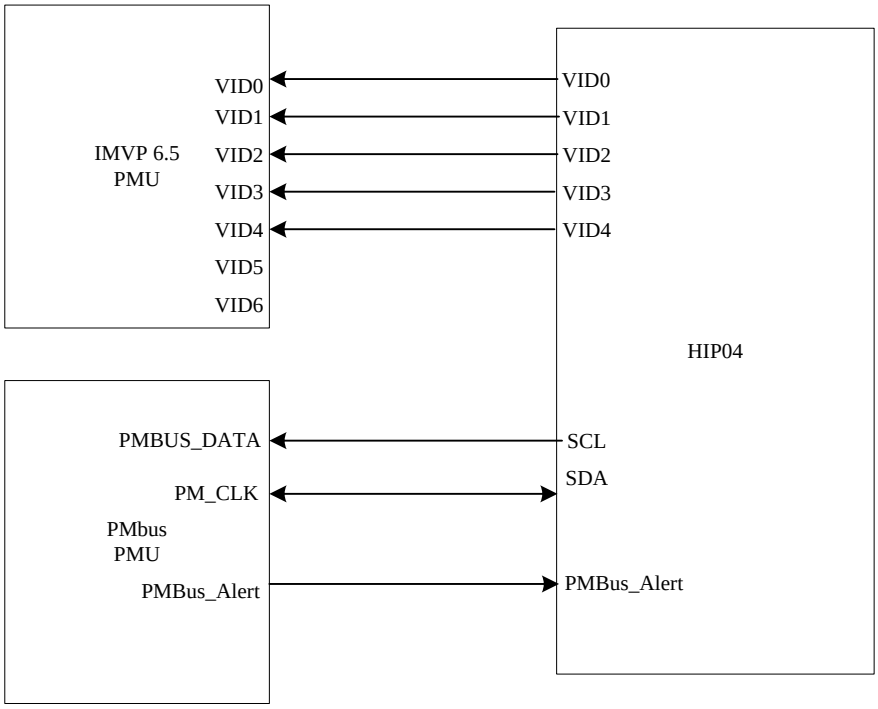


图6-12 Hip04 PMU 接口

6.6.8.2 IMVP 6.5 PMU 接口

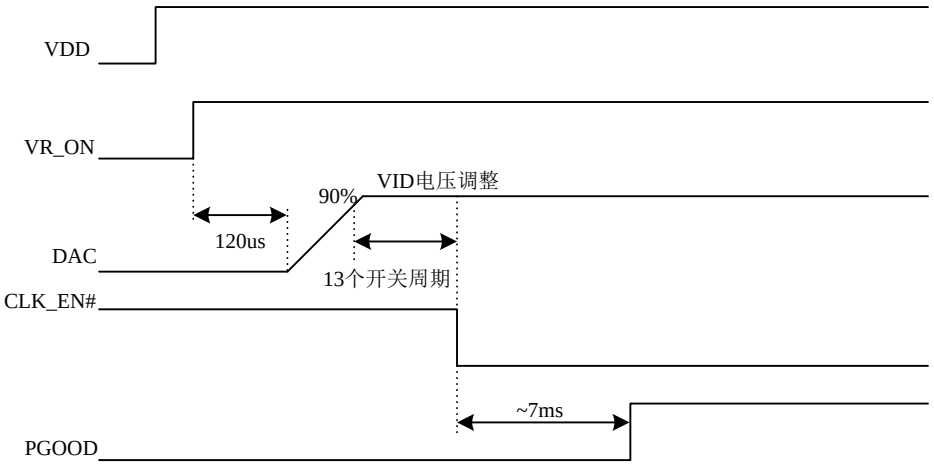
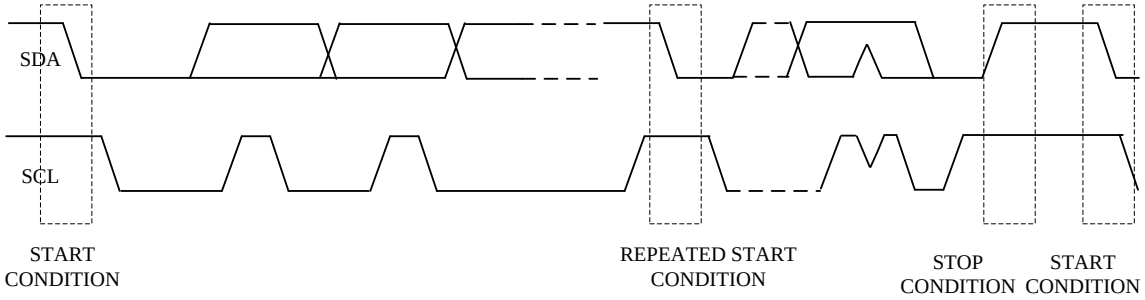


图6-13 IMVP 6.5 PMU 上电时序

6.6.8.3 PMbus1.1 接口



S	SLAVE ADDRESS	Wr	A	DATA BYTE	A	P

S START CONDITION
Sr REPEATED START CONDITION
Rd READ (BIT VALUE OF 1)
Wr WRITE(BIT VALUE OF 0)
x SHOWN UNDER A FIELD INDICATES THAT THAT FIELD IS REQUIRED TO HAVE THE VLAUE OF x
A ACKNOWLEDGE (THIS BIT POSITION MAY BE 0 FOR AN ACK OR 1 OF A NACK)
P STOP CONDITION
PEC PACKET ERROR CODE

MASTER TO SLAVE

SLAVE TO MASTER

图6-14 PMbus PMU 接口时序和协议包格式

6.7 工作方式

6.7.1 静态 power gating

系统上电 boot 完成后，根据不同的业务场景，按照如下顺序配置 PM 寄存器完成静态 power gating：

1) 软件配置静态Power Gating的复位请求寄存器PM_SRST_REQ，复位相应的加速引擎；
2) 软件配置静态Power Gating的时钟门控寄存器PM_SCLK_EN，门控相应的加速引擎时钟；
3) 软件配置静态Power Gating的隔离单元使能寄存器PM_SISO_EN，隔离相应的加速引擎的输出；
4) 软件配置静态Power Gating的MTCMOS使能寄存器PM_SMTCMOS_EN，相应加速引擎被power gating。

软件可以通过查询静态 Power Gating 的 MTCMOS 状态寄存器 PM_SMTCMOS_STAT，获取相应的加速引擎的 power gating 信息。

6.7.2 动态 Power Gating

1） 软件配置SEC0/1进入低功耗状态周期计数寄存器PM_SEC0/1_IDLE_DLY，设置SEC0/1进入IDLE模式后多久可以进入Power Gating例程；
2） 软件配置SEC动态Power Gating请求寄存器PM_PD_REQ，请求SEC0/1Power Gating；

软件可以实时查询 SEC 动态 Power Gating 状态寄存器 PM_PD_STAT，获取当前 SEC 是否处于工作还是 IDLE 状态。软件可以通过 SEC 动态 Power Gating 的 MTCMOS 状态寄存器 PM_MTCMOS_STAT 查询 SEC Power Gating 的状态，软件也可以通过查询 SEC 动态 Power Gating 状态机状态寄存器，获取 PM 当前 Power Gating 的状态。

6.7.3 AVS 动态温度补偿配置流程 1 – IMVP 接口

1） 软件配置AVS动态温度补偿HPM使能屏蔽寄存器PM_AVS_DTC_HPM_EN，确定AVS动态温度补偿所使用的HPM类型及其分布。
2） 软件配置AVS动态温度补偿HPM时钟分频寄存器PM_AVS_DTC_HPM_CLKDIV，设置HPM内部工作时钟是当前子系统时钟的几分频。
3） 软件配置AVS动态温度补偿参数寄存器0 PM_AVS_DTC_PARA0，设置HPM原始性能码移位值，AVS动态温度补偿的IR Drop余量，AVS动态温度补偿参考校准码和AVS动态温度补偿原始性能码偏移值；

4） 软件配置AVS动态温度补偿参数寄存器5 PM_AVS_DTC_PARA5，设置AVS动态温度补偿的最低电压和最高电压；

5） 软件配置AVS动态温度补偿参数寄存器6 PM_AVS_DTC_PARA6，设置AVS动态温度补偿HPM原始性能码的取样次数；

6） 软件配置PMU升降压时间寄存器PM_PMU_UPDN_TIME，设置AVS动态温度补偿其间外部Voltage Regulator器件的升降压时间；

7） 软件配置AVS动态温度补偿重新启动调压时间寄存器PM_TRIGGER_TIME，设置AVS动态温度补偿重新启动调压时间；

8） 软件配置AVS动态温度补偿使能寄存器PM_AVS_DTC_EN，启动AVS动态温度补偿流程。

软件可以通过查询 AVS 动态温度补偿状态机状态寄存器 PM_AVS_DTC_FSM_STAT，察看 AVS 动态温度补偿的当前状态。AVS 动态温度补偿模块 HPM 有效值寄存器 PM_DTC_XXX_HPMSL/H 寄存器反映实时 HPM 的取样值。

6.7.4 AVS 动态温度补偿配置流程 1 – PMBus 接口

1） 软件配置PMBus控制寄存器PM_PMBUS_CTRL的pmbus_en，启动PMBus控制器；

2） 软件配置PMBUS Slave器件配置寄存器PM_PMBUS_SVL_CFG，配置PMBus命令模式，PMBus Slave器件地址和PMBus命令VOUT模式底数；

3） 软件配置I2C控制寄存器PM_I2C_CON，启动DW_I2C_APB主模式，选择相应速度；

4） 软件根据PM_I2C_CON的设置配快速/标准速度下的SCL时钟低电平和高电平时间寄存器PM_I2C_S/FS_SCL_LCNT和PM_I2C_S/FS_SCL_HCNT，设置快速/标准速度下SCL时钟高、低电平时间；

5） 软件配置I2C访问Slave器件的地址寄存器PM_I2C_TAR，配置Slave器件的访问地址和模式；

6） 软件配置I2C工作模式使能寄存器PM_I2C_ENABLE，启动I2C；

7） 软件配置AVS动态温度补偿HPM使能屏蔽寄存器PM_AVS_DTC_HPM_EN，确定AVS动态温度补偿所使用的HPM类型及其分布。

8） 软件配置AVS动态温度补偿HPM时钟分频寄存器PM_AVS_DTC_HPM_CLKDIV，设置HPM内部工作时钟是当前子系统时钟的几分频。

9） 软件配置AVS动态温度补偿参数寄存器0 PM_AVS_DTC_PARA0，设置HPM原始性能码移位值，AVS动态温度补偿的IR Drop余量，AVS动态温度补偿参考校准码和AVS动态温度补偿原始性能码偏移值；

10） 软件配置AVS动态温度补偿参数寄存器5 PM_AVS_DTC_PARA5，设置AVS动态温度补偿的最低电压和最高电压；

11） 软件配置AVS动态温度补偿参数寄存器6 PM_AVS_DTC_PARA6，设置AVS动态温度补偿HPM原始性能码的取样次数；

12） 软件配置PMU升降压时间寄存器PM_PMU_UPDN_TIME，设置AVS动态温度补偿其间外部Voltage Regulator器件的升降压时间；

13） 软件配置AVS动态温度补偿重新启动调压时间寄存器PM_TRIGGER_TIME，设置AVS动态温度补偿重新启动调压时间；

14） 软件配置PMBus控制寄存器PM_PMBUS_CTRL的pmbus_en，启动PMBus控制器，PMBus AVS调压使能和PMBus PEC使能信号；

15） 软件配置DW_I2C_APB IO控制寄存器PM_I2C_IO_CTRL，设置I2C数据管脚延迟；

16） 软件配置AVS动态温度补偿使能寄存器PM_AVS_DTC_EN，启动AVS动态温度补偿流程；

软件可以通过查询 AVS 动态温度补偿状态机状态寄存器 PM_AVS_DTC_FSM_STAT，察看 AVS 动态温度补偿的当前状态。AVS 动态温度补偿模块 HPM 有效值寄存器 PM_DTC_XXX_HPMSL/H 寄存器反映实时 HPM 的取样值。

6.7.5 AVS0 HPM 静态测量

1） 软件配置寄存器PM_AVS_SPC_HPM_CLKDIV，用来设置HPM的测试时钟周期；

2） 软件配置寄存器PM_SPC_HPM_EN，启动对应模块的不同VT的HPM；

3） 读取寄存器PM_SPC_*module*_HPM*S(L/H)*_VAL获取当前PVT条件下的HPM数据。

如果需要多次读取 HPM 的静态值，需要把寄存器 PM_SPC_HPM_EN 先去使能，然后再使能。